

CS162 Operating Systems & Systems Programming

Lecture 1: What is an Operating System? (Detailed Exam Notes)

1. Defining the Operating System: Roles & Paradigms

Historically, the tasks of organizing and routing work on computer hardware transitioned from human **switchboard operators** to physical **computer operators**, and eventually into a software layer: the **Operating System**.

There is no universally accepted industry definition of an operating system. Pragmatically, it can be defined as "**everything a vendor ships when you order an operating system**" (though this varies wildly). Strictly speaking, the **kernel** is the one program running at all times on the computer. Everything else is either a **system program** (ships with the OS) or an **application program**.

To understand how an OS operates, systems programmers analyze it through three conceptual roles: the **Referee**, the **Illusionist**, and the **Glue**.

The Referee

The OS acts as an arbiter to manage resource allocation, communication, protection, and isolation.

- **Resource Allocation:** Decides which processes receive hardware resources (CPU cycles, physical memory pages, network bandwidth).
- **Isolation and Protection:** Enforces a secure boundary to prevent buggy or malicious programs from crashing other programs or the kernel itself.

The Illusionist

The OS virtualizes physical hardware to provide applications with clean, simple, and seemingly limitless abstractions of physical resources.

- **Infinite Memory:** Gives each process the illusion of having a vast, contiguous address space (Virtual Memory) despite physical RAM constraints.
- **Dedicated Machine:** Gives each program the illusion that it has exclusive access to its own CPU core, hiding context-switching details.
- **Virtualization:** Masks physical limitations and hardware-specific quirks, translating raw sectors into structured files.

The Glue

The OS acts as a standardizing agent, providing common services and look-and-feel APIs.

- **I/O Services:** Standardizes interfaces for persistent storage (File Systems), network streams (Sockets), display windows, and user inputs (Mice/Keyboards).
 - **Decoupling Application from Hardware:** Applications do not need to write custom drivers for every hard drive or network adapter; the OS offers standardized, high-level system APIs.
-

2. What Makes a "System"?

A **system** is defined as an entity composed of multiple interrelated, interacting parts. Developing robust systems programming relies on a rigorous engineering mindset:

- 1. **Meticulous Error Handling:** Designing defensively for physical machine limitations and unpredictable failures.
- 2. **Defensive Programming:** Anticipating malicious or careless actions from users or external systems.
- 3. **Concrete Hardware Awareness:** Treating the underlying hardware as a concrete physical machine with real performance costs, bottlenecks, and error rates, rather than a perfect mathematical abstraction.

3. The Scale of Timescales (Jeff Dean's Numbers)

An operating system acts as a scheduler and resource allocator across radically different timescales. To build efficient code, systems programmers must develop an intuitive grasp of latency scales:

Operation	Latency (Nanoseconds)	Latency (Standard Time)	Human-Scale Analogy (Scaled up: 0.5 ns L1 Cache = 1 Heartbeat)
L1 Cache Reference	0.5 ns	0.5 ns	1 second (1 heartbeat)
Branch Mispredict	5 ns	5 ns	10 seconds
L2 Cache Reference	7 ns	7 ns	14 seconds
Mutex Lock/Unlock	25 ns	25 ns	50 seconds
Main Memory Reference (DRAM)	100 ns	100 ns	3.3 minutes
Compress 1KB (Zippy)	3,000 ns	3 μs	1.6 hours
Send 2KB over 1 Gbps Network	20,000 ns	20 μs	11.1 hours
Read 1MB Sequentially from Memory	250,000 ns	250 μs	5.7 days
Round Trip in Same Datacenter	500,000 ns	500 μs	11.5 days
Disk Seek	10,000,000 ns	10 ms	231.5 days (≈ 7.7 months)
Read 1MB Sequentially from Disk	20,000,000 ns	20 ms	1.3 years
Send Packet (CA → Netherlands → CA)	150,000,000 ns	150 ms	9.5 years

Human-Scale Mathematical Derivation

Let S be the scale factor. If we scale a 0.5 ns L1 Cache reference to equal 1 second: $S = \frac{1\text{ s}}{0.5 \times 10^{-9}\text{ s}} = 2 \times 10^9$

Applying this scale factor to a physical **Disk Seek** (10 ms): Scaled Time = $10 \times 10^{-3}\text{ s} \times (2 \times 10^9) = 2 \times 10^7$ seconds $\approx$ 231.5 days

Exam Takeaway: This massive gap in speeds explains why the OS implements caching, speculative execution, prefetching, and asynchronous I/O. Waiting for a physical disk seek is equivalent to a human waiting over 7 months for a document!

4. The "Process" Abstraction

To run programs safely and efficiently, the OS abstracts physical hardware details into a secure execution environment: the **Process**.

What is a Process?

A process is an execution environment with restricted rights provided by the OS. It consists of three primary components:

1. **An Address Space:** The range of memory locations the process is allowed to read and write. This is managed via page tables.
2. **One or More Threads of Control:** The actual execution units running instructions inside that address space.
3. **Additional System State:** Resources allocated to the process by the OS, including:
 - File descriptors (Open files).
 - Sockets (Network connections).

Process View Comparison

- **System Programmer's View:** Programs compile down to machine instructions linked against system libraries (`System Libs`). The process acts as the target execution environment, exposing Threads, Address Spaces, Files, and Sockets.
- **Operating System's View:** The OS manages multiple processes (Process 1, Process 2) simultaneously. It translates calls from the application interface down to raw physical components (CPU cores, MMU/Page Tables, physical memory, storage Controllers, network interfaces).

5. Basic OS Mechanics: Protection, Switching, and Dual-Mode Operation

To prevent user applications from compromising the system, modern computer architectures enforce a **hardware boundary** managed by **Dual-Mode Operation**.

Dual-Mode Operation

The hardware CPU must provide at least two distinct execution modes:

1. **User Mode (Mode Bit = 1):** Applications execute with restricted instructions. They cannot directly access physical hardware, modify page table registers, or disable CPU interrupts.
2. **Kernel Mode / Supervisor Mode (Mode Bit = 0):** The OS kernel runs with full access to privileged CPU instructions and the entire physical system memory.

Transition Boundaries

Applications must transition from user mode to kernel mode to perform any protected operations (e.g., reading a file, sending a network packet). These transitions are highly controlled:

- **System Calls:** Program-initiated requests to the kernel (e.g., `read()` , `write()` , `sys_fork`).
- **Exceptions:** Synchronous events caused by programmer errors (e.g., divide by zero, page fault, segmentation fault).
- **Interrupts:** Asynchronous hardware-driven events (e.g., a timer interrupt, keyboard press, network packet arrival).

When an application calls a system call, it triggers a **Trap** instruction. The hardware switches the mode bit from 1 to 0, jumps to a pre-defined address in the OS's system call table (handling standard arguments), executes the code safely in kernel mode, and executes a return-from-system-call instruction to switch the mode bit back to 1.

Code Example: Illustrating System Call vs. Standard Library Call in C

This simple program demonstrates how standard memory management and I/O translate to system-level resource requests.

```
c

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/stat.h>
#include <fcntl.h>

int main(void) {
    // 1. Standard Library Memory Allocation (User Space)
    // malloc manages memory in user space via standard heap tracking structures.
    // When more memory is needed, it issues a system call (sbrk or mmap) behind the scenes.
    char *buffer = (char *)malloc(100 * sizeof(char));
    if (buffer == NULL) {
        perror("Failed to allocate memory");
        return 1;
    }

    // 2. Issuing a direct system call (write)
    // Standard printf() uses buffers in user space. Standard write() is a direct system call
    // that immediately traps into the kernel (Mode Bit -> 0) to output bytes to File Descriptor 1
    (stdout).
    char msg[] = "Writing directly using a system call!\n";

    // Trap into Kernel Mode
    write(STDOUT_FILENO, msg, sizeof(msg) - 1);

    // Clean up heap allocation
    free(buffer);
    return 0;
}
```

Context Switching Mechanics

When switching execution from Process 1 to Process 2:

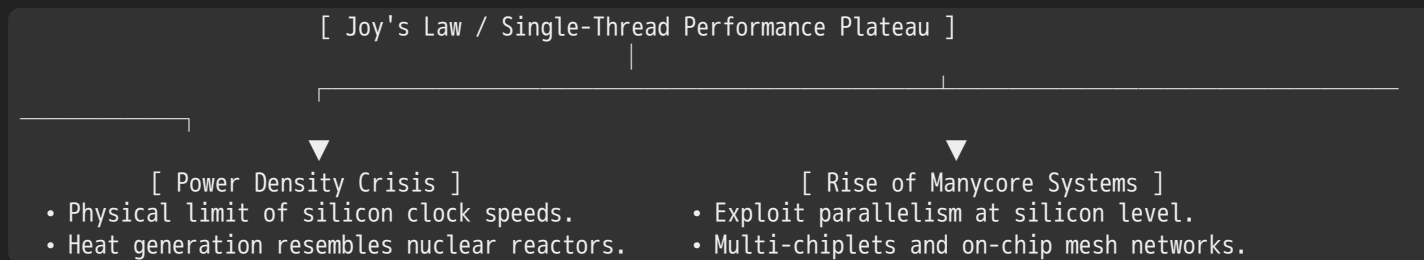
1. The CPU receives a timer interrupt or a blocking system call.
2. The hardware traps into **Kernel Mode**.
3. The OS saves the state (program counter, registers, page table pointers) of **Process 1** into memory.
4. The OS loads the previously saved hardware state of **Process 2** into the physical registers.
5. The OS changes the hardware mode bit back to **User Mode** and executes from the restored Program Counter of Process 2.

Fault Containment (Segmentation Faults)

If Process 2 attempts to write to the physical memory of Process 1, or to protected OS kernel memory, the hardware Memory Management Unit (MMU) catches this instruction before it executes because the page table registers do not map that physical memory inside Process 2's address space. The hardware immediately traps to the kernel, which halts the offender and throws a `Segmentation fault (core dumped)` error. This isolation guarantees system reliability.

6. Technological Trends: Why OS Design is Constantly Evolving

The landscape of computer systems is continually shifting due to scaling trends and physical constraints:



Bell's Law & The Internet of Things (IoT)

Bell's Law states that a completely new class of computer systems emerges roughly every 10 years. This evolution has fundamentally shifted the ratio of computers per person:

- **1960s:** Mainframes (1 : 10^6 ratio—thousands of people sharing a single machine).
- **1980s:** Personal Computers (1 : 1 ratio—one workstation/PC per person).
- **2010s:** Mobile and Smartphones (surpassed PC shipments in 2011, establishing mobile-first systems design).
- **2020s and beyond:** The Internet of Things / Motes (10^3 : 1 ratio—thousands of embedded microcontrollers per human being).

Moore's Law: Not Dead, but Evolving

Originally, Gordon Moore predicted in 1965 that transistor density on integrated circuits would double roughly every 18 months.

- Around **February 2016**, the classical physical scaling of transistors (2D scaling) faced severe physical limits.
- However, manufacturers pivoted to **3D stacked chips**, multi-layer chip packaging, and advanced architectures (like RibbonFET and PowerVia) to target chip designs containing over **1 trillion transistors** by 2030.

The Power Density Barrier

Historically, shrinking transistors meant they became faster and consumed less power (Dennard Scaling). But as sizes dropped below 10 nm, physical leakage current caused heat generation to skyrocket, leading to a massive power density wall.

- Unabated scaling extrapolation showed chip temperatures reaching levels comparable to a **nuclear reactor, rocket nozzle, or the sun's surface**.
- This put an end to classical clock speed increases (plateauing around 3 GHz to 5 GHz) and shifted the focus of design to multi-core efficiency and battery life optimization.

Joy's Law & Multi-Core Systems

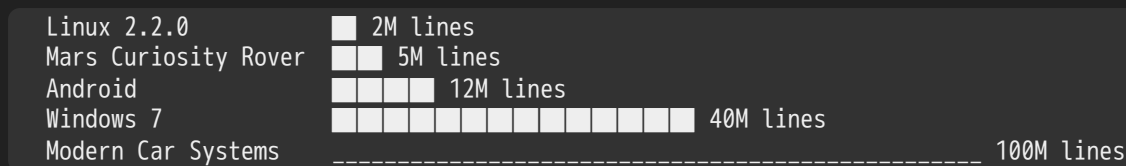
Because single-threaded performance stopped scaling exponentially around 2002 (Joy's Law plateau), chip designs shifted toward multi-core parallelism.

- Modern enterprise microprocessors like **Intel Sapphire Rapids (2023)** run up to 60 cores and 120 threads per socket.
- They use a physical on-chip **Mesh Interconnect Network** to link up to 4 distinct chiplets, running up to 4 TiB of DDR5 memory with dedicated hardware accelerators for encryption, AI workloads, and security.

Exam Takeaway: Modern operating systems cannot treat the computer as a simple sequential instruction processor. An OS must act as a massive distributed network coordinator managing complex thread scheduling, cache coherence, NUMA memory access, and heterogeneous hardware accelerators.

7. Software Complexity Case Study: The Mars Rover Pathfinder

Modern software systems are incredibly complex. Visualizing the sheer scale of lines of code highlights this issue:



The Pathfinder Constraints

The 1997 Mars Pathfinder mission faced significant operating challenges:

- **Hardware Constraints:** Running a 20 MHz processor, 128 MB of physical DRAM, and the VxWorks Real-Time Operating System (RTOS).
- **System Design:** Multiple processes must run concurrently (camera systems, locomotion, solar panels, and scientific telemetry).
- **Inaccessibility:** You cannot physically push a reset button on Mars; the system must detect hangs and reboot itself automatically.

The Famous Pathfinder Priority Inversion Bug (Syllabus Extension)

Although not explicitly detailed on slide 73, the Mars Pathfinder mission is famous in OS academic history for experiencing a critical **Priority Inversion** failure:

1. A high-priority task (the information bus manager) needed to read/write system telemetry.
2. A low-priority task (the weather monitoring system) acquired a mutual exclusion lock (Mutex) on a shared memory segment.
3. An intermediate-priority task (the communication task) started executing, preempting the low-priority task because its priority was higher.
4. Consequently, the low-priority task could not finish execution to release its mutex. The high-priority task became permanently blocked waiting for the mutex, while the medium-priority task hogged the CPU.
5. The OS **watchdog timer** noticed that the critical high-priority task had not run for a long time, assumed the system had crashed, and forced an immediate hardware reboot.

The Fix: The team enabled **Priority Inheritance** in VxWorks. Under priority inheritance, when a high-priority task blocks on a lock held by a lower-priority task, the lower-priority task temporarily inherits the high-priority level, allowing it to execute quickly, release the lock, and prevent intermediate tasks from preempting it.

8. Essential C Language & GDB Checklist

As a systems programming course, CS162 requires complete comfort with memory management, pointer manipulation, and diagnostic debugging in C. Use this exam study sheet to test your understanding:

Pointer Casts & Function Pointers

Modern OS systems rely heavily on polymorphism using raw `void *` pointer casting and dynamic callback arrays.

```
c
#include <stdio.h>

// A simple callback function matching our function pointer signature
int add_nums(int a, int b) {
```

```

    return a + b;
}

// Struct representing a polymorphic task
typedef struct task {
    int id;
    void *data;                // Polymorphic container pointer
    int (*func)(int, int);     // Function pointer callback
} task_t;

int main(void) {
    int secret_num = 42;

    // Assigning task details
    task_t my_task;
    my_task.id = 101;
    my_task.data = &secret_num; // void* points to our integer
    my_task.func = add_nums;     // Assign function pointer

    // Dereferencing the void* container pointer requires an explicit cast
    int dereferenced_val = *(int *)(my_task.data);

    // Calling the function pointer callback
    int calculation = my_task.func(dereferenced_val, 8);

    printf("Task ID: %d, Result: %d\n", my_task.id, calculation);
    return 0;
}

```

Memory Allocation: Stack vs. Heap

- **Stack Memory:** Super-fast allocation managed directly by the compiler. It stores function call frames, local variables, and return pointers. Memory is automatically cleaned up when a function returns.
- **Heap Memory:** Dynamic memory requested at runtime via `malloc()`. Managed manually by the programmer, it remains allocated until explicitly freed using `free()`. Failing to call `free()` leads to **memory leaks**; dereferencing after freeing causes a **Use-After-Free** security bug.

9. CS162 Course Blueprint & Administration

Syllabus Breakdown

1. **Concurrency:** Threads, process isolation, locks, semaphores, scheduling, deadlock, and coordination.
2. **Address Spaces:** Virtual Memory, address translation tables, page allocation, and physical protection.
3. **File Systems:** Storage hardware virtualization, file systems, caching layers, and database transactions.
4. **Distributed Systems:** Sockets, networking protocols, RPC, distributed hash tables (DHTs), and consistency models.

Projects & Homeworks

- **Individual Homeworks:** Designed to establish foundational systems skills:
 - *HW0:* Developing in the standard VM environment and checking code with GDB.
 - *HW1:* Creating dynamic Doubly Linked Lists in C.
 - *HW2 (BYOS):* Building your own shell from scratch.

- *HW3*: Designing a multi-threaded web server utilizing Sockets.
- **Group Projects**: Built in groups of exactly 4 students to teach real-world collaborative development.
 - *Project 1*: Implementing system calls and user program execution (`exec` & `syscall`).
 - *Project 2*: Thread management and CPU scheduling algorithms.
 - *Project 3*: Designing a functioning system-level File System.

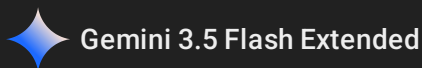
Course Evaluation

- **Three Midterms**: 36% of the grade (12% each, held on 2/24, 3/31, 4/30).
- **Group Projects**: 36%.
- **Individual Homeworks**: 18%.
- **Participation**: 10%.
- **No Final Exam!**

Academic Policy & LLMs

To build the deep mental models required to write, debug, and understand complex concurrent systems, **all use of Large Language Models (LLMs) is strictly forbidden** in CS162.

Why LLMs are disallowed: LLMs are statistical models that can hallucinate and write buggy, unoptimized systems code. This is especially dangerous for concurrency, where subtle synchronization bugs can leak sensitive user data. To prevent cheating, the course staff uses similarity checking against historic project repositories and conducts mandatory **in-person oral exams** to test students' understanding of their submitted code.



CS162 Operating Systems and Systems Programming

Lecture 1: What is an Operating System? – Comprehensive, Exam-Ready Study Notes

Section 1: Introduction, Course Administration & Academic Policies

1.1 Course Objectives & Overview

The ultimate goal of CS162 is to train you to think like a **systems programmer**. Operating systems are among the most complex artifacts created by humanity; they must manage diverse, asynchronous hardware devices, maximize resource utilization, and provide absolute isolation to protect applications from each other and the kernel itself. This course covers four fundamental blocks of software engineering:

1. **Concurrency & Synchronization**: Threads, CPU scheduling, locks, semaphores, coordination, and deadlocks.
2. **Address Spaces & Virtual Memory**: Memory translation, page allocation, paging networks, protection, and shared memory mapping.
3. **Persistent Storage & File Systems**: Hardware virtualization of storage, naming devices, caching layers, performance optimization, and transactional safety.
4. **Distributed Systems & Networking**: Sockets, networking protocols, Remote Procedure Calls (RPC), distributed hash tables (DHTs), consistency, and scalability.

1.2 Administrative Logistics & Grade Breakdown

- **Midterm Exams (36% total):** Composed of three midterms (12% each), completely in-person. No class is held on exam days to allow extra office hours.
 - *Midterm 1:* Tuesday, February 24th (7-10 PM)
 - *Midterm 2:* Tuesday, March 31st (7-10 PM)
 - *Midterm 3:* Thursday, April 30th (7-10 PM)
- **Group Projects (36% total):** Done in strictly regulated teams of **exactly member units**. Teams with 3 members require rigorous departmental justification; teams of 5 are entirely prohibited. Code submission via `git push` triggers the remote automated test suite.
- **Individual Homework Assignments (18% total):** Weekly or bi-weekly technical assignments targeting foundational coding and tool fluency.
- **Class Participation (10% total):** Enforced through discussion section attendance and active engagement.
- **Final Exam:** There is **no final exam** in CS162.

1.3 Course Infrastructure & Drop Deadlines

- **Course Site:** `https://cs162.org`
- **Discussion Forum:** EdStem platform.
- **Primary Textbooks:** *Operating Systems: Principles and Practice (2nd Edition)* by Anderson & Dahlin, supplemented by *Operating Systems: Three Easy Pieces (OSTEP)* by Arpaci-Dusseau (available free online) and *Linux Kernel Development (3rd Edition)* by Robert Love.
- **Drop Policy:** CS162 is an **Early Drop Deadline** course. The final date to drop without penalty is **January 30th**. If you are on the waitlist, department policy dictates the automated enrollment sequence; you must complete all coursework and homeworks to remain eligible for admission.

1.4 Dynamic C Language and GDB Diagnostics Checklist

The standalone code footprint of the CS162 codebase (Pintos/Nachos frameworks) is vastly more sophisticated than the lightweight codebases examined in computer architecture pre-requisites. To navigate examinations and project benchmarks successfully, you must master the following idioms:

1. **Dynamic Memory Allocation Contexts:** Complete ownership of the physical differences between Stack (local stack frames, auto-cleanup on function exit) and Heap (`malloc()` , `calloc()` , `realloc()` , and mandatory tracking of matching `free()` operations to prevent resource exhaustion or system leaks).
2. **Polymorphic Pointer Tracking:** Total comfort parsing untyped references (`void *`) and casting memory layouts dynamically inside operating systems data blocks.
3. **Function Pointers:** The core mechanism used by kernels to instantiate dynamic jump tables, device driver interrupt vectors, and mock object-oriented polymorphism in clean C structures.

Code Example: Polymorphic C Jump Tables & Function Pointers

```
c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

// Typedef definitions for an explicit kernel callback interface
typedef int (*device_operation_t)(void *device_context, char *buffer, size_t length);
```

```

typedef struct device_driver {
    char driver_name[32];
    device_operation_t read_channel;
    device_operation_t write_channel;
} device_driver_t;

// Concrete implementation mimicking raw terminal character ingestion
int terminal_read(void *context, char *buffer, size_t length) {
    int *port_id = (int *)context; // Explicit pointer cast from untyped container
    printf("[KERNEL] Reading from hardware terminal port %d\n", *port_id);
    strncpy(buffer, "INJECT_BYTE_STREAM", length);
    return (int)strlen(buffer, length);
}

int main(void) {
    int physical_port = 4f3; // Raw mock hardware address mapping

    // Construct dynamic device configuration block
    device_driver_t serial_driver;
    strncpy(serial_driver.driver_name, "Intel_UART_8250", 32);
    serial_driver.read_channel = terminal_read; // Assigning function address directly

    // Invoke polymorphic read block simulating kernel interface layers
    char kernel_sys_buffer[64];
    int bytes_read = serial_driver.read_channel(&physical_port, kernel_sys_buffer, 64);

    printf("[USER APPLN] Received payload size: %d, Data: %s\n", bytes_read, kernel_sys_buffer);
    return 0;
}

```

1.5 Academic Integrity: The Large Language Model (LLM) Ban Policy

The use of **Large Language Models (LLMs)** is strictly forbidden in all dimensions of CS162.

Rationale for the Ban

Operating systems are concurrent systems. LLMs are statistical probability estimators trained on loose public software engineering corpora. While they excel at syntax completion, they regularly produce unoptimized code or code that contains hidden concurrency flaws (such as race conditions, atomicity violations, and resource deadlocks). In a phone application, a small bug might cause a graphical glitch; in an operating system kernel, a synchronization error can create privilege escalation vulnerabilities, leak user data, or crash the entire system.

Enforcement Protocols

To protect academic integrity and verify absolute personal competence, the course staff employs two rigorous mechanisms:

1. **Automated Structural Similarity Engine Plagiarism Scanning:** Submissions are verified against a cross-generational repository of student solutions and public implementations to catch code reuse.
2. **Mandatory In-Person Technical Oral Examinations:** Throughout the term, teaching assistants conduct individual oral exams where you must explain, line by line, the design documents, correctness arguments, and internal mechanics of your submitted codebase. If you submit AI-generated code without a deep internal model of its execution, you will fail the oral verification.

Section 2: Defining the Operating System (The Three Roles)

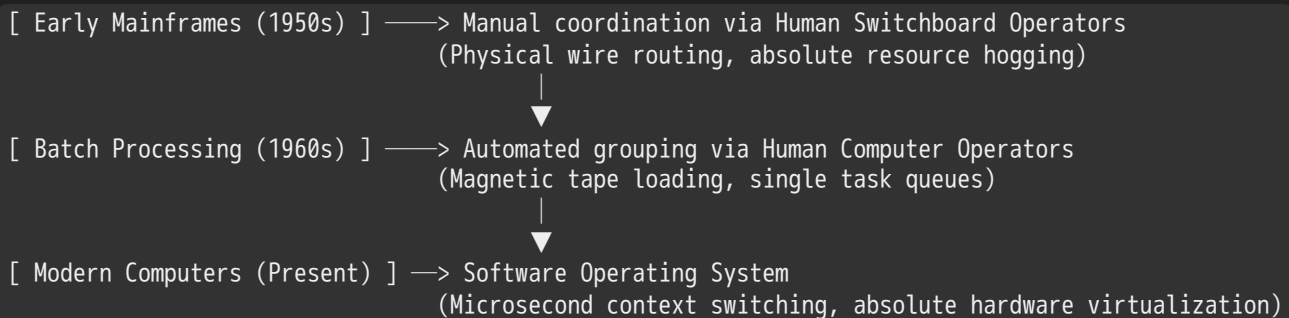
2.1 Pragmatic vs. Formal Boundary Formulations

There is no universally accepted industry definition of what constitutes an operating system.

- **The Vendor Definition (Pragmatic Approach):** "An operating system is everything a commercial distributor bundles together when you order the product". This pragmatic definition is fluid; it can include peripheral layers like system utility programs, graphical windows managers, file explorers, media players, and web browsers.
- **The Pure System Architecture Definition (Formal Approach):** "The operating system is the **kernel**, which is the single program running at all times on the underlying CPU hardware with absolute supervisor privileges". Every other piece of execution software is categorically divided into either a **system program** (ancillary utilities shipped alongside the kernel, executing in user-space, such as shells, network managers, and compilation linkers) or an **application program** (any third-party workspace tool instantiated by a user).

2.2 Historical Paradigm Shift

The fundamental necessity of an OS mirrors the evolution of workforce logistics in data centers:



In early mainframes, a single program completely controlled the hardware. Today, the operating system manages sharing and abstraction automatically through software engineering.

2.3 The Conceptual Roles: Referee, Illusionist, Glue

To understand an operating system, we analyze its operations through three core roles:

1. The Referee

The OS acts as an arbiter that manages resource allocation, isolation, protection, and inter-process communications:

- **Resource Allocation:** Dynamically balances hardware resource requests (such as CPU instruction cycles, physical DRAM allocations, disk block access, and network bandwidth) across competing workloads.
- **Isolation and Protection:** Enforces absolute software protection boundaries to prevent flawed or malicious application code from overwriting kernel space or corrupting other running programs.

2. The Illusionist

The OS provides clean, virtualized abstractions of complex physical hardware:

- **Infinite Memory Illusion:** Gives each individual program the illusion of possessing a massive, contiguous memory address space, even if the underlying physical RAM is fragmented and highly constrained.
- **Exclusive Processor Illusion:** Multiplexes CPU execution time so that each thread operates as if it has permanent, exclusive access to its own dedicated virtual CPU core.
- **Hardware Abstraction:** Masks the low-level physical realities of device controllers (like cylinder heads, disk sectors, flash block wear-leveling, and network radio bands) behind simple, high-level abstractions like structured files, unique network sockets, and persistent data streams.

3. The Glue

- The OS provides a uniform layer of common system services that decouples hardware variations from application development:
- **Standardized System Services:** Exposes generic APIs for network communication stacks, storage organization formats, and interactive window systems.
 - **Interface Uniformity:** Establishes a uniform look and feel, allowing a single compiled binary file to run on different physical storage media and drive models without modifying the application code.

2.4 The Systems Engineering Mindset

A software structure qualifies as a **system** when it contains multiple interrelated components that interact dynamically. Building system-level code requires a rigorous engineering mindset:

1. **Meticulous Error Handling:** You cannot assume a reliable physical environment. Every system call, hardware access check, pointer dereference, and hardware packet response must explicitly validate return states.
2. **Defensive Programming Boundaries:** The system must proactively defend against buggy, careless, or malicious users. Inputs from user space must be treated as untrusted data until validated by the kernel memory management unit.
3. **Hardware Awareness:** You must treat the underlying hardware as a concrete physical entity with specific resource limitations, physical latencies, and real-world failure rates, rather than a clean, idealized mathematical abstraction.

Section 3: The Scale of Physical Latencies (Jeff Dean's Numbers)

An operating system acts as a resource manager across timescales that span nine orders of magnitude. To design efficient concurrent software, systems programmers must understand the physical latency costs of hardware operations.

The following table presents data adapted from Jeff Dean's benchmark metrics, normalized to a human scale where a single high-speed L1 cache reference (0.5 ns) is equivalent to a single human heartbeat (1 second).

Hardware Latency Analogy Table

Physical Operation	Raw Latency (Nanoseconds)	Normalized Time Context	Human-Scale Analogy (0.5 ns ≡ 1 second)
L1 Cache Reference	0.5 ns	0.5 ns	1 Second (Single Heartbeat)
Branch Misprediction	5.0 ns	5.0 ns	10 Seconds (Short pause)
L2 Cache Reference	7.0 ns	7.0 ns	14 Seconds (Checking notes)
Mutex Lock / Unlock Execution	25.0 ns	25.0 ns	50 Seconds (Waiting in line)
Main Memory Reference (DRAM Access)	100.0 ns	100.0 ns	3.33 Minutes (Brewing espresso)
Compress 1KB using Zippy Engine	3,000.0 ns	3.0 μs	1.66 Hours (Watching a film)
Send 2KB over 1 Gbps Network Fabric	20,000.0 ns	20.0 μs	11.11 Hours (Transatlantic flight)
Read 1MB Sequentially from DRAM Memory	250,000.0 ns	250.0 μs	5.78 Days (Standard work week)

Physical Operation	Raw Latency (Nanoseconds)	Normalized Time Context	Human-Scale Analogy (0.5 ns ≡ 1 second)
Network Round Trip inside Same Data Center	500,000.0 ns	500.0 μs	11.57 Days (An extended holiday)
Physical Disk Seek Latency	10,000,000.0 ns	10.0 ms	7.71 Months (An academic year)
Read 1MB Sequentially from Hard Disk Sector	20,000,000.0 ns	20.0 ms	1.26 Years (A complete calendar year)
Packet Round Trip (California to Netherlands)	150,000,000.0 ns	150.0 ms	9.51 Years (An entire decade of life)

Mathematical Derivation of the Scaling Analogy

To build an intuitive model of these latencies, we derive our scaling factor S : Scaled Cache Reference = 1.0 second
Physical Cache Latency = 0.5×10^{-9} seconds $S = \frac{1.0 \text{ s}}{0.5 \times 10^{-9} \text{ s}} = 2 \times 10^9$

Applying this scaling factor to a physical disk seek operation (10.0 ms): Physical Disk Latency = 10.0×10^{-3} seconds
Human Scaled Value = $(10.0 \times 10^{-3} \text{ s}) \times (2 \times 10^9) = 2.0 \times 10^7$ seconds

Converting this scaled value into days and months: Total Days = $\frac{2.0 \times 10^7 \text{ s}}{86400 \text{ s/day}} \approx 231.48$ Days
Total Months = $\frac{231.48 \text{ Days}}{30 \text{ Days/Month}} \approx 7.71$ Months

Exam Design Interpretation: This massive latency gap explains why operating systems use complex software architectures for prefetching, asynchronous scheduling, and multi-tier memory caching structures. If a thread halts execution to wait for a physical mechanical disk seek, it is computationally equivalent to a human worker sitting completely idle for over seven months.

Section 4: Physical Evolution & Hardware Technology Scaling Trends

Operating system design is driven by shifting physical hardware constraints and resource scaling boundaries.

4.1 Bell’s Law

Bell’s Law states that a completely new class of computer systems emerges roughly every 10 years, driven by exponential advances in component density, cost reductions, and form-factor optimization. This law dictates the historical ratio of human beings to active computing nodes:

- **1960s Mainframes:** A ratio of $1 : 10^6$ (Thousands of human operators multiplexed across a single central processing node).
- **1980s Personal Computers:** A ratio of $1 : 1$ (A dedicated desktop workstation or laptop allocated per user).
- **2010s Smart Devices:** A ratio of $1 : \text{Multiple}$ (The absolute proliferation of mobile computing platforms, smartphones, and mobile infrastructure).
- **2020s The Internet of Things (IoT):** A ratio of $10^3 : 1$ (Thousands of embedded microcontroller smart-motes, environmental sensors, and remote telemetry nodes allocated per person).

[Image illustrating Bell's Law and the shifting human-to-computer ratio over time]

4.2 Moore’s Law: Physical Dimensions and the Pivot to 3D Stacking

Moore's Law is a physical scaling trend observed by Gordon Moore in 1965. It states that the number of transistors packed onto a monolithic semiconductor integrated circuit doubles roughly every 18 to 24 months.

In **February 2016**, traditional planar (2D) silicon geometric scaling reached hard atomic limits at sub-10nm feature sizes. At these micro-scales, quantum tunneling effects make it difficult to completely shut off transistors, leading to excessive current leakage.

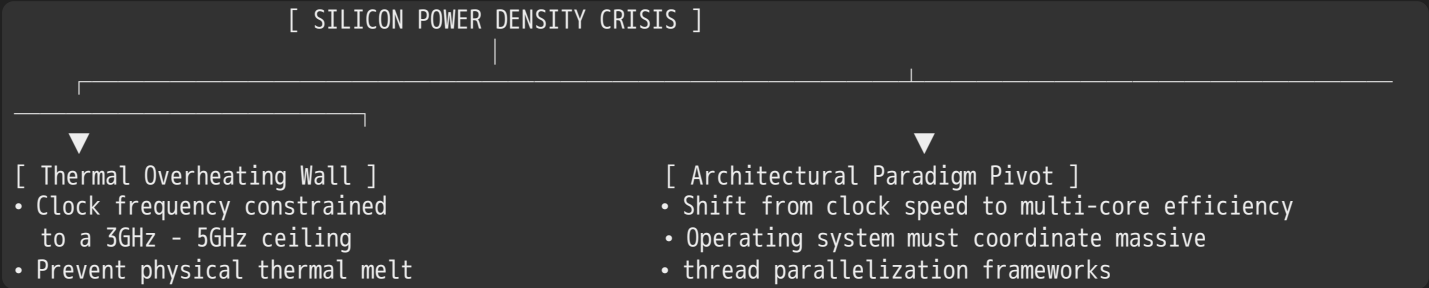
To sustain scaling, the semiconductor industry pivoted to alternative architectural models:

- **3D Stacked Silicon Topologies:** Layering multiple active memory and processing circuits vertically on a single package.
- **Advanced Packaging Innovations:** Utilizing High-NA extreme ultraviolet lithography, RibbonFET gate-all-around architectures, and PowerVia backside power delivery networks to target packages containing over **1 trillion transistors** by 2030.

4.3 The Power Density Barrier & The Demise of Dennard Scaling

Historically, Dennard scaling meant that as transistors shrank, they became faster and consumed less power, keeping the total power per unit area of a silicon chip constant. Around 2005, this scaling model collapsed due to leakage currents and heat dissipation limits.

Extrapolating Moore's Law without Dennard scaling predicted that chip power density would surpass the heat generation limits of a **standard hot plate, a commercial nuclear reactor core, and eventually the surface of the sun.**



4.4 Multi-Core / Many-Core Processing Topologies

Because single-threaded processor performance plateaued around 2002 (the slowing of Joy's Law) , chip design shifted toward parallel architectures on a single die.

Case Study: Intel Sapphire Rapids Architecture (2023)

The modern Sapphire Rapids microprocessor demonstrates this shift toward parallel, highly integrated architectures:

- **Transistor Density:** Packs over **100 billion transistors** onto a single processing package using an advanced 4-chiplet architecture bonded via ultra-fast multi-die interconnects.
- **Compute Capacity:** Scales up to **60 physical cores and 120 simultaneous execution threads** per individual socket slot.
- **Memory Subsystem Layout:** Supports up to **4 TiB of multi-channel DDR5 memory**, with local cache lines broken down into 80 KB L1 cache configurations and 2 MB L2 cache maps per individual core.
- **On-Chip Networking:** Integrates a physical on-chip **Mesh Interconnect Network Router** directly onto the silicon to handle cache coherence traffic and high-speed data routing across different processing elements.

Section 5: The Process Abstraction & Systems Software Complexity

5.1 The Software Complexity Landscape

As underlying hardware architectures became increasingly parallel, software complexity grew exponentially. The chart below tracks software complexity measured in total Lines of Code (LoC) across various systems:

```
[ Linux Kernel 2.2.0 (Historical Baseline) ] ———> 2 Million LoC
[ Mars Curiosity Rover Mission Stack ] —————> 5 Million LoC
[ Android Mobile Operating Platform OS ] —————> 12 Million LoC
[ Modern Desktop Windows 7 System ] —————> 40 Million LoC
[ Contemporary Autonomous Connected Car ] —————> 100 Million LoC
```

Modern cars contain a complex mesh of independent embedded microcontrollers managing real-time driver control, engine performance, brake safety, infotainment systems, and wireless telematics. This massive software footprint makes testing all execution states impossible, shifting the focus of software engineering from basic bug elimination to active **fault containment, isolation, and system recovery**.

5.2 Case Study: Mars Pathfinder Priority Inversion Failure

The 1997 Mars Pathfinder mission illustrates how subtle, unhandled synchronization issues inside a real-time operating system can cascade into mission-critical system failures.

System Constraints

The Pathfinder rover operated under tight resource limits: a 20 MHz central processor, 128 MB of physical DRAM, and the VxWorks Real-Time Operating System (RTOS). The system ran multiple tasks concurrently, including scientific instrumentation pipelines, locomotion drivers, and a critical wireless telemetry tracking system.

The Failure Mechanism (Priority Inversion)

The system experienced a catastrophic failure due to an interaction between three threads of varying priorities:

1. **High-Priority Thread (T_H)**: The core information bus management task, responsible for routing telemetry data. It executed on a strict real-time deadline.
2. **Medium-Priority Thread (T_M)**: A long-running communications task that handled data processing and network encoding without real-time deadlines.
3. **Low-Priority Thread (T_L)**: A lightweight meteorological telemetry logging task that periodically read basic atmospheric sensors.

Step 1: Low-Priority Task (T_L) acquires the Mutex Lock on the shared memory bus.



Step 2: High-Priority Task (T_H) awakes, preempts T_L , and requests the Bus Mutex. Since Mutex is held by T_L , T_H is blocked and enters a suspended state.



Step 3: Medium-Priority Task (T_M) awakes. Since T_M 's priority is higher than T_L , the scheduler runs T_M directly. T_M completely blocks T_L from running.



[CRITICAL BREAKDOWN]: Since T_L cannot run, it can never release the Mutex. T_H remains permanently starved. The Watchdog Timer trips due to T_H 's missed deadline and forces a hard hardware reset.

The Architectural Solution (Priority Inheritance)

To resolve this priority inversion, the engineers enabled **Priority Inheritance** inside the VxWorks kernel's mutex tracking code.

Under this protocol, when a high-priority thread (T_H) blocks on a resource or lock held by a lower-priority thread (T_L), the lower-priority thread **temporarily inherits the priority level of the blocked high-priority thread (T_H)**. This allows T_L to run ahead of intermediate threads (T_M), quickly complete its critical section, and release the lock. Once the lock is released, T_L returns to its base priority, and T_H safely acquires the resource, preventing system timeouts.

5.3 The Dual Views of a Process Space

To manage this complexity, the operating system isolates applications within a secure execution environment called a **Process**.

1. The System Programmer's View

An application consists of source code compiled into machine instructions and linked against standard system libraries. To the programmer, the process provides a clean, abstract machine interface composed of sequential instruction execution blocks, a private address space, open file descriptors, and communication sockets.

[Image contrasting the System Programmer's View of a process with the Operating System's View]

2. The Operating System's View

The operating system multiplexes multiple independent processes across shared physical hardware. The OS translates abstract application system calls into direct operations on physical hardware components (such as CPU registers, hardware MMU translation tables, flash memory sectors, and network controllers) while maintaining strict protection boundaries between processes.

Section 6: Basic Operating System Mechanics & Execution Abstractions

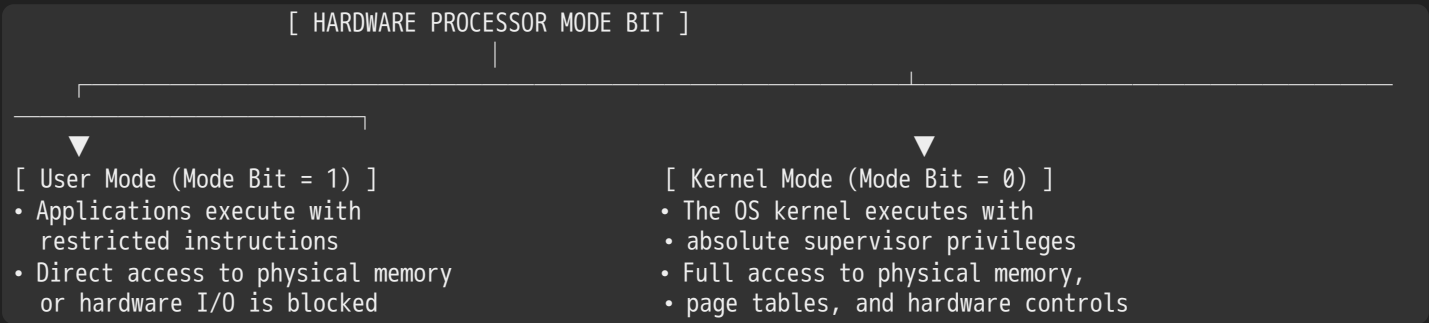
6.1 Anatomy of the Process Abstraction

A process is a protected, isolated execution environment with restricted access rights managed by the operating system kernel. A valid process structure consists of three primary components:

- 1. **A Private Virtual Address Space:** The specific range of memory addresses that the process can read or write. This segment isolates application data from other processes and the kernel.
- 2. **One or More Threads of Control:** The active units of concurrency executing instructions sequentially within that address space. Each thread tracks its own architectural state, including its Program Counter (PC), stack pointer, and internal register values.
- 3. **Associated System State:** Private resources allocated to the process by the kernel, including indexed collections of open **File Descriptors** and network communication **Sockets**.

6.2 Dual-Mode Operation Boundaries

To protect the system from application errors or malicious behavior, modern processor architectures enforce a hardware protection boundary called **Dual-Mode Operation**.



6.3 Transition Vectors: Traps, Interrupts, and Exceptions

Applications must transition from user mode to kernel mode to perform any protected operations, such as reading files, allocating memory, or sending network packets. These transitions are strictly regulated by hardware vectors:

- 1. **System Calls (Traps):** Synchronous, program-controlled requests initiated by an application via a specialized hardware instruction (such as `syscall` or `sysenter`) to request specific services from the operating system kernel.
- 2. **Hardware Interrupts:** Asynchronous, external hardware events independent of the currently executing process (such as a timer tick interval, a keyboard keypress, or the arrival of a network packet at a controller interface) that signal the kernel to handle hardware I/O.

3. **Exceptions:** Synchronous fault events triggered by an application error during instruction execution (such as an integer division by zero, an unaligned memory access, or a page fault).

When a transition event occurs, the hardware automatically switches the mode bit to 0, saves the thread's execution context, and jumps to a pre-defined address in the kernel's **Interrupt Vector Table** to execute the appropriate handler safely.

6.4 Context Switching Architecture & Fault Containment

When the scheduler switches execution from Process 1 to Process 2, it performs a **Context Switch** to save and restore execution states:

1. **Context Ingestion:** The CPU receives a timer interrupt or a blocking system call, forcing a trap into kernel mode (Mode Bit drops to 0).
2. **State Preservation:** The kernel saves the caller's architectural state (including the program counter, stack pointer, and general-purpose registers) into a dedicated memory block called the **Thread Control Block (TCB)**.
3. **Translation Switch:** The kernel updates internal hardware translation registers to point to the virtual address space and page tables of the next process.
4. **Context Restoration:** The kernel loads the saved register values from the target process's TCB into the physical CPU core.
5. **Execution Resumption:** The kernel executes a return instruction, resetting the hardware mode bit to 1 and jumping to the restored program counter location of the target process.

Fault Containment and Memory Protection

If an application attempts to bypass system APIs and directly access unauthorized memory locations or write to kernel space, the hardware Memory Management Unit (MMU) intercepts the instruction. Because the target address is not mapped within the process's page table, the hardware blocks the write and triggers a trap into kernel mode. The kernel immediately terminates the offending process to protect system stability, printing a `Segmentation fault (core dumped)` error to the terminal.

Code Example: System Calls vs. Standard User Library Functions

This program illustrates the structural difference between standard user-space helper functions and direct kernel system calls that trigger hardware mode shifts.

```
c

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/stat.h>
#include <fcntl.h>

int main(void) {
    size_t array_allocation_size = 1024;

    // 1. User-Space Memory Request (Standard Library Call)
    // malloc() executes entirely within user space, managing allocation lists
    // on the application heap. It only issues an underlying system call (brk/sbrk)
    // when it needs to request additional physical memory pages from the kernel.
    int *user_heap_array = (int *)malloc(array_allocation_size * sizeof(int));
    if (user_heap_array == NULL) {
        perror("[ERROR] User space memory allocation failed");
        return 1;
    }
}
```

```

// Initialize array memory within user-space boundaries
for (size_t index = 0; size_t index < array_allocation_size; index++) {
    user_heap_array[index] = (int)(index * 2);
}

// 2. Direct Kernel System Request (System Call)
// Unlike printf(), which buffers data in user space, write() is a direct system call.
// It issues a hardware trap instruction, switching the CPU mode bit to 0 and handing
// control to the kernel to output bytes to standard output (File Descriptor 1).
char log_message[] = "[SYSTEM CALL] Trapping into kernel to write telemetry data.\n";

// Hardware Mode Transition: Mode Bit flips from 1 to 0
write(STDOUT_FILENO, log_message, sizeof(log_message) - 1);
// Mode Bit returns from 0 to 1

// Clean up heap resources
free(user_heap_array);
return 0;
}

```

Section 7: Key Exam Concepts Mastery Cheat Sheet

Use these summary verification items to check your understanding of the core concepts from Lecture 1:

1. Referee vs. Illusionist vs. Glue Core Definitions

- **Referee:** Enforces protection, security isolation, and hardware resource allocation among competing processes.
- **Illusionist:** Virtualizes hardware to present simplified, robust abstractions like virtual memory spaces and dedicated virtual CPU threads to applications.
- **Glue:** Exposes uniform, standardized system utility APIs that hide hardware differences from application developers.

2. Physical Latency Hierarchy Scaling Analogy

- CPU cache access operates on a scale of fractions of a nanosecond (0.5 ns), main memory DRAM access takes roughly 100 ns, and physical hard disk mechanical seeks require roughly 10,000,000 ns.
- Scaled to human-scale time terms where an L1 cache reference represents **1 second**, a main memory access takes **3.33 minutes**, while a mechanical disk seek is equivalent to **7.71 months** of absolute execution starvation.

3. Hardware Component Technology Scaling Laws

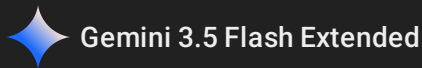
- **Bell's Law:** A completely new class of computing devices emerges every 10 years as dimensions and deployment costs shrink, scaling from ancient room-sized mainframes to modern smart devices and dense IoT mote networks.
- **Moore's Law:** Transistor integration density per package chip area historically doubled every 18 to 24 months. Planar silicon geometric boundaries forced a pivot from raw clock frequency scaling to multi-core parallelism and advanced 3D packaging strategies.

4. Anatomy of the Process Execution Block

- A process consists of a **private virtual address space** (for memory isolation), **one or more execution threads** (tracking program counters and register states), and **kernel resource mappings** (such as file descriptor slots and open sockets).

5. Dual-Mode Transition Boundaries

- The processor uses a hardware mode bit to enforce protection boundaries. User mode (mode bit = 1) runs applications with restricted instructions, while kernel mode (mode bit = 0) grants full supervisor access to physical infrastructure.
- Execution transitions between user mode and kernel mode are strictly controlled through **traps (system calls), hardware interrupts, and execution exceptions**.



CS162 Operating Systems & Systems Programming

Lecture 3: Abstractions 1: Threads and Processes (Programmer's Viewpoint)

Section 1: Introduction & High-Level Goals

The transition from understanding basic hardware execution to developing real-world concurrent systems requires a strong foundation in execution abstractions. The primary goal of this lecture is to examine the **Thread** and **Process** abstractions from a programmer's perspective.

1.1 Core Lecture Objectives

- **Understanding Threads:** Define what a thread is, what it is not, and the core motivations for using them.
- **Thread Programming Fluency:** Learn how to write concrete, multi-threaded applications using the standardized **POSIX Threads (pthreads)** library.
- **Architectural Trade-offs:** Examine alternatives to multi-threading (such as multi-processing) and understand when each model is appropriate.
- **Concurrency vs. Parallelism:** Distinguish between overlapping instruction streams (concurrency) and simultaneous physical execution (parallelism).

Section 2: The Thread Abstraction

2.1 Programmer's View: Threads as Virtual Cores

To a programmer, a thread is an abstraction of a **virtual CPU core (vCPU)**.

If you have a single physical processor, the operating system can run multiple threads by **multiplexing them in time**. The OS runs one thread for a short period, saves its state, loads another thread's state, and resumes execution. This switch happens so quickly (on the scale of microseconds) that it creates the illusion that multiple processors are executing concurrently.

2.2 What Comprises a Thread?

A thread is the smallest schedulable unit of execution in an operating system. A thread is considered to be "executing" on a physical processor when its execution context is loaded into the processor's physical registers.

Each virtual core (thread) maintains its own private execution state, which includes:

- **Program Counter (PC):** Points to the memory address of the next instruction the thread will execute.
- **Stack Pointer (SP):** Points to the top of the thread's private activation stack, which manages local variables and function calls.
- **Registers:** Holds active data values, including general-purpose, floating-point, and status flags.

2.3 Where Does a Thread Reside?

A thread's execution state exists in one of two physical locations:

- 1. **On the Physical Core:** The thread's state is loaded directly into the CPU's registers, program counter, and stack pointer.
- 2. **In Memory (Thread Control Block - TCB):** When a thread is not currently running on a physical processor, the operating system preserves its state in a dedicated kernel memory structure called the **Thread Control Block (TCB)**.

Section 3: The Process Abstraction

3.1 What is a Process?

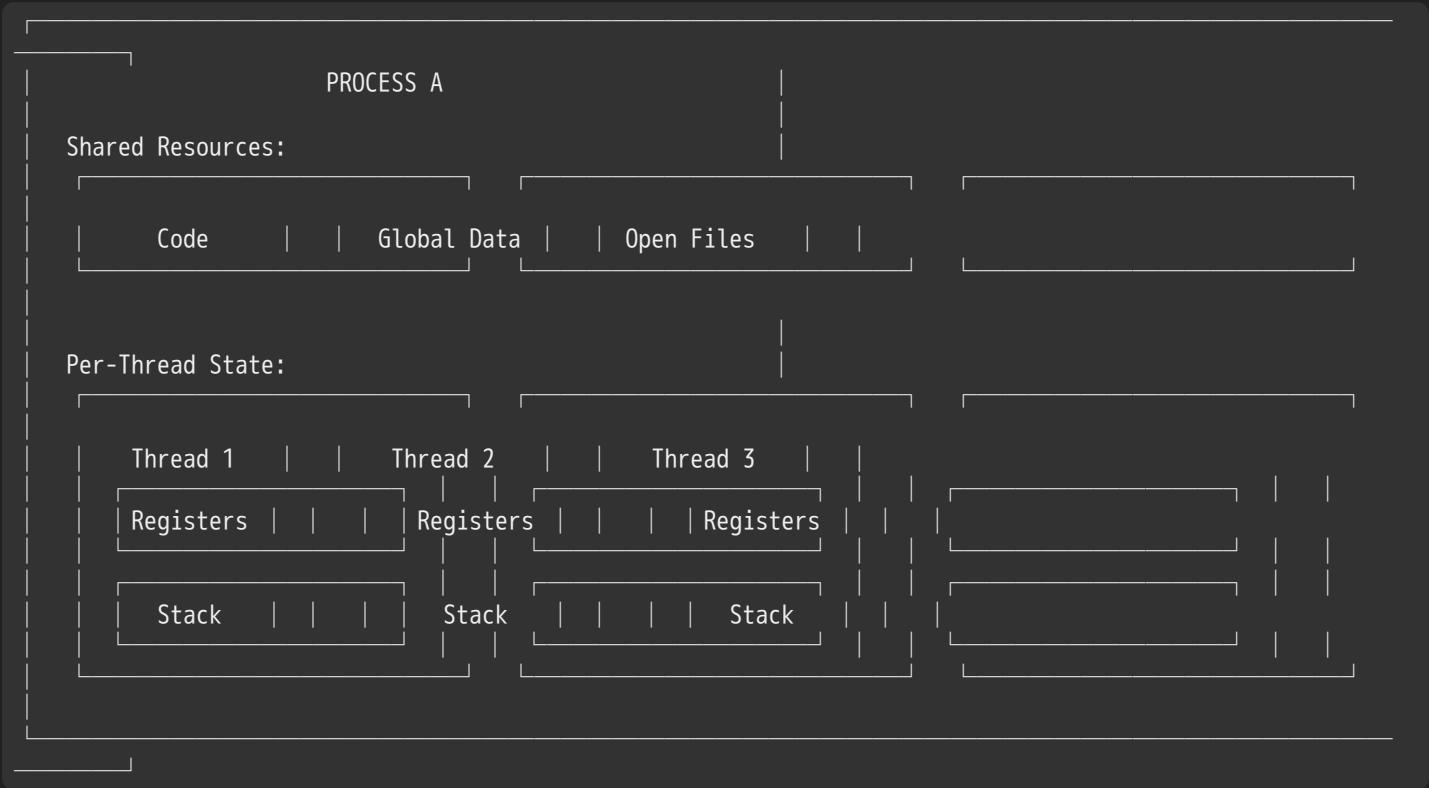
A process is an **instance of a running program**. It represents a protected execution environment with **restricted rights** managed by the operating system kernel.

A process contains:

- **A Protected Address Space:** A private range of memory addresses that the process can read or write.
- **One or More Threads:** The active units of execution running within that address space.
- **Associated OS Resources:** Private system handles allocated to the process, such as open file descriptors, network socket references, and user identity credentials.

3.2 Multi-Threaded Process Structure

In a multi-threaded process, the threads **share** most of the process's resources but retain some **private** state:



- **Shared State:** Code segments, static global data, the heap, and system assets (like file descriptors and network connections) are shared by all threads within the process.
- **Per-Thread Private State:** Each thread has its own private set of register values (including the PC and SP) and its own private execution stack to manage function local frames.

3.3 Why Processes?

The operating system uses the process abstraction to enforce security and reliability through **fault containment**:

- **Isolation:** A bug or crash (like a segmentation fault) in one process cannot corrupt the memory or state of another process.
- **Kernel Protection:** Applications run outside the kernel in restricted environments. This ensures that if an application crashes, the operating system itself remains stable.

Section 4: Dual-Mode Operation & Transition Mechanics

4.1 Dual-Mode Operation

To protect the system from application errors or malicious software, hardware architectures enforce a security boundary called **Dual-Mode Operation**.

The hardware CPU provides at least two execution modes, tracked by a hardware **Mode Bit** or **Sysmode** register:

1. **User Mode (Mode Bit = 1):** The processor executes instructions with restricted rights. Applications running in user mode cannot access physical hardware devices directly, modify memory translation structures (like page tables), or disable hardware interrupts.
2. **Kernel Mode / Supervisor Mode (Mode Bit = 0):** The CPU has unrestricted access to all hardware resources, physical memory maps, and privileged CPU instructions.

4.2 System Architecture Layering (The UNIX Case Study)

The UNIX operating system organizes these boundaries into distinct architectural layers:

Layer	Privilege Level	Components
User Mode	Restricted	Applications, Shells, Compilers, Interpreters, Standard System Libraries (e.g., <code>libc</code>)
Kernel Mode	Full Privileges	System-Call Interface, File Systems, Virtual Memory, CPU Schedulers, Page Replacement, Character/Block I/O, Device Drivers
Hardware Layer	Raw Physical Components	Terminal/Device Controllers, Hard Disks, Memory Controllers, Physical RAM, CPUs

4.3 Transition Vectors between User and Kernel Mode

Applications transition from user mode to kernel mode through three controlled entry points:

- **System Calls (Traps):** Synchronous, programmer-initiated requests to the kernel (e.g., writing a file or spawning a thread).
- **Hardware Interrupts:** Asynchronous external events (e.g., a timer interrupt or a network packet arrival) that signal the kernel to handle hardware I/O.
- **Exceptions:** Synchronous events caused by application errors (e.g., divide-by-zero or page faults).

Section 5: Multi-Process Multiplexing & Base and Bound Protection

5.1 The Process Control Block (PCB)

The operating system represents each running process with a kernel data structure called the **Process Control Block (PCB)**.

The PCB serves as the central record for a process's state and contains:

- **Process State:** Current status (e.g., Running, Ready, Blocked).
- **Process Identifier (PID):** A unique integer identifying the process.

- **Register Save State:** Space to store the CPU's registers (including the PC and SP) when the process is context-switched out of execution.
- **Memory Management:** Page table pointers or Base and Bound registers.
- **I/O & File State:** A list of open file descriptors and active network socket configurations.

5.2 Process Context Switching Timeline

When the operating system switches execution from Process P_0 to Process P_1 , it performs a **Context Switch**:

1. **Trap/Interrupt:** An interrupt or system call triggers a switch from user mode to kernel mode.
2. **Save State (P_0):** The OS saves P_0 's execution registers into its corresponding Process Control Block (PCB_0).
3. **Execute Kernel Loop:** The scheduler selects the next process (P_1) to run.
4. **Restore State (P_1):** The OS loads P_1 's execution registers from its Process Control Block (PCB_1).
5. **Return to User Mode (RTU):** The OS switches the CPU mode bit back to user mode and resumes executing P_1 .

Section 6: Memory Virtualization: Base and Bound (B&B)

To run multiple processes safely, the operating system must isolate each process's memory. One of the simplest historical mechanisms to achieve this is the **Base and Bound (B&B)** relocation register model.

6.1 Hardware Translation Mechanics

Under the Base and Bound model, every memory address generated by an application (the virtual/program address) is translated on the fly by hardware before it reaches physical RAM:

1. **Hardware Check:** The hardware checks if the virtual address is within the process's allocated limits:
Virtual Address < Bound
2. **Relocation:** If the address is within bounds, the hardware translates it to a physical address: Physical Address = Virtual Address + Base
3. **Fault Trigger:** If the virtual address is greater than or equal to the Bound register, the hardware triggers a memory fault trap into the kernel, terminating the application with a segmentation fault.

6.2 Step-by-Step Base and Bound Execution Lifecycle

Let's trace how the operating system loads, executes, and switches processes using the Base and Bound model:

Step 1: OS Loads the Process State

Initially, the CPU runs in kernel mode ($\text{sysmode} = 1$). The Base and Bound registers are ignored when running in kernel mode, giving the OS full access to physical memory.

```
[Hardware State]
sysmode: 1 (Kernel)
PC      : Points to OS Bootloader Code
SP      : Points to Kernel Stack
```

The OS writes the executable's code, static variables, and initial stack space into physical memory (for example, starting at address `0x1000` with a size of `0x0100` bytes).

Step 2: Preparing the Hardware Registers

The OS prepares the CPU registers for user-mode execution. The OS sets the Base and Bound registers to define the process's address space:

[Hardware Registers Configured by OS]

```
sysmode: 1 (Kernel)
Base   : 0x1000 (Physical address where process memory begins)
Bound  : 0x0100 (Maximum offset the process can access)
uPC    : 0x0001 (The process's entry point, e.g., main)
SP     : 0x00FF (The top of the process's private virtual stack)
```

Step 3: Return to User Mode (RTU)

The kernel executes an **RTU (Return to User Mode)** instruction. This instruction is a privileged operation that performs three actions atomically:

1. Flips the mode bit from 0 (Kernel) to 1 (User).
2. Loads the user Program Counter (`PC`) from the prepared value (`uPC`).
3. Activates Base and Bound address translation.

[Hardware State running User Code]

```
sysmode: 0 (User Mode)
PC      : 0x0001 (Translated to physical address 0x1001)
SP      : 0x00FF (Translated to physical address 0x10FF)
```

The application now runs in its isolated address space. Any attempt to access an address outside `0x0000` - `0x0100` is blocked by the CPU's memory management unit.

Step 4: Reclaiming Control via Interrupt

The user application runs until an external event (like a hardware timer interrupt or an I/O request) occurs.

When the timer interrupt fires, the CPU hardware performs the following actions:

1. Flips the mode bit back to 0 (Kernel Mode).
2. Saves the current program counter value into a temporary hardware register (`uPC`).
3. Loads the Program Counter with the address of the kernel's interrupt handler from the interrupt vector table (`PC = IntrpVector[i]`).

[Hardware State running Kernel Interrupt Handler]

```
sysmode: 1 (Kernel Mode)
PC      : Points to Kernel Scheduler Loop
uPC     : Holds suspended process's last PC
```

Step 5: Process Context Switch

The kernel scheduler decides to switch execution to another process. It saves the suspended process's state into its Thread Control Block (TCB_2 , which may reside inside its PCB) and loads the state of the next process (TCB_1):

1. Save registers (PC, SP, general-purpose registers) of current thread to TCB_2 .
2. Load registers of next thread from TCB_1 .
3. Update Base and Bound registers to point to the new process's memory region.

Step 6: Resuming the New Process

The kernel executes an RTU instruction, resetting the mode bit to 0 (User Mode) and jumping to the new process's program counter.

[Hardware State running Process B]

```
sysmode: 0 (User Mode)
Base   : 0x3000 (Process B's Base)
Bound  : 0x0080 (Process B's Bound)
PC     : 0x0002 (Translated to physical address 0x3002)
```

6.3 Limitations of Base and Bound Relocation

While simple and fast, the Base and Bound relocation model has several major limitations for general-purpose computing:

- **Inflexible Allocation & Waste:** A process must be allocated contiguous memory for its peak potential stack and heap sizes, which wastes space in between.
- **Physical Memory Fragmentation:** Over time, loading and terminating processes of different sizes creates holes of unallocated memory (fragmentation), making it difficult to fit new processes into contiguous blocks.
- **No Memory Sharing:** Because processes are strictly confined to their own contiguous base and bounds window, they cannot directly share memory pages or read-only library code with other processes.

6.4 The Alternative: Translation via Page Tables

To solve these limitations, modern operating systems use **Page Tables** instead of Base and Bound registers.

Instead of mapping a process to a single contiguous block of physical memory, page tables divide virtual and physical memory into fixed-size chunks called **pages** (typically 4 KB). A process's virtual pages can be mapped to non-contiguous physical pages anywhere in memory, eliminating physical fragmentation and allowing processes to easily share memory pages.

Section 7: Hardware Realities & Threading Models

Modern operating systems are designed around the physical parallel architecture of the underlying hardware.

7.1 Case Study: Intel Sapphire Rapids Architecture (2023)

The modern Sapphire Rapids microprocessor demonstrates this shift toward parallel, highly integrated architectures:

- **Compute Capacity:** Scales up to **60 physical cores and 120 simultaneous threads** per physical socket.
- **Chiplet Design:** Integrates up to **4 distinct chiplets** on a single physical package, connected via high-speed on-chip interconnects.
- **Memory Subsystem Layout:** Supports up to **4 TiB of DDR5 memory**.
- **On-Chip Networking:** Uses an on-chip **Mesh Interconnect Network** to handle cache-coherence traffic across cores, scaling up to 480 cores in a single shared-memory domain across 8 sockets.

7.2 Simultaneous Multi-Threading (SMT) vs. Hyper-Threading

To maximize hardware utilization, modern processors implement hardware-level scheduling known as **Simultaneous Multi-Threading (SMT)** or **Hyper-Threading**:

Planar Instruction Cycles Timeline:

a) Classical Single-Core Superscalar:
[Cycle 1: T0] [Cycle 2: Idle] [Cycle 3: T0] [Cycle 4: Idle] (Stall on memory read)

b) Hyper-Threading Core:
[Cycle 1: T0] [Cycle 2: T1] [Cycle 3: T0] [Cycle 4: T1] (T1 executes during T0 stall)

- **How it works:** A single physical CPU core contains multiple execution units (adders, multipliers, etc.) but often stalls waiting for memory accesses. SMT duplicates the core's register state (including the program counter), exposing two "logical threads" to the operating system.
- **Operating System View:** The OS schedules threads onto each logical CPU as if they were independent, physical cores.
- **Performance:** SMT provides a **sub-linear speedup**. While it does not double processing power (since execution units are still shared), it improves performance by keeping execution units busy when one thread stalls.

Section 8: POSIX Threads (pthreads) Programming API

In modern operating systems, anything that runs outside of the kernel runs inside a process. When a process starts, it initially executes with a single thread. To run tasks concurrently, the application makes system calls to create additional threads that

share the process's address space.

The industry standard API for thread management is **POSIX Threads (pthreads)**.

8.1 Core API Functions

C

```
int pthread_create(pthread_t *thread, const pthread_attr_t *attr,
                  void *(*start_routine) (void*), void *arg);
```

- Spawns a new thread executing `start_routine` with `arg` as its sole argument.
- Returns `0` on success, or an error code.

C

```
void pthread_exit(void *value_ptr);
```

- Terminates the calling thread and makes `value_ptr` available to any thread that joins it.

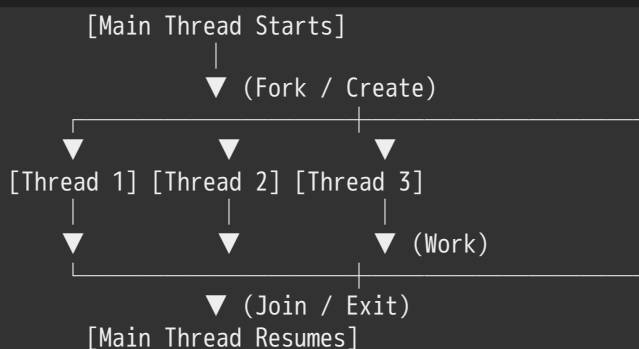
C

```
int pthread_join(pthread_t thread, void **value_ptr);
```

- Suspends execution of the calling thread until the target `thread` terminates.
- If `value_ptr` is non-NULL, it receives the exit status passed to `pthread_exit()`.

8.2 Fork-Join Concurrency Pattern

The Fork-Join pattern is a common multi-threading paradigm where a main thread spawns (forks) a set of child threads to perform tasks in parallel, and then waits (joins) for them to complete before continuing:



8.3 Complete Code Example: The Fork-Join Pattern (With Data Race)

This example compiles and runs a fork-join application. However, it contains a critical **data race** because multiple threads attempt to modify a shared global variable without synchronization.

C

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>

// Shared global variable
int common = 162; //
```

```

void *threadfun(void *threadid) {
    long tid = (long)threadid; //

    // CRITICAL SECTION WITHOUT LOCKING (Data Race!)
    // Multiple threads read, increment, and write to 'common' concurrently.
    int local_val = common;
    common = local_val + 1; // Equivalent to common++

    printf("Thread #%lx stack: %lx common: %d\n",
           tid, (unsigned long)&tid, local_val); //

    pthread_exit(NULL); //
}

int main(int argc, char *argv[]) {
    int nthreads = 4; // Default to 4 threads
    if (argc > 1) {
        nthreads = atoi(argv[1]); //
    }

    pthread_t *threads = malloc(nthreads * sizeof(pthread_t)); //

    printf("Main stack: %lx, common: %d\n",
           (unsigned long)&nthreads, common); //

    // Fork Phase: Create threads
    for (long t = 0; t < nthreads; t++) {
        int rc = pthread_create(&threads[t], NULL, threadfun, (void *)t); //
        if (rc) {
            printf("ERROR: return code from pthread_create() is %d\n", rc); //
            exit(-1); //
        }
    }

    // Join Phase: Wait for all threads to complete
    for (int t = 0; t < nthreads; t++) {
        pthread_join(threads[t], NULL); //
    }

    free(threads);
    printf("Final common value: %d\n", common);
    pthread_exit(NULL); //
}

```

8.4 Analyzing the Data Race

In this example, the output value of the shared variable `common` is **non-deterministic** because the threads execute concurrently.

If four threads execute, the final value of `common` might be `163` or `164` instead of the expected `166`. This occurs because multiple threads can read the initial value of `common` (e.g., `162`), increment it in their local registers, and write back the same value (`163`), overwriting each other's updates.

Section 9: Detailed Execution Stack Trace Walkthrough

To understand how threads manage separate execution paths, we must look at how function calls manipulate the **Execution Stack**.

9.1 Stack Activation Record Mechanics

An execution stack holds **activation records** (stack frames) for active function calls. Each frame contains:

- 1. **Function Parameters:** Input values passed to the function.
- 2. **Local Variables:** Temporary variables declared inside the function.
- 3. **Return Address (PC):** The memory address to jump back to when the function returns.

9.2 Tracing a Recursive Stack Lifecycle

Let's trace the stack frames for the following recursive C code:

```
c

void A(int tmp) {
    if (tmp < 2)
        B();
    printf("%d ", tmp);
}

void B() {
    C();
}

void C() {
    A(2);
}

int main() {
    A(1);
}
```

Step 1: Execution Begins (main)

The program starts by calling A(1) from main (return address: exit).

[Stack Layout] (Stack Pointer SP points to Top)

A: tmp = 1	<--- SP
Return PC: exit	

Step 2: A(1) Calls B()

Since tmp = 1 < 2 , A calls B() (return address: instruction A + 2, which is printf).

[Stack Layout]

A: tmp = 1	
Return PC: exit	
B:	<--- SP
Return PC: A+2	

Step 3: B() Calls C()

Function B immediately calls C() (return address: instruction $B + 1$, the return point of B).

[Stack Layout]

A: tmp = 1 Return PC: exit	
B: Return PC: A+2	
C: Return PC: B+1	<--- SP

Step 4: C() Calls A(2)

Function C calls A recursively, passing 2 as the argument (return address: instruction $C + 1$, the return point of C).

[Stack Layout]

A: tmp = 1 Return PC: exit	
B: Return PC: A+2	
C: Return PC: B+1	
A: tmp = 2 Return PC: C+1	<--- SP

Step 5: Evaluating A(2)

For this call frame, tmp = 2 , which is not less than 2. The if condition is skipped, and the program executes printf(2) , outputting 2 .

Step 6: A(2) Returns

The function returns to C+1 . Its stack frame is popped from the stack.

[Stack Layout]

A: tmp = 1 Return PC: exit	
B: Return PC: A+2	
C: Return PC: B+1	<--- SP

Step 7: Unrolling the Stack

C() completes and returns to B+1 . B() completes and returns to $A + 2$ (the print statement in the first call of A).

[Stack Layout]

A: tmp = 1 Return PC: exit	<--- SP
-------------------------------	---------

Step 8: Execution Completes

The program executes the print statement for `tmp = 1` in the first call of `A` , outputting `1` . The total printed output is `2 1` . `A(1)` returns to `exit` , and the program terminates.

9.3 Memory Layout with Multiple Stacks

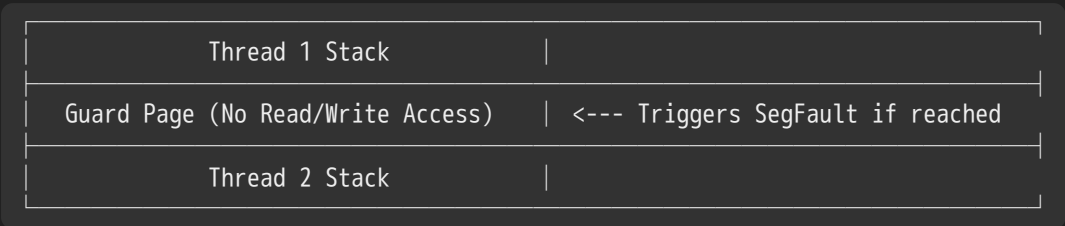
When a process runs multiple threads, each thread must have its own private execution stack to manage function calls independently:

This layout introduces several architectural challenges:

- **Stack Positioning:** The operating system must arrange multiple stacks within the process's single virtual address space.
- **Choosing Maximum Size:** The OS must allocate a maximum size for each stack. If a stack is allocated too much space, virtual memory is wasted. If it is allocated too little, the stack can easily overflow.
- **Stack Overflow Risks:** If a thread's stack grows too large, it can silently overwrite the heap or another thread's stack, corrupting the process's data.

9.4 Catching Stack Violations (Guard Pages)

To prevent threads from overwriting neighboring memory, modern operating systems use **Guard Pages**:



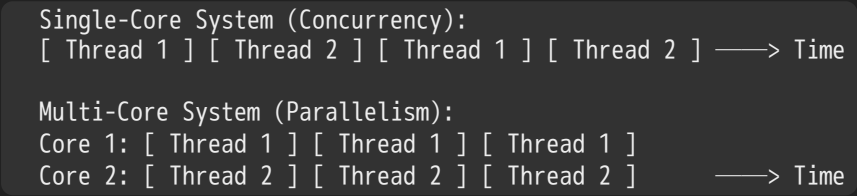
- **How it works:** The operating system inserts an unmapped virtual memory page (a guard page with no read or write permissions) directly between each thread's stack space.
- **Violation Detection:** If a thread's stack grows beyond its limit and attempts to access the guard page, the CPU's memory management unit intercepts the access and triggers a page fault, terminating the process immediately with a segmentation fault instead of allowing silent data corruption.

Section 10: Interleaving, Non-Determinism, & Race Conditions

Writing concurrent programs is challenging because the operating system's scheduler can run threads in any order and switch between them at any time.

10.1 Concurrency vs. Parallelism

- **Concurrency:** Managing multiple tasks at once. On a single-core system, threads execute concurrently by interleaving their instructions over time.
- **Parallelism:** Executing multiple tasks simultaneously on separate physical processors.



10.2 Tracing a Race Condition

A **Race Condition** occurs when multiple threads read and write shared data concurrently, and the final outcome depends on the order in which their instructions interleave.

Let's analyze this example where two threads modify shared variables:

Initial State: $x = 0, y = 0$

Thread A
 $x = y + 1;$

Thread B
 $y = 2;$
 $y = y * 2;$

Depending on how the scheduler schedules these instructions, x can result in one of three possible values:

Interleaving 1 (Thread A Runs First)

```
1. Thread A:  $x = y + 1 \implies x = 0 + 1 = 1$ 
2. Thread B:  $y = 2$ 
3. Thread B:  $y = y * 2 \implies y = 4$ 
```

Result: $x = 1$

Interleaving 2 (Thread B Runs First)

```
1. Thread B:  $y = 2$ 
2. Thread B:  $y = y * 2 \implies y = 4$ 
3. Thread A:  $x = y + 1 \implies x = 4 + 1 = 5$ 
```

Result: $x = 5$

Interleaving 3 (Thread B Suspended Mid-Execution)

```
1. Thread B:  $y = 2$ 
2. Thread A:  $x = y + 1 \implies x = 2 + 1 = 3$ 
3. Thread B:  $y = y * 2 \implies y = 3 * 2 = 6$ 
```

Result: $x = 3$

This variation demonstrates that concurrent code can produce different results on different runs, making bugs difficult to reproduce and debug.

Section 11: Synchronization and Locks

To prevent race conditions, cooperating threads must synchronize their access to shared resources.

11.1 Key Terminology

- **Synchronization:** Coordinating the execution of multiple threads to ensure correct behavior when accessing shared data.
- **Mutual Exclusion:** Ensuring that only one thread can access a shared resource or execute a critical section of code at any given time.
- **Critical Section:** A block of code that accesses shared resources and must not be executed by more than one thread concurrently.
- **Lock:** An object used to enforce mutual exclusion.

11.2 The Lock Abstraction

A lock provides two primary atomic operations:

1. **Acquire:** A thread attempts to acquire the lock. If the lock is free, the thread marks it as busy and continues execution. If the lock is held by another thread, the calling thread blocks until the lock is released.
2. **Release:** The thread holding the lock releases it, marking it as free and allowing blocked threads to acquire it.

11.3 Resolving Race Conditions with pthreads Mutexes

The pthreads library provides mutual exclusion objects called **mutexes** to synchronize thread access to shared data.

C

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>

int common = 162; // Shared resource

// Initialize the mutex lock
pthread_mutex_t common_lock = PTHREAD_MUTEX_INITIALIZER; //

void *threadfun(void *threadid) {
    long tid = (long)threadid; //

    // Acquire the lock to enter the critical section safely
    pthread_mutex_lock(&common_lock); //

    // Critical Section: Only one thread can execute this block at a time
    int my_common = common++; //

    // Release the lock to allow other threads to enter
    pthread_mutex_unlock(&common_lock); //

    printf("Thread #%lx stack: %lx common: (%d)\n",
           tid, (unsigned long)&tid, my_common); //

    pthread_exit(NULL); //
}

int main(int argc, char *argv[]) {
    int nthreads = 4;
    if (argc > 1) {
        nthreads = atoi(argv[1]);
    }

    pthread_t *threads = malloc(nthreads * sizeof(pthread_t));

    for (long t = 0; t < nthreads; t++) {
        pthread_create(&threads[t], NULL, threadfun, (void *)t);
    }

    for (int t = 0; t < nthreads; t++) {
        pthread_join(threads[t], NULL);
    }

    free(threads);
    printf("Safe final common value: %d\n", common);
    return 0;
}
```

By wrapping the modification of `common` inside a `pthread_mutex_lock` and `pthread_mutex_unlock` block, we guarantee that only one thread can increment the variable at a time, ensuring a deterministic and correct final outcome.

Section 12: Practice Exam Problems with Detailed Solutions



Use these practice problems to test your understanding of the concepts covered in Lecture 3.

Question 1: Base and Bound Relocation Address Mapping

A CPU runs a process under a Base and Bound relocation register system. The hardware registers are currently set to:

- **Base:** 0x4000
- **Bound:** 0x1200

For each of the following virtual memory addresses issued by the application, determine whether the access is valid. If it is valid, calculate the corresponding physical memory address.

1. Virtual Address 0x0050
2. Virtual Address 0x1200
3. Virtual Address 0x1500

Solution:

The hardware enforces the check: $\text{Virtual Address} < \text{Bound}$. If valid, the physical address is: $\text{Physical Address} = \text{Virtual Address} + \text{Base}$.

1. Virtual Address 0x0050 :

- Check: $0x0050 < 0x1200$ (Valid).
- Translation: $0x0050 + 0x4000 = 0x4050$.
- **Answer:** Valid; maps to physical address 0x4050 .

2. Virtual Address 0x1200 :

- Check: $0x1200 < 0x1200$ (Invalid; the virtual address must be strictly less than the Bound).
- **Answer:** Invalid; triggers a memory protection fault (Segmentation Fault).

3. Virtual Address 0x1500 :

- Check: $0x1500 < 0x1200$ (Invalid; address is out of bounds).
- **Answer:** Invalid; triggers a memory protection fault (Segmentation Fault).

Question 2: Stack Execution Frames Trace

Given the following C program:

```
c

void X(int val) {
    if (val > 1) {
        X(val - 1);
    }
    printf("%d", val);
}

int main() {
    X(3);
}
```

1. Sketch the layout of the execution stack when it reaches its maximum depth during execution.
2. What is the printed output of this program?

Solution:

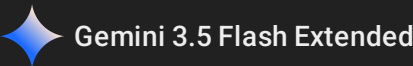
1. Stack trace at maximum depth:

- `main()` calls `X(3)` (Return PC: `exit`).
- `X(3)` calls `X(2)` recursively (Return PC: print statement in `X`).
- `X(2)` calls `X(1)` recursively (Return PC: print statement in `X`).
- For `X(1)` , the condition `val > 1` is false, so it does not make any further recursive calls. This is the maximum depth of the stack:

X: val = 3 Return PC: exit	
X: val = 2 Return PC: X print	
X: val = 1 Return PC: X print	<--- SP (Max Stack Depth)

2. Program Output:

- At maximum depth, `X(1)` executes `printf` and outputs `1` . It then returns to `X(2)` .
- `X(2)` resumes execution at its print statement, outputs `2` , and returns to `X(3)` .
- `X(3)` resumes execution at its print statement, outputs `3` , and returns to `main` .
- **Answer:** The program outputs `123` .



CS162 Operating Systems & Systems Programming

Lecture 4: Abstractions 2: Threads (Con't) & Process Management (Exam-Ready Notes)

Section 1: Concurrency vs. Parallelism Revisited

To build high-performance systems, we must distinguish between how threads are executed logically versus physically.

- **Concurrency:** The logical grouping and management of multiple tasks over overlapping time frames. It is about **handling multiple things at once (MTAO)**. On a single-core processor, concurrency is achieved by interleaved execution (time-slicing), where the operating system's scheduler dynamically switches between threads.
- **Parallelism:** The physical execution of multiple tasks at the exact same physical instant. It is about **doing multiple things simultaneously**. Parallelism requires multiple physical processor cores or execution units.

not Concurrent	not Parallel	[Thread A] —————> [Thread B] —————> (Sequential execution)
Concurrent	not Parallel	[A][B][A][B][A] ———> (Interleaved on 1 Core)
Concurrent	and Parallel	Core 1: [Thread A] —————> Core 2: [Thread B] —————> (Simultaneous on 2 Cores)

Based on the conceptual execution models of threads.

Schedulability and Time-Slicing

When running threads concurrently, the scheduler is free to execute them in any order and with any granularity of interleaving. A thread might run to absolute completion without interruption, or it might be preempted and time-sliced in large or extremely small instruction chunks. Programs must be designed to behave correctly regardless of the scheduling order.

Section 2: Thread Memory Layout & Stack Management

In a multi-threaded process, all threads reside inside a single, shared address space. While they share global data and the heap, each thread must have its own private execution context, which includes its own set of CPU registers and its own private execution stack.

Key Engineering Challenges in Multi-threaded Layouts

When laying out multiple thread stacks inside a single process address space, systems programmers must address several design challenges:

1. **Stack Positioning:** Where do we position the stacks relative to each other? In a single-threaded process, the stack grows downwards from the top of the address space, and the heap grows upwards from the bottom. With multiple threads, the OS must allocate multiple stacks inside the address space, leaving virtual memory gaps between them.
2. **Determining Stack Size:** What maximum size should we choose for each stack? If the allocated stack size is too small, a thread executing deeply nested or recursive function calls will quickly run out of stack space. If it is too large, the process's virtual address space becomes fragmented, limiting the maximum number of threads that can be spawned.
3. **Handling Stack Violations:** What happens if a thread violates its stack boundary? If a thread's stack grows too large, it can overflow into the memory region allocated for another thread's stack or the heap. This leads to silent data corruption, which is exceptionally difficult to debug.
4. **Catching Violations (Guard Pages):** How can we catch stack overflows? Modern operating systems place a read-and-write-protected virtual memory page—called a **Guard Page**—directly below each thread's stack. If a thread exceeds its stack limit and touches this page, the MMU triggers a page fault exception, allowing the kernel to catch the violation immediately and terminate the offending thread (producing a Segmentation Fault) rather than allowing silent memory corruption.

Section 3: Thread Interleaving, Nondeterminism, & Race Conditions

3.1 The Illusion vs. Physical Reality

The programmer is presented with the illusion of an infinite number of processors, where each thread executes sequentially on its own dedicated virtual core.

In reality, the physical processor core(s) multiplex threads dynamically. Threads execute at variable "speeds" because of hardware interrupts, cache misses, page faults, and scheduling decisions. Programs must be designed to work correctly under any possible scheduling timeline.

3.2 Programmer vs. Processor View of Execution

Consider the following sequence of instructions:

```
c
x = x + 1;
y = y + x;
```

```
z = x + 5 * y;
```

To the programmer, these instructions appear to execute atomically and sequentially. However, the processor can suspend a thread at any point (even mid-instruction at the assembly level) to run other threads, resuming it later.

Possible Execution Timeline #1:

[Thread 1: $x = x+1$] $\longrightarrow$ [Thread 1: $y = y+x$] $\longrightarrow$ [Thread 1: $z = x+5y$] (Runs uninterrupted)

Possible Execution Timeline #2:

[Thread 1: $x = x+1$] $\longrightarrow$ (Thread 1 Suspended / Other Threads Run) $\longrightarrow$ [Thread 1: $y = y+x$] $\longrightarrow$ [Thread 1: $z = x+5y$]

Possible Execution Timeline #3:

[Thread 1: $x = x+1$] $\longrightarrow$ [Thread 1: $y = y+x$] $\longrightarrow$ (Thread 1 Suspended / Other Threads Run) $\longrightarrow$ [Thread 1: $z = x+5y$]

Thread suspension can occur between any two instructions, altering the interleaving of operations.

3.3 Independent vs. Cooperating Threads

- **Independent Threads:** These threads do not share any state or memory with other threads. Their execution is completely deterministic and reproducible, meaning the same inputs will always produce the same outputs.
- **Cooperating Threads:** These threads share memory, data structures, or hardware resources. They are non-deterministic and their behavior depends on the exact scheduling order of instructions, making them highly susceptible to race conditions if not properly synchronized.
- **The Goal:** Build cooperating systems with **Correctness by Design**.

3.4 Race Condition Case Studies

Case Study 1: Independent Threads

Initially, $x = 0$ and $y = 0$.

- **Thread A:** $x = 1$;
- **Thread B:** $y = 2$;

Question: What are the possible values of x after both threads finish?

Answer: x must be 1. Because Thread A and Thread B are independent and do not write to the same memory locations, Thread B cannot interfere with the value of x .

Case Study 2: Cooperating Threads (Race Condition)

Initially, $x = 0$ and $y = 0$.

- **Thread A:** $x = y + 1$;
- **Thread B:** $y = 2; y = y * 2$;

Question: What are the possible values of x after both threads finish?

Answer: x can non-deterministically be 1, 3, or 5 depending on instruction interleaving:

- **Scenario 1** ($x = 1$): Thread A executes before Thread B starts. $x = y + 1 \implies x = 0 + 1 = 1$
- **Scenario 2** ($x = 3$): Thread B executes its first instruction ($y = 2$), then Thread A preempts Thread B and runs ($x = y + 1$), and finally Thread B finishes ($y = y * 2$). $x = 2 + 1 = 3$
- **Scenario 3** ($x = 5$): Thread B runs to completion (y becomes 4), and then Thread A runs. $x = 4 + 1 = 5$

This is a classic **Race Condition**, where Thread A races against Thread B to access the shared variable y .

Section 4: Mutual Exclusion & Locks

To prevent race conditions, cooperating threads must synchronize their access to shared data structures.

4.1 Key Synchronization Definitions

- **Synchronization:** The coordination of thread execution to ensure correct behavior when accessing shared data.
- **Mutual Exclusion:** A mechanism that ensures only one thread can perform a specific operation or access a shared resource at any given time.
- **Critical Section:** A block of code that accesses a shared resource and must not be entered by more than one thread concurrently.
- **Lock:** A software object that provides mutual exclusion, allowing only one thread to hold it at a time.

4.2 The Lock Interface

A lock exposes two fundamental atomic operations:

- `Lock.acquire()` : Attempts to acquire the lock. If the lock is currently free, the calling thread marks it as busy and continues executing. If the lock is busy, the calling thread is blocked and must wait until the lock becomes free.
- `Lock.release()` : Marks the lock as free, allowing one of the waiting threads to acquire it. This operation should only be called by the thread currently holding the lock.

4.3 Synchronizing a Concurrent Insert

In the tree-based set data structure shown above, we can guarantee thread safety by wrapping our operations in acquire and release calls:

Thread A	Thread B
<code>Lock.acquire();</code>	<code>Lock.acquire();</code> (blocks if Thread A has it)
<code>Insert(3) into tree;</code>	<code>Insert(4) into tree;</code>
<code>Lock.release();</code>	<code>Lock.release();</code>

Ensuring sequential execution of inserts through mutual exclusion.

Section 5: POSIX Mutex API & Fixing Race Conditions in C

5.1 The pthreads Mutex Interface

The standard POSIX threads library provides mutual exclusion objects called **mutexes**:

```
c

// Dynamic initialization of a mutex
int pthread_mutex_init(pthread_mutex_t *mutex, const pthread_mutexattr_t *attr); //

// Locking and unlocking operations
int pthread_mutex_lock(pthread_mutex_t *mutex); //
int pthread_mutex_unlock(pthread_mutex_t *mutex); //
```

5.2 Analyzing a Vulnerable Concurrent Program

The following program spawns a user-defined number of threads that concurrently increment a shared global variable without locking:

```
c
```

```

#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <string.h>

int common = 162; // Shared global variable

void *threadfun(void *threadid) {
    long tid = (long)threadid; //

    // VULNERABLE CRITICAL SECTION: 'common++' is not atomic!
    // It compiles down to: Read common, Increment register, Write back common.
    printf("Thread #%lx stack: %lx common: %lx (%d)\n",
        tid, (unsigned long)&tid, (unsigned long)&common, common++); //

    pthread_exit(NULL); //
}

int main(int argc, char *argv[]) {
    long t; //
    int nthreads = 2; //
    if (argc > 1) {
        nthreads = atoi(argv[1]); //
    }

    pthread_t *threads = malloc(nthreads * sizeof(pthread_t)); //

    printf("Main stack: %lx, common: %lx (%d)\n",
        (unsigned long)&t, (unsigned long)&common, common); //

    for (t = 0; t < nthreads; t++) {
        int rc = pthread_create(&threads[t], NULL, threadfun, (void *)t); //
        if (rc) {
            printf("ERROR; return code from pthread_create() is %d\n", rc); //
            exit(-1); //
        }
    }

    for (t = 0; t < nthreads; t++) {
        pthread_join(threads[t], NULL); //
    }

    pthread_exit(NULL); //
}

```

Why This Code Is Broken

Because `common++` is not atomic, the compiled code reads the value from memory into a register, increments the register, and writes it back. If Thread 0 and Thread 1 interleave during these steps, they can read the same initial value and write back the same incremented value, causing one of the increments to be lost.

This leads to non-deterministic outputs like the following:

```
Main stack: 7ffee2c6b6b8, common: 10cf95048 (162)
Thread #1 stack: 70000d83bef8 common: 10cf95048 (162) <-- Thread 1 read 162
Thread #3 stack: 70000d941ef8 common: 10cf95048 (164) <-- Increments are lost/unordered!
Thread #2 stack: 70000d8beef8 common: 10cf95048 (165)
Thread #0 stack: 70000d7b8ef8 common: 10cf95048 (163)
```

The output depends entirely on the scheduler's order.

5.3 Correcting the Program Using pthreads Mutexes

To fix this race condition, we must serialize access to the critical section using a mutex:

```
c

#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>

int common = 162; //

// Statically initialize the mutex
pthread_mutex_t common_lock = PTHREAD_MUTEX_INITIALIZER; //

void *threadfun(void *threadid) {
    long tid = (long)threadid; //

    // Acquire the lock to enter the critical section
    pthread_mutex_lock(&common_lock); //

    // CRITICAL SECTION (Protected)
    int my_common = common++; //

    // Release the lock immediately after completing the shared access
    pthread_mutex_unlock(&common_lock); //

    printf("Thread #%lx stack: %lx common: %lx (%d)\n",
           tid, (unsigned long)&tid, (unsigned long)&common, my_common); //

    pthread_exit(NULL); //
}
```

By wrapping the read and modification of `common` inside `pthread_mutex_lock` and `pthread_mutex_unlock` statements, we ensure that only one thread can execute this block at a time, guaranteeing correct and deterministic behavior.

Section 6: User-to-Kernel Mode Transitions

Operating systems isolate user applications within a restricted environment (User Mode) and handle privileged operations within a secure boundary (Kernel Mode).

6.1 The Three Transition Vectors

There are three distinct mechanisms that trigger a transfer of control from user space to kernel space:

Vector	Trigger Type	Conceptual Model	Key Examples
System Call (Syscall)	Synchronous / Intentional	The process explicitly requests a system service. This is like a function call but operates outside the process's address space.	<code>fork()</code> , <code>exec()</code> , <code>write()</code> , <code>exit()</code>
Hardware Interrupt	Asynchronous / External	An external hardware event triggers a context switch. This occurs independently of the currently running process.	Timer expiration, network packet arrival, disk write completion
Trap or Exception	Synchronous / Error	An internal processor event or error within the process triggers a context switch.	Segmentation faults, division by zero, page faults

Section 7: Implementing Safe Kernel Transfers & Separate Stacks

To prevent user programs from compromising system security or stability, the operating system must strictly enforce safe kernel transitions.

7.1 Security Requirements for Kernel Transfers

- **Atomic Mode Switches:** Entering kernel mode must automatically switch the mode bit and jump to a pre-defined kernel instruction handler in a single, atomic step.
- **Indirection Tables:** The CPU must restrict entry points into the kernel. It does this using lookup tables—such as the **System Call Table** and **Interrupt Vector Table**—which map syscall or interrupt IDs to specific kernel functions. This prevents user programs from jumping to arbitrary locations in kernel memory.
- **Preserving Process State:** The kernel handler must save the entire hardware state (registers, program counter, and flags) of the suspended process before starting its own work.

7.2 The Two-Stack Model

The kernel **cannot** write to or use the user-space stack for its own execution:

- **Security risk:** The kernel cannot trust the user's stack pointer to point to a valid, aligned memory location. If it did, a malicious user could pass an invalid pointer to trigger a kernel crash.
- **Concurrency risk:** In a multi-threaded application, other running threads could modify data on the user stack while the kernel is using it, leading to TOCTOU (Time-of-Check to Time-of-Use) vulnerabilities.

As a result, each thread has both a **User Stack** (in user space) and a **Kernel Stack** (in protected kernel space).

7.3 Hardware Register Transitions During a Transition

When an interrupt or system call occurs, the hardware automatically saves the user process's state onto the thread's **Kernel Stack**:

Before Interrupt:
[Registers]
SS:ESP —> User Stack pointer
CS:EIP —> User Instruction
EFLAGS —> CPU flags

During Interrupt (Saved onto Kernel Stack):

SS	(User Stack Segment)
ESP	(User Stack Pointer)
EFLAGS	(User CPU flags)
CS	(User Code Segment)
EIP	(User Instruction Pointer)
Error Code	(Optional)

The hardware automatically pushes these registers onto the kernel stack to preserve the user-space context.

7.4 Handling Arguments Safely

- **Locating Arguments:** The kernel locates system call arguments passed in the registers or on the user stack.
- **Copying Arguments:** The kernel copies these arguments from user memory into kernel memory before validation. This protects the kernel from malicious programs trying to alter arguments mid-check.
- **Validating Arguments:** The kernel validates all inputs to prevent errors in user code from causing kernel crashes.
- **Copying Results Back:** After executing the system call, the kernel copies any output values back into user memory.

Section 8: Hardware Interrupt Routing & Controllers

Hardware interrupts are routed from external devices to the CPU through a dedicated hardware component called the **Interrupt Controller**.

8.1 Key Hardware Components

- **Interrupt Lines:** Physical wires connected to hardware devices (like network cards, storage controllers, or timers).
- **Software Interrupts:** Special registers that allow the operating system to trigger software-defined interrupt paths.
- **Interrupt Mask:** A bitmask inside the controller that allows the operating system to selectively enable or disable individual interrupt lines.
- **Priority Encoder:** Evaluates all active, enabled interrupt lines and selects the one with the highest priority to forward to the CPU.
- **Global Interrupt Disable Flag:** An internal flag in the CPU that can disable all maskable interrupts.
- **Non-Maskable Interrupt (NMI):** A dedicated hardware line for critical errors (such as memory parity faults or hardware failures) that cannot be disabled by the CPU.

8.2 The Interrupt Vector Dispatch Pattern

When an interrupt is accepted by the CPU, the hardware uses the **Interrupt ID** as an index into the **Interrupt Vector Table** to look up the address of the corresponding handler and jump to it.

This lookup pattern is also used to dispatch system calls and exceptions.

Section 9: The Process Lifecycle & Management API

Everything running outside the kernel executes within a process. Processes are managed dynamically through a set of core operating system APIs.

9.1 Bootstrapping: The "INIT" Process

Since all processes are created by other processes, how does the very first process start?

- **The First Process:** The kernel starts the first user-space process directly when the system boots.
- **INIT Process:** This process is called `init` or the "system bootstrap" process. All other processes on the system are spawned as descendants of this initial process.

9.2 The Process Management API

- `fork()` : Copies the current process to create a new child process.
- `exec()` : Replaces the current process's program image with a new executable.
- `wait()` : Suspends the calling process until one of its child processes terminates.
- `exit()` : Terminates the current process.
- `kill()` : Sends an asynchronous signal to a process.
- `sigaction()` : Registers a custom signal handler function within a process.

Section 10: Process Management Case Studies & Code Walkthroughs

10.1 Case Study 1: The OS Library Exit Wrapper

Consider the following program (`pid.c`):

```
c

#include <sys/types.h>
#include <unistd.h>
#include <stdio.h>
#include <stdlib.h>

int main(int argc, char *argv[]) {
    pid_t pid = getpid(); // Retrieve process ID
    printf("My pid: %d\n", pid); //
    exit(0); //
}
```

Question: What happens if `main` returns without explicitly calling `exit()` ?

Answer: The program still terminates cleanly. This is because standard executables are wrapped by system libraries (like `libc`). The library defines the true entry point of the executable, which calls `main` :

```
c

// Conceptual runtime bootstrap code inside standard system library (libc)
void _start() {
    // ... initialize environment ...
    int status = main(argc, argv);
    exit(status); // If main returns, the library calls exit automatically
}
```

The OS library wraps the main function to guarantee clean termination.

10.2 Case Study 2: Process Creation via `fork()`

The `fork()` system call duplicates the calling process, creating an identical child process.

```
c

pid_t cpid = fork(); //
```

- **Return Values:**

- **In the Parent Process:** `fork()` returns the positive Process ID (PID) of the newly created child process.
- **In the Child Process:** `fork()` returns `0`.
- **Error:** If the call fails, `fork()` returns a negative value, and no child process is created.

- **Shared State:** The child process starts as a single-threaded copy of the parent, duplicating its address space (memory), open file descriptors, and register values.

Walkthrough of `fork1.c`

This program demonstrates how to use `fork()` to create a child process and run separate code paths in the parent and child:

```
c

#include <stdlib.h>
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>

int main(int argc, char *argv[]) {
    pid_t cpid, mypid;
    pid_t pid = getpid(); //

    printf("Parent pid: %d\n", pid); //

    cpid = fork(); // Process splits into Parent and Child

    if (cpid > 0) {
        /* Parent Process Path */ //
        mypid = getpid(); //
        printf("[%d] parent of [%d]\n", mypid, cpid); //
    } else if (cpid == 0) {
        /* Child Process Path */ //
        mypid = getpid(); //
        printf("[%d] child\n", mypid); //
    } else {
        perror("Fork failed"); //
    }

    return 0;
}
```

Output Analysis:

```
Parent pid: 45210
[45210] parent of [45211]
[45211] child
```

Note: The parent and child processes execute concurrently, so the order of their print statements is non-deterministic and depends on the scheduler.

10.3 Case Study 3: Concurrent Loops (`fork_race.c`)

This program demonstrates how parent and child processes run concurrently as independent execution streams after a `fork()` :

```
c

#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>

int main() {
    int i;
    pid_t cpid = fork(); //

    if (cpid > 0) {
        for (i = 0; i < 10; i++) {
            printf("Parent: %d\n", i); //
        }
    } else if (cpid == 0) {
        for (i = 0; i > -10; i--) {
            printf("Child: %d\n", i); //
        }
    }
    return 0;
}
```

Question 1: What does this print?

Answer: The output will be a non-deterministic mix of "Parent: X" and "Child: Y" print statements. This is because the parent and child are scheduled independently.

Question 2: Would adding a call to `sleep(1)` inside each loop change the behavior?

Answer: Yes. Calling `sleep(1)` forces the running process to yield the CPU and enter a blocked state. This allows the scheduler to switch to the other process, resulting in a more evenly interleaved output (typically alternating between parent and child prints).

10.4 Case Study 4: Starting New Programs (`execv` & `wait`)

To run a different program inside a child process, we use the `exec()` family of system calls alongside `wait()` to synchronize execution.

Process Management Example (`fork2.c`)

This program shows how a parent process forks a child, the child executes a new program (the `ls` command), and the parent waits for the child to finish before exiting:

```
c

#include <stdio.h>
#include <stdlib.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <unistd.h>

int main() {
```

```

int status; //
pid_t tcpid; //
pid_t cpid = fork(); //

if (cpid > 0) {
    /* Parent Process Path */ //
    pid_t mypid = getpid(); //
    printf("[%d] parent of [%d]\n", mypid, cpid); //

    // Wait for the child process to terminate and collect its exit status
    tcpid = wait(&status); //
    printf("[%d] bye %d (%d)\n", mypid, tcpid, WEXITSTATUS(status)); //

} else if (cpid == 0) {
    /* Child Process Path */ //
    pid_t mypid = getpid(); //
    printf("[%d] child\n", mypid); //

    // Prepare arguments for the 'ls' command
    char *args[] = {"ls", "-l", NULL}; //

    // Replace the child's program image with '/bin/ls'
    execv("/bin/ls", args); //

    // execv only returns if the call fails
    perror("execv failed"); //
    exit(1); //
}

return 0;
}

```

Why `execv` Does Not Return

On success, `execv()` replaces the process's address space, code, and stack with the new program. The original code after the `execv` call is completely gone, so it only executes if the system call fails (e.g., if the file path is incorrect).

Section 11: The Shell Pattern (Fork-Exec-Wait)

Command-line shells (such as Bash) use a fundamental process management pattern called the **Fork-Exec-Wait** loop.

Steps of the Shell Loop

1. **Fork:** The shell forks a copy of itself, creating a child process.
2. **Exec:** The child process calls `exec()` to replace its program image with the command entered by the user (e.g., `ls` or `grep`).
3. **Wait:** The parent shell process calls `wait()` to block and yield the CPU until the child process finishes executing.
4. **Resume:** Once the child process terminates, the `wait()` call returns, and the parent shell resumes to prompt the user for the next command.

Section 12: Asynchronous Signals & Signal Handling

Signals are asynchronous notifications sent by the kernel to a user-space process. They behave like software interrupts, preempting the application's normal execution to run a signal handler.

12.1 Registering Custom Signal Handlers

We can use the `sigaction` API to catch specific signals and run custom handler functions:

```
c

#include <stdio.h>
#include <stdlib.h>
#include <sys/types.h>
#include <unistd.h>
#include <signal.h>

// Custom signal handler callback function
void signal_callback_handler(int signum) { //
    printf("Caught signal %d!\n", signum); //
    exit(1); // Terminate process safely
}

int main() {
    struct sigaction sa; //
    sa.sa_flags = 0; //

    // Initialize the signal mask to block other signals during execution
    sigemptyset(&sa.sa_mask); //

    // Assign our custom callback function to the handler
    sa.sa_handler = signal_callback_handler; //

    // Register the handler for SIGINT (Control-C)
    sigaction(SIGINT, &sa, NULL); //

    while (1) {
        // Infinite loop waiting for a signal
    }
    return 0;
}
```

Question: What happens if a process receives a `SIGINT` (Control-C) but has not registered a custom signal handler?

Answer: The process is terminated immediately. The operating system defines default handler behaviors for all signals, and the default action for `SIGINT` is termination.

12.2 Common POSIX Signals

- `SIGINT` : Sent when the user presses Control-C. Its default action is process termination.
- `SIGTERM` : The standard termination signal sent by administrative tools (like the `kill` command) to request a clean exit.
- `SIGTSTP` : Sent when the user presses Control-Z. Its default action is to suspend (stop) process execution.
- `SIGKILL` : Forcefully terminates a process.
- `SIGSTOP` : Forcefully suspends a process.

12.3 Uncatchable Signals: SIGKILL and SIGSTOP

The `SIGKILL` and `SIGSTOP` signals **cannot** be caught, blocked, or ignored using `sigaction` .

The Rationale

If processes could catch or block `SIGKILL` and `SIGSTOP`, buggy or malicious programs could run forever without any way for system administrators to stop them. Restricting these signals ensures that administrators can always forcefully terminate or suspend runaway processes, maintaining control over system resources.

Section 13: High-Yield Exam Concept Review

Verify your understanding of these core concepts before the exam:

1. Concurrency vs. Parallelism

- Concurrency is about logical execution overlap (handling multiple tasks at once). Parallelism is about physical execution overlap (running multiple tasks simultaneously on different cores).

2. Thread Stack Layout

- While threads share heap and global data, they must have private execution stacks to maintain their own function call contexts. Overflows are caught using protected virtual memory zones called **Guard Pages**.

3. Mutual Exclusion

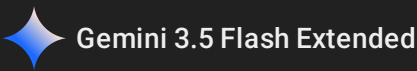
- Protecting shared data structures requires mutual exclusion. POSIX mutexes (`pthread_mutex_lock` / `unlock`) serialize access to critical sections, eliminating non-deterministic race conditions.

4. Separate Kernel Stacks

- The kernel must use a separate, secure kernel stack for each thread because it cannot trust user-space stack pointers. On an interrupt, the hardware automatically saves the user state (SS, ESP, EFLAGS, CS, EIP) onto the kernel stack.

5. Process Lifecycle APIs

- `fork()` copies the calling process and returns `0` in the child and the child's PID in the parent. `exec()` replaces the process's code image and never returns on success. `wait()` suspends the parent process until the child terminates.

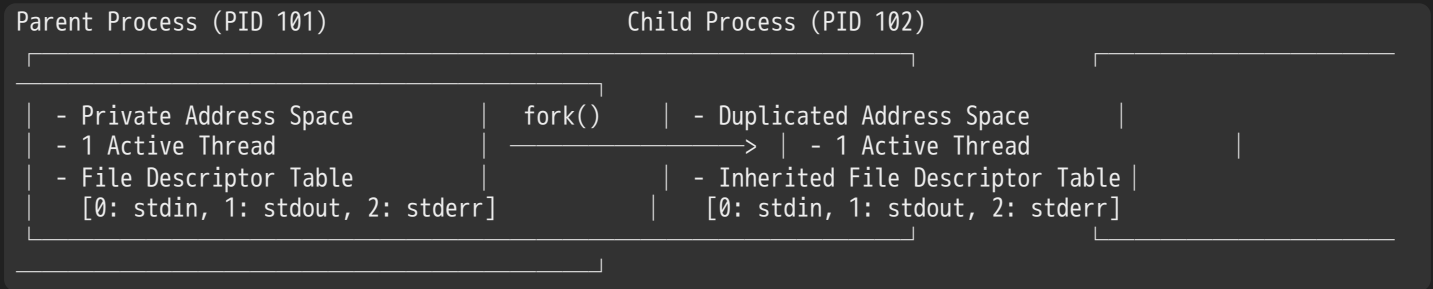


CS162 Operating Systems & Systems Programming

Lecture 5: Abstractions 3: Files and I/O, Sockets and IPC (Detailed Exam Notes)

1. Review of Process Creation and Duplication (`fork`)

To understand the UNIX input/output model, we must first recall how processes are created and how their resource states are inherited.



- **The `fork()` System Call:** Duplicates the calling parent process to create a new child process.
 - **Execution State:** The child process starts with a single active thread of control.
 - **Return Values:** * If the return value is > 0 , execution is continuing in the context of the original parent process, and the returned value is the Process ID (PID) of the new child.
 - If the return value is $== 0$, execution is running in the context of the newly created child process.
 - If the return value is < 0 , an allocation error occurred, and no child process was created.
 - **Resource Duplication:** The entire state of the original process is duplicated in both parent and child. This includes the private virtual address space, memory segments, environmental variables, and the process's file descriptor table. For system programmers, the mental model of `fork()` must be treated as a complete duplication of the parent's resources.
-

2. The UNIX Philosophy: "Everything is a File"

One of the most defining design patterns of UNIX and POSIX-compliant operating systems is the radical abstraction: "Everything is a File".

- **Unified Interface:** Rather than exposing unique system call interfaces for every distinct hardware device, the operating system exposes an identical software interface for vastly different physical abstractions:
 - Regular text or binary files stored on a physical hard disk or SSD.
 - Hardware devices (terminals, serial consoles, system printers).
 - Network communication channels (Sockets).
 - Local interprocess communication (IPC) lines (Pipes, Unix domain sockets).
 - **Core API System Calls:** This unified model is driven by four fundamental low-level system calls: `open()` , `read()` , `write()` , and `close()` .
 - **The Custom Escape Hatch (`ioctl`):** For device-specific configurations that do not fit nicely into standard stream read/write models (such as ejecting a disk tray or changing a terminal's baud rate), the system provides the `ioctl()` (I/O Control) system call.
 - **Historical Context:** Exposing devices as files was proposed by Dennis Ritchie and Ken Thompson in their 1974 paper "*The UNIX Time-Sharing System*".
-

3. The POSIX Standard Interface

To prevent fragmentation across various operating systems, vendors established a unified standard.

- **Definition:** **POSIX** stands for *Portable Operating System Interface*.
 - **Purpose:** It defines a standardized application programming interface (API) to bring order to many Unix-derived systems. This ensures that software written on one compliant system can easily compile and run on another without requiring a complete rewrite.
 - **Cross-Platform Availability:** While native to Unix and Linux systems, POSIX interfaces are partially available on non-Unix platforms, such as Windows (via compatibility layers like subsystem utilities).
 - **Architectural Requirement:** POSIX compliance requires the operating system to support a standardized system call interface.
-

4. The File System Abstraction

An operating system uses a directory structure to translate physical disk sectors into a logical file system.

4.1 What is a File?

- **Sequence of Bytes:** To POSIX systems, a file is a named collection of data stored in a file system, represented as a simple, unstructured sequence of bytes. The kernel does not enforce any internal structure on file contents; it can represent raw text, binary code, database records, or serialized structures.
- **File Metadata:** Information *about* the file is stored separately from the byte sequence. This includes the file size, modification/access times, owner identity, security descriptors, and Access Control Lists (ACLs).

4.2 What is a Directory?

- **Hierarchical Naming Graph:** A directory is a "folder" that maps logical names to file references, allowing directories to be nested within other directories.
- **Unique Paths:** A path through this directory graph uniquely identifies a specific file or folder. For example, the path `/home/ff/cs162/public_html/fa14/index.html` traces from the root directory `/` down to a specific HTML page.

4.3 Paths and Working Directories

- **Current Working Directory (CWD):** Every active process maintains a current working directory, which acts as the reference point for relative path lookups.
- **Modifying CWD:** A process can change its working directory using the `chdir()` system call:

```
c

int chdir(const char *path); // Changes the process's CWD
```

- **Absolute Paths:** Paths starting with a leading slash `/` are absolute paths and ignore the process's CWD (e.g., `/home/oski/cs162`).
- **Relative Paths:** Paths that do not start with `/` are evaluated relative to the process's CWD (e.g., `index.html` or `./index.html`).
 - `.` refers to the current directory.
 - `..` refers to the parent directory (one level up).
 - `~` (and patterns like `~cs162/`) are shell-expanded to point to a user's home directory.

5. The Layered System I/O Architecture

An operating system uses several layers to route a user's high-level read request down to physical disk hardware.

1. **Application / Service:** The user-level software program requesting file operations.
2. **High-Level I/O (Buffered I/O Streams):** The standard user-space library layer (such as `stdio.h` in C). It processes data in user-managed chunks (streams) and buffers reads and writes to reduce system call overhead.
3. **Low-Level I/O (File Descriptors):** The raw kernel interface. Applications make system calls (`open`, `read`, `write`, `close`) using an integer file descriptor.
4. **System Call Interface:** Transition boundary between user mode and kernel mode.
5. **File System (File Descriptions / Inodes):** The kernel layer that manages file lookup tables, directory paths, and file descriptions. It translates logical byte offsets into physical disk block offsets.
6. **I/O Device Driver:** Hardware-specific kernel code that coordinates with physical device controllers using Direct Memory Access (DMA) or programmed I/O commands to transfer data.

6. High-Level C File API (Streams & Buffering)

The standard C library (`stdio.h`) provides a high-level file API based on the **FILE** data structure. This API operates on "streams," which are buffered sequences of bytes.

6.1 Stream Lifecycle and Modes

```
c
#include <stdio.h>

FILE *fopen(const char *filename, const char *mode); // Opens a file stream
int fclose(FILE *fp);                               // Closes a file stream
```

When opening a stream with `fopen()` , you must specify the access mode:

Text Mode	Binary Mode	Description
"r"	"rb"	Open an existing file for reading. Returns NULL if the file does not exist.
"w"	"wb"	Open for writing. Creates the file if it does not exist, and truncates its length to zero if it does.
"a"	"ab"	Open for appending. Creates the file if it does not exist, and writes all data to the end of the file.
"r+"	"rb+"	Open an existing file for reading and writing.
"w+"	"wb+"	Open for reading and writing. Creates the file if it does not exist, and truncates its length to zero if it does.
"a+"	"ab+"	Open for reading and writing. Creates the file if it does not exist. Reads start at the beginning, but all writes append to the end.

6.2 Predefined Standard Streams

When a program starts, three standard streams are opened implicitly:

- `FILE *stdin` : Standard input stream.
- `FILE *stdout` : Standard output stream.
- `FILE *stderr` : Standard error stream.

Composition via Pipelines

In UNIX systems, these standard streams can be redirected or piped together:

```
bash
cat hello.txt | grep "World!"
```

The standard output (`stdout`) of the `cat` process is connected directly to the standard input (`stdin`) of the `grep` process, allowing processes to be chained together.

7. Stream Operations & C Code Examples

7.1 High-Level Stream APIs

```
c

// Character-oriented operations
int fgetc(FILE *fp);           // Read a single character
int fputc(int c, FILE *fp);    // Write a single character
char *fgets(char *buf, int n, FILE *fp); // Read a line (up to n-1 bytes)
int fputs(const char *s, FILE *fp); // Write a null-terminated string

// Block-oriented operations
size_t fread(void *ptr, size_t size, size_t nobj, FILE *fp); // Read blocks of bytes
size_t fwrite(const void *ptr, size_t size, size_t nobj, FILE *fp); // Write blocks of bytes

// Formatted operations
int fprintf(FILE *stream, const char *format, ...); // Write formatted output
int fscanf(FILE *stream, const char *format, ...); // Read formatted input
```

7.2 Code Example: Character-by-Character Copy

This program copies the contents of `input.txt` to `output.txt` one character at a time.

```
c

#include <stdio.h>
#include <stdlib.h>

int main(void) {
    FILE* input = fopen("input.txt", "r"); // Open for reading
    FILE* output = fopen("output.txt", "w"); // Open for writing
    int c;

    if (input == NULL || output == NULL) {
        perror("Failed to open files");
        return 1;
    }

    c = fgetc(input); // Read first character
    while (c != EOF) { // Loop until End-Of-File
        fputc(c, output); // Write character to output
        c = fgetc(input); // Read next character
    }

    fclose(input); // Close input stream
    fclose(output); // Close output stream
    return 0;
}
```

7.3 Code Example: Block-by-Block Copy

This program performs the same copy operation but processes data in larger blocks to reduce overhead.

```
c
```

```

#include <stdio.h>
#include <stdlib.h>

#define BUFFER_SIZE 1024 //

int main(void) {
    FILE* input = fopen("input.txt", "r");    //
    FILE* output = fopen("output.txt", "w");  //
    char buffer[BUFFER_SIZE];                //
    size_t length;                           //

    if (input == NULL || output == NULL) {
        perror("Failed to open files");
        return 1;
    }

    // Read up to BUFFER_SIZE bytes into buffer
    length = fread(buffer, sizeof(char), BUFFER_SIZE, input); //
    while (length > 0) {                                     //
        // Write exactly the number of bytes read to output
        fwrite(buffer, sizeof(char), length, output);        //
        // Read next block
        length = fread(buffer, sizeof(char), BUFFER_SIZE, input); //
    }

    fclose(input);      //
    fclose(output);     //
    return 0;
}

```

7.4 Defensive Programming and Error Handling

Systems programmers must verify the return values of all I/O functions to prevent silent errors and bugs.

C

```

FILE* input = fopen("input.txt", "r"); //
if (input == NULL) {                  // Check if open failed
    // perror automatically prints your custom message followed by
    // the system error message corresponding to global 'errno'
    perror("Failed to open input file"); //
    exit(EXIT_FAILURE);
}

```

7.5 Stream Positioning (Seeking)

Every active stream tracks its current byte offset, which can be modified using seeking functions.

C

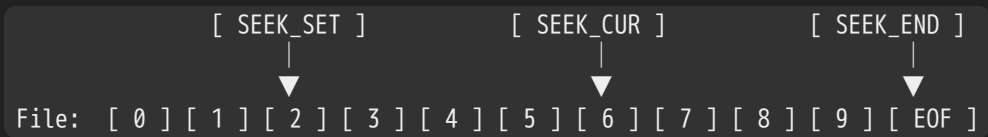
```

int fseek(FILE *stream, long int offset, int whence); // Reposition stream pointer
long int ftell(FILE *stream);                        // Get current byte offset
void rewind(FILE *stream);                           // Reset stream position to 0

```

The `whence` parameter specifies how `offset` is interpreted:

- `SEEK_SET` : Apply `offset` relative to the beginning of the file (position 0).
- `SEEK_CUR` : Apply `offset` relative to the current position.
- `SEEK_END` : Apply `offset` relative to the end of the file (seeking backward).



Visual interpretation of stream pointer seeking.

8. Low-Level RAW File I/O (The System Call Interface)

Unlike high-level streams, low-level I/O operates directly on raw integer **File Descriptors (FD)** and triggers a context switch into the kernel with every operation.

```
c
#include <fcntl.h>
#include <unistd.h>
#include <sys/types.h>

int open(const char *filename, int flags [, mode_t mode]); // Opens a file descriptor
int close(int filedес); // Closes a file descriptor
```

8.1 System Call Arguments

- **File Descriptor:** A non-negative integer returned by `open()` on success. On error, `open()` returns a negative value and sets the global variable `errno`.
- **Flags (Bit Vector):** A bitwise OR combination of access and creation modes:
 - Access Modes: `O_RDONLY` (Read only), `O_WRONLY` (Write only), or `O_RDWR` (Read & Write).
 - Creation Flags: `O_CREAT` (Create file if it does not exist), `O_TRUNC` (Truncate size to 0 if it exists).
 - Operating Modes: `O_APPEND` (All writes are appended to the end of the file).
- **Mode (Permissions Bit Vector):** Used with `O_CREAT` to define the new file's read, write, and execute permissions across User, Group, and Other identities (e.g., `S_IRUSR | S_IWUSR` for owner read/write permissions).

8.2 Standard File Descriptors

By convention, POSIX systems define three standard descriptors that are pre-opened for every process:

Standard Descriptor	Macro Definition	Raw Integer Value
Standard Input	<code>STDIN_FILENO</code>	0
Standard Output	<code>STDOUT_FILENO</code>	1
Standard Error	<code>STDERR_FILENO</code>	2

Converting Streams to Descriptors

```
c
```

```
int fileno(FILE *stream); // Retrieve underlying FD from FILE* stream
FILE *fdopen(int filedес, const char *opentype); // Wrap a raw file descriptor inside a FILE*
stream
```

8.3 Core Low-Level APIs (`read` , `write` , `lseek`)

C

```
ssize_t read(int filedес, void *buffer, size_t maxsize); // Read up to maxsize bytes
```

- **Behavior:** Reads up to `maxsize` bytes from `filedes` into `buffer` .
- **Return Values:** Returns the number of bytes read on success, `0` if it reached End-of-File (EOF), or `-1` on error.
- **Crucial Note:** `read()` may read fewer bytes than requested (a *short read*). Systems programmers must use a loop to ensure all expected data is read.

C

```
ssize_t write(int filedес, const void *buffer, size_t size); // Write size bytes
```

- **Behavior:** Writes `size` bytes from `buffer` to `filedes` .
- **Return Values:** Returns the number of bytes written, or `-1` on error.
- **Crucial Note:** Like `read()` , `write()` may write fewer bytes than requested (a *short write*), requiring a loop to send the remaining bytes.

C

```
off_t lseek(int filedес, off_t offset, int whence); // Reposition kernel offset
```

- **Behavior:** Changes the offset tracking position inside the kernel's file description. This position is completely independent of any high-level FILE stream buffering.

8.4 Code Example: Low-Level I/O (`lowio.c`)

This program demonstrates opening a file, reading its contents into a local buffer, and writing those bytes to standard output using raw system calls.

C

```
#include <fcntl.h>
#include <unistd.h>
#include <sys/types.h>
#include <stdio.h>

int main() {
    char buf[1000]; //

    // Open file using read-only mode and owner read/write permissions
    int fd = open("lowio.c", O_RDONLY, S_IRUSR | S_IWUSR); //
    if (fd < 0) {
        perror("Open failed");
        return 1;
    }
}
```

```

// Read up to 1000 bytes into buf
ssize_t rd = read(fd, buf, sizeof(buf)); //
if (rd < 0) {
    perror("Read failed");
    close(fd);
    return 1;
}

// Close the file descriptor
int err = close(fd); //

// Write the read bytes to standard output
ssize_t wr = write(STDOUT_FILENO, buf, rd); //

return 0;
}

```

9. POSIX I/O Design Patterns & Kernel Buffering

9.1 Core Design Patterns

- **Open Before Use:** Before a process can interact with a file, it must open it. During the `open()` call, the kernel performs access control checks and sets up internal tracking structures.
- **Byte-Oriented Interface:** The OS exposes files as simple byte streams. The kernel is responsible for hiding hardware-specific details, such as the fact that hard drives store and retrieve data in fixed-size blocks (e.g., 512 bytes or 4 KB).
- **Explicit Close:** Processes should close files when they are finished with them to free up file descriptors and kernel resources.

9.2 Kernel Buffering and Caching

Low-level read and write system calls do not directly access physical hardware; instead, they interact with a **Kernel Buffer Cache**.



- **Read Buffering:** When a process requests a read, the kernel reads larger blocks of data from the hardware device into kernel memory. Subsequent reads can then be served immediately from this cache. If a process requests data that is not cached, it blocks (yielding the CPU to let other processes run) while the kernel fetches the data from the hardware.
- **Write Buffering:** When an application calls `write()`, the kernel copies the data into its internal buffers and returns success immediately, before the data is written to physical disk. The kernel flushes these buffered writes to physical devices in the background, which improves application performance.
- **Disk Block Alignment:** The kernel manages caching and I/O operations in fixed-size units matching physical disk block sizes, hiding these details from user-space programs.

10. Advanced Low-Level File Descriptor Operations

POSIX systems provide several systems programming APIs to manipulate and duplicate file descriptors.

10.1 Descriptor Duplication (`dup` and `dup2`)

C

```
int dup(int oldfd);           // Duplicate oldfd, returning the lowest unused descriptor
int dup2(int oldfd, int newfd); // Duplicate oldfd to newfd, closing newfd first if open
```

These calls copy file descriptor entries within the process's descriptor table, allowing multiple descriptors to point to the same open file description in the kernel. This is commonly used to redirect standard output or standard error (e.g., redirecting `STDOUT_FILENO` to a file).

10.2 Pipes

A pipe is a unidirectional, local communication channel with a read end and a write end.

C

```
int pipe(int pipefd[2]); // Creates a pipe, returning two file descriptors
```

- `pipefd[1]` : The write end of the pipe.
- `pipefd[0]` : The read end of the pipe. Any data written to `pipefd[1]` can be read from `pipefd[0]`. This is the mechanism used by shells to connect pipeline processes.

10.3 File Locking

POSIX provides file locking mechanisms to prevent race conditions when multiple processes attempt to access the same file concurrently.

10.4 Memory-Mapped Files (`mmap`)

Memory mapping maps a file's contents directly into the process's virtual address space. This allows the application to read and write file data using standard pointer operations rather than system calls, improving I/O performance for large files.

11. High-Level vs. Low-Level I/O Comparison

An operating system uses different buffering strategies depending on whether an application uses high-level streams or low-level file descriptors.

11.1 Buffering and System Call Overhead

- **Low-Level Direct Access (`read` / `write`)**: Every single read or write call issues a system call and traps into the kernel, regardless of how few bytes are transferred. If a program reads a file one byte at a time using `read()`, it triggers a system call for each byte, creating massive execution overhead.
- **High-Level Buffered Access (`fread` / `fwrite`)**: The `FILE` structure allocated in user space contains an internal buffer (typically 1 KB to 8 KB).
 - The first call to `fread()` triggers a `read()` system call to fetch a large block of data (e.g., 1024 bytes) into this user-space buffer.
 - It returns the requested bytes to the application.

- Subsequent `fread()` calls are served directly from this user-space buffer without triggering system calls, only trapping into the kernel again when the buffer runs dry.

11.2 Comparing Library Internals

The internal implementation of these APIs illustrates how high-level calls avoid system call overhead:

```
c

// Low-Level direct read system call
ssize_t read(int fd, void *buf, size_t count) {
    // 1. Marshall fd, buf, and count arguments into CPU registers
    // 2. Write the sys_read call identifier into register EAX
    // 3. Execute trap/syscall instruction to enter kernel mode
    // 4. Kernel processes I/O, stores return status in register EAX
    // 5. Read returns immediately to the caller
}

// High-Level buffered read library call
size_t fread(void *ptr, size_t size, size_t nmemb, FILE *stream) {
    // 1. Check if the requested data is already present in user-space buffer
    // 2. If present, copy data to ptr and return immediately
    // 3. If empty, execute trap/syscall to read a large block of data from file
    // 4. Copy the requested bytes to ptr
    // 5. Store any excess read data in the user-space buffer for future calls
}
```

11.3 Code Example: Output Flushing Difference

This example demonstrates how user-space buffering can delay output compared to raw system calls.

```
c

// High-Level Stream Example (Buffered)
printf("Beginning of line "); // Data is stored in user-space buffer
sleep(10);                    // Program sleeps for 10 seconds
printf("and end of line\n"); // '\n' triggers a flush of the entire buffer
// Result: Both strings are printed together after 10 seconds.

// Low-Level FD Example (Unbuffered)
write(STDOUT_FILENO, "Beginning of line ", 18); // Writes directly to standard output
sleep(10);                                     // Program sleeps for 10 seconds
write(STDOUT_FILENO, "and end of line\n", 16); // Writes next string
// Result: "Beginning of line " is printed immediately, and "and end of line"
// is printed 10 seconds later.
```

12. Under the Hood: File Descriptors, File Descriptions, and Inodes

To prevent security vulnerabilities and allow file sharing, the operating system kernel maps file descriptors to physical files using a three-level translation structure.

12.1 Why Use File Descriptors?

Why does the kernel return an integer index to the process instead of a raw pointer to the file structure in kernel memory?

- **Security & Isolation:** A raw memory pointer would allow a buggy or malicious user program to modify kernel state directly.

- **Pointer Manipulation Prevention:** Restricting user space to integer indices ensures that the kernel retains complete control over all file access checks.

12.2 The Three-Tier Translation Structure

1. **File Descriptor Number (Per-Process):** An integer that indexes into the process's private **File Descriptor Table**. Each entry contains a pointer to an entry in the system-wide open file description table.
2. **Open File Description Object (System-Wide):** A kernel structure representing an active file open operation. It tracks:
 - The current **byte offset** for this open instance.
 - The access permissions and status flags used during `open()` .
 - A reference count tracking how many file descriptors in active processes point to this description.
3. **File System Inode/Vnode Object (System-Wide):** A structure that represents the actual physical file. It tracks:
 - The file's physical location (mapping logical offsets to sectors on disks or SSDs).
 - The file's metadata (size, owner, permissions).

13. System Call Dispatch Reference

The system call interface functions as a synchronous exception vector. The table below lists some core Linux system calls, their system call numbers, arguments, and source code definitions:

Syscall Name	System Call ID (EAX)	Parameter 1 (EBX)	Parameter 2 (ECX)	Parameter 3 (EDX)	Source Definition
sys_restart_syscall	0x00	—	—	—	kernel/signal.c
sys_exit	0x01	int error_code	—	—	kernel/exitt.c
sys_read	0x03	unsigned int fd	char __user *buf	size_t count	fs/read_write.c
sys_write	0x04	unsigned int fd	const char __user *buf	size_t count	fs/read_write.c
sys_open	0x05	const char __user *filename	int flags	int mode	fs/open.c
sys_close	0x06	unsigned int fd	—	—	fs/open.c
sys_waitpid	0x07	pid_t pid	int __user *stat_addr	int options	kernel/exitt.c
sys_creat	0x08	const char __user *pathname	int mode	—	fs/open.c
sys_link	0x09	const char __user *oldname	const char __user *newname	—	fs/namei.c

Low-level library wrappers place these arguments into the corresponding registers and issue a syscall instruction to trap into the kernel.

14. Practice Exam Problems with Detailed Solutions

Use these practice problems to test your understanding of the concepts covered in Lecture 5.

Question 1: Stream vs. File Descriptor Performance Comparison

An application needs to read a 1 MB file sequentially.

1. If the application reads the file 4 bytes at a time using raw `read()` system calls, how many times does it trap into kernel mode?
2. If the application instead uses `fread()` with an internal 1 KB stream buffer, how many times does it trap into kernel mode?

Solution:

1. Using `read()` :

- Every single call to `read()` triggers a system call.
- File Size = 1 MB = 1,048,576 bytes.
- Traps = $\frac{1,048,576 \text{ bytes}}{4 \text{ bytes/call}} = \mathbf{262,144}$ traps.

2. Using `fread()` with a 1 KB buffer:

- The first call to `fread()` triggers a system call to read a full 1 KB block (1024 bytes) into the user-space stream buffer.
- Subsequent `fread()` calls are served from the user-space buffer without triggering system calls.
- The application only traps into the kernel when the 1 KB buffer runs dry.
- Traps = $\frac{1,048,576 \text{ bytes}}{1024 \text{ bytes/trap}} = \mathbf{1024}$ traps.

Question 2: Redirection and Duplication

Explain how a shell redirects standard output (`stdout`) to a file `log.txt` during a `fork()` - `exec()` sequence using system calls.

Solution:

The shell uses the following sequence to redirect standard output:

1. **Fork:** The shell calls `fork()` to create a child process. The child process inherits copies of the parent's file descriptors, including standard output (`STDOUT_FILENO` = 1) pointing to the terminal.
2. **Open:** In the child process, the shell opens `log.txt` in write mode:

```
c
int fd = open("log.txt", O_WRONLY | O_CREAT | O_TRUNC, S_IRUSR | S_IWUSR);
```

3. **Redirect:** The child process calls `dup2()` to duplicate the open file descriptor `fd` to standard output `1`, which closes the terminal reference first:

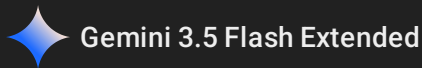
```
c
dup2(fd, STDOUT_FILENO); // STDOUT_FILENO (1) now points to log.txt
```

4. **Close:** The child process closes the original temporary descriptor `fd` :

```
c
```

```
close(fd);
```

5. **Exec:** The child process calls `exec()` to run the target application. The new program inherits the open file descriptors, and any data it writes to standard output is written directly to `log.txt`.



CS162 Operating Systems & Systems Programming

Lecture 6: Abstractions 4: Sockets, I/O, IPC (Finished)

Module 1: High-Level vs. Low-Level I/O Abstractions & Kernel Buffering

Operating systems bridge the gap between user-level application logic and physical hardware devices. This is managed through a layered system architecture, isolating applications from hardware details.

1.1 Core API Execution Models

Systems programmers distinguish between the direct, raw system call interface and standard user-space buffering libraries.

Low-Level File API (`read` , `write` , `open` , `close`)

- **Interface Mechanics:** Operates directly on raw integer **File Descriptors (FD)**.
- **Kernel Ownership:** File descriptors are integer indices into a per-process **File Descriptor Table** (also called a Process Descriptor Table). This table contains pointers to a system-wide kernel-level structure called the **Open File Description**.
- **Performance Profile:** Every single low-level read or write call forces a system call trap into the kernel, regardless of the size of the request (even a single byte). If an application reads a file 4 bytes at a time in a loop using `read()`, each iteration incurs the performance overhead of a full kernel context switch.

High-Level File API (`fopen` , `fread` , `fwrite` , `fclose`)

- **Interface Mechanics:** Exposes a pointer to a **FILE structure** allocated in user space.
- **User-Space Buffering:** The `FILE` structure contains an internal, user-space chunk of memory acting as a buffer. It also wraps the underlying low-level raw integer file descriptor.
- **Performance Profile:** Library functions filter requests through this user-space buffer. For example, a loop reading 4 bytes at a time via `fread()` will call the underlying `read()` system call on its first iteration to fetch a much larger block (e.g., 1024 bytes) from the kernel. This data is loaded into the user-space buffer, and the first 4 bytes are returned to the user. Subsequent `fread()` iterations pull data directly from this buffer without invoking any system calls, only trapping into the kernel again once the buffer runs dry.

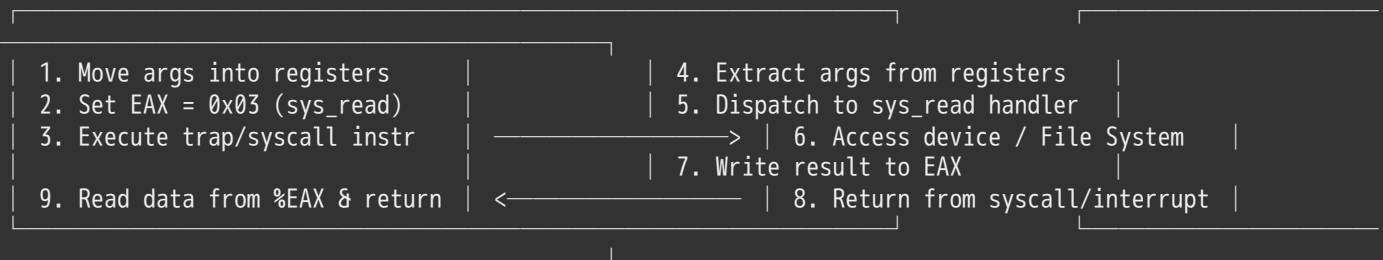
1.2 Under the Hood: Low-Level vs. High-Level API Calls

The following assembly-level and kernel-transition walkthrough highlights the path of both calls:

Low-Level API: `read(fd, buf, 4)`

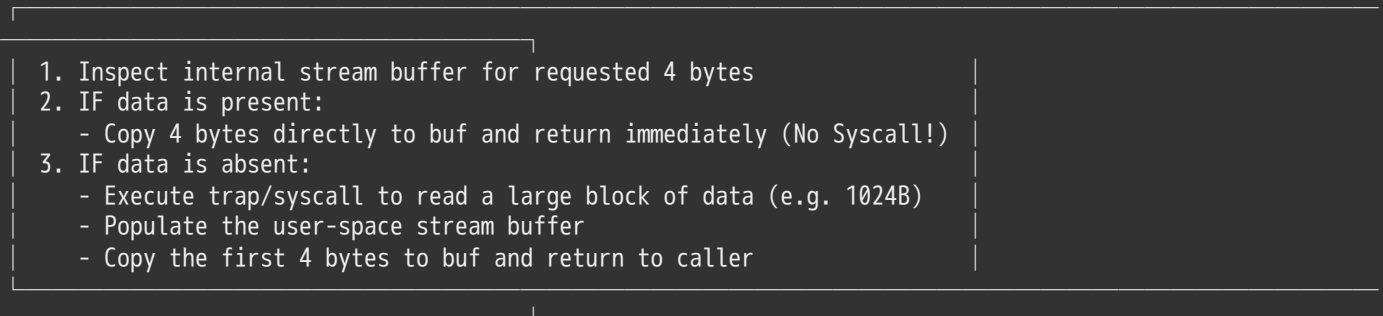
User Space

Kernel Space



High-Level API: `fread(buf, 4, 1, stream)`

User Space



Module 2: Kernel Internals: Open File Descriptions and `vfs_read` Code Analysis

Every active process maintains an isolated view of files, which is translated by the kernel into concrete data layout records.

2.1 The Open File Description Structure

Within the kernel (as defined in `linux/fs.h`), each open file is represented by a `struct file` object. While this structure contains many technical parameters, its primary roles are tracking:

1. **Physical Location:** Where to find the file data on disk (via the `f_path` and `f_inode` parameters).
2. **File Pointer Position:** The current read/write offset position within the file (`f_pos`).

Key structural elements of `struct file` in the Linux kernel include:

- `f_path`: The file path representation.
- `f_inode`: Pointer to the unique kernel inode structure containing physical metadata.
- `f_op`: Pointer to a table of device-specific file operations (`struct file_operations`).
- `f_count`: The reference count of file descriptors pointing to this description.
- `f_flags`: Access and open status flags (e.g., `O_RDONLY`, `O_CREAT`, `O_NONBLOCK`).
- `f_mode`: The open mode permissions (e.g., `FMODE_READ`, `FMODE_WRITE`).
- `f_pos`: The current byte offset for read/write tracking.

2.2 Deep Dive: Kernel Source Code Walkthrough of `vfs_read`

The `vfs_read` function inside `fs/read_write.c` is the kernel's entry point for handling read operations. It validates arguments, checks permissions, and dispatches requests to device-specific file system drivers.

```
c
ssize_t vfs_read(struct file *file, char __user *buf, size_t count, loff_t *pos)
{
    ssize_t ret;
```

```

// 1. Validate that the file description was opened with read permissions
if (!(file->f_mode & FMODE_READ))
    return -EBADF; // Bad File Descriptor error

// 2. Validate that the file operations and read function pointers are populated
if (!file->f_op || (!file->f_op->read && !file->f_op->aio_read))
    return -EINVAL; // Invalid argument error

// 3. Verify that the destination user-space buffer is writable by the user process
// unlikely() is a compiler optimization hint indicating this path is rarely taken
if (unlikely(!access_ok(VERIFY_WRITE, buf, count)))
    return -EFAULT; // Bad address fault

// 4. Verify that the requested file range is valid and within security boundaries
ret = rw_verify_area(READ, file, pos, count); //
if (ret >= 0) {
    count = ret;

    // 5. Polymorphic Dispatch: If the driver provides a read method, use it;
    // otherwise, fall back to do_sync_read()
    if (file->f_op->read)
        ret = file->f_op->read(file, buf, count, pos); //
    else
        ret = do_sync_read(file, buf, count, pos); //

    // 6. Post-execution bookkeeping and event notifications
    if (ret > 0) {
        // Notify parent directory that this file was accessed (for monitoring)
        fsnotify_access(file->f_path.dentry); //

        // Update accounting metrics: bytes read by the current task
        add_rchar(current, ret); //
    }

    // Update accounting metrics: count of total read system calls by current task
    inc_syscr(current); //
}

return ret; // Return actual bytes read or negative error code
}

```

Module 3: Device Drivers: Top Half vs. Bottom Half and I/O Lifecycles

3.1 Device Driver Concept

A **Device Driver** is device-specific code in the kernel that interacts directly with physical device controllers. Drivers register with the kernel to expose a standard, uniform interface to the virtual file system, allowing the same generic kernel I/O system to interact with completely different physical hardware. Special device configurations are handled via the polymorphic `ioctl()` system call.

3.2 Split-Half Design

To keep system calls responsive and handle asynchronous hardware, modern device drivers are split into two halves:

TOP HALF (Synchronous)

- Runs in the execution path of system calls (process context).
- Implements standard file operations (open, read, write, ioctl).
- Initiates device operations and can put the calling thread to sleep.

Asynchronous Hardware I/O



BOTTOM HALF (Asynchronous)

- Runs as an interrupt service routine (ISR) in interrupt context.
- Cannot block, sleep, or wait on resources.
- Handles incoming data blocks or output completions from hardware.
- Wakes up sleeping threads waiting in the top half once I/O is done.

3.3 The File Operations Interface (struct file_operations)

Device driver methods are registered with the kernel using the `file_operations` struct:

C

```
struct file_operations {
    struct module *owner;
    loff_t (*llseek) (struct file *, loff_t, int);
    ssize_t (*read) (struct file *, char __user *, size_t, loff_t *); //
    ssize_t (*write) (struct file *, const char __user *, size_t, loff_t *); //
    ssize_t (*aio_read) (struct kiocb *, const struct iovec *, unsigned long, loff_t); //
    ssize_t (*aio_write) (struct kiocb *, const struct iovec *, unsigned long, loff_t); //
    int (*readdir) (struct file *, void *, filldir_t); //
    unsigned int (*poll) (struct file *, struct poll_table_struct *); //
    int (*ioctl) (struct inode *, struct file *, unsigned int, unsigned long); //
    int (*mmap) (struct file *, struct vm_area_struct *); //
    int (*open) (struct inode *, struct file *); //
    int (*flush) (struct file *, fl_owner_t id); //
    int (*release) (struct inode *, struct file *); //
    int (*fsync) (struct file *, struct dentry *, int datasync); //
    int (*fasync) (int, struct file *, int); //
    int (*flock) (struct file *, int, struct file_lock *); //
};
```

3.4 Lifecycle of an I/O Request

When a user program reads or writes to a device, the request transitions through several layers of the operating system.

The lifecycle of an I/O request follows this path:

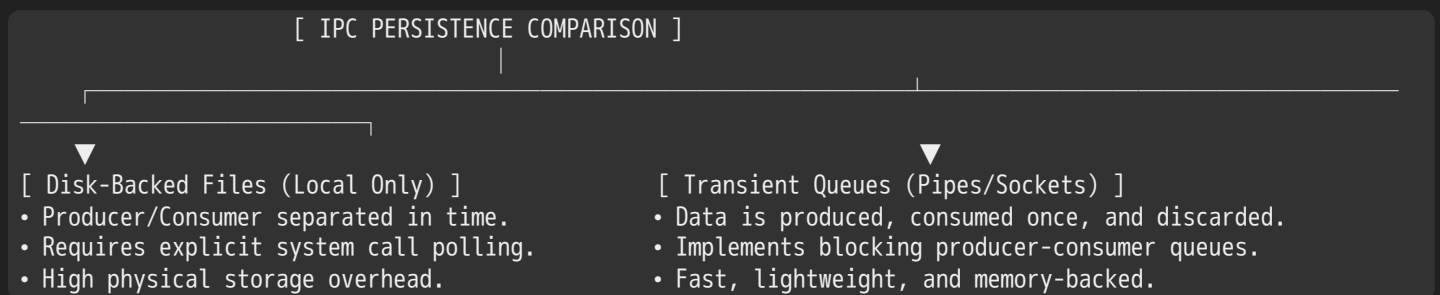
1. **Initiation:** The user program invokes a standard system call (such as `read()`), trapping into the **Kernel I/O Subsystem**.
2. **Subsystem Verification:** The kernel checks if the request can be satisfied immediately (e.g., using data cached in kernel memory).
 - *If Yes:* The data is copied to the process, and the system call returns immediately.
 - *If No:* The request is forwarded to the **Device Driver Top Half**.

3. **Driver Initiation:** The driver's top half translates the request into device-controller commands and configures the hardware controller.
4. **Process Suspension:** If the operation is blocking, the driver puts the calling thread to sleep to yield the CPU.
5. **Hardware Monitoring:** The physical **Device Controller** monitors the hardware and executes the read/write operation.
6. **Hardware Completion:** Once the hardware operation completes, the device controller asserts a physical interrupt signal line to the CPU.
7. **Interrupt Service (Bottom Half):** The **Device Driver Bottom Half** handles the interrupt. It retrieves the data from the device, stores it in a kernel-space buffer, and signals to unblock the sleeping process.
8. **Top Half Return:** The driver's top half wakes up, determines that the operation completed successfully, and passes status info to the I/O subsystem.
9. **Finalization:** The subsystem copies the data from kernel-space buffers into the user program's memory and returns control to the user process.

Module 4: Inter-Process Communication (IPC) & The Socket Abstraction

4.1 Process Isolation vs. Communication

By default, processes run inside isolated, protected virtual address spaces. While this protection prevents bugs or crashes in one application from affecting other programs, it also means that communication between processes must be configured explicitly through operating system channels.



4.2 The Socket Interface

A **Socket** is a software abstraction representing an endpoint of a network or local connection.

- **Unified Abstraction:** Sockets allow communication across the network to look like standard file I/O. Sockets are represented as file descriptors, allowing applications to use standard read and write system calls.
- **Historical Origins:** Standardized by POSIX, the socket interface was originally developed and introduced in **4.2 BSD (Berkeley Standard Distribution) UNIX**.
- **Network Independence:** Sockets expose the same abstraction regardless of the underlying network layer, whether local (Unix Domain Sockets) or external (TCP/IP, UDP/IP).
- **Queue Abstraction:** Sockets model bidirectional communication using two queues:
 - `write()` adds data to the local output queue (destined for the peer).
 - `read()` removes data from the local input queue (sent by the peer).
- **VFS Differences:** Unlike files, sockets do not support positioning operations; calling `lseek()` on a socket descriptor is invalid and returns an error.

Module 5: Sockets Deep-Dive: Namespaces, 5-Tuples, and Connection Setup

5.1 Communication Abstractions Comparison

Operating systems provide several abstractions for I/O and communication:

Dimension	Files	Pipes	Sockets
Lifetime	Permanent, exist independently of process lifecycles.	Transient, cleaned up when the process terminates.	Transient, cleaned up when closed.
Name Space	Structured Directory Tree.	No formal namespace; represented as anonymous descriptors.	IP addresses and Port numbers.
Setup Mechanics	Opened via path strings.	Created via <code>pipe()</code> ; descriptors are inherited by child processes.	Configured via hostnames, IP addresses, and ports.
Topology	Many-to-Many shared storage.	One-way communication queues spanning related processes.	Bidirectional communication queues spanning arbitrary machines.

5.2 IP Namespaces

To establish a network connection, applications must identify the target machine and service using the IP namespace:

- **Hostname:** Human-readable domain name (e.g., `www.eecs.berkeley.edu`).
- **IP Address:** Unique network identifier:
 - *IPv4:* 32-bit integer (e.g., `128.32.244.172`).
 - *IPv6:* 128-bit integer (e.g., `2607:f140:0:81::f`).
- **Port Numbers:** 16-bit integers (0 to 65535) that route network packets to specific application processes:
 - *System / Well-Known Ports (0-1023):* Reserved for standard system services (e.g., HTTP is port 80, HTTPS is port 443). Binding to these ports requires superuser privileges.
 - *Registered Ports (1024-49151):* Assigned by IANA for specific user-space services.
 - *Dynamic / Private / Ephemeral Ports (49152-65535):* Automatically allocated by the operating system for client connections.

5.3 TCP/IP Connections and the 5-Tuple

On the Internet, every active TCP connection is uniquely identified by a **5-Tuple**:

{Source IP, Destination IP, Source Port, Destination Port, Protocol}

This 5-Tuple allows a server to multiplex multiple client connections on the same port. For example, a web server running on port 80 can handle thousands of simultaneous client connections because each connection has a unique client IP or ephemeral port.

5.4 Server Socket Mechanics

To accept incoming connections, servers use a specialized **Server Socket**:

- **Interface Restrictions:** Although represented as a file descriptor, a server socket cannot be used to read or write data directly.
- **Core Operations:**
 1. `listen()` : Configures the server socket to listen for incoming connection requests.
 2. `accept()` : Pops an incoming request off the connection queue, creates a new connection socket for that client, and returns its file descriptor.

Module 6: Socket Programming in C

6.1 Complete Code Example: The Echo Client

This program establishes a connection to a remote server, prompts the user for text, sends it via a TCP socket, and prints the echoed response.

```
c

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/socket.h>
#include <netdb.h>

#define MAXIN 1024
#define MAXOUT 1024

// Resolve hostnames to IP addresses
struct addrinfo *lookup_host(char *host_name, char *port_name) {
    struct addrinfo *server;
    struct addrinfo hints;

    memset(&hints, 0, sizeof(hints));
    hints.ai_family = AF_UNSPEC;    // Support both IPv4 and IPv6
    hints.ai_socktype = SOCK_STREAM; // Establish TCP byte streams

    int rv = getaddrinfo(host_name, port_name, &hints, &server); //
    if (rv != 0) {
        fprintf(stderr, "getaddrinfo failed: %s\n", gai_strerror(rv)); //
        return NULL;
    }
    return server;
}

void run_client(int sockfd) {
    char sndbuf[MAXIN];
    char rcvbuf[MAXOUT];
    int n;

    while (1) {
        printf("Enter string: ");
        if (fgets(sndbuf, MAXIN, stdin) == NULL) break; //

        // Send string (including null terminator) to server
        write(sockfd, sndbuf, strlen(sndbuf) + 1); //

        memset(rcvbuf, 0, MAXOUT); //

        // Block until echoed data is received
        n = read(sockfd, rcvbuf, MAXOUT); //
        if (n <= 0) {
            printf("Connection closed by server.\n");
        }
    }
}
```

```

        break;
    }

    // Print response to standard output
    write(STDOUT_FILENO, rcvbuf, n); //
}
}

int main(int argc, char *argv[]) {
    if (argc < 3) {
        fprintf(stderr, "Usage: %s <host> <port>\n", argv[0]);
        return 1;
    }

    char *host_name = argv[1];
    char *port_name = argv[2];

    struct addrinfo *server = lookup_host(host_name, port_name); //
    if (server == NULL) return 1;

    // 1. Create client socket descriptor
    int sock_fd = socket(server->ai_family, server->ai_socktype, server->ai_protocol); //
    if (sock_fd < 0) {
        perror("Socket creation failed");
        freeaddrinfo(server);
        return 1;
    }

    // 2. Connect to server
    if (connect(sock_fd, server->ai_addr, server->ai_addrlen) < 0) { //
        perror("Connect failed");
        close(sock_fd);
        freeaddrinfo(server);
        return 1;
    }

    run_client(sock_fd); // Execute application logic

    close(sock_fd); //
    freeaddrinfo(server);
    return 0;
}

```

6.2 Stream Framing Considerations

Because TCP is stream-oriented, it does not preserve application message boundaries.

If a client makes two sequential calls to `write()` to send strings "A" and "B", the operating system may package them into a single TCP segment. As a result, a single call to `read()` on the server side might retrieve both strings "AB" at once, or retrieve a partial chunk of "A".

To handle this, application protocols must implement **framing**:

- **Delimiter-based:** Scan incoming bytes sequentially for a terminator character (such as a null byte `\0` or a newline `\n`) to identify message boundaries.

- **Length-prefixed:** Prefix each message with its byte length, allowing the reader to read the exact length before parsing the message.

Module 7: Concurrent Server Architectures

To handle multiple clients concurrently, servers use different execution models to keep blocking operations on one connection from stalling others.

7.1 Web Server: Iterative Model (Single Process, No Protection)

This model handles connections sequentially.

```
c

//
struct addrinfo *setup_address(char *port) {
    struct addrinfo *server;
    struct addrinfo hints;
    memset(&hints, 0, sizeof(hints));
    hints.ai_family = AF_UNSPEC;
    hints.ai_socktype = SOCK_STREAM;
    hints.ai_flags = AI_PASSIVE; // Accept connections on any local IP address

    getaddrinfo(NULL, port, &hints, &server); //
    return server;
}

int main(int argc, char *argv[]) {
    char *port_name = "8080";
    struct addrinfo *server = setup_address(port_name); //

    int server_socket = socket(server->ai_family, server->ai_socktype, server->ai_protocol); //
    bind(server_socket, server->ai_addr, server->ai_addrlen); //
    listen(server_socket, 128); //

    while (1) {
        // Block until a client connects
        int conn_socket = accept(server_socket, NULL, NULL); //

        // Serve client sequentially; blocks other incoming requests
        serve_client(conn_socket); //
        close(conn_socket); //
    }
    close(server_socket);
    return 0;
}
```

7.2 Web Server: Multi-Process Model (With Protection, No Concurrency)

This model sandboxes client requests by handling each connection in a separate process. However, calling a blocking `wait()` in the main loop means the server still handles requests sequentially.

```
c
```

```

while (1) {
    int conn_socket = accept(server_socket, NULL, NULL); //
    pid_t pid = fork(); // Duplicate parent process

    if (pid == 0) {
        /* Child Process Context */
        close(server_socket); // Close the inherited listening socket
        serve_client(conn_socket); // Service client request in sandbox
        close(conn_socket); // Close child's reference to connection socket
        exit(0); // Terminate child process
    } else {
        /* Parent Process Context */
        close(conn_socket); // Close parent's reference to connection socket
        wait(NULL); // Blocks accept loop until child terminates (Iterative
execution)
    }
}

```

7.3 Web Server: Concurrent Multi-Process Model (With Protection & Concurrency)

To handle clients concurrently, the parent process skips the blocking `wait()` inside the loop. This allows the parent to immediately accept the next client while the child handles the previous one.

```

c

while (1) {
    int conn_socket = accept(server_socket, NULL, NULL); //
    pid_t pid = fork(); //

    if (pid == 0) {
        /* Child Process Context */
        close(server_socket); //
        serve_client(conn_socket); //
        close(conn_socket); //
        exit(0); //
    } else {
        /* Parent Process Context */
        close(conn_socket); //
        // wait(NULL); is commented out to prevent blocking the accept loop
    }
}

```

Note: To clean up child processes without blocking, the parent must register a non-blocking `SIGCHLD` signal handler to call `waitpid(-1, NULL, WNOHANG)` whenever a child terminates.

7.4 Web Server: Concurrent Multi-Threaded Model (No Protection, Concurrency)

Spawning processes incurs significant memory and CPU overhead. To improve performance, the server can handle connections in separate threads using `pthread_create()`.

```

c

void *thread_worker(void *arg) {
    int conn_socket = (int)(long)arg;
    pthread_detach(pthread_self()); // Automatically reclaim resources on thread exit
}

```

```

    serve_client(conn_socket);
    close(conn_socket);
    pthread_exit(NULL);
}

int main(void) {
    // Socket initialization elided...
    while (1) {
        int conn_socket = accept(server_socket, NULL, NULL);
        pthread_t tid;
        // Spawn thread to handle client; main loop returns immediately to accept
        pthread_create(&tid, NULL, thread_worker, (void *) (long)conn_socket); //
    }
}

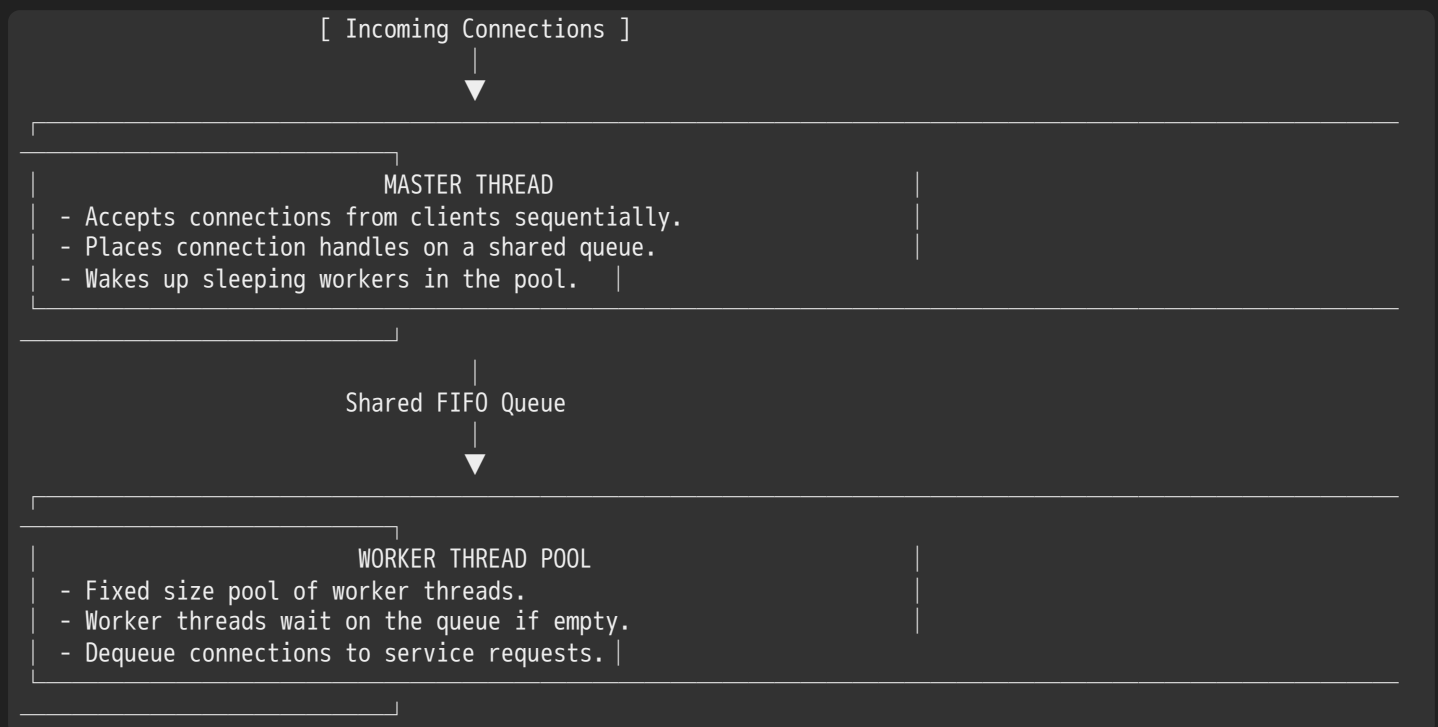
```

- **Performance Profile:** Fast connection handling with low memory overhead.
- **Security Risk:** No protection isolation. A bug or crash in one client handler thread can corrupt shared memory or crash the entire server process.

7.5 Bounded Concurrency: The Thread Pool Model

While the concurrent multi-threaded model is fast, spawning unbounded threads under high traffic can overwhelm the system, causing memory exhaustion and thread thrashing.

To prevent this, servers use the **Thread Pool** pattern, allocating a fixed number of worker threads to handle connections.



Thread Pool Execution Algorithm

```

c
// Master Thread Algorithm
master() {
    allocThreads(worker, queue); // Instantiate bounded worker pool
    while (TRUE) {
        con = AcceptCon();        // Wait for client connection
    }
}

```

```

        Enqueue(queue, con);    // Place connection handle on shared queue
        wakeUp(queue);         // Signal sleeping worker threads
    }
}

// Worker Thread Algorithm
worker(queue) {
    while (TRUE) {
        con = Dequeue(queue);    // Attempt to pop connection off queue
        if (con == null)
            sleepOn(queue);      // Block if queue is empty
        else
            ServiceWebPage(con); // Process I/O request
    }
}

```

Module 8: Comprehensive Practice Problems with Detailed Solutions

Use these practice problems to test your understanding of Lecture 6.

Problem 1: File Descriptor Duplication and Reference Counting

A server process executes the following code block:

```

c

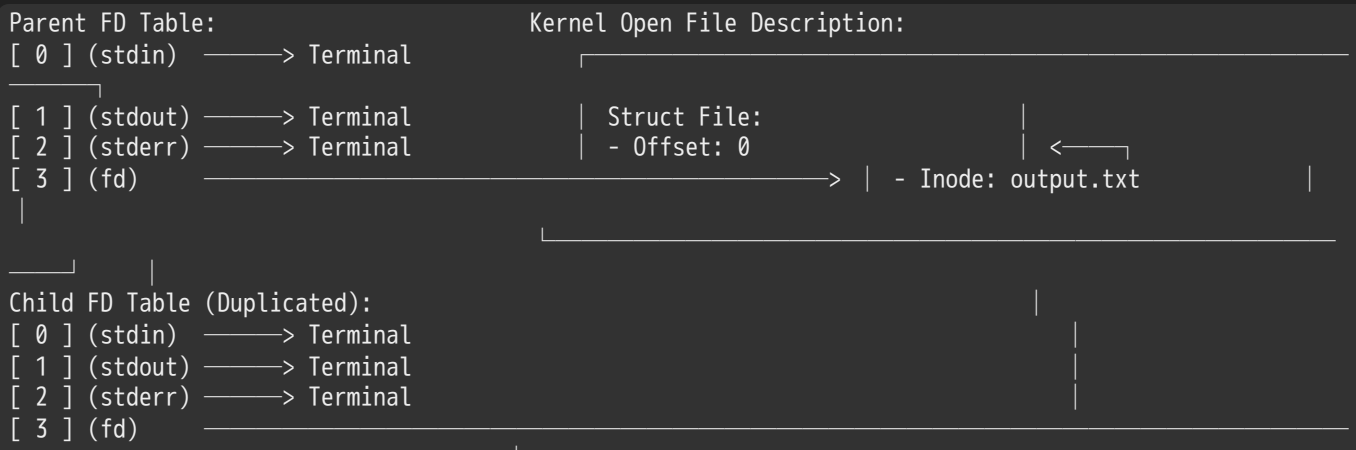
int fd = open("output.txt", O_RDWR); // Returns File Descriptor index 3
pid_t pid = fork();
if (pid == 0) {
    dup2(fd, STDOUT_FILENO); // Duplicate fd to stdout (FD index 1)
    close(fd);
    exit(0);
} else {
    close(fd);
    wait(NULL);
}

```

1. Sketch the file descriptor tables for both the parent and child processes immediately after the `fork()` call, showing how they map to kernel structures.
2. Calculate the reference count (`f_count`) of the kernel's **Open File Description** at each of the following points in execution:
 - Immediately before the `fork()` call.
 - Immediately after the `fork()` call, but before `dup2()` .
 - After the child process executes `dup2()` and `close(fd)` .
 - After the child process terminates and the parent executes `close(fd)` .

Solution:

1. **File Descriptor Table Mapping:** Immediately after the `fork()` call, both processes have descriptor tables mapping files to the same shared Open File Description in the kernel:



2. Reference Count (`f_count`) Calculations:

- **Before `fork()`** : Only the parent process has descriptor `3` pointing to the file description. `f_count = 1`
- **After `fork()`** : The child process duplicates the parent's descriptor table, so both the parent's FD `3` and the child's FD `3` point to the same file description. `f_count = 2`
- **After child `dup2()` and `close()`** :
 - `dup2(3, 1)` duplicates FD `3` to FD `1` inside the child process. Both FD `3` and FD `1` in the child now point to the description, incrementing `f_count` to 3.
 - `close(fd)` (equivalent to `close(3)`) removes the child's FD `3` reference, decrementing `f_count` to 2.
 - The file description is still referenced by FD `1` in the child and FD `3` in the parent. `f_count = 2`
- **After child exit and parent `close()`** :
 - When the child process terminates, the operating system automatically closes its remaining open descriptors, removing FD `1`'s reference and decrementing `f_count` to 1.
 - When the parent process executes `close(fd)` (equivalent to `close(3)`), the parent's reference is removed.
 - With `f_count` reaching 0, the kernel deallocates the Open File Description and closes the file. `f_count = 0`

Problem 2: Sockets and 5-Tuple Demultiplexing

A web server is bound to wildcard IP address `0.0.0.0` on port `80`. The server currently has three active client connections.

The active sockets are identified by the following parameters:

- **Server Listening Socket:** File Descriptor `3` (Bound to `*:80`).
- **Connection Sockets:**
 - Socket A: File Descriptor `4`
 - Socket B: File Descriptor `5`
 - Socket C: File Descriptor `6`

The server's network card receives three TCP segments with the following headers:

1. Segment 1: [Src: 198.51.100.12:45001, Dst: 128.32.10.5:80, Protocol: TCP]
2. Segment 2: [Src: 198.51.100.12:45002, Dst: 128.32.10.5:80, Protocol: TCP]
3. Segment 3: [Src: 203.0.113.84:53211, Dst: 128.32.10.5:80, Protocol: TCP]

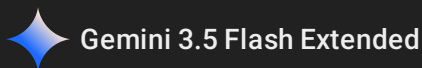
The kernel's socket table maps connections to file descriptors as follows:

```
{198.51.100.12:45001, 128.32.10.5:80, TCP}  ———> File Descriptor 4
{198.51.100.12:45002, 128.32.10.5:80, TCP}  ———> File Descriptor 5
{203.0.113.84:53211, 128.32.10.5:80, TCP}  ———> File Descriptor 6
```

1. Route each of the incoming segments to its corresponding socket descriptor.
2. If a new client sends a connection request (`SYN` packet) with headers `[Src: 192.0.2.55:36524, Dst: 128.32.10.5:80, Protocol: TCP]` , which file descriptor receives this request? What system call must the server execute to retrieve the new connection socket?

Solution:

1. **Routing Inbound Segments:** The kernel maps incoming segments to the appropriate file descriptors using their 5-Tuples:
 - **Segment 1** matches the 5-tuple for Socket A: routes to **File Descriptor 4** .
 - **Segment 2** matches the 5-tuple for Socket B: routes to **File Descriptor 5** .
 - **Segment 3** matches the 5-tuple for Socket C: routes to **File Descriptor 6** .
2. **Handling New Connection Requests:**
 - **Target File Descriptor:** The `SYN` connection request is routed to the server's listening socket, **File Descriptor 3** .
 - **System Call:** To accept the connection, retrieve the client details, and generate a new connection socket, the server process must execute the `accept()` system call.



CS162 Operating Systems & Systems Programming

Lecture 7: Synchronization 1: Concurrency, Mutual Exclusion, and Atomic Operations (Detailed Exam Notes)

Section 1: Architectural Paradigms of Network Servers

The design of a network server illustrates the classic progression of operating system abstractions, transitioning from isolated multi-process protection to lightweight, shared-memory multi-threading.

1.1 Multi-Process Server with Strict Protection (Iterative)

A baseline multi-process server can isolate client connections into independent address spaces using the `fork()` system call.

```
c

// Pre-sockets setup code executed by parent process
listen(server_socket, MAX_QUEUE); //

while (1) {
    // Accept a new incoming client connection, allocating a new socket descriptor
    int conn_socket = accept(server_socket, NULL, NULL); //

    pid_t pid = fork(); // Duplicate the parent process
```



```

if (pid == 0) {
    /* Child Process Context */
    close(server_socket); // Close the inherited listening socket to preserve boundaries
    serve_client(conn_socket); // Handle client request within a sandboxed address space
    close(conn_socket); // Release the client connection socket
    exit(0); // Terminate the child process safely
} else {
    /* Parent Process Context */
    close(conn_socket); // Release the parent's copy of the client socket
    wait(NULL); // Block parent until the child process finishes execution
}
}
close(server_socket); //

```

Operational Bottlenecks

This design follows the standard command shell pattern. Because the parent process invokes a blocking `wait(NULL)` inside the primary loop, it cannot return to the `accept()` system call until the current client finishes and the child process terminates. While the network stack can queue incoming requests up to `MAX_QUEUE`, this server behaves **iteratively** and cannot handle multiple clients concurrently.

1.2 Multi-Process Concurrent Server

To convert the iterative server into a concurrent one, the parent process skips the blocking `wait(NULL)` call. This allows the parent to immediately loop back and accept the next client while the child handles the previous one.

```

c

// Concurrent Multi-Process Server Loop
while (1) {
    int conn_socket = accept(server_socket, NULL, NULL); //
    pid_t pid = fork(); //

    if (pid == 0) {
        /* Child Process Context */
        close(server_socket); //
        serve_client(conn_socket); //
        close(conn_socket); //
        exit(0); //
    } else {
        /* Parent Process Context */
        close(conn_socket); // Parent returns immediately to accept()
        // wait(NULL); Omitted to achieve execution concurrency
    }
}
}

```

Architectural Trade-offs

- **Advantage:** Provides fault containment and protection. If a bug or crash occurs while servicing a client, the failure is isolated to that child's private address space, leaving the primary server process unaffected.
- **Disadvantage:** High system overhead. Allocating a full process requires duplicating page tables, tracking data structures, and memory maps. Furthermore, these independent processes must be careful not to introduce data races when interacting through shared resources like the global file system.

1.3 Lightweight Concurrent Server (Multi-Threaded)

To avoid process creation overhead, modern high-performance servers handle each connection in a separate thread within the same process address space using POSIX threads (`pthread_create`).

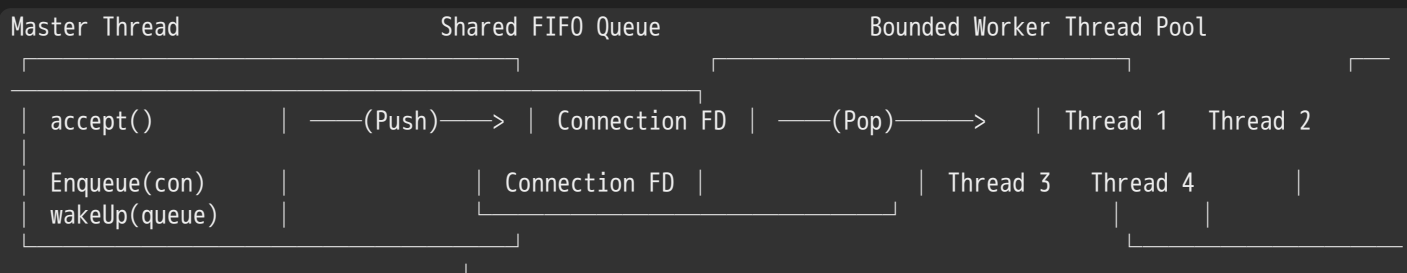
```
c
// Main loop of a Multi-Threaded Concurrent Server
while (1) {
    int conn_socket = accept(server_socket, NULL, NULL); //
    pthread_t thread_id;

    // Spawn a lightweight thread sharing the process address space
    pthread_create(&thread_id, NULL, serve_client_wrapper, (void *) (long)conn_socket); //
}
```

- **Performance Profile:** Threads are much more efficient to create and context-switch than full processes because they do not require changing the active address space or reloading memory translation structures.
- **The Cost of Giving Up Protection:** Because all threads share the same address space and global file descriptors, they are highly susceptible to data races. A single memory corruption bug or synchronization error in one thread can corrupt shared structures or crash the entire server process.

1.4 The Thread Pool Architecture

The multi-threaded design above features **unbounded thread allocation**, which introduces a major stability risk: if the site faces a sudden surge in traffic, spawning an unlimited number of threads will exhaust system memory and degrade overall throughput. To prevent this, professional systems employ a **Thread Pool** model, bounding the maximum level of multiprogramming with a fixed set of worker threads.



Master and Worker Thread Algorithms

```
c
// Master Thread Loop
master() {
    allocThreads(worker, queue); // Allocate a fixed pool of worker threads
    while(TRUE) {
        con = AcceptCon(); // Block waiting for a new client connection
        Enqueue(queue, con); // Append connection handle to the shared queue
        wakeup(queue); // Signal and wake up a sleeping worker thread
    }
}

// Worker Pool Routine
worker(queue) {
    while(TRUE) {
        con = Dequeue(queue); // Attempt to pop a connection off the queue
        if (con == null) {

```

```

        sleepOn(queue);           // Block and yield the CPU if no work is available
    } else {
        ServiceWebPage(con); // Handle client I/O requests
    }
}
}

```

Section 2: Kernel Management of File Descriptors & Aliasing under `fork()`

To manage process communication and sharing safely, the operating system kernel separates user-level file identifiers from the underlying file state.

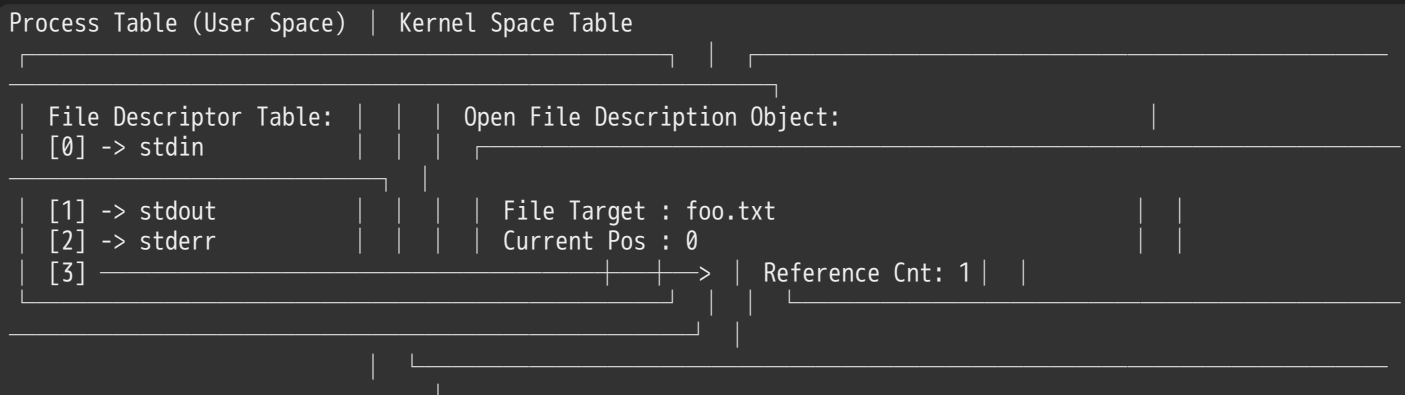
2.1 The Process Control Block (PCB)

The kernel represents each individual process using a central record called the **Process Control Block (PCB)**. This structure tracks the process's execution status (Running, Ready, Blocked), saved register states, Process ID (PID), user ownership, scheduling priorities, memory space boundaries, and an indexed list of open files. The scheduler uses these PCBs to allocate CPU time and hardware resources based on its internal policy decisions.

2.2 Lifecycle of an Open File Descriptor

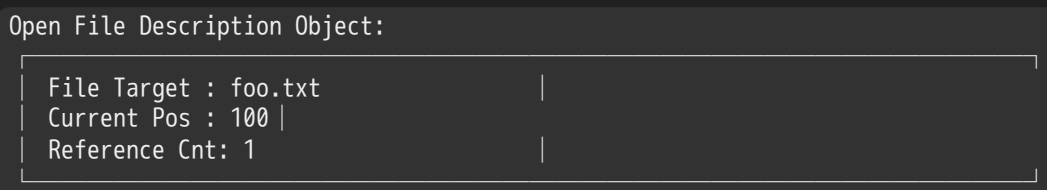
When an application calls `open("foo.txt")`, the kernel verifies permissions, allocates a system-wide **Open File Description** object, and returns its index from the per-process file descriptor table (e.g., index 3).

Step 1: Initial Allocation



Step 2: Offset Tracking During Reads

When the process executes `read(3, buf, 100)`, the kernel reads 100 bytes from disk, copies them into user space, and automatically increments the file position pointer inside the Open File Description object to `100`.

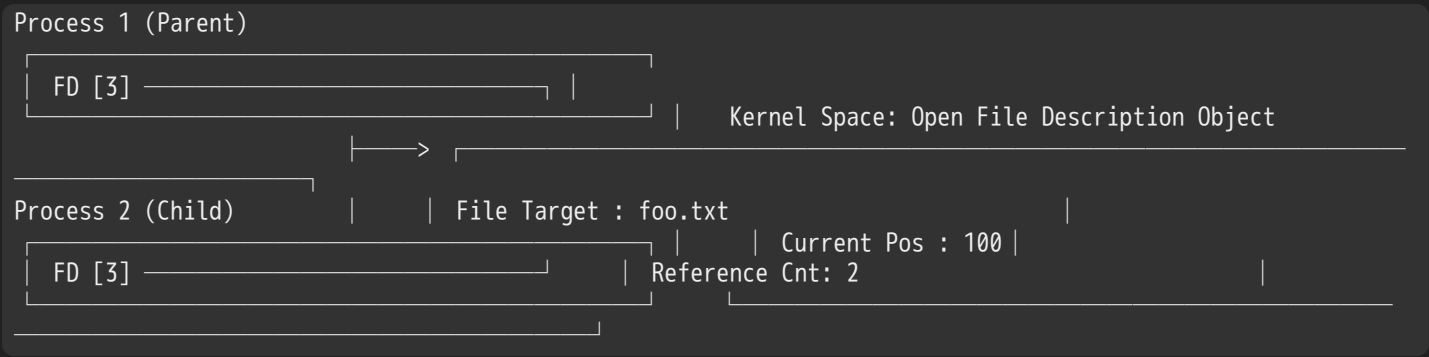


Step 3: Descriptor Release

When the process executes `close(3)`, the kernel clears entry 3 from the process's file descriptor table and decrements the Open File Description's reference count. Once the reference count drops to zero, the object is deallocated.

2.3 File Descriptor Duplication and Sharing under `fork()`

When a process calls `fork()`, the child process inherits an exact copy of the parent's file descriptor table. However, the underlying entries are not duplicated; instead, they are **aliased**, meaning both the parent and child process tables point to the *same* system-wide Open File Description object.



Dynamic Behavior of Shared Offsets

Because the parent and child processes share the same Open File Description object, updates to the file pointer are visible to both:

- 1. If Process 1 executes `read(3, buf, 100)`, it reads data from offset 100 to 200, and the shared file pointer automatically advances to 200.
- 2. If Process 2 executes a read call next, `read(3, buf, 100)`, it does *not* read from the beginning of the file; instead, it reads from offset 200 to 300, and advances the shared pointer to 300.

Close Semantics on Aliased Fields

If Process 1 executes `close(3)`, its local entry is cleared and the object's reference count decrements from 2 to 1. The file remains open and accessible within Process 2 via its descriptor 3. The underlying Open File Description object remains alive until every process referencing it has closed its respective descriptor.

2.4 Practical Applications of File Descriptor Aliasing

This aliasing behavior is essential for sharing system resources across process boundaries.

Shared Terminal Emulators

When a parent shell spawns a child application via `fork()`, the standard I/O streams— `stdin` (0), `stdout` (1), and `stderr` (2)—are copied directly to the child's descriptor table. Because these descriptors are aliased to the same terminal device driver object, both the parent and child can write output using `printf` to the same physical terminal window without clobbering each other's configurations. If one process closes its terminal descriptor, it remains open and fully functional in the other.

Section 3: Pipe Architecture & Inter-Process Communication

A **Pipe** is a unidirectional inter-process communication channel implemented as an internal queue inside kernel memory.

```
c
#include <unistd.h>
int pipe(pipe_fd[2]); // Allocates two file descriptors
```

- `pipe_fd[0]` : The **Read End** of the channel.
- `pipe_fd[1]` : The **Write End** of the channel.

3.1 Single-Process Pipe Lifecycle

While a single process can read and write to its own pipe, this pattern is rarely useful outside of multithreaded coordination.

```

c
#include <unistd.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define BUFSIZE 256 //

int main(void) {
    char *msg = "Message in a pipe.\n"; //
    char buf[BUFSIZE]; //
    int pipe_fd[2]; //

    if (pipe(pipe_fd) == -1) { // Instantiate the kernel queue
        fprintf(stderr, "Pipe failed.\n"); //
        return EXIT_FAILURE; //
    }

    // Write bytes into the pipe's write end (pipe_fd[1])
    ssize_t writelen = write(pipe_fd[1], msg, strlen(msg) + 1); //
    printf("Sent: %s [%ld %ld]\n", msg, (long)(strlen(msg) + 1), (long)writelen); //

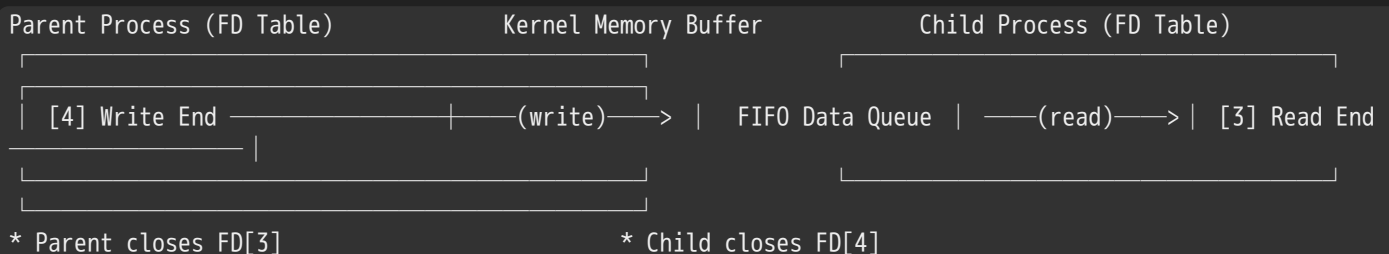
    // Read bytes out of the pipe's read end (pipe_fd[0])
    ssize_t readlen = read(pipe_fd[0], buf, BUFSIZE); //
    printf("Rcvd: %s [%ld]\n", buf, (long)readlen); //

    close(pipe_fd[0]); //
    close(pipe_fd[1]); //
    return EXIT_SUCCESS;
}

```

3.2 Inter-Process Pipelines (`fork()` Integration)

To use a pipe for communication across process boundaries, developers combine `pipe()` and `fork()`. After the fork, the parent and child close their unused ends of the pipe to create a clean, unidirectional data channel.



```

c
// Process Pipeline Implementation
int pipe_fd[2];
pipe(pipe_fd); // 1. Create the pipe first so descriptors are inherited

pid_t pid = fork(); // 2. Duplicate the process

if (pid > 0) {
    /* Parent Process Context */
    close(pipe_fd[0]); // Close the unused read end of the pipe
}

```

```

    write(pipe_fd[1], msg, strlen(msg) + 1); // Send data to child
    close(pipe_fd[1]); // Close the write end when finished to flush EOF
} else if (pid == 0) {
    /* Child Process Context */
    close(pipe_fd[1]); // Close the unused write end of the pipe

    ssize_t readlen = read(pipe_fd[0], buf, BUFSIZE); // Blocks until data arrives
    printf("Child Rcvd: %s [%d]\n", buf, (int)readlen); //
    close(pipe_fd[0]);
    exit(0);
}

```

Section 4: The Dispatch Loop & Thread Execution Lifecycles

At its core, thread scheduling and execution inside an operating system kernel boils down to a continuous loop called the **Dispatch Loop**.

```

c
// Conceptual execution model of the kernel scheduler
Loop {
    RunThread();           // Transfer control of the core to a thread
    ChooseNextThread();    // Execute scheduling policy choices
    SaveStateOfCPU(curTCB); // Save current thread state to its control block
    LoadStateOfCPU(newTCB); // Load the next thread's context into CPU registers
}

```

This dispatch loop is an infinite loop that runs for the entire lifetime of the operating system. The kernel never exits this loop unless the machine is shut down or faces a critical hardware failure.

4.1 Transferring CPU Control to User Space (`RunThread`)

To execute a thread, the kernel loads its saved context (registers, stack pointers, and environment details) directly into the physical CPU core. It then performs a jump instruction to the thread's saved Program Counter (`PC`).

At this exact instant, the kernel **yields complete physical control** of the processor core to the user-space code. While user instructions are executing, the operating system kernel is completely inactive on that core. To regain control and run the dispatch loop again, the kernel relies on either **Internal Events** or **External Events**.

4.2 Internal Events (Voluntary Relinquishment)

Internal events are triggered directly by the running thread voluntarily yielding its execution time. This occurs through three main paths:

1. **Blocking on I/O operations:** Requesting files or network data implicitly blocks the thread and yields the CPU.
2. **Synchronization waits:** A thread yields control when it blocks on a lock or waits for a signal from another thread.
3. **Explicit yielding:** Calling the `yield()` system call voluntarily relinquishes the remainder of the thread's time slice.

```

c
// Example of a voluntary yielding loop
computePI() {
    while(TRUE) {

```

```

    ComputeNextDigit();
    yield(); // Voluntarily hand control back to the OS dispatcher
}
}

```

Section 5: Thread Synchronization Primitives & Internal Context Switching Mechanics

When a thread calls `yield()`, the system transitions from the user-level context down to the kernel-level stack to perform a switch.

5.1 The Two-Stack Execution Model

To prevent user code from compromising system stability, every thread has both a **User-Level Stack** and a **Kernel-Level Stack**.

- **User-Level Stack:** Tracks standard function execution frames while running in user space (Mode Bit = 1).
- **Kernel-Level Stack:** Used when a thread traps into the kernel (via a system call, exception, or interrupt). The kernel cannot trust the user-space stack pointer, so it switches to this protected kernel stack to handle operations safely.

```

c
// Kernel workflow to switch executing threads
run_new_thread() {
    newThread = PickNextRunnableThread(); // Policy Choice
    switch(curThread, newThread);         // Low-level assembly context switch
    ThreadHouseKeeping();                  // Deallocate terminated resources
}

```

5.2 Low-Level Assembly Context Switching Mechanics

The assembly-level `switch()` function unloads the register context from the active CPU core, saves it into the current thread's TCB, and loads the saved context of the next thread.

```

assembly

# Conceptual implementation of switch(tCur, tNew)
switch:
    # 1. PRESERVE ACTIVE CONTEXT INTO OLD TCB (tCur)
    Store CPU.r0    -> TCB[tCur].regs.r0    # Save general-purpose registers
    Store CPU.r7    -> TCB[tCur].regs.r7    #
    Store CPU.sp    -> TCB[tCur].regs.sp    # Save active Stack Pointer
    Store CPU.retpc -> TCB[tCur].regs.retpc # Save Return Address pointer

    # 2. LOAD NEW CONTEXT FROM TARGET TCB (tNew)
    Load TCB[tNew].regs.r0 -> CPU.r0    # Restore target register frames
    Load TCB[tNew].regs.r7 -> CPU.r7    #
    Load TCB[tNew].regs.sp -> CPU.sp    # Switch to the new thread's stack pointer
    Load TCB[tNew].regs.retpc -> CPU.retpc # Load the target return address

    return                                # Jump to instructions at CPU.retpc

```

5.3 Technical Challenges and Risks in Switch Code

Writing context-switching code is exceptionally risky; because it manages low-level CPU registers directly, even a minor mistake can introduce silent, catastrophic bugs.

For example, if the implementation forgets to save or restore a single register, the application will not crash immediately. Instead, the system will experience intermittent, non-deterministic failures that occur only when a context switch happens to interrupt a thread while it is actively using that specific register. Testing this code is incredibly difficult; because there are an infinite number of possible instruction interleavings, you cannot build an exhaustive test suite to catch every edge case.

Cautionary Tale: The Topaz Kernel Failure

To optimize execution speeds, engineers building the classic Topaz operating system kernel optimized their assembly context switch to save a single CPU instruction. This optimization was well-documented and worked perfectly—but it relied on a hard-coded assumption that the total kernel footprint would remain under 1 MB.

As time passed, new developers added features to the kernel without removing old ones, eventually pushing its size past the 1 MB boundary. Because the original optimization constraint was forgotten, the system began experiencing completely unpredictable failures and data corruption. This serves as a warning to systems programmers: **always design for simplicity over clever optimizations.**

Section 6: Preemption, External Events, & the Interrupt Architecture

6.1 The Need for Preemption

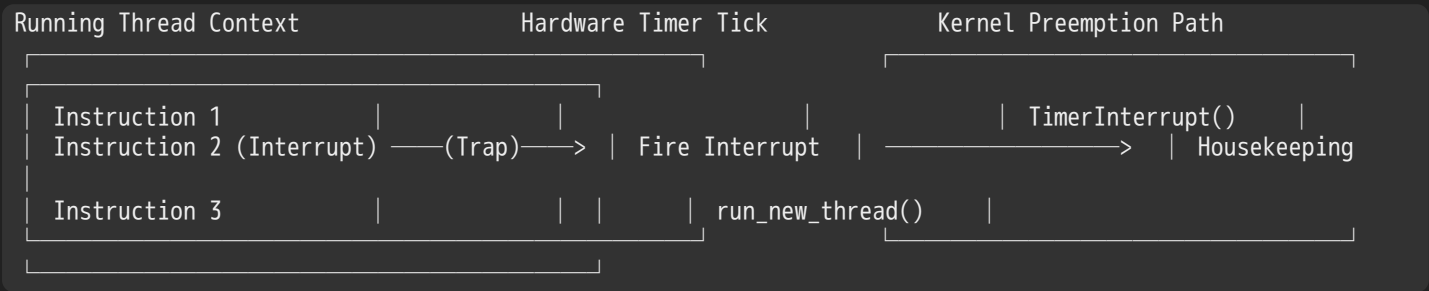
If an operating system relied solely on voluntary internal events, a buggy or malicious application could easily hijack the computer. For example, if a program computing PI falls into an infinite loop without performing any I/O or calling `yield()`, it would hog the processor core indefinitely and lock up the system.

To keep any single thread from monopolizing resources, the operating system uses **External Events** to forcefully reclaim control.

6.2 Preemption via Timer Interrupts

The primary tool for preemption is the hardware **Timer**. The timer acts like a physical alarm clock, periodically sending an interrupt signal to the CPU at a fixed interval (typically every few milliseconds).

When the timer fires, the CPU hardware intercepts execution, flips the mode bit to enter kernel mode, and branches directly to the kernel's timer interrupt handler. This handler runs periodic housekeeping tasks and invokes `run_new_thread()` to force a scheduling decision, ensuring the system remains responsive.



6.3 The Hardware Interrupt Controller

Hardware interrupts are managed and routed using a specialized component called the **Interrupt Controller**.

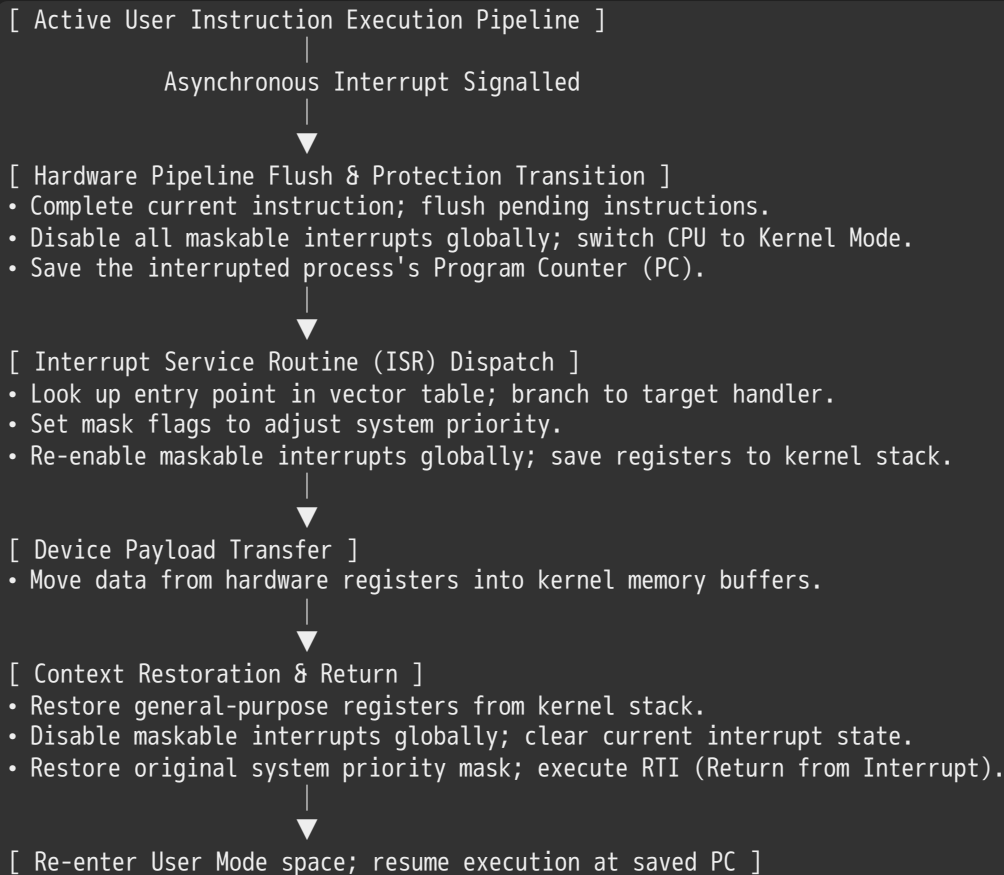
The interrupt controller features several core mechanisms to manage device signals:

- **Interrupt Mask:** A configurable bitmask that allows the operating system to selectively enable or disable individual device interrupt lines.
- **Priority Encoder:** Evaluates all active, enabled hardware signals and selects the one with the highest priority to forward to the CPU.

- **Global Interrupt Disable Flag:** An internal CPU register bit that allows the kernel to temporarily turn off all maskable interrupts during critical code sections.
- **Non-Maskable Interrupt (NMI):** A dedicated, high-priority hardware line reserved for critical errors (like RAM parity failures or hardware faults) that cannot be disabled by the CPU.

6.4 Lifecycle of a Hardware Interrupt Context Switch

Unlike a system call, a hardware interrupt is an unexpected, involuntary context switch that handles requests immediately without evaluating what to run next. When an external device (such as a network card) sends an interrupt signal, the CPU handles it through a strict hardware and software lifecycle:



Section 7: Thread Lifecycle Initialization (ThreadFork & ThreadRoot)

To spawn a new thread, the operating system executes an internal allocation and initialization routine called `ThreadFork()`.

7.1 The ThreadFork Routine

`ThreadFork()` takes three primary arguments: a pointer to the function routine (`fcnPtr`), a pointer to its argument array (`fcnArgPtr`), and the requested stack allocation size.

The allocation routine executes the following steps:

1. Validates the arguments in user space, traps into kernel mode, and performs the security checks again.
2. Allocates a new physical memory block to serve as the thread's execution stack and instantiates a new TCB.
3. Configures the saved register values within the TCB so that its stack pointer points to the newly allocated stack block.
4. Sets the TCB's saved program counter (`retpc`) to point to the address of the kernel assembly stub function `ThreadRoot`.
5. Places the target function address (`fcnPtr`) and argument address (`fcnArgPtr`) into the TCB fields corresponding to the system's argument registers (e.g., `r0` and `r1` or `a0` and `a1`).

6. Appends the newly initialized TCB to the scheduler's ready queue.

7.2 Bootstrapping via ThreadRoot

The new thread remains suspended on the ready queue until the scheduler invokes `run_new_thread()` and selects it. The assembly context switch loads the prepared values into the CPU, changing the stack pointer and branching directly into `ThreadRoot`.

```
c
// Conceptual execution flow of the ThreadRoot bootstrap stub
ThreadRoot(fcnPtr, fcnArgPtr) {
    DoStartupHousekeeping(); // Record start times and log thread statistics
    UserModeSwitch();        // Flip the mode bit to enter standard User Mode

    // Execute the user's application function with its arguments
    Call fcnPtr(fcnArgPtr);   // The stack grows and shrinks dynamically below this frame

    ThreadFinish();          // Clean up resources and wake up any waiting threads
}
```

Section 8: Core Challenges of Concurrency, Nondeterminism, & Atomic Operations

To build correct multi-threaded applications, systems programmers must understand the differences between concurrency and true physical parallelism.

8.1 Concurrency vs. Parallelism

- **Concurrency:** The logical overlap of multiple execution streams over time. On a single CPU core, threads run concurrently through time-slicing and interleaving. The scheduler is free to run instructions in any order, meaning programs must be designed to work correctly under any possible timeline.
- **Parallelism:** The physical execution of multiple instruction streams at the exact same instant, which requires multiple processing cores or processors.

8.2 Shared State Corruption: The ATM Bank Server Case Study

Consider an un-synchronized bank server designed to handle balance modifications across an automated teller network.

```
c
// Vulnerable balance modification routine
void Deposit(acctId, amount) {
    acct = GetAccount(acctId); // Fetches account details (triggers blocking disk I/O)
    acct->balance += amount;    // Critical Section
    StoreAccount(acct);         // Saves updated account details back to disk
}
```

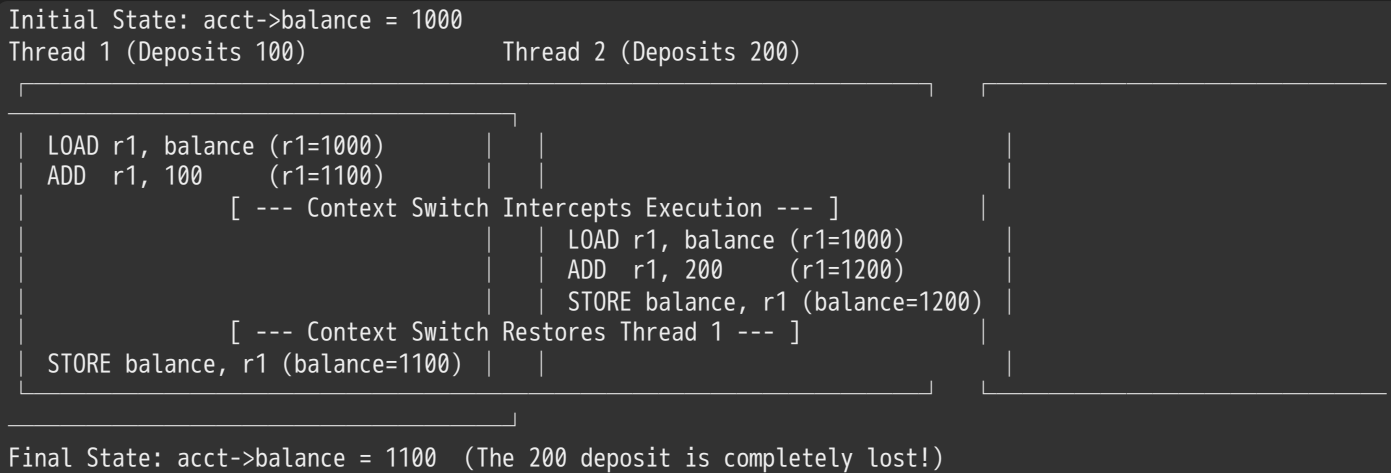
The Assembly-Level Breakdown of Lost Updates

While `acct->balance += amount` looks like an atomic assignment in C, it compiles down to three independent assembly instructions:

```
assembly
```

```
# Assembly sequence for a balance update
LOAD  r1, acct->balance # 1. Read balance from memory into register r1
ADD    r1, amount        # 2. Add deposit value to register r1
STORE r1, acct->balance  # 3. Write updated value back to memory
```

If two threads attempt to deposit money into the same account simultaneously, a context switch can interleave these instructions and cause updates to be lost:



8.3 Mathematical Interleaving Analysis Examples

Mathematical Interleaving Case Study 1

Initial State: $y = 12$.

- **Thread A instructions:** $I_{A1} : x = 1;$ $I_{A2} : x = y + 1;$
- **Thread B instructions:** $I_{B1} : y = 2;$ $I_{B2} : y = y \times 2;$

Depending on how the scheduler interleaves these instructions, the variable x can result in three different final values:

- **Scenario A ($x = 13$):** Thread A runs to absolute completion before Thread B starts. $x = y + 1 \implies x = 12 + 1 = 13$
- **Scenario B ($x = 5$):** Thread B runs to absolute completion before Thread A executes its second instruction. $y = 2 \implies y = 2 \times 2 = 4$ $x = y + 1 \implies x = 4 + 1 = 5$
- **Scenario C ($x = 3$):** Thread B executes its first instruction ($y = 2$), then Thread A preempts it and runs completely ($x = y + 1$), and finally Thread B finishes ($y = y \times 2$). $x = 2 + 1 = 3$

Mathematical Interleaving Case Study 2 (Word Aggregation Bounds)

Initial State: Shared variable $x = 0$.

- **Thread A:** `x = 1;`
- **Thread B:** `x = 2;`

On modern architectures with atomic word writes, the final value of x will non-deterministically be either `1` or `2`.

However, on older or non-standard architectures with non-atomic multi-word writes, a split write sequence could produce a garbled middle value. For example, if Thread A writes the upper bits and is interrupted by Thread B before writing the lower bits, the interleaved execution could result in an impossible final value like `3`.

8.4 Atomic Operations

To prevent these corruptions, concurrent programs must be built on top of **Atomic Operations**. An atomic operation is an indivisible unit of execution that always runs completely to success or behaves as if it never ran at all; it cannot be stopped

mid-execution, and its intermediate state cannot be modified by other threads.

On most modern processors, basic memory references and word assignments (standard loads and stores) are guaranteed to be atomic by the hardware, which prevents the garbled bit-interleaving example described above. However, complex instructions—such as double-precision floating-point operations or hardware-level array copies—are often not atomic, requiring developers to build synchronization structures using high-level **Lock APIs** (`acquire()` and `release()`).

Section 9: Course Administration, Midterm Reference, & Project Requirements

9.1 Midterm Exam Logistics

- **Date & Time:** Tuesday, February 24th, from 7:00 PM to 10:00 PM.
- **Format:** Strictly closed-book and closed-device. Students are permitted to bring **one double-sided, hand-written page of reference notes**.
- **Class Schedule:** No formal lecture or class session is held on the day of the exam, freeing up time for study and extra office hours.

9.2 Project 1 Design Document Requirements

- **Deadline:** Due Tuesday, February 17th. The course staff enforces a strict policy: **no extensions of any sort are granted for design documents**.
- **Design Review Format:** Teams must attend an upcoming high-level design review with their TA to defend their technical approach. Students do not need to present fully finalized code with every semicolon in place, but they must prove they thoroughly understand the complexities of their design.

Group Verification Checklist

1. **Algorithmic Clarity:** Teams must explicitly detail exactly what functions they intend to modify and outline the core algorithms they will use.
2. **Structural Mapping:** Explain how different system modules will link together and trace data flows across kernel layers.
3. **Shared Contribution:** Every single team member must fully understand their portion of the code. Individual understanding is verified through direct questioning during the review.
4. **Testing Strategy:** Teams must explain their test cases and prove they understand the testing framework, including identifying edge cases that are not covered by the baseline tests provided by the staff.
5. **Academic Honesty:** All work must be original. The course staff scans submissions and monitors repositories for any code copied from historic semesters or generated by AI tools.



CS162 Operating Systems & Systems Programming

Lecture 8: Synchronization 2: Lock Implementation, Atomic Instructions, Futex, and Higher-Level Locking

Section 1: Thread Scheduling & Process Lifecycle Architecture

An operating system kernel manages execution contexts by continuously mapping virtual threads to physical processing cores. This orchestration relies on a core kernel component called the scheduler.

1.1 Context Recall: The Execution Environment

A thread represents an isolated execution stream tracking its own Program Counter (PC), stack pointer (SP), and machine register states. When a running thread is suspended (e.g., when a thread executes a voluntary `yield()` call inside a processing loop), its current execution state is saved to its **Thread Control Block (TCB)**. The kernel scheduler can then place that TCB onto different execution tracking queues:

- **Ready Queue:** A collection of TCBs that are fully operational and waiting to be allocated a CPU core.
- **Wait/Blocked Queue:** A collection of TCBs that are suspended and cannot run until a specific event or condition is resolved (such as I/O data returning or a lock being released).

1.2 The Complete Process/Thread Lifecycle State Machine

As an instruction stream executes, its operational status changes, transitioning through five distinct architectural states:



Visual layout of lifecycle state transitions.

1. **New:** The process or thread is currently being initialized by the kernel (allocating memory blocks, instantiating TCB/PCB records); it is not yet eligible to run.
2. **Ready:** The thread is fully operational and waiting in the ready queue for the scheduler to allocate it a CPU core.
3. **Running:** The thread has been allocated a CPU core; its instructions are executing directly on the physical hardware.
4. **Waiting:** The thread has issued a blocking request (e.g., an I/O read or a synchronization lock request) and has yielded the CPU. It is placed on a wait queue until the event completes.
5. **Terminated:** The thread has finished executing its code. It enters a dead or "zombie" state until its parent process reclaims its exit status, at which point its kernel resources are freed.

1.3 Per-Device Queue Multiplexing

The kernel does not maintain a single, global queue for blocked threads. Instead, to avoid lock contention and optimize scheduling, it distributes blocks into **separate queues for each physical device, interrupt vector, or condition variable**.

- **Queue Layout:** Hardware drivers (such as USB Controller Unit 0, Storage Disk Unit 0, Disk Unit 2, or Ethernet Network Adapter 0) each maintain a private queue tracking the Process Control Blocks (PCBs) and TCBs waiting on that specific device.
- **Policy Isolation:** Because these queues are isolated from each other, the kernel can enforce different scheduling policies (such as First-In-First-Out, Priority Scheduling, or Round-Robin) on different queues to match the performance needs of each device.

Section 2: Concurrency Paradigms & Low-Level Nondeterminism

When applications share data across multiple threads, scheduling choices can introduce complex, non-deterministic bugs.

2.1 Server Concurrency Abstractions Comparison

Consider a banking server processing transactions across an automated teller network. Systems engineers can organize this application using three different architectural paradigms:

- 1. **Iterative Single-Threaded Model:** Processes requests sequentially. While simple, this model forces the entire server to block whenever a request waits for disk I/O, preventing the system from overlapping computation and hardware latencies.
- 2. **Event-Driven Architecture:** Uses a single thread that loops over non-blocking events via a central coordinator call like `WaitForNextEvent()` . This model overlaps computation and I/O without the overhead of threads, but it significantly increases code complexity. Developers must split their application logic into hundreds of distinct, non-blocking callback fragments (`StartOnRequest` , `ContinueRequest` , `FinishRequest`), and a single accidental blocking I/O step can stall the entire server.
- 3. **Multi-Threaded Shared Address Model:** Allocates a dedicated virtual thread to handle each client request. This allows code to be written cleanly and sequentially from start to finish, as the operating system automatically handles blocking and context switches behind the scenes. However, because these threads share the same process address space, any shared state can easily be corrupted if access is not properly synchronized.

Thread 1 (Transaction A)

Thread 2 (Transaction B)

load r1, acct->balance		
add r1, amount1		
[--- Context Switch Interrupts Execution ---]		
	load r1, acct->balance	
	add r1, amount2	
	store r1, acct->balance	
[--- Context Switch Restores Thread 1 ---]		
store r1, acct->balance		

Result: Transaction B's balance update is completely overwritten and lost!

2.2 Memory Interleaving at the Hardware Layer

When multiple threads read and write to the same memory locations concurrently, the final state of the program depends entirely on the exact order in which the scheduler interleaves their instructions.

Non-Atomic Assignment Interference

Initial State: Shared variable *y* = 12.

- Thread A: `x = 1; x = y + 1;`
- Thread B: `y = 2; y = y * 2;`

Depending on where context switches occur, the variable *x* can non-deterministically result in one of three values:

- **Scenario 1** (*x* = 13): Thread A runs to absolute completion before Thread B begins execution (*x* = 12 + 1 = 13).
- **Scenario 2** (*x* = 5): Thread B runs to completion first (*y* becomes 4), and then Thread A executes (*x* = 4 + 1 = 5).
- **Scenario 3** (*x* = 3): Thread B executes its first assignment (*y* = 2), then Thread A preempts it and runs completely (*x* = 2 + 1 = 3), and finally Thread B finishes (*y* = 2 × 2 = 4).

Explaining Word Corruption Math

Initial State: Shared variable *x* = 0.

- Thread A: `x = 1;` (Binary: 0001)
- Thread B: `x = 2;` (Binary: 0010)

On standard modern architectures, word-sized loads and stores are guaranteed to be atomic by the hardware, meaning x will cleanly result in either `1` or `2`.

However, on older architectures with multi-word writes (such as double-precision floating-point values or bulk array copies), a write operation can be split across multiple bus cycles. If a context switch occurs mid-write, the bits from both threads can interleave. For example, if Thread A writes the lower bits (`...0001`) and Thread B overwrites the upper bits (`...0010`), the combined result could be `0011`, producing an impossible final value of `3`.

2.3 Case Study: Non-Atomic Global Counters

Consider a program where two threads compete to modify a single shared counter variable `i`:

- **Thread A:** Loops while `i < 10`, executing `i = i + 1`;
- **Thread B:** Loops while `i > -10`, executing `i = i - 1`;

Register Interference Trace

At the assembly level, incrementing and decrementing memory variables are non-atomic operations that require loading data into registers, modifying the value, and writing it back:

Thread A Context (r1)	Shared Memory (i)	Thread B Context (r1)
1. load r1, M[i] (r1 = 0)	i = 0	
2. add r1, r1, 1 (r1 = 1)	i = 0	
[i] (r1 = 0)	i = 0	3. load r1, M
1)	i = 0	4. sub r1, r1, 1 (r1 = -
5. store M[i], r1	i = 1	6. store M[i], r
1	i = -1	

Lost update trace on a multiprocessor or uniprocessor with hyperthreading.

In this scenario, Thread A increments the counter in its local register, but before it can write the update back to memory, Thread B reads the original value (`0`). When Thread A writes its updated value (`1`), Thread B immediately overwrites it with `-1`.

This data race can cause updates to be lost indefinitely on both uniprocessors and multiprocessors, making the final outcome of the loop completely non-deterministic.

2.4 Real-World Impact: The Therac-25 Disasters

Failing to synchronize shared variables can have catastrophic real-world consequences, as seen in the infamous **Therac-25 radiation therapy machine accidents**.

The Therac-25 was a computer-controlled radiation therapy system where an electron accelerator was managed entirely by software. The system suffered from a series of race conditions on shared configuration variables due to poor software design.

If an operator updated prescription data rapidly on the control console, the high-speed data entry triggered a race condition. The software failed to detect that the physical turntable had moved into an unshielded position, delivering raw, massive radiation overdoses that directly caused the deaths of several patients.

Section 3: Synchronization Definitions & High-Level Lock Interfaces

To prevent data races and ensure deterministic behavior, cooperating threads must use synchronization primitives.

3.1 Core Definitions

- **Synchronization:** The use of atomic operations to coordinate the execution of multiple threads to ensure correct behavior when accessing shared data.
- **Mutual Exclusion:** A mechanism that guarantees only one thread can perform a specific operation or access a shared resource at any given time.
- **Critical Section:** A block of code that accesses a shared resource and must not be entered by more than one thread concurrently. Mutual exclusion is the mechanism used to protect a critical section.

3.2 The Lock Abstraction

A **Lock** (or Mutex) is a software object used to enforce mutual exclusion. It exposes a clean, simple interface with two fundamental atomic operations:

- `acquire(&mylock)` : Attempts to acquire the lock. If the lock is free, the calling thread marks it as busy and continues executing. If the lock is busy, the calling thread blocks and waits until the lock becomes available.
- `release(&mylock)` : Marks the lock as free. This operation should only be called by the thread currently holding the lock, allowing waiting threads to proceed.

```
c

// Fixing the Bank Transaction Race Condition using Locks
void Deposit(int acctId, double amount) {
    acquire(&mylock); // Block other threads from entering the critical section

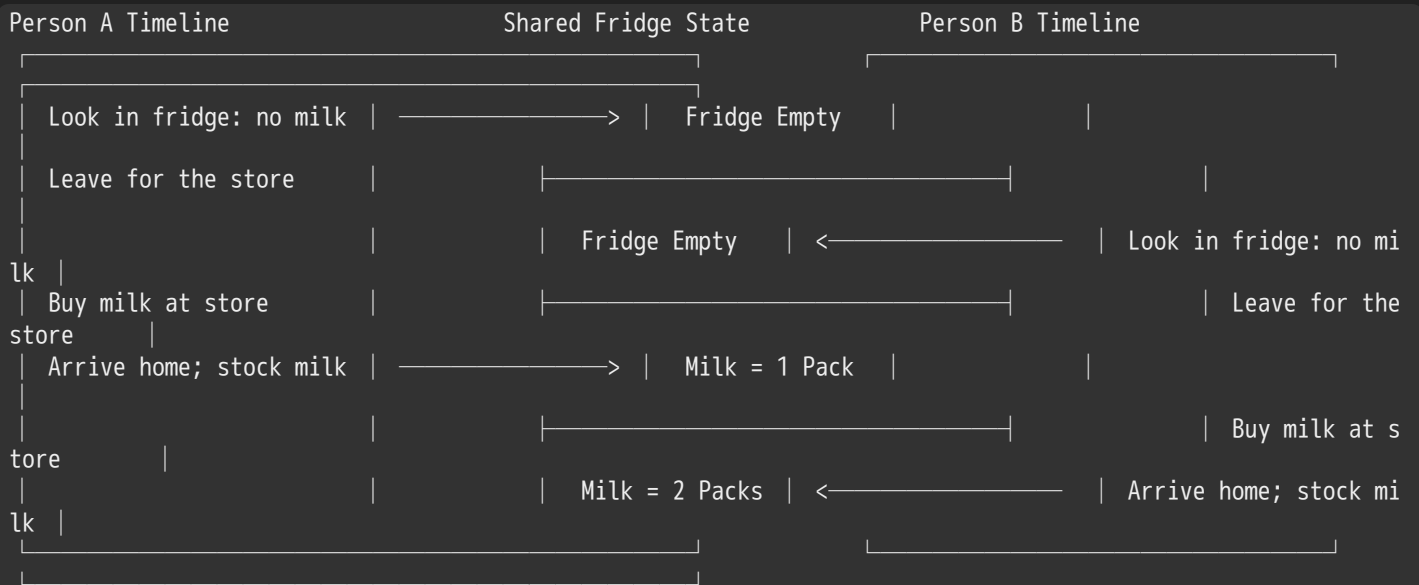
    /* --- START CRITICAL SECTION --- */
    acct = GetAccount(acctId);
    acct->balance += amount; // Protected allocation update
    /* ---- END CRITICAL SECTION ---- */

    release(&mylock); // Allow the next waiting thread to enter
    StoreAccount(acct);
}
```

For this synchronization model to work correctly, **all cooperating functions (such as Withdraw or Transfer) must share and use the exact same lock variable** when accessing the underlying data structure.

Section 4: The Classical "Too Much Milk" Coordination Dilemma

To understand how synchronization primitives are built, we look at a classic coordination problem: ensuring room-mates do not accidentally buy too much milk.



Result: The roommates have failed to coordinate, resulting in too much milk!

4.1 Defining Correctness Properties

Any valid solution to this coordination problem must guarantee two core correctness properties:

1. **Safety:** Never more than one person buys milk at the same time.
2. **Liveness:** Someone will always buy milk if it is needed.

We will try to solve this problem using only basic, atomic memory reads and writes (loads and stores) as our building blocks. Note that using a physical key lock on the refrigerator door is a poor abstraction; it over-locks the resource and prevents roommates from accessing other items like orange juice.

4.2 Solution #1: Note as a Lock (Check-Then-Leave)

In this attempt, a roommate leaves a note to signal they are going to buy milk, and removes it once they return.

```
c
// Solution #1 Implementation Code
if (noMilk) {
    if (noNote) {
        leave Note;
        buy milk;
        remove note;
    }
}
```

- **Failure Analysis:** This attempt fails because checking the conditions and leaving the note are separate, non-atomic actions. Thread A can check that there is no milk and no note, but if a context switch occurs right before it executes `leave Note;`, Thread B can run, see that there is no note, and proceed to buy milk as well. This leads to intermittent, unpredictable duplication bugs that are incredibly difficult to debug.

4.3 Solution #1.5: Leave Note First (Leave-Then-Check)

To fix Solution #1, we try leaving the note first before checking the fridge status.

```
c
// Solution #1.5 Implementation Code
leave Note;
```

```

if (noMilk) {
    if (noNote) {
        buy milk;
    }
}
remove Note;

```

- **Failure Analysis:** This version causes complete starvation and livelock. If Thread A leaves a note, it will immediately fail its own internal check because `noNote` is now false, causing it to remove the note and exit without ever buying milk. As a result, no thread will ever buy milk.

4.4 Solution #2: Labeled Private Notes

Next, we try using distinct, labeled notes for each thread so they can track each other's status independently.

```

c
// Thread A Code      // Thread B Code
leave note A;          leave note B;
if (noNote B) {        if (noNote A) {
    if (noMilk) {        if (noMilk) {
        buy Milk;        buy Milk;
    }
}
remove note A;          remove note B;

```

- **Failure Analysis:** This approach introduces a serious **Starvation / Livelock** vulnerability. If Thread A leaves note A and is immediately context-switched out before checking note B, Thread B can run and leave note B. When both threads resume, they will see each other's notes, assume the other roommate is handling the errand, and remove their own notes without anyone ever buying milk.

4.5 Solution #3: Rigorous Two-Note Asymmetric Coordination

To build a fully working solution using only atomic loads and stores, we must break the symmetry between the two threads, making their execution paths structurally distinct.

```

c
// Thread A Implementation // Thread B Implementation
leave note A;               leave note B;
while (note B) {            if (noNote A) {
    /* busy-wait loop: do nothing */    if (noMilk) {
}                                     buy milk;
if (noMilk) {
    buy milk;
}
remove note A;              remove note B;

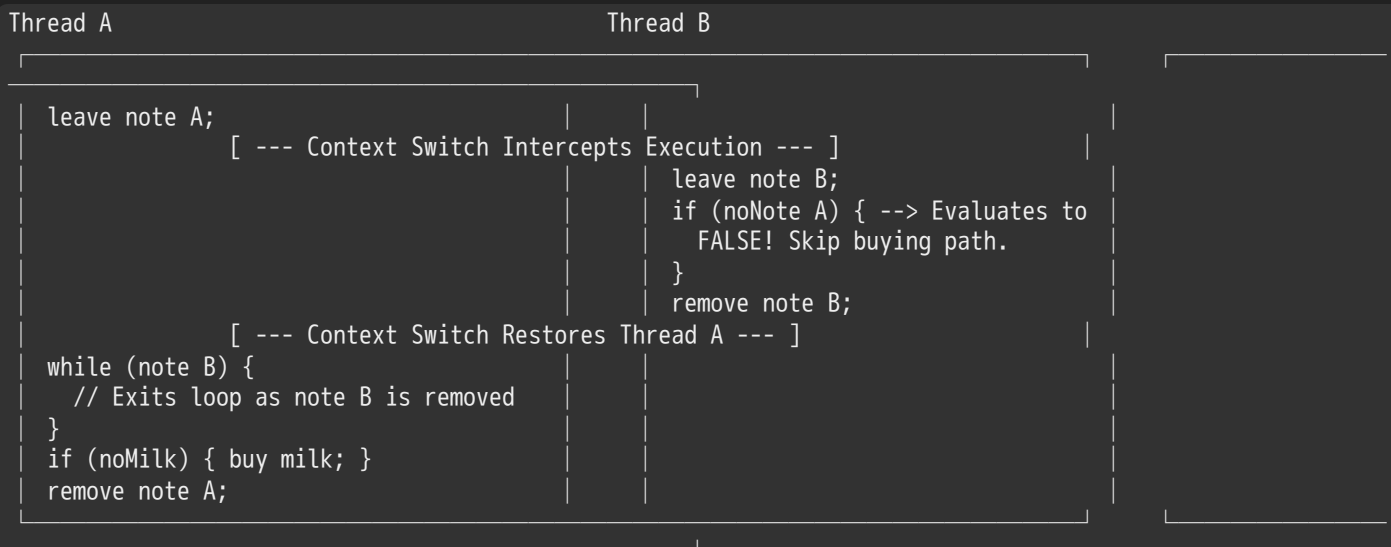
```

Complete Mathematical Safety Proof Walkthrough

We verify that Solution #3 is perfectly safe by tracing all possible execution timelines across two exhaustive cases:

Case 1: Thread A Leaves its Note First

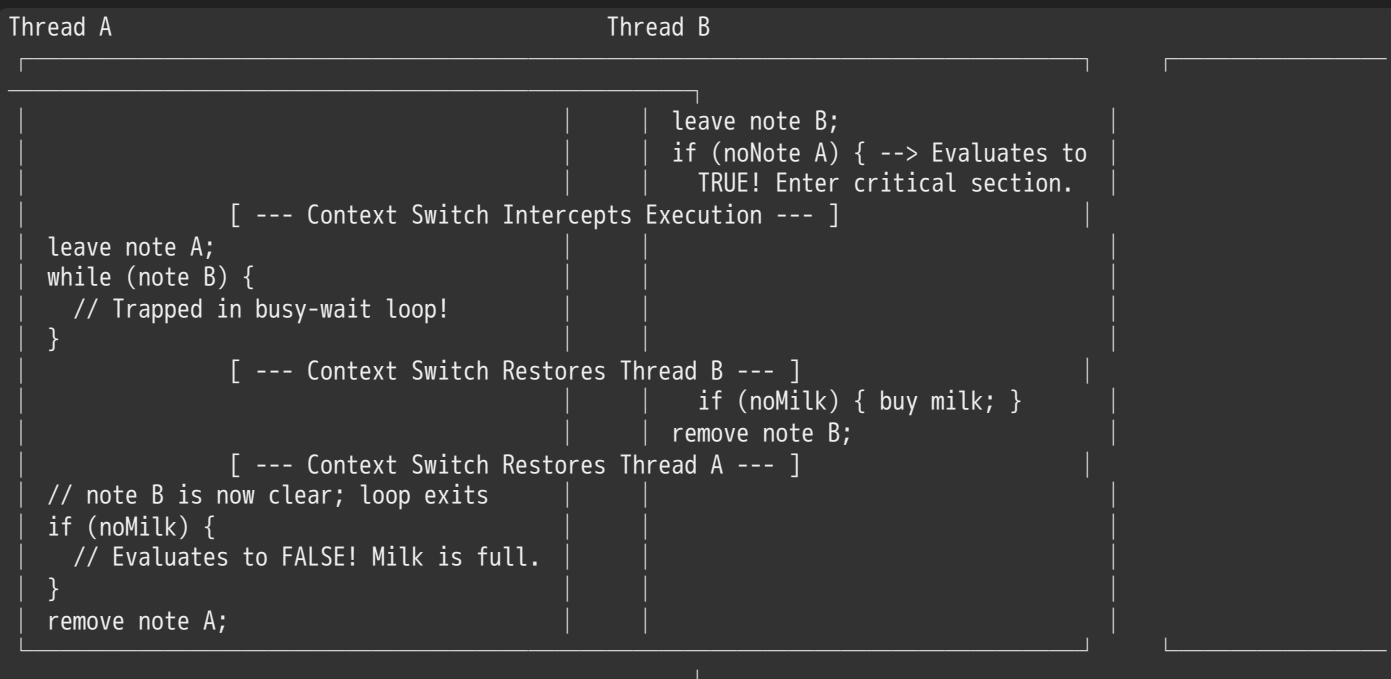
Thread A executes `leave note A` before Thread B evaluates its conditional check `if (noNote A)`.



Result: Safe. Only Thread A buys milk.

Case 2: Thread B Reaches its Conditional Check First

Thread B evaluates `if (noNote A)` before Thread A executes its initialization assignment `leave note A`.



Result: Safe. Only Thread B buys milk; Thread A busy-waits and then skips.

This asymmetric approach can be generalized to support n competing threads using **Leslie Lamport's Bakery Algorithm (1974)**.

Downsides of Pure Load/Store Coordination

While Solution #3 is correct, it has significant engineering drawbacks for real-world development:

- **High Complexity:** The logic is highly asymmetric and difficult to reason about, making it prone to errors as systems scale.
- **Lack of Generalization:** The code must be tailored differently for each participating thread, meaning you cannot easily run the same code block across an arbitrary number of worker threads.
- **Resource Waste:** While Thread A waits for Thread B, it traps the CPU core in a continuous, high-speed spin loop (`while (note B)`). This is known as **busy-waiting**, and it wastes significant clock cycles and power.

To avoid these limitations, modern architectures provide specialized hardware instructions to build clean, efficient synchronization primitives like locks.

Section 5: Uniprocessor Lock Implementations (Interrupt Disabling)

On single-core processors, concurrency is achieved entirely by time-slicing threads via interrupts. We can exploit this behavior to build uniprocessor-safe synchronization primitives.

5.1 Naïve Interrupt Disabling

Because the operating system scheduler can only regain control of a single core through internal events or external hardware interrupts, a thread can achieve absolute atomicity by temporarily disabling all maskable hardware interrupts.

```
c

// Naïve Uniprocessor Lock Model
void LockAcquire() { disableInterrupts(); } //
void LockRelease() { enableInterrupts(); } //
```

Operational Risks

This naïve approach has severe system security and stability flaws:

- **Denial of Service:** If the kernel exposes raw interrupt controls to user mode, a malicious or buggy program could call `LockAcquire()` and enter an infinite loop (`while(TRUE);`). Because the timer interrupt is disabled, the kernel can never regain control, completely freezing the computer.
- **Real-Time I/O Failure:** Disabling interrupts delays all incoming hardware notifications. Critical safety alerts—such as an automated industrial warning packet like `"Reactor about to meltdown. Help?"` —will be lost or delayed, creating severe real-world hazards.

5.2 The Guarded-Variable Internal Lock Model

To implement locks safely on a uniprocessor, the kernel restricts interrupt disabling to its own internal functions, keeping the time spent with interrupts disabled incredibly short.

```
c

// Guarded Variable Internal Uniprocessor Lock
int value = FREE; // System-wide lock tracking variable

void Acquire() {
    disableInterrupts(); // Enter a short internal "meta"-critical section
    if (value == BUSY) {
        putThreadOnWaitQueue(current_thread); // Suspend execution
        sleep();                               // Yield the CPU
    } else {
        value = BUSY;                          // Grab the lock cleanly
    }
    enableInterrupts(); // Exit the internal "meta"-critical section
}

void Release() {
    disableInterrupts();
    if (anyoneOnWaitQueue()) {
        thread = removeThreadFromWaitQueue();
        placeOnReadyQueue(thread);           // Wake up a waiting thread
    } else {
        value = FREE;                        // Mark the lock as available
    }
}
```

```
enableInterrupts();
}
```

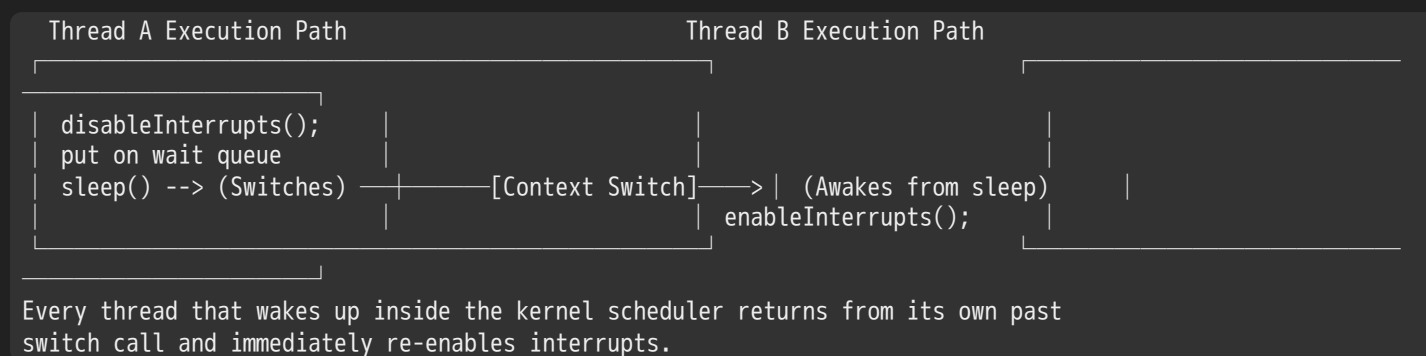
5.3 The Enable Position Problem & Inter-Thread Interrupt Hand-off

When a thread must go to sleep inside `Acquire()`, choosing exactly when to re-enable interrupts introduces a subtle and dangerous race condition.

- **Re-enabling before putting the thread on the queue:** If interrupts are enabled before updating the wait queue, an incoming interrupt can trigger a context switch. A concurrent thread could call `Release()`, find the wait queue empty, and exit. When the original thread resumes, it will proceed to place itself on the queue and go to sleep, missing the wakeup signal entirely and hanging permanently.
- **Re-enabling after queuing but before calling `sleep()`:** If interrupts are enabled after updating the queue but before entering the scheduler, a concurrent `Release()` can pull the thread off the wait queue and place it on the ready queue. However, the original thread still thinks it needs to sleep, so it will continue into the scheduler and block anyway, causing a permanent deadlock.

The Solution: The Next-Thread Interrupt Hand-off Protocol

To solve this race condition, the thread must call `sleep()` with interrupts **fully disabled**. The responsibility for re-enabling interrupts is then shifted to the *next* thread scheduled by the kernel.



5.4 Step-by-Step Simulation Trace

Let's trace two threads, A and B, interacting through this uniprocessor lock implementation:

Timeline Trace State Matrix

Step	Active Thread	Executing Action	value State	Wait Queue	Thread A State	Thread B State
1	Thread A	Calls <code>Acquire()</code> ; traps inside meta-section	0	Empty	Running	Ready
2	Thread A	Sets <code>value = 1</code> ; exits meta-section and enters critical section	1	Empty	Running	Ready
3	—	Timer Interrupt Fires; Scheduler Context Switch occurs	1	Empty	Ready	Running
4	Thread B	Calls <code>Acquire()</code> ; disables interrupts and checks <code>value</code>	1	Empty	Ready	Running
5	Thread B	Finds <code>value == 1</code> ; pushes itself onto wait queue and calls <code>sleep()</code>	1	[Thread B]	Ready	Waiting

Step	Active Thread	Executing Action	value State	Wait Queue	Thread A State	Thread B State
6	Thread A	Context switch returns to Thread A; it finishes its critical section and calls <code>Release()</code>	1	[Thread B]	Running	Waiting
7	Thread A	Disables interrupts, pops Thread B off wait queue, and moves it to ready queue	1	Empty	Running	Ready

Section 6: Multiprocessor Lock Implementations (Atomic Instructions & Spinlocks)

On multi-core and multiprocessor hardware, disabling interrupts is not sufficient to enforce synchronization. Disabling interrupts on one core does not stop other cores from executing instructions and accessing shared memory simultaneously. Broadcasting interrupt-disable messages across all cores is far too slow and resource-intensive to be practical.

Instead, multiprocessor systems rely on hardware-level **Atomic Read-Modify-Write (RMW) Instructions**. These instructions read, modify, and write back a memory location in a single, indivisible hardware operation, coordinated across cores via cache coherence protocols.

6.1 Hardware Read-Modify-Write Semantic Blocks

Modern CPU architectures provide several RMW primitives:

```
c
// 1. Test-And-Set Architecture Semantic Block
int test&set(int *address) {
    int result = *address; // Read current value from memory
    *address = 1;          // Set value to 1 atomically
    return result;         // Return the original value
}

// 2. Atomic Swap Architecture Semantic Block (x86 Idiom)
void swap(int *address, int *register_val) {
    int temp = *address;
    *address = *register_val;
    *register_val = temp; // Swap values between register and memory atomically
}

// 3. Compare-And-Swap Architecture Semantic Block
bool compare&swap(int *address, int expected_val, int new_val) {
    if (*address == expected_val) {
        *address = new_val; // Update value if it matches expectation
        return SUCCESS;
    }
    return FAILURE;
}
```

6.2 Basic Multiprocessor Spinlock via `test&set`

Using these hardware primitives, we can build a simple multiprocessor lock called a **Spinlock** that operates entirely in user space without requiring kernel context switches.

```
c
```

```
// Basic test&set Spinlock Implementation
int thelock = 0; // 0 = FREE, 1 = BUSY

void acquire(int *thelock) {
    while (test&set(thelock)); // Spin loop: busy-wait until thelock becomes 0
}

void release(int *thelock) {
    *thelock = 0; // Reset the lock value atomically
}
```

Operational Bottlenecks: Cache Line Ping-Ponging

While correct, basic spinlocks have severe performance flaws when highly contended:

- **High CPU Overhead:** Waiting threads consume 100% of their CPU time spinning in the `while` loop, wasting clock cycles that could be used by other threads.
- **Cache Line Ping-Ponging:** Because `test&set` is an atomic write instruction, every single iteration of the spin loop attempts to write to memory. This forces the underlying cache coherence protocol to invalidate that cache line across all other CPU cores. The cache line is forced to constantly bounce back and forth between cores ("ping-ponging"), consuming massive amounts of interconnect network bandwidth and slowing down the entire system.
- **Priority Inversion Vulnerability:** Spinlocks are highly susceptible to priority inversion. If a high-priority thread attempts to acquire a spinlock held by a low-priority thread, it will spin at 100% CPU utilization. On a single-core system, this completely starves the low-priority thread, preventing it from ever running to release the lock, locking up the system indefinitely. This exact bug famously crippled the **Mars Pathfinder Rover mission in 1997**.

6.3 The Optimized `test&test&set` Spinlock

To eliminate excessive interconnect network overhead, systems developers use an optimized pattern called a `test&test&set lock`.

```
c

// Optimized test&test&set Spinlock Implementation
void acquire(int *thelock) {
    do {
        while (*thelock); // 1. Pure Read Spin: Loop locally in cache without issuing writes
    } while (test&set(thelock)); // 2. Atomic Try: Only attempt an actual write when the lock
    appears free
}
```

- **Performance Profile:** By spinning on a plain memory read (`while (*thelock)`), the cache line remains in a shared, read-only state across all waiting CPU cores. Waiting threads read the value directly from their local L1/L2 caches without generating any network or bus traffic. Only when the lock holder writes `0` during release is the cache line invalidated, prompting the waiting cores to drop out of the inner loop and execute `test&set` to compete for the lock.

6.4 Guarded Sleep Lock with `test&set`

To completely eliminate long-term busy-waiting, we can combine a short spinlock (called a `guard`) with an internal kernel block queue.

```
c

// Guarded Sleep Lock Implementation
int guard = 0; // Short spinlock protecting access to the lock's internal metadata
```

```
int thelock = FREE; // Actual lock status variable

void acquire(int *thelock) {
    while (test&set(&guard)); // Spin briefly to acquire the guard

    if (*thelock == BUSY) {
        putThreadOnWaitQueue(current_thread); // Queue the thread
        go_to_sleep_and_release_guard(&guard); // Atomically sleep and reset guard = 0
        // On wakeup, the guard is guaranteed to be 0!
    } else {
        *thelock = BUSY; // Acquire the actual lock
        guard = 0;      // Release the guard
    }
}

void release(int *thelock) {
    while (test&set(&guard)); // Spin briefly to acquire the guard

    if (anyoneOnWaitQueue()) {
        thread = removeThreadFromWaitQueue();
        placeOnReadyQueue(thread); // Wake up a waiting thread; hold thelock as BUSY
    } else {
        *thelock = FREE;          // Mark the lock as available
    }
    guard = 0;                    // Release the guard
}
```

- **Note on Guard Reset:** The guard variable must be reset *atomically* as part of the sleep call. Resetting the guard *before* calling sleep creates a race condition where a concurrent release can check the queue and miss the wakeup. Resetting it *after* sleep is impossible, as the thread is already asleep and cannot execute instructions.

6.5 Architectural Lock Mechanics Comparison Matrix

The table below compares the implementation trade-offs of the different locking mechanisms covered so far:

Dimension	Raw Interrupt Lock	Guarded Interrupt Lock	Raw Test-And-Set Lock	Guarded Test-And-Set Lock
Multiprocessor Safe?	No	No	Yes	Yes
Long Busy-Waiting?	No	No	Yes (Consumes 100% CPU)	No (Short spin on guard only)
System Impact	Severe; blocks all system activity and scheduling.	Minimal; interrupts are disabled only for a few instructions.	Wasteful; causes severe cache invalidation traffic.	Efficient; balances short spin times with kernel sleeping.
Argument Utilization	Lock argument is completely ignored.	Lock variable tracks state changes.	Operates entirely in user space.	Integrates user space tracking with kernel blocking.

Section 7: Modern OS Synchronization (Linux Futex Architecture)

To maximize performance, modern operating systems avoid entering the kernel for locking unless absolutely necessary. This optimization is achieved using the Linux **Futex** (Fast Userspace Mutex) architecture.

7.1 The Futex System Call API

```
c
#include <linux/futex.h>
int futex(int *uaddr, int futex_op, int val, const struct timespec *timeout); //
```

- `uaddr` : A pointer to a standard 32-bit integer alignment tracking variable located entirely within user-space memory.
- `futex_op` : The requested operational command:
 - `FUTEX_WAIT` : If the value at `uaddr` is equal to `val`, the kernel puts the current thread to sleep until a corresponding `FUTEX_WAKE` call is issued. This check is performed atomically inside the kernel to prevent race conditions.
 - `FUTEX_WAKE` : Wakes up at most `val` threads currently blocked on the `uaddr` memory location.
- `timeout` : An optional pointer to a timeout structure.

7.2 Evolutionary Futex Implementations

Implementation Variant 1: Naïve Futex Integration

This initial attempt uses `test&set` alongside `FUTEX_WAIT` to eliminate long-term busy-waiting.

```
c
// Futex Variant 1 Implementation
int mylock = 0; //

void acquire(int *thelock) {
    while (test&set(thelock)) {
        // If the lock is busy, call futex to put the thread to sleep conditionally
        futex(thelock, FUTEX_WAIT, 1, NULL); //
    }
}

void release(int *thelock) {
    *thelock = 0; // Unlock the state variable
    futex(thelock, FUTEX_WAKE, 1, NULL); // Wake up a waiting thread
}
```

- **Performance Profile:** While this eliminates long-term busy-waiting, it introduces a new performance issue: every single call to `release()` must invoke the `futex` system call to wake up potential waiters, even if no other threads are waiting. This creates unnecessary system call overhead for uncontended locks.

Implementation Variant 2: The Waiter-Flag Optimization

To avoid unnecessary system calls, we introduce a boolean flag `maybe_waiters` to track if any threads are currently blocked on the lock.

```
c
```

```
// Futex Variant 2 Implementation
int mylock = 0;
bool maybe_waiters = false; // Flag to track potential waiters

void acquire(int *thelock, bool *maybe) {
    while (test&set(thelock)) {
        *maybe = true; // Mark that a thread is about to go to sleep
        futex(thelock, FUTEX_WAIT, 1, NULL); // Sleep on the lock variable
        *maybe = true; // Ensure subsequent sleepers are not permanently stuck
    }
}

void release(int *thelock, bool *maybe) {
    *thelock = 0; // Unlock the state variable
    if (*maybe) {
        *maybe = false; // Reset the waiter flag
        futex(thelock, FUTEX_WAKE, 1, NULL); // Issue the system call to wake up a thread
    }
}
```

- **Performance Profile:** This implementation successfully eliminates system calls when the lock is uncontended, as `release()` will skip the `futex` call if `maybe_waiters` is false. However, managing concurrent updates to both `thelock` and `maybe_waiters` introduces subtle edge-case race conditions.

Implementation Variant 3: The Optimized Three-State Futex Architecture

To resolve the coordination issues of Variant 2, we pack both the lock status and the waiter tracking into a single, unified 32-bit integer that supports three distinct states:

1. **UNLOCKED** (0): The lock is fully available; no threads hold it.
2. **LOCKED** (1): A single thread currently holds the lock; no other threads are waiting.
3. **CONTESTED** (2): A thread holds the lock, and one or more additional threads are currently sleeping in the kernel waiting to acquire it.

```
c

// Optimized Three-State Futex Implementation
typedef enum { UNLOCKED, LOCKED, CONTESTED } Lock; //
Lock mylock = UNLOCKED; //

void acquire(Lock *thelock) {
    // Fast Path: Attempt to grab the lock assuming it's completely uncontended
    if (compare&swap(thelock, UNLOCKED, LOCKED))
        return; // Success! No system call or overhead incurred

    // Slow Path: The lock is busy; loop and prepare to sleep in the kernel
    while (swap(thelock, CONTESTED) != UNLOCKED) {
        // Sleep using futex unless another thread releases the lock right before we wait
        futex(thelock, FUTEX_WAIT, CONTESTED, NULL); //
    }
}

void release(Lock *thelock) {
    // Atomically reset the lock to UNLOCKED and check its previous state
    if (swap(thelock, UNLOCKED) == CONTESTED) {
```

```

        // If the lock was CONTESTED, we must issue a system call to wake up a sleeper
        futex(thelock, FUTEX_WAKE, 1, NULL); //
    }
}

```

This multi-state design is the gold standard for high-performance libraries, as it ensures zero system call overhead in the common, uncontended case while seamlessly falling back to kernel-level sleeping when thread contention occurs.

Section 8: Lock-Free Concurrency Design Pattern

While locks ensure correctness, they can still introduce bottlenecks due to thread blocking and scheduling overhead. For highly performance-critical structures, developers can use atomic instructions to build **Lock-Free Data Structures** that coordinate threads without ever using explicit locks.

8.1 Example: Lock-Free Concurrent Queue Insertion

The following function shows how to use the `compare&swap` instruction to safely append a new node to the head of a shared singly linked list without using any locks:

```

c
// Lock-Free Queue Enqueue Implementation
void addToQueue(Node *object) {
    do {
        // 1. Read the current head of the queue into a local register
        Node *current_head = M[root]; //

        // 2. Set our new node's next pointer to point to this expected head
        object->next = current_head; //

        // 3. Atomically update the root pointer to our new node IF the head hasn't changed
        // If another thread updated the root in the meantime, compare&swap fails and we retry
    } while (!compare&swap(&root, current_head, object)); //
}

```

By using an optimistic retry loop with `compare&swap`, this pattern guarantees that at least one thread will make forward progress on every iteration, achieving high-throughput concurrency without the risk of deadlocks or priority inversions.

Section 9: Practice Exam Problems with Detailed Solutions

Use these practice problems to test your understanding of Lecture 8.

Question 1: Uniprocessor Interrupt Enable/Disable Placement Analysis

An engineer proposes an alternative implementation for the internal kernel `Acquire()` lock function on a uniprocessor, placing the `enableInterrupts()` call directly before the `sleep()` command to simplify the code:

```

c
void BrokenAcquire() {
    disableInterrupts();
    if (value == BUSY) {
        putThreadOnWaitQueue(current_thread);
        enableInterrupts(); // Alternate placement position
        sleep();
    }
}

```

```

    } else {
        value = BUSY;
        enableInterrupts();
    }
}

```

Provide a detailed interleaving trace demonstrating exactly how this modified function can break the system's correctness guarantees.

Solution:

This alternative placement introduces a critical race condition known as a **Lost Wakeup Fault**, which can lead to permanent system deadlocks.

Consider the following execution timeline involving two threads where Thread A currently holds the lock (`value == BUSY`):

1. **Thread B** calls `BrokenAcquire()` , disables interrupts, discovers that `value == BUSY` , and places its TCB on the lock's wait queue.
2. **Thread B** executes the line `enableInterrupts()` .
3. Right before Thread B can execute the next instruction (`sleep()`), a **hardware timer interrupt fires**, intercepting execution and forcing a context switch to Thread A.
4. **Thread A** completes its work and calls `Release()` .
5. Inside `Release()` , Thread A disables interrupts and checks the wait queue. It finds Thread B on the queue, pops it off, places it on the scheduler's ready queue, and then re-enables interrupts and exits.
6. The scheduler selects **Thread B** to run next.
7. Thread B resumes execution exactly where it left off: it calls `sleep()` .
8. **The Breakdown:** Because Thread B enters the sleep state anyway, it completely misses the wakeup signal that was already issued by Thread A. Thread B will sleep indefinitely on the wait queue, and since it still technically holds the lock state, no other thread can ever acquire the lock, deadlocking that subsystem permanently.

Question 2: Spinlock Cache Contention Analysis

Suppose a four-core multiprocessor system runs four threads that are heavily competing to acquire a single basic spinlock protected by a hardware `test&set` instruction. The lock variable is initially busy (`thelock == 1`).

1. If all four threads are executing a basic spinlock loop (`while (test&set(thelock));`), describe the impact on the shared hardware interconnect bus.
2. Explain how converting the lock to a `test&test&set` architecture (`while (*thelock);`) alters the cache line states and reduces bus overhead.

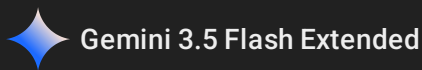
Solution:

1. **Basic Spinlock Impact:** The basic `test&set` instruction is treated as a write operation by the processor hardware, regardless of the value it reads.
 - As a result, every single iteration of the spin loop across all four cores attempts to write to the lock variable's memory location.
 - Under standard cache coherence protocols (like MESI), a write operation requires the core to acquire exclusive ownership of the cache line, forcing it to send invalidation messages to all other cores.
 - This causes the cache line containing `thelock` to constantly bounce back and forth between the four cores' local caches ("cache line ping-ponging"). The shared interconnect bus becomes completely saturated with invalidation and

data transfer traffic, severely degrading the performance of all other memory accesses on the system.

2. **Optimized test&test&set Alteration:** By modifying the lock loop to first check the value using a plain memory read (`while (*thelock);`), we change how the cache lines are managed:

- A plain read instruction only requires shared, read-only access to the cache line.
- This allows the cache line containing `thelock` to reside simultaneously in a Shared (S) state across the local L1/L2 caches of all four cores.
- As long as the lock remains busy, all four cores read the value `1` directly from their own local caches without generating any network or interconnect bus traffic. The bus remains completely free until the lock holder releases the lock, making this design significantly more efficient.



CS162 Operating Systems & Systems Programming

Lecture 9: Synchronization 3: Semaphores, Monitors, and Readers/Writers Problem (Exam-Ready Notes)

Section 1: Review of Hardware Atomic Primitives

To construct dependable high-level synchronization APIs, computer architectures must provide underlying hardware-level operations that run to completion indivisibly. The following four foundational atomic primitives form the baseline for system synchronization:

1.1 Test-and-Set (`test&set`)

- **Hardware Logic:** Reads a value from a memory address, sets the value at that address to `1`, and returns the original value.
- **Conceptual Pseudocode:**

```
c
int test_and_set(int *address) {
    int result = *address; // Read original value
    *address = 1;          // Set value to 1 atomically
    return result;         // Return original value
}
```

1.2 Swap (`swap`)

- **Hardware Logic:** Atomically exchanges the contents of a CPU register with a value in memory.
- **Conceptual Pseudocode:**

```
c
int swap(int *address, int register_value) {
    int temp = *address; // Read current value from memory
    *address = register_value; // Write register value to memory
}
```

```

    return temp;                // Return memory value back to register
}

```

1.3 Compare-and-Swap (`compare&swap`)

- **Hardware Logic:** Compares the contents of a memory location to an expected register value (`reg1`). If they are equal, the memory location is updated with a new value (`reg2`) and returns success. Otherwise, memory remains unchanged and returns failure.
- **Conceptual Pseudocode:**

```

c

int compare_and_swap(int *address, int reg1, int reg2) {
    if (reg1 == *address) {
        *address = reg2; // Put new value into memory
        return SUCCESS;  // Return success status
    } else {
        return FAILURE;  // Memory remains unchanged
    }
}

```

1.4 Load-Linked & Store-Conditional (`ll` & `sc`)

- **Hardware Logic:** Used together to achieve lock-free synchronization across modern RISC architectures. `load-linked` (`ll`) reads a memory location and registers its address in a hardware tracking table. `store-conditional` (`sc`) only writes a new value to that address if no other thread has modified it since the `ll` instruction. If a modification is detected, the store fails.
- **Conceptual Assembly Loop:**

```

assembly

loop:
    ll    r1, M[address]    # Load value and establish hardware link
    movi  r2, 1              # Prepare tentative new value
    sc    r2, M[address]    # Store conditionally; sets r2 to 1 on success, 0 on failure
    beqz  r2, loop           # If the store conditional failed (r2 == 0), retry the loop

```

Section 2: The Semaphore Abstraction

A **Semaphore** is a high-level synchronization variable initialized to a non-negative integer value, offering a clean alternative to the complexity of raw load and store operations.

2.1 Core Mechanics and Primitives

Invented by Edsger Dijkstra, a semaphore exposes an internal integer state through two restricted, atomic system operations:

1. The $P()$ Operation (`down()` , `wait()`)

- **Behavior:** Attempts to decrement the internal semaphore value. If the value is strictly greater than zero, it is decremented, and the thread continues executing immediately. If the value is zero, the calling thread is suspended and placed on the semaphore's internal wait queue until a wakeup signal arrives.
- **Dijkstra Origin:** Named after the Dutch word *Proberen* (meaning "to test").

2. The $V()$ Operation (`up()` , `signal()`)

- **Behavior:** Increments the internal semaphore value. If any threads are currently suspended on the wait queue, the kernel removes one thread and places it onto the scheduler's ready queue to resume execution.
- **Dijkstra Origin:** Named after the Dutch word *Verhogen* (meaning "to increment").

2.2 Critical Invariants and Structural Safety

- **Atomic Enforcement:** The execution of both $P()$ and $V()$ must be fully serialized. If two threads call $P()$ on a semaphore with a value of `1` simultaneously, the internal kernel mechanics ensure that only one thread decrements it to `0` and proceeds, while the second thread blocks.
- **Non-Negative Value Invariant:** A semaphore's value can never drop below zero. This behavior provides a reliable foundation for enforcing scheduling constraints across multiple processes.

2.3 Categorization: Binary vs. Counting Semaphores

- **Binary Semaphore:** Initialized with an integer value of exactly `1`. It behaves identically to a standard mutual exclusion lock (Mutex), guaranteeing that at most one thread can enter a protected critical section at any given time.
- **Counting Semaphore:** Initialized to an arbitrary positive integer value N . This represents a pool of N identical, available resources (such as slots in a memory buffer or open connections), allowing up to N threads to access the resource pool concurrently.

Section 3: Solving the Bounded Buffer (Producer-Consumer) Problem

The Bounded Buffer problem is a classic synchronization challenge where a set of `Producer` threads populate slots in a fixed-size queue, and a set of `Consumer` threads remove items from it.

3.1 Coordination Constraints

To prevent data corruption, a correct solution must enforce three synchronization rules:

1. **Mutual Exclusion:** Only one thread can modify the buffer's internal pointers or data arrays at any given time.
2. **Buffer Overflow Prevention:** A producer thread must block and yield the CPU if it tries to add an item when all buffer slots are full.
3. **Buffer Underflow Prevention:** A consumer thread must block and yield the CPU if it tries to remove an item when the buffer is completely empty.

3.2 Semaphore Synchronization Scheme

We can enforce these rules cleanly by using three separate semaphores, each dedicated to a specific constraint:

- `semaphore mutex = 1;` — A binary semaphore providing mutual exclusion over buffer modifications.
- `semaphore empty = N;` — A counting semaphore initialized to the total buffer capacity N , tracking the number of vacant slots available for producers.
- `semaphore full = 0;` — A counting semaphore initialized to `0`, tracking the number of populated items available for consumers.

3.3 Implementation Code in C

```
c
#include <stdio.h>
#include <pthread.h>
```



```
        consume_item(item);
    }
}
```

Section 4: The Monitor Abstraction

While semaphores are powerful, managing multiple variables spread across complex application paths can easily introduce bugs. To make synchronization more manageable, software systems use an object-oriented paradigm called a **Monitor**.

4.1 Definition and Core Components

A monitor is a high-level programming language construct that encapsulates shared data structures, a mutual exclusion lock, and one or more **Condition Variables** to orchestrate thread interactions.

1. **The Monitor Mutex Lock:** A thread must acquire the monitor's lock before it can execute any internal method or access encapsulated variables. The monitor guarantees that at most one thread can be active inside its boundaries at any time.
2. **Condition Variables (CV):** Explicit synchronization queues inside the monitor that allow threads to safely sleep and release the monitor lock when they cannot make forward progress.

4.2 Condition Variable Operations

A condition variable ignores standard integer increments and instead exposes three atomic operations:

- `wait(&cond, &lock)` : Called when a thread cannot proceed. It performs three actions atomically:
 1. Puts the calling thread to sleep on the condition variable's wait queue.
 2. Releases the monitor's lock so other threads can enter.
 3. On wakeup, it re-acquires the monitor lock before returning to the application code.
- `signal(&cond)` : Wakes up a single thread waiting on that condition variable's queue. If no threads are waiting, the signal is a no-op and is discarded.
- `broadcast(&cond)` : Wakes up all threads currently sleeping on that condition variable's queue.

Section 5: Hoare vs. Mesa Monitor Semantics

When a thread executes a `signal()` operation on a condition variable, a choice must be made: which thread runs next, the signaler or the waiter? This choice is defined by two different monitor semantics.

5.1 Mesa Semantics (Signal-and-Continue)

- **Execution Flow:** When a thread issues a `signal()`, the awakened thread is moved from the condition variable's queue to the scheduler's ready queue. The signaling thread retains the monitor lock and continues running until it exits the monitor or hits its own wait condition.
- **The Guarding `while` Loop Imperative:** Because the signaling thread keeps running, it could modify the shared state before the awakened thread gets a chance to execute. As a result, the awakened thread cannot assume the condition is still true when it resumes. It **must re-evaluate the condition inside a `while` loop** before proceeding:

```
c

while (resource_is_unavailable) {
    cond_wait(&condition, &lock); // Re-checks the condition upon re-acquiring the lock
}
```

- **Production Context:** Mesa semantics are used by modern production systems, including POSIX threads (`pthread`), Linux, Java, and C#.

5.2 Hoare Semantics (Signal-and-Urgent-Wait)

- **Execution Flow:** When a thread issues a `signal()`, the monitor lock is transferred immediately and atomically to the awakened thread. The signaling thread is suspended and placed on an internal "urgent queue," allowing the awakened thread to run immediately without interference.
- **The Guarding `if` Statement:** Because the lock is transferred atomically, no other threads can modify the state in between. The awakened thread is guaranteed that the condition is true when it wakes up, allowing it to use a simple `if` statement:

```
c
if (resource_is_unavailable) {
    cond_wait(&condition, &lock); // Can proceed safely immediately upon wakeup
}
```

- **Production Context:** Hoare semantics are conceptually elegant but introduce significant scheduling overhead, making them rare in production operating system kernels.

Section 6: Deep Dive: The Readers/Writers Problem

The Readers/Writers problem models access to a shared resource, like a database, where multiple threads need access concurrently.

6.1 System Optimization Constraints

To maintain data consistency while maximizing performance, the solution must enforce two rules:

1. **Reader Concurrency:** Multiple reader threads can safely read the database at the same time, as they do not modify the data.
2. **Writer Mutual Exclusion:** Only a single writer thread can access the database at a time. While a writer is active, all other readers and writers must be blocked from entering.

6.2 Monitor Tracking Variables

To implement this cleanly under Mesa semantics, the monitor tracks the system's execution state using four internal integers and two condition variables:

- `AR` (Active Readers): The number of threads currently reading inside the database.
- `WR` (Waiting Readers): The number of reader threads blocked outside, waiting to enter.
- `AW` (Active Writers): The number of writer threads inside the database (can only be 0 or 1).
- `WW` (Waiting Writers): The number of writer threads blocked outside, waiting to enter.
- `cond_t okToRead;` — Condition variable used to suspend reader threads.
- `cond_t okToWrite;` — Condition variable used to suspend writer threads.

6.3 Complete Implementation Code (Mesa Semantics)

The code blocks below implement the Readers/Writers coordination loops using standard `pthread` and Mesa monitor tracking state variables:

```
c
```

```

#include <pthread.h>
#include <stdio.h>

// Monitor State Registers
pthread_mutex_t lock = PTHREAD_MUTEX_INITIALIZER;
pthread_cond_t okToRead = PTHREAD_COND_INITIALIZER;
pthread_cond_t okToWrite = PTHREAD_COND_INITIALIZER;

int AR = 0; // Active Readers
int WR = 0; // Waiting Readers
int AW = 0; // Active Writers
int WW = 0; // Waiting Writers

// --- READER OPERATIONS ---

void Reader_CheckIn() {
    pthread_mutex_lock(&lock);

    // Under Mesa semantics, we must check conditions inside a while loop.
    // A reader must block if a writer is active (AW > 0) or if a writer is waiting (WW > 0).
    // Prioritizing waiting writers (WW > 0) prevents writer starvation.
    while (AW > 0 || WW > 0) {
        WR++; // Increment waiting readers counter
        pthread_cond_wait(&okToRead, &lock); // Sleep and release lock
        WR--; // Decrement waiting readers counter on wakeup
    }

    AR++; // Reader is now active!
    pthread_mutex_unlock(&lock);
}

void Reader_CheckOut() {
    pthread_mutex_lock(&lock);
    AR--; // Reader has finished and left

    // If this was the last active reader, check if any writers are waiting
    if (AR == 0 && WW > 0) {
        pthread_cond_signal(&okToWrite); // Wake up a single waiting writer
    }

    pthread_mutex_unlock(&lock);
}

// --- WRITER OPERATIONS ---

void Writer_CheckIn() {
    pthread_mutex_lock(&lock);

    // A writer must block if any readers are active (AR > 0) or another writer is active (AW > 0)
    while ((AW + AR) > 0) {
        WW++; // Increment waiting writers counter
        pthread_cond_wait(&okToWrite, &lock); // Sleep on the writer condition variable
        WW--; // Decrement waiting writers counter on wakeup
    }
}

```

```

    AW++; // Writer is now active!
    pthread_mutex_unlock(&lock);
}

void Writer_CheckOut() {
    pthread_mutex_lock(&lock);
    AW--; // Writer has finished and left

    // Give priority to waiting writers to prevent starvation
    if (WW > 0) {
        pthread_cond_signal(&okToWrite); // Wake up a single waiting writer
    }
    // If no writers are waiting, wake up all waiting readers concurrently
    else if (WR > 0) {
        pthread_cond_broadcast(&okToRead); // Wake all readers
    }

    pthread_mutex_unlock(&lock);
}

// --- EXECUTION IMAGE WRAPPER ---

void *Reader_Task(void *arg) {
    Reader_CheckIn();
    AccessDatabase(ReadOnly); // Safe concurrent reading
    Reader_CheckOut();
    return NULL;
}

void *Writer_Task(void *arg) {
    Writer_CheckIn();
    AccessDatabase(ReadWrite); // Safe exclusive writing
    Writer_CheckOut();
    return NULL;
}

```

Section 7: Practice Exam Problems with Detailed Solutions

Use these practice problems to test your understanding before the exam.

Question 1: Constructing Semaphores from Monitors

Can we build a fully functional Dijkstra Counting Semaphore using only a Monitor (a single Mutex Lock and one Condition Variable)?

If yes, write out the complete, valid C implementation for the initialization, $P()$, and $V()$ functions, ensuring compliance with Mesa semantics. If no, provide a technical proof detailing why it is mathematically impossible.

Solution:

Yes, it is entirely possible to implement a counting semaphore using a monitor framework. We track the semaphore's non-negative state using an internal integer variable, wrapped inside a `while` loop to comply with Mesa semantics.

```

c

```

```

#include <pthread.h>

typedef struct {
    pthread_mutex_t lock;        // Monitor Lock
    pthread_cond_t cond;        // Condition Variable
    int value;                  // Internal Semaphore State
} monitor_semaphore_t;

// Initialization Routine
void sem_monitor_init(monitor_semaphore_t *s, int initial_value) {
    pthread_mutex_init(&s->lock, NULL);
    pthread_cond_init(&s->cond, NULL);
    s->value = initial_value; // Must be non-negative
}

// P() Operation (Wait / Down)
void sem_monitor_P(monitor_semaphore_t *s) {
    pthread_mutex_lock(&s->lock);

    // Mesa Semantics Requirement: we must use a while loop,
    // as the condition might change before a thread wakes up and re-acquires the lock.
    while (s->value == 0) {
        pthread_cond_wait(&s->cond, &s->lock); // Sleep until a V() signal arrives
    }

    s->value--; // Decrement the internal counter atomically
    pthread_mutex_unlock(&s->lock);
}

// V() Operation (Signal / Up)
void sem_monitor_V(monitor_semaphore_t *s) {
    pthread_mutex_lock(&s->lock);

    s->value++; // Increment the internal counter
    pthread_cond_signal(&s->cond); // Wake up a single waiting thread

    pthread_mutex_unlock(&s->lock);
}

```

Question 2: Starvation Hazards in Readers/Writers Design

Analyze the `Writer_CheckOut()` routine provided in Section 6.3:

```

c

if (WW > 0) {
    pthread_cond_signal(&okToWrite); // Prioritize waiting writers
} else if (WR > 0) {
    pthread_cond_broadcast(&okToRead); // Wake waiting readers
}

```

1. What strategy does this coordination logic use to prioritize threads?

2. What are the starvation risks for reader threads under high write traffic? Explain how you would modify the design to guarantee fairness across both readers and writers.

Solution:

1. **Prioritization Strategy:** This coordination scheme directly **prioritizes writers over readers**. When a writer exits the database, it checks if any writers are waiting (`WW > 0`) before checking for waiting readers. If a writer is found, it signals that writer immediately, ignoring any readers that may have been waiting on the queue.
2. **Starvation Risks:** If the system faces high write traffic where new writer threads continue to arrive and increment `WW` , reader threads will be starved indefinitely. They will remain trapped on the `okToRead` condition variable queue, unable to enter.
3. **The Fix (Fairness Modification):** To guarantee fairness and prevent reader starvation, we can track arrival order using a FIFO timestamp queue or use an integer counter to limit how many consecutive writers can pass through before waiting readers must be served. Alternatively, we can use a state bit flag that blocks arriving writers if reader threads have been waiting on the queue for more than a set threshold.

✦ Gemini 3.5 Flash Extended

CS162 Operating Systems & Systems Programming

Lecture 10: Synchronization 4: Readers/Writers & Scheduling Intro (Pintos Concurrency)

Section 1: High-Level Synchronization Mechanics (Semaphores & Monitors)

1.1 Semaphore Recap: The Counting Semaphore Bounded Buffer

A **Semaphore** is a core synchronization primitive containing an internal non-negative integer value managed via two atomic operations: $P()$ (`sem_wait` , which blocks if the value is 0, then decrements) and $V()$ (`sem_post` , which increments and awakens a sleeping thread).

The classical Bounded Buffer (often called the Coke Machine problem) manages safe communication between `Producer` and `Consumer` threads over a circular queue of fixed size `bufSize` . To enforce synchronization without busy-waiting, the architecture splits coordination across three distinct semaphores:

- `Semaphore fullSlots = 0;` — Tracks the total number of populated items currently available for consumers to drain.
- `Semaphore emptySlots = bufSize;` — Tracks the total number of vacant slots currently available for producers to fill.
- `Semaphore mutex = 1;` — A binary semaphore enforcing mutual exclusion over internal buffer pointer modifications.

```
C
// Bounded Buffer Semaphore Implementation

void Producer(Item item) {
    semaP(&emptySlots); // Decrement empty slots; blocks if machine is full
    semaP(&mutex);       // Acquire exclusive access to the storage queue

    Enqueue(item);      // Critical Section: modify pointer state registers

    semaV(&mutex);       // Release exclusive access
```

```

    semaV(&fullSlots); // Increment full slots; wake up a waiting consumer
}

Item Consumer() {
    semaP(&fullSlots); // Decrement full slots; blocks if machine is empty
    semaP(&mutex);      // Acquire exclusive access to the storage queue

    Item item = Dequeue(); // Critical Section: remove item and advance pointers

    semaV(&mutex);        // Release exclusive access
    semaV(&emptySlots); // Increment empty slots; notify producer space is clear

    return item;
}

```

1.2 Monitors & Condition Variable Core Mechanics

A **Monitor** is a programming paradigm consisting of a mutual exclusion lock and zero or more **Condition Variables** designed to manage concurrent access to shared data.

- **Condition Variable (CV)**: An explicit queue of threads waiting for a specific logical condition to become true inside a critical section. Unlike a semaphore, a condition variable is **memoryless**; it does not maintain an internal counter. If a signal is issued when no threads are waiting, the signal has no effect and is permanently lost.
- **Wait(&lock)** : Atomically releases the monitor lock and places the calling thread to sleep on the CV's queue. When the thread is awakened, it automatically re-acquires the monitor lock before returning from the wait call.
- **Signal()** : Awakens a single thread waiting on the condition variable's queue, if one exists.
- **Broadcast()** : Awakens all threads currently sleeping on the condition variable's queue.
- **The Structural Rule**: A thread **must** hold the associated monitor lock when invoking any condition variable operation.

```

c

// Infinite Synchronized Buffer via Condition Variables
lock buf_lock;           // Initially unlocked
condition buf_CV;        // Initially empty queue
queue storage_queue;     // Actual queue data structure

void Producer(Item item) {
    acquire(&buf_lock);    // Acquire monitor lock
    enqueue(&storage_queue, item); // Critical section: append data node
    cond_signal(&buf_CV);  // Notify a waiting consumer thread
    release(&buf_lock);    // Release monitor lock
}

Item Consumer() {
    acquire(&buf_lock);    // Acquire monitor lock
    while (isEmpty(&storage_queue)) {
        cond_wait(&buf_CV, &buf_lock); // Atomically sleep and release lock if empty
    }
    Item item = dequeue(&storage_queue); // Remove item from head of queue
    release(&buf_lock);    // Release monitor lock
    return item;
}

```

1.3 Deep Architectural Contrast: Mesa vs. Hoare Scheduling

The design of a monitor depends heavily on the scheduling choices made when a thread executes a `Signal()` operation. The core difference between **Mesa** and **Hoare** monitor semantics centers on which thread takes control of the monitor lock immediately after a signal is issued.

1. Hoare Scheduling (Signal-and-Urgent-Wait)

- **Mechanics:** When a thread calls `Signal()`, the signaler immediately gives up the monitor lock and yields the CPU to a waiting thread. The awakened thread runs immediately. Once that awakened thread exits the critical section or blocks again, ownership of the lock and CPU returns back to the original signaler.
- **Condition Verification:** Because the lock is transferred atomically from the signaler to the waiter, no other thread can execute in between. The logical condition is guaranteed to remain true when the waiter wakes up, allowing developers to use a simple `if` statement to check conditions:

```
c
if (isEmpty(&queue)) {
    cond_wait(&buf_cv, &buf_lock); // Wakes up and immediately dequeues
}
```

- **Engineering Trade-offs:** While conceptually intuitive, Hoare scheduling is highly inefficient. It forces multiple back-to-back kernel context switches between the signaler and waiter, degrading system performance.

2. Mesa Scheduling (Signal-and-Continue)

- **Mechanics:** When a thread calls `Signal()`, the signaler retains both the monitor lock and control of the processor. It continues executing until it naturally completes its critical section and releases the lock. The awakened thread is simply placed onto the scheduler's standard ready queue with no special execution priority.
- **Condition Verification:** Because the awakened thread must compete for the lock like any other thread, another thread could "sneak in" and modify the shared state before the awakened thread acquires the lock. As a result, the logical condition may no longer be true upon wakeup, and the thread **must re-verify the condition inside a `while` loop**:

```
c
while (isEmpty(&queue)) {
    cond_wait(&buf_cv, &buf_lock); // Re-checks the condition upon re-acquiring the lock
}
```

- **Engineering Trade-offs:** Mesa monitors are far more efficient and easier to implement than Hoare monitors, as they avoid unnecessary context switches and preserve the signaler's CPU cache states. Nearly all production operating systems and language libraries (including POSIX threads, Java, and C#) utilize Mesa scheduling semantics.

1.4 Bounded Buffer via Monitors

Using Mesa-style monitors, a fully synchronized, bounded circular buffer can be implemented safely using two separate condition variables to avoid buffer overflows and underflows.

```
c
// Bounded Buffer Monitor Configuration (Mesa Semantics)
lock buf_lock = UNLOCKED;           // Monitor entry exclusion lock
condition producer_cv = EMPTY;      // Queue for producers waiting for vacant space
condition consumer_cv = EMPTY;      // Queue for consumers waiting for available data
```



```

void Producer(Item item) {
    acquire(&buf_lock);          // Enter monitor
    while (BufferIsFull()) {
        cond_wait(&producer_CV, &buf_lock); // Sleep if full; releases lock atomically
    }

    Enqueue(item);               // Critical Section: append item

    cond_signal(&consumer_CV);    // Signal a waiting consumer thread
    release(&buf_lock);           // Exit monitor
}

Item Consumer() {
    acquire(&buf_lock);          // Enter monitor
    while (BufferIsEmpty()) {
        cond_wait(&consumer_CV, &buf_lock); // Sleep if empty; releases lock atomically
    }

    Item item = Dequeue();       // Critical Section: pull item

    cond_signal(&producer_CV);    // Signal a waiting producer thread
    release(&buf_lock);           // Exit monitor
    return item;
}

```

Section 2: The Readers/Writers Optimization Paradigm

2.1 Problem Formulation and Motivation

A common systems programming challenge involves managing access to a shared resource, such as a database, that is accessed by two distinct classes of threads: **Readers** and **Writers**.

- **Readers:** Threads that only read data and never modify the underlying structures.
- **Writers:** Threads that both read and modify the shared data.

If the system uses a single global mutual exclusion lock over the entire resource, performance drops significantly because only one thread can access the data at a time, forcing all readers to execute sequentially.

To optimize throughput, a specialized synchronization layout must enforce two core constraints:

1. **Reader Concurrency:** Multiple reader threads must be allowed to access the database simultaneously when no writer is active, maximizing read throughput.
2. **Writer Mutual Exclusion:** Only a single writer thread can access the resource at any given time. While a writer is active, all other readers and writers must be blocked from entering.

2.2 State Register Definitions

To coordinate access safely using Mesa monitors, the system tracks execution status using four state integers and two distinct condition variables:

- `int AR = 0;` — The count of active readers currently accessing the database.
- `int WR = 0;` — The count of waiting readers suspended on the entry condition variable.
- `int AW = 0;` — The count of active writers currently inside the database (can only be 0 or 1).
- `int WW = 0;` — The count of waiting writers suspended on the entry condition variable.

- `condition okToRead;` — Queue where reader threads sleep when access is unsafe.
- `condition okToWrite;` — Queue where writer threads sleep when access is unsafe.
- `lock lock;` — The master mutual exclusion lock protecting updates to these state registers.

2.3 Reader & Writer Code Implementation

```
c

// Readers/Writers Monitor Coordination Interface

void Reader() {
    acquire(&lock); // Secure exclusive access to state variables

    // Safety check: block if a writer is active OR if a writer is waiting.
    // Checking WW > 0 prioritizes waiting writers to prevent writer starvation.
    while ((AW + WW) > 0) { //
        WR++; // Increment waiting readers tally
        cond_wait(&okToRead, &lock); // Sleep and release lock
        WR--; // Decrement waiting readers tally on wakeup
    }

    AR++; // Transition from waiting to active reader
    release(&lock); // Release state lock before accessing data

    // --- Execute Concurrently with Other Readers ---
    AccessDatabase(ReadOnly); //

    acquire(&lock); // Re-acquire state lock for checkout bookkeeping
    AR--; // Terminate active reader status

    // If this was the final active reader and a writer is waiting, wake it up
    if (AR == 0 && WW > 0) { //
        cond_signal(&okToWrite); // Wake up a single waiting writer
    }
    release(&lock); // Final checkout release
}

void Writer() {
    acquire(&lock); // Secure exclusive access to state variables

    // Safety check: block if any active readers OR active writers exist
    while ((AW + AR) > 0) { //
        WW++; // Increment waiting writers tally
        cond_wait(&okToWrite, &lock); // Sleep on writer condition variable
        WW--; // Decrement waiting writers tally on wakeup
    }

    AW++; // Transition from waiting to active writer
    release(&lock); // Release state lock before modifying data

    // --- Execute with Absolute Mutual Exclusion ---
    AccessDatabase(ReadWrite); //

    acquire(&lock); // Re-acquire state lock for checkout bookkeeping
```

```

AW--; // Terminate active writer status

// Prioritize waiting writers over waiting readers to prevent writer starvation
if (WW > 0) { //
    cond_signal(&okToWrite); // Wake up a single waiting writer
} else if (WR > 0) { // If no writers are waiting, wake up all waiting readers
    cond_broadcast(&okToRead); // Wake all readers concurrently
}
release(&lock); // Final checkout release
}

```

2.4 Complete Step-by-Step Simulation Checkpoint Matrix

To verify the execution of this monitor solution, we trace a sequence of thread arrivals: **Reader 1 (R1), Reader 2 (R2), Writer 1 (W1), and Reader 3 (R3)**. The table below tracks the step-by-step changes to the state registers and condition variable queues across the entire execution lifecycle .

Trace Step	Thread Event Context	Active Thread Action	A R	W R	A W	W W	okTo Read Queue	okTo Write Queue	Operational Notes / State Changes
0	Baseline State	System Initialization	0	0	0	0	Empty	Empty	All queues clear; database idle.
1	R1 Arrival	R1 calls <code>acquire(&lock)</code>	0	0	0	0	Empty	Empty	R1 secures master state variables lock.
2	R1 Evaluation	R1 evaluates <code>while (AW+WW > 0)</code>	0	0	0	0	Empty	Empty	Condition evaluates to <code>0</code> . Loops skipped.
3	R1 Entry	R1 executes <code>AR++</code> and <code>release</code>	1	0	0	0	Empty	Empty	R1 marks itself active; leaves state loop.
4	R1 Processing	R1 executes <code>AccessDatabase(ReadOnly)</code>	1	0	0	0	Empty	Empty	R1 actively reads database.
5	R2 Arrival	R2 calls <code>acquire(&lock)</code>	1	0	0	0	Empty	Empty	R2 secures master state variables lock.
6	R2 Evaluation	R2 evaluates <code>while (AW+WW > 0)</code>	1	0	0	0	Empty	Empty	Condition evaluates to <code>0</code> . Loops skipped.
7	R2 Entry	R2 executes <code>AR++</code> and <code>release</code>	2	0	0	0	Empty	Empty	R2 marks itself active; loops clear.

Trace Step	Thread Event Context	Active Thread Action	A R	W R	A W	W W	okTo Read Queue	okTo Write Queue	Operational Notes / State Changes
8	Shared Reading	R1 & R2 access data concurrently	2	0	0	0	Empty	Empty	Concurrent Read Mode Active.
9	W1 Arrival	W1 calls <code>acquire(&lock)</code>	2	0	0	0	Empty	Empty	W1 secures master state variables lock.
10	W1 Evaluation	W1 evaluates <code>while (AW+AR > 0)</code>	2	0	0	0	Empty	Empty	Condition evaluates to <code>2 > 0</code> . W1 must block.
11	W1 Blocked	W1 executes <code>WW++</code> and <code>cond_wait</code>	2	0	0	1	Empty	[W1]	W1 falls asleep; releases monitor lock.
12	R3 Arrival	R3 calls <code>acquire(&lock)</code>	2	0	0	1	Empty	[W1]	R3 secures master state variables lock.
13	R3 Evaluation	R3 evaluates <code>while (AW+WW > 0)</code>	2	0	0	1	Empty	[W1]	Condition <code>1 > 0</code> (due to W1 waiting). R3 must block.
14	R3 Blocked	R3 executes <code>WR++</code> and <code>cond_wait</code>	2	1	0	1	[R3]	[W1]	R3 falls asleep; releases monitor lock.
15	Contented Hold	Readers active; Writer/Reader blocked	2	1	0	1	[R3]	[W1]	R1 & R2 reading; W1 and R3 are queued.
16	R2 Exit	R2 calls <code>acquire(&lock)</code> ; executes <code>AR--</code>	1	1	0	1	[R3]	[W1]	R2 finishes reading; decrements active count.
17	R2 Checkout	R2 evaluates <code>if (AR==0 && WW > 0)</code>	1	1	0	1	[R3]	[W1]	<code>AR == 1</code> (R1 still active). R2 exits.
18	R1 Exit	R1 calls <code>acquire(&lock)</code> ; executes <code>AR--</code>	0	1	0	1	[R3]	[W1]	R1 finishes reading; active readers hit 0.

Trace Step	Thread Event Context	Active Thread Action	A R	W R	A W	W W	okTo Read Queue	okTo Write Queue	Operational Notes / State Changes
19	R1 Checkout	R1 evaluates if (AR==0 && WW > 0)	0	1	0	1	[R3]	[W1]	Conditions match (0==0 and 1>0). Signal issued.
20	W1 Wakeup Call	R1 calls cond_signal(&okToWrite)	0	1	0	1	[R3]	Empty	W1 moved from sleep queue to ready queue.
21	W1 Lock Grab	W1 wakes up; re-acquires master lock	0	1	0	1	[R3]	Empty	W1 exits wait call; re-evaluates entry loop.
22	W1 Loop Exit	W1 drops out of loop; runs WW--	0	1	0	0	[R3]	Empty	W1 decrements waiting writers count.
23	W1 Entry	W1 executes AW++ and release	0	1	1	0	[R3]	Empty	W1 marks itself active; exits monitor.
24	W1 Processing	W1 executes AccessDataBase(Read Write)	0	1	1	0	[R3]	Empty	Writer Exclusive Mode Active.
25	W1 Complete	W1 calls acquire(&lock); executes AW--	0	1	0	0	[R3]	Empty	W1 finishes writing; active writers hit 0.
26	W1 Priority Check	W1 evaluates checkout conditions	0	1	0	0	[R3]	Empty	WW == 0. Cascades to else if (WR > 0) check.
27	W1 Release	W1 calls cond_broadcast(&okToRead)	0	1	0	0	Empty	Empty	W1 wakes up all waiting readers concurrently.
28	R3 Lock Grab	R3 wakes up; re-acquires master lock	0	1	0	0	Empty	Empty	R3 exits wait call; re-evaluates entry loop.
29	R3 Loop Exit	R3 drops out of loop; runs WR--	0	0	0	0	Empty	Empty	R3 decrements waiting readers count.

Trace Step	Thread Event Context	Active Thread Action	A R	W R	A W	W W	okTo Read Queue	okTo Write Queue	Operational Notes / State Changes
30	R3 Entry	R3 executes AR++ and release	1	0	0	0	Empty	Empty	R3 marks itself active; exits monitor.
31	R3 Processing	R3 executes AccessDatabase(ReadOnly)	1	0	0	0	Empty	Empty	R3 actively reads database.
32	R3 Exit	R3 calls acquire(&lock); executes AR--	0	0	0	0	Empty	Empty	R3 finishes reading; active reader's hit 0.
33	Final State	System returns to idle baseline	0	0	0	0	Empty	Empty	All threads finished; system clear.

2.5 Edge Cases, Starvation, and Single CV Analysis

- Reader Starvation Risk:** Because the entry code for readers checks while (AW + WW > 0), readers will block if any writer is active or if any writer is waiting. Consequently, if a continuous stream of writer threads arrives, WW will remain greater than zero, and waiting readers can be starved indefinitely on the okToRead queue.
- Impact of Altering Exit Semantics:** If a developer removes the safety check if (AR == 0 && WW > 0) from the reader exit path and simply calls cond_signal(&okToWrite), the signal could be wasted. If active readers are still inside the database (AR > 0), the awakened writer will wake up, evaluate while (AW + AR > 0), find AR > 0, and immediately go back to sleep, wasting CPU cycles.

The Single Condition Variable (okContinue) Deadlock Hazard

A common mistake is trying to simplify the solution by replacing the two condition variables (okToRead and okToWrite) with a single condition variable, named okContinue, where both readers and writers sleep.

```
c
// Vulnerable Single Condition Variable Design (Deadlock Flag)
void Reader() {
    acquire(&lock);
    while ((AW + WW) > 0) {
        WR++;
        cond_wait(&okContinue, &lock); // Sleep on shared CV
        WR--;
    }
    AR++;
    release(&lock);

    AccessDatabase(ReadOnly);
}
```

```

    acquire(&lock);
    AR--;
    if (AR == 0 && WW > 0) {
        cond_signal(&okContinue); // VULNERABLE SIGNAL!
    }
    release(&lock);
}

```

The Deadlock Scenario

1. Reader 1 (`R1`) enters the monitor and starts reading (`AR = 1`).
2. While `R1` is active, Writer 1 (`W1`) arrives. Because `AR = 1` , `W1` enters the entry loop, sets `WW = 1` , and goes to sleep on the shared `okContinue` CV.
3. Next, Reader 2 (`R2`) arrives. Because `WW = 1` , `R2` enters the entry loop, sets `WR = 1` , and also goes to sleep on the shared `okContinue` CV.
4. When `R1` finishes reading, it executes its checkout path, sees `AR == 0` and `WW == 1` , and calls `cond_signal(&okContinue)` .
5. **The Lost Wakeup Breakdown:** Under standard queue mechanics, the signal is delivered to `R2` instead of `W1` . `R2` wakes up and re-evaluates its entry condition: `while (AW + WW > 0)` . Because `W1` is still waiting (`WW == 1`), the condition is true, so `R2` increments `WR` and **goes back to sleep**.
6. **The Result:** Because `cond_signal` only wakes up a single thread, and that signal was wasted on a thread type that couldn't proceed, `W1` is left sleeping permanently. No other thread will ever be signaled, resulting in a permanent deadlock.
7. **The Fix:** To use a single condition variable safely, developers **must change the signal call to a broadcast call** (`cond_broadcast(&okContinue)`) on all checkout paths. This ensures that all waiting threads wake up and re-evaluate their conditions, preventing lost wakeups at the cost of higher scheduling overhead.

Section 3: Language-Level Synchronization Ecosystems

3.1 The C Language Synchronization Ecosystem

The standard C language provides no native compiler-level synchronization keyword controls; developers must manage synchronization explicitly using system libraries like POSIX threads (`pthread`). This requires carefully tracking all execution paths to ensure locks are released before a function returns.

Non-Local Jump Anomalies (`setjmp` / `longjmp`)

Using low-level stack manipulation functions like `setjmp()` and `longjmp()` introduces severe resource leak hazards in multi-threaded C programs.

```

Call Stack Execution Path:
Proc A —> Proc B (setjmp) —> Proc C (acquire lock) —> Proc D —> Proc E (longjmp)
                                     |
Unrolled Return Path:
Proc B (Execution resumes here; Proc C's lock is leaked!) <—————

```

Visual workflow of a non-local jump lock leak anomaly.

If `Procedure C` acquires a lock and calls `Procedure D` , which then calls `Procedure E` , executing a `longjmp()` inside `Procedure E` will immediately unroll the stack and transfer control back to the saved jump buffer inside `Procedure B` . This completely bypasses the cleanup code in `Procedure C` , leaking the lock and causing all subsequent threads to block permanently.

The Clean Goto Error Cleanup Pattern

To handle multiple sequential locks safely without duplicating cleanup code, systems programmers use a structured `goto` design pattern.

c

```
// Standard Linux Kernel Error Cascading Goto Pattern
int ReadDeviceBuffers() {
    pthread_mutex_lock(&lock1);
    if (EncounterDeviceError()) {
        goto release_lock1_and_return; // Clean single-exit diversion
    }

    pthread_mutex_lock(&lock2);
    if (EncounterBufferError()) {
        goto release_both_and_return; // Cascading cleanup diversion
    }

    // Perform synchronized operations...
    pthread_mutex_unlock(&lock2);
    pthread_mutex_unlock(&lock1);
    return SUCCESS;

release_both_and_return: // Cleans up step 2 allocations
    pthread_mutex_unlock(&lock2);

release_lock1_and_return: // Cleans up step 1 allocations
    pthread_mutex_unlock(&lock1);
    return ERROR_CODE;
}
```

3.2 C++ RAII Lock Guard Mechanics

In languages that support exceptions, like C++, traditional manual locking can easily lead to leaked resources. If an exception is thrown inside a critical section, the stack unwinds immediately, bypassing any manual `release()` calls at the end of the function.

To prevent this, C++ uses an object-oriented pattern called **Resource Acquisition Is Initialization (RAII)** via the `std::lock_guard` template class.

cpp

```
// Safe RAII Synchronization in C++
#include <mutex>

int global_counter = 0; // Shared state resource
std::mutex global_mutex; // Master exclusion lock

void safe_increment() {
    // The constructor automatically calls global_mutex.lock() on creation
    std::lock_guard<std::mutex> lock(global_mutex); //

    global_counter++; // Critical Section

    // When the function exits (via normal return or an exception), the lock object
```



```
// goes out of scope, and its destructor automatically calls global_mutex.unlock().  
}
```

3.3 Python Context Control Contexts

Python provides a similar RAII-like mechanism using the `with` keyword to manage scope and synchronization automatically.

python

```
# Safe Context Synchronization in Python  
import threading  
  
lock = threading.Lock() # Establish primitive object  
  
def update_resource():  
    # The 'with' statement automatically calls lock.acquire() on entry  
    with lock:  
        some_var += 1 # Critical Section  
    # lock.release() is guaranteed to be called however the block is exited
```

3.4 Java Compiler Built-in Monitors

Java integrates the monitor paradigm directly into its language specification using the `synchronized` keyword.

- **Implicit Object Locks:** Every single object instance in Java contains an internal monitor lock and a single implicit condition variable.
- **Method Synchronization:** Marking a method as `synchronized` forces a thread to acquire that object instance's lock before entering the method, and automatically releases it when the method returns or throws an exception.

java

```
// Native Java Monitor Pattern  
class Account {  
    private int balance; // Encapsulated state variable  
  
    public Account(int initialBalance) {  
        this.balance = initialBalance; //  
    }  
  
    // Automatically acquires object lock on entry; releases on exit  
    public synchronized void deposit(int amount) {  
        balance += amount; // Critical Section  
    }  
  
    public synchronized int getBalance() {  
        return balance; // Protected read access  
    }  
}
```

To implement condition variable sleep and wakeup paths within a Java synchronized block, developers use built-in object methods:

- `void wait();` — Equivalent to `cond_wait()` ; releases the object lock and sleeps.
- `void notify();` — Equivalent to `cond_signal()` ; wakes up a single thread.
- `void notifyAll();` — Equivalent to `cond_broadcast()` ; wakes all waiting threads.

Section 4: Advanced Monitor Reconstruction from Semaphores

While constructing semaphores from monitors is straightforward, building condition variables out of semaphores is significantly harder because their underlying mechanics are fundamentally different.

4.1 Commutative vs Memoryless Variance

- **Semaphores are Commutative:** The final state of a semaphore depends on the balance of operations, not their exact order. Calling `semaV()` followed by `semaP()` leaves the semaphore count unchanged, and the calling thread does not block. This means semaphores preserve state history.
- **Condition Variables are Memoryless:** Condition variables are completely non-commutative and do not preserve state history. If a thread calls `Signal()` when the wait queue is empty, the signal is dropped immediately. If another thread calls `Wait()` a moment later, it blocks indefinitely, oblivious to the past signal.

4.2 The Slip-in Race Condition

A common, broken attempt at building a condition variable from semaphores involves simply releasing the lock and calling `semaP` :

```
c
// Broken Condition Variable Implementation from Semaphores
void Wait(Lock *thelock, Semaphore *thesema) {
    release(thelock); //
    semaP(thesema);   // VULNERABLE STEP: Thread blocks here
    acquire(thelock); //
}

void Signal(Semaphore *thesema) {
    semaV(thesema);   //
}
```

- **The Failure Mechanism:** This design creates a critical **Slip-In Race Condition**. After the waiter releases the monitor lock but *before* it can execute `semaP(thesema)` , a context switch can occur. A signaling thread can slip in, acquire the monitor lock, perform its updates, and call `Signal()` , which executes `semaV(thesema)` .
- Because semaphores have memory, this increments the semaphore count to `1` . When the waiter resumes execution and finally calls `semaP(thesema)` , it consumes that history token and drops straight through without blocking, completely violating condition variable wait semantics.

4.3 Correct Monitor Construction Code (Mesa Semantics)

To implement a Mesa-style condition variable using semaphores correctly, the system must explicitly track the number of waiting threads to ensure signals are only delivered when waiters are active, preventing slip-in bugs.

```
c
// Correct Monitor Condition Variable from Semaphores
typedef struct {
    int waiter_count;           // Explicit state variable to track active sleepers
    Semaphore waiter_sema;      // Semaphore acting as the thread sleep block channel
    pthread_mutex_t count_lock; // Private internal lock protecting waiter_count
} verified_cond_t;

void cond_verify_init(verified_cond_t *cv) {
```

```

cv->waiter_count = 0;
sema_init(&cv->waiter_sema, 0); // Initialized to 0 so callers always block
pthread_mutex_init(&cv->count_lock, NULL);
}

void cond_verify_wait(verified_cond_t *cv, pthread_mutex_t *monitor_lock) {
    // 1. Secure exclusive access to the waiter count and increment it
    pthread_mutex_lock(&cv->count_lock);
    cv->waiter_count++;
    pthread_mutex_unlock(&cv->count_lock);

    // 2. Safely release the outer monitor lock to allow other threads to enter
    pthread_mutex_unlock(monitor_lock);

    // 3. Block on the internal semaphore; guaranteed to park the thread
    semaP(&cv->waiter_sema);

    // 4. Upon wakeup, re-acquire the master monitor lock to restore invariants
    pthread_mutex_lock(monitor_lock);
}

void cond_verify_signal(verified_cond_t *cv) {
    pthread_mutex_lock(&cv->count_lock);

    // Memoryless Guard: only execute a wakeup if waiters actually exist.
    // If the queue is empty, the signal is safely discarded as a no-op.
    if (cv->waiter_count > 0) {
        cv->waiter_count--; // Decrement count before waking the thread
        semaV(&cv->waiter_sema); // Wake up exactly one thread safely
    }

    pthread_mutex_unlock(&cv->count_lock);
}

```

Section 5: Core Kernel Architecture, Interrupt Processing, & Scheduling

5.1 CPU Thread Multiplexing and the Dispatch Loop

The operating system core runs an infinite loop—called the **Dispatch Loop**—to manage thread execution and schedule tasks onto the CPU:

```

c

// Core Dispatch Loop Execution Logic
void DispatchLoop() {
    while (TRUE) {
        Thread *nextThread = ChooseNextThread(); // Select thread based on scheduling policy
        Thread *curThread = GetActiveThread();

        SaveStateOfCPU(curThread->TCB); // Save context registers to old TCB
        LoadStateOfCPU(nextThread->TCB); // Load next thread context into CPU registers

        RunThread(nextThread); // Yield physical core control to user thread
    }
}

```

```
}
```

Transferring Control to User Space

When the kernel scheduler calls `RunThread()`, it loads a thread's saved register context directly into the physical CPU core and jumps to its saved Program Counter (`PC`).

At this exact instant, the operating system kernel becomes completely inactive on that core. The running thread has absolute control over the processor until a **Kernel Crossing** event occurs, which can be triggered by internal events (like a system call or voluntary `yield()`) or external events (like a hardware timer interrupt).

5.2 The x86 Hardware Context Switch Primitives

To manage transitions between user and kernel execution spaces safely, the x86 architecture implements strict hardware-enforced protection boundaries.

- **Privilege Levels:** The processor enforces distinct execution modes, ranging from Privilege Level 3 (PL 3 / User Mode) to Privilege Level 0 (PL 0 / Kernel Mode / Supervisor Mode).
- **The Task-State Segment (TSS):** A dedicated hardware structure in kernel memory that tracks privilege execution frames. The TSS stores the target kernel-space stack pointer (`SS:ESP`) used whenever a thread traps into the kernel from user space.
- **The Interrupt Descriptor Table (IDT):** A system indirection table containing 256 explicit exception slots. Each slot maps a specific interrupt or exception ID to the exact address of its kernel handler routine, preventing user applications from jumping to arbitrary locations in kernel memory.

5.3 Pintos Combined TCB and Stack Topography

The Pintos real-world learning kernel implements an elegant allocation layout to keep context switches fast and efficient:

- **Unified Page Object:** For every active process or thread, Pintos allocates a single, contiguous **4 KB page of physical memory** that contains both the Thread Control Block (TCB) and the kernel execution stack.
- **Layout Structure:** The actual Thread Control Block structure (storing metadata like the thread status, priority level, name, and thread ID `tid`) sits at the very bottom (lowest memory address) of the 4 KB page. The remainder of the page is used as the thread's **Kernel Stack**, which grows downwards from the very top of the page toward the TCB.
- **The Magic Number Guard:** Because the stack grows down toward the TCB metadata, a deeply nested function call or massive local array allocation can overflow the stack and overwrite the TCB structure, corrupting critical state data. To catch this, the very first field in the TCB struct contains a unique identifier called the **magic #** . The kernel checks this field during scheduling choices; if the value does not match the expected magic constant, the kernel immediately flags a stack overflow fault and halts the system to prevent silent corruption.

5.4 Complete Workflow Trace of a Pintos Timer Interrupt

When the physical hardware timer fires, the system executes a precise sequence of steps to handle the interrupt, update metrics, and trigger a context switch if needed:

[Active User-Space Thread Instruction Stream (PL 3)]

Asynchronous Hardware Timer Interrupt Line Asserts (0x20)

[Hardware State Save & Privilege Transition (PL 3 -> PL 0)]

- CPU reads the Task-State Segment (TSS) to locate the kernel stack pointer.
- Switches CPU execution context to the target kernel stack line.
- Pushes user state registers ('SS', 'ESP', 'EFLAGS', 'CS', 'EIP') onto kernel stack.
- Flips Privilege Mode to PL 0; branches to target handler in IDT slot 0x20.

[Low-Level Assembly Vector Ingestion ('stubs.S')]

- 'intr20_stub()' pushes the raw interrupt number ('0x20') onto the stack.
- Jumps to the unified kernel entry assembly block 'intr_entry'.
- 'intr_entry' saves all remaining general-purpose registers as an explicit frame.
- Sets up the protected kernel segment environment; calls C function 'intr_handler'.

[Kernel Dispatch and Tracking Management ('interrupt.c')]

- 'intr_handler()' parses the frame, logs metrics, and acknowledges the hardware IRQ.
- Dispatches the request to the registered timer callback 'timer_interrupt()' inside 'timer.c'.

[Operating System Metrics & Quantum Accounting ('timer.c')]

- 'timer_interrupt()' increments the system global tick counter.
- Invokes 'thread_tick()', which updates execution counters for the current thread.
- If the running thread's allocation quantum has expired, it sets a global internal 'yield' flag.

[Relinquishing and Thread Rescheduling ('thread.c')]

- On the return path from the interrupt, the kernel checks the 'yield' flag and calls 'thread_yield()'.
- 'thread_yield()' marks the active thread as 'READY' and pushes its TCB onto the ready list.
- Invokes 'schedule()' to select the next thread to run from the ready queue.
- 'schedule()' calls the assembly routine 'switch_threads()' inside 'switch.S'.

[Low-Level Register Context Switch ('switch.S')]

- 'switch_threads' saves the current kernel registers onto the active kernel stack.
- Updates the CPU's stack pointer register ('ESP') to point to the new thread's kernel stack.
- Pops the new thread's saved register context off its stack, restoring its hardware state.
- Returns from the new thread's historical call to 'switch_threads', resuming its kernel path.

[Returning to User-Space Execution Space ('iret')]

- The system unrolls back through 'intr_exit', which restores the remaining general registers.
- Executes the privileged '**iret' (Interrupt Return)** assembly instruction.
- 'iret' pops the saved user context ('CS:EIP', 'SS:ESP', 'EFLAGS') off the stack.
- Resets the privilege level back to PL 3, resuming the target process's user instruction stream.

5.5 Dennis Ritchie Context Exploration

The historical complexity of low-level context-switching code is perfectly captured by a famous, ironic quote written by UNIX co-creator Dennis Ritchie in the source code for early **UNIX Version 6** (`slp.c`):

C

```
/*
 * If the new process paused because it was swapped out,
 * set the stack level to the last call to savu(u_ssav).
 * This means that the return which is executed immediately
 * after the call to aretu actually returns from the last
 * routine which did the savu.
```

```
*/
```

```
/* You are not expected to understand this. */
```

The Systems Architecture Context

In early PDP-11 computer architectures, the `savu` and `aretu` functions were the historical equivalents of modern context-switching and stack restoration routines.

Because the system had to manually manipulate CPU register offsets and activation frames to trick the hardware into returning to a completely different thread's call stack, the assembly instructions were incredibly dense and counter-intuitive. Ritchie added this comment as a humorous warning to developers, highlighting that context-switching involves low-level hardware manipulations that are notoriously difficult to reason about on first glance.

5.6 CPU Scheduling Classifications & Criteria

Once the context-switching mechanics are in place, the operating system must implement a scheduling policy to decide exactly *which* thread should be selected from the ready queue next.

A scheduler balances four primary performance goals, which can conflict depending on the active workload:

1. **Minimize Response Time:** Optimization vector dedicated to user-facing applications (such as text editors or interactive web browsers). It measures the latency from when a user inputs a command to when the system produces its initial visual response, prioritizing human interaction over raw throughput.
2. **Maximize Throughput:** Optimization vector dedicated to heavy computational and background batch processing tasks. It measures the total amount of computational work completed per unit of time, minimizing the performance overhead of context switches to maximize raw instruction processing.
3. **Fairness:** Policy directive ensuring that every active thread receives an equitable share of the shared CPU resource over time, preventing lower-priority background threads from being starved or resource-starved.
4. **Predictability:** Policy directive dedicated to specialized **Real-Time Systems**. It guarantees that critical software operations (such as automated flight controls or industrial robotics loops) always execute and complete within strict, predictable time windows, preventing system failures.

Section 6: High-Yield Concept Review (Exam Study Guide)

Before the exam, make sure you understand these core concepts from Lecture 10:

1. Condition Variable Memory Variance

- Semaphores have memory and preserve state history; condition variables are memoryless. A `Signal()` call issued on an empty CV queue is lost immediately.

2. The Mesa Loop Requirement

- Under Mesa semantics, threads awakened by a signal are placed on the standard ready queue and must compete for the lock. Because other threads can run first and modify the state, the waiting condition **must** be re-checked inside a `while` loop.

3. Readers/Writers Priority Layout

- To prevent writer starvation, the entry path blocks arriving readers if a writer is currently active or if a writer is waiting (`while (AW + WW > 0)`). When a writer exits, it checks `if (WW > 0)` to prioritize waiting writers before waking up waiting readers.

4. Single Condition Variable Risk

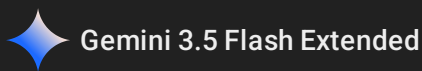
- Replacing specialized condition variables with a single shared CV (`okContinue`) can lead to deadlocks if a `cond_signal` wakes up a thread type that cannot currently proceed. To use a single CV safely, you **must use** `cond_broadcast` on all wakeup paths.

5. Pintos 4KB Thread Layout

- In Pintos, each thread is allocated a single 4KB page combining both the TCB (at the bottom) and the kernel stack (growing downwards from the top). Stack overflows are caught by validating a unique `magic #` constant located at the start of the TCB structure.

6. The User-to-Kernel Pipeline Sequence

- Hardware interrupts step through a strict execution path: **Hardware Vector Line** → **IDT Vector Index Lookup** → **Assembly Wrapper** (`stubs.S`) → **Unified Interrupt Handler** (`interrupt.c`) → **Timer Callback** (`timer.c`) → **Scheduler** (`switch_threads`) (`switch.S`).



CS162 Operating Systems & Systems Programming

Lecture 11: Scheduling 1: Concepts and Classic Policies (Comprehensive Exam Notes)

Section 1: Mesa Monitor Architecture & Pintos Thread Layout Review

1.1 Algorithmic Framework of Mesa Monitor Programs

An operating system uses the **Monitor** abstraction as a primary paradigm to manage concurrent access to shared data structures. A Mesa monitor combines a mutual exclusion lock with one or more condition variables.

The structural pattern of a Mesa monitor relies on synchronization logic where threads check state variables, wait if a resource is unavailable, and signal when a state change allows other threads to proceed.

The standard programmatic implementation of the Mesa monitor pattern is structured as follows :

```
c

/* Standard Mesa Monitor Synchronization Design Pattern */

void Monitor_Wait_Routine() {
    acquire(&lock); // Establish mutual exclusion over monitor state variables

    // Critical Mesa Semantic: The condition MUST be verified inside a while loop.
    // When a thread is woken up by signal(), it is simply placed on the ready queue.
    // Another thread could be scheduled first and change the state variables,
    // making the condition false again by the time the waiter executes.
    while (need_to_wait) {
        condvar.wait(&cond, &lock); // Atomically release lock and go to sleep
        // Upon waking, the thread automatically re-acquires the lock before returning
    }
}
```

```

    // Check and/or update state variables safely
    UpdateStateVariables();

    release(&lock); // Relinquish monitor control
}

void Monitor_Signal_Routine() {
    acquire(&lock); // Establish mutual exclusion

    // Modify shared state variables to eliminate the wait condition
    ModifyStateVariables();

    // Under Mesa semantics, the signaler keeps the lock and the processor
    // The waiter is placed on the ready queue with no special priority
    condvar.signal(&cond); // Wake up at most one waiter, if any exist

    release(&lock); // Release monitor control
}

```

1.2 Pintos / x86 Multi-Threaded Kernel Single Thread Process Layout

In the Pintos operating system running on x86 hardware, every active user process or thread is directly bound to an associated kernel thread.

This execution context is implemented using a unified, fixed-size hardware structure:

- **The 4KB Allocation Boundary:** Each kernel thread is described by a single **4KB page memory object**. This single page co-locates both the Thread Control Block (TCB) and the kernel execution stack.
- **The Bottom Element (TCB):** The Thread Control Block sits at the lowest physical address of the 4KB page allocation. It tracks vital thread parameters, including:
 - **magic #** : A guard value positioned at the edge of the TCB struct to detect stack overflows. If the kernel stack pointer moves past its boundary, it overwrites this magic number, allowing the kernel to catch the violation before scheduling the thread.
 - **list** : Kernel list pointers used to insert the thread into scheduler queues (such as the **ready_list** or device wait lists).
 - **priority** : An integer value tracking the thread's scheduling precedence.
 - **stack** : A pointer tracking the top of the kernel stack when the thread is suspended.
 - **status** : The current execution lifecycle state (e.g., **RUNNING** , **READY** , **BLOCKED**).
 - **tid** : The unique Thread Identifier integer.
- **The Top Element (Kernel Stack):** The thread's kernel execution stack occupies the remainder of the 4KB page, growing downwards from the top of the page toward the TCB.
- **Registers:** The processor maps execution contexts using hardware registers, such as the Program Counter (**PC**) and Stack Pointer (**SP**).

Section 2: x86 Privilege Escalation, Kernel Crossings, & Interrupt Processing

Operating systems isolate applications within user space and handle privileged hardware operations within the kernel using strict hardware boundaries.

2.1 Hardware Execution Modes

- **User Mode (Privilege Level 3):** Applications execute instructions with restricted access. The CPU uses user-space stacks (`ss:esp`) and maps virtual memory using the Page Table Base Register (`PTBR` or register `CR3`). Direct access to physical memory blocks or hardware control structures is blocked.
- **Kernel Mode (Privilege Level 0):** The CPU runs with unrestricted supervisor privileges. It operates on protected kernel stacks while the user thread's state is preserved, standing by until execution returns to user space.
- **Pure Kernel Threads:** These specialized execution contexts run entirely within kernel space at `PL: 0` . They execute using a small stack within their thread structure and have no corresponding user-mode component.

2.2 Privilege Transition Lifecycle (`User` → `Kernel`)

When a user application triggers a system call, trap, or hardware interrupt, the CPU executes a hardware-enforced transition sequence:

1. **Vector Indexing:** The interrupt or trap indexes into a system lookup table called the **Interrupt Vector Table** (also known as the Interrupt Descriptor Table or IDT). Each slot in this table maps to a specific exception type (such as vector `0x20` for the system timer).
2. **Stack Switching:** The x86 hardware reads the **Task-State Segment (TSS)** to find the kernel-space stack pointer (`ss:esp`) assigned to the incoming privilege level (`PL 0`).
3. **Context Preservation:** The CPU pushes the suspended user-space stack registers (`old SS:ESP`) onto the newly activated kernel stack. It then pushes the user execution state, including the flags register and instruction pointer (`old EFLAGS` , `CS:EIP`), alongside any optional hardware error codes.
4. **Handler Branching:** The CPU sets the code segment and instruction pointer (`CS:EIP`) to point to the kernel target handler address defined in the IDT slot, shifting execution to kernel space.

2.3 Pintos Interrupt Vector Ingestion Routing (`stubs.S` & `interrupt.c`)

In Pintos, low-level trap and interrupt vectors are routed through a combination of assembly stubs and generic C dispatch handlers:

```
assembly

# Conceptual low-level assembly layout inside stubs.S

.global intr20_stub
intr20_stub:
    push $0x20          # 1. Push the unique hardware interrupt vector ID onto the stack
    jmp intr_entry      # 2. Jump to the unified kernel entry point wrapper

intr_entry:
    # 3. Save all general-purpose registers as an explicit execution frame
    pusha
    push %ds; push %es; push %fs; push %gs

    # 4. Initialize the protected kernel execution segment environment
    mov $0x10, %ax; mov %ax, %ds; mov %es, %ax

    # 5. Call the generic C handler, passing a pointer to the saved frame on the stack
    pushl %esp
    call intr_handler    # Transfers execution to interrupt.c

.global intr_exit
intr_exit:
    # 6. Restore the preserved register context and return from the interrupt
```

```

popa
iret                # Executes iret instruction to return to user space

```

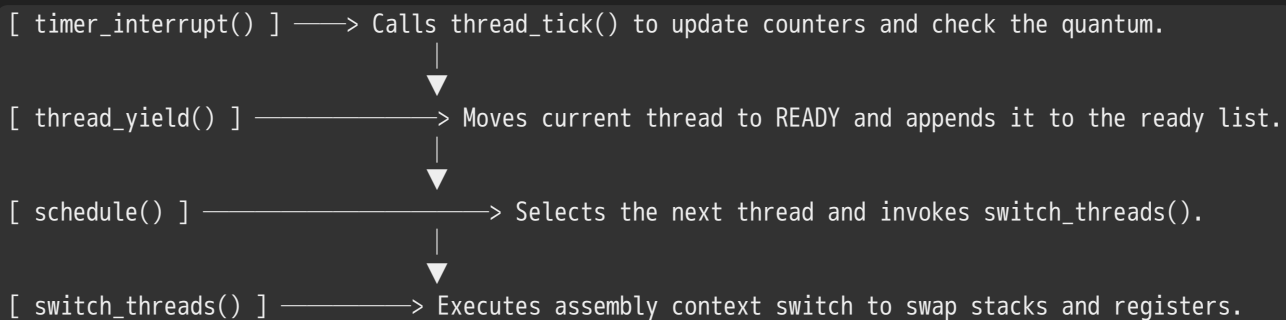
Inside `interrupt.c`, the generic `intr_handler(struct intr_frame *frame)` function processes the incoming request:

1. **Classification & Triage:** It parses the frame vector ID, classifies the interrupt type, and acknowledges the hardware Interrupt Request (IRQ) line.
2. **Polymorphic Dispatch:** It forwards the frame to the registered callback handler. For a timer interrupt (`0x20`), control jumps directly to `timer_interrupt()` inside `timer.c`.
3. **Yield Verification:** On the return path, the handler checks if a context switch is required. If the active thread's time slice has expired, it invokes `thread_yield()` to switch threads.

Section 3: Pintos Rescheduling Framework & Historical Context Switches

3.1 The Thread Rescheduling Mechanism

Pintos manages thread execution states through a structured sequence of function calls during a timer interrupt:



- **thread_tick(void)** : Increments the active thread's runtime counters. If the current thread has exhausted its allocated time quantum, the function sets an internal `yield` flag to signal that preemption is required.
- **thread_yield(void)** : Executed on the return path from an interrupt handler if preemption was flagged. It changes the active thread's status back to `READY`, appends its TCB back onto the system `ready_list`, and invokes `schedule()` to select the next thread.
- **schedule(void)** : Runs the kernel's scheduling algorithm to select the next thread to run. It changes the selected thread's status to `RUNNING` and calls the low-level assembly routine `switch_threads` to swap execution contexts.
- **switch_threads(struct thread *cur, struct thread *next)** : A low-level assembly routine in `switch.S` that saves the active registers onto the current thread's kernel stack, updates the CPU's stack pointer register (`SP`) to point to the next thread's stack, and restores the new thread's saved register context.

3.2 Historical Perspective: Dennis Ritchie's Unix V6 Context Switch Context

The underlying complexity of low-level context switching is highlighted by a famous comment written by Dennis Ritchie in the source code of early **Unix V6** (`slp.c`):

```

c
/*
 * If the new process paused because it was
 * swapped out, set the stack level to the last call
 * to savu(u.ssav).
 * This means that the return which is executed immediately
 * after the call to aretu
 * actually returns from the last routine which did
 * the savu.

```

```
*  
* You are not expected to understand this.  
*/
```

In early Unix systems, `savu` and `aretu` performed the historical equivalent of saving and restoring thread stack pointers during a context switch.

Because the code manually manipulated activation frames to make the processor return to a completely different function context, the execution flow was incredibly counter-intuitive. Ritchie added the famous warning "*You are not expected to understand this*" as an ironic acknowledgement of how difficult it can be to reason about assembly-level stack manipulations.

Section 4: Administrivia & Course Academic Integrity Policies

4.1 Midterm I Overview

- **Grading Matrix:** Midterm I grading is actively underway, and official solutions will be posted to the course resources page shortly.
- **Exam Assessment:** The exam was long, and students found the coding and implementation problems particularly challenging. Reviewing and understanding the solutions to Problems 3 and 5 is highly recommended to improve code comprehension and architectural tracing skills.
- **Class Schedule:** No formal lecture is held on the day of the exam, and class time is repurposed for extended office hours.

4.2 Large Language Model (LLM) Ban Enforcement Policy

The use of **Large Language Models (LLMs)** is **strictly forbidden** in all dimensions of CS162.

```
[ Submissions Table ] —> [ Structural Similarity Engine ] —> Cross-Generational History  
                                                                —> Public Code Solutions  
                                                                —> AI Corpus Signatures
```

To protect academic integrity and verify student competence, the course staff enforces a strict verification ecosystem:

- **Multi-Generational Similarity Scanning:** Every project submission is verified against an extensive repository containing solutions from prior semesters, public code repositories, and AI-generated code baselines.
- **Policy Enforcement:** Submitting AI-generated code violates course policy. Violations trigger formal academic disciplinary actions, as detailed on the course overview page.

Section 5: Foundations of Scheduling & The CPU-I/O Burst Execution Model

5.1 Introduction to Scheduling

A core responsibility of an operating system is **Scheduling**: deciding which threads are given access to shared resources from moment to moment. While typically evaluated in terms of CPU time allocations, scheduling principles apply equally to other shared system resources, such as network bandwidth allocation or persistent disk access.

5.2 The CPU-I/O Burst Execution Model

The design of scheduling algorithms is driven by the physical reality that programs alternate between phases of computation and hardware waits:

- **CPU Burst:** A phase where the thread executes instructions sequentially on the processor core (such as loops, data updates, or memory calculations).
- **I/O Burst:** A phase where the thread blocks to wait for a hardware device operation to complete (such as reading a file from disk or waiting for a network packet).

- **The Scheduling Context:** A scheduling decision is made whenever the active thread finishes a CPU burst and blocks on an I/O request, freeing up the processor to run another thread. Under a time-slicing system, the operating system can also use timer interrupts to preempt a thread mid-burst, forcing it to yield the CPU before its computation finishes.
- **Temporal Locality and Distribution:** Real-world workloads are heavily **weighted toward small bursts**. Most threads execute brief computational phases before quickly blocking on an I/O operation or a user interaction. Long, continuous CPU bursts are rare and typically isolated to heavy numerical or background rendering tasks.

5.3 Core Optimization Criteria

An operating system balances several conflicting metrics when allocating CPU time:

1. **Minimize Response Time:** The elapsed time required to complete a specific operation or job. This metric represents what the user experiences, such as the delay to echo a keystroke in a text editor or the time required to compile a program. Real-time systems face strict deadlines where tasks must finish within specific windows imposed by the physical world.
2. **Maximize Throughput:** The total number of operations or jobs completed per unit of time. Throughput is closely related to response time but has different optimization requirements. For example, minimizing average response time often requires frequent context switching to keep systems responsive, which introduces performance overhead that reduces total throughput. Maximizing throughput requires minimizing context-switching overhead and keeping hardware resources highly utilized.
3. **Fairness:** Ensuring that the CPU is shared among competing users or processes in an equitable way. Fairness often conflicts with average response time optimizations; a system can regularly achieve a lower average response time across all tasks by intentionally choosing to be less fair to long-running background jobs.

Section 6: Classic Scheduling Policies: FCFS vs. Round-Robin

6.1 First-Come, First-Served (FCFS) / FIFO Scheduling

First-Come, First-Served is a non-preemptive scheduling policy where the thread that requests the CPU first retains control until it either blocks or runs to absolute completion.

Mathematical Trace Example 1 (Standard Arrival)

Consider three processes arriving in the system in the exact order P_1, P_2, P_3 at time index 0:

- P_1 : Burst duration = 24 ms
- P_2 : Burst duration = 3 ms
- P_3 : Burst duration = 3 ms

The resulting Gantt chart traces execution as follows:

Gantt Chart Layout:



We evaluate the performance metrics for this schedule:

- **Waiting Times:** $P_1 = 0$ ms; $P_2 = 24$ ms; $P_3 = 27$ ms
- **Average Waiting Time:** $AWT = \frac{0+24+27}{3} = 17.0$ ms
- **Completion Times:** $P_1 = 24$ ms; $P_2 = 27$ ms; $P_3 = 30$ ms
- **Average Completion Time:** $ACT = \frac{24+27+30}{3} = 27.0$ ms

Mathematical Trace Example 2 (Altered Arrival Order)

Now consider the same three processes, but suppose they happen to arrive in the alternative order P_2, P_3, P_1 at time index 0. The Gantt chart shifts:

Gantt Chart Layout:



Re-evaluating our performance metrics:

- **Waiting Times:** $P_1 = 6$ ms; $P_2 = 0$ ms; $P_3 = 3$ ms
- **Average Waiting Time:** $AWT = \frac{6+0+3}{3} = 3.0$ ms
- **Completion Times:** $P_2 = 3$ ms; $P_3 = 6$ ms; $P_1 = 30$ ms
- **Average Completion Time:** $ACT = \frac{3+6+30}{3} = 13.0$ ms

The Convoy Effect Fault

These examples demonstrate that FCFS performance varies wildly based on the exact order tasks are submitted.

Because FCFS is non-preemptive, it suffers from a performance bottleneck known as the **Convoy Effect**: a long, CPU-bound process can monopolize the processor, forcing a long line of short, I/O-bound processes to sit completely idle in the ready queue. This behaves like a supermarket line where a customer buying a single carton of milk is trapped behind a large cart full of items, dragging down average response times across the entire system.

6.2 Round-Robin (RR) Preemptive Scheduling

To protect short jobs from the convoy effect, **Round-Robin** scheduling introduces preemption. Each thread is allocated a small, fixed window of CPU time called a **Time Quantum** (or time slice) q , typically ranging between 10 and 100 milliseconds.

When a thread's time slice expires, a hardware timer interrupt intercepts execution, and the kernel moves the thread to the back of a First-In-First-Out ready queue to let the next thread run.

Invariant Allocation Guarantees

If there are n active processes waiting in the ready queue with a configured time quantum q :

- Each process receives a strict, equitable $1/n$ share of the total CPU time.
- The CPU time is allocated in continuous chunks of at most q time units.
- No individual process ever waits more than $(n - 1)q$ time units before its next execution slice.

Mathematical Trace Example 3 (Round Robin Processing)

Consider four processes arriving simultaneously at time 0, executed under a Round-Robin policy with a time quantum $q = 20$ ms:

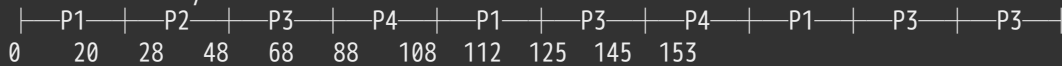
- P_1 : Burst duration = 53 ms
- P_2 : Burst duration = 8 ms
- P_3 : Burst duration = 68 ms
- P_4 : Burst duration = 24 ms

The system steps through time slices, tracking the remaining burst requirements for each process:

- Interval 0-20 : P_1 runs for 20 ms (requires 33 ms remaining).
- Interval 20-28 : P_2 runs for 8 ms and terminates cleanly.
- Interval 28-48 : P_3 runs for 20 ms (requires 48 ms remaining).
- Interval 48-68 : P_4 runs for 20 ms (requires 4 ms remaining).

- Interval 68-88 : P_1 returns to run for 20 ms (requires 13 ms remaining).
- Interval 88-108 : P_3 returns to run for 20 ms (requires 28 ms remaining).
- Interval 108-112 : P_4 returns to run for its final 4 ms and terminates.
- Interval 112-125 : P_1 returns to run for its final 13 ms and terminates.
- Interval 125-145 : P_3 runs for 20 ms (requires 8 ms remaining).
- Interval 145-153 : P_3 runs for its final 8 ms and terminates.

Gantt Chart Layout:



We calculate the precise process waiting times by subtracting execution intervals from completion marks:

- Waiting Time P_1 :** $(68 - 20) + (112 - 88) = 72$ ms
- Waiting Time P_2 :** $(20 - 0) = 20$ ms
- Waiting Time P_3 :** $(28 - 0) + (88 - 48) + (125 - 108) = 85$ ms
- Waiting Time P_4 :** $(48 - 0) + (108 - 68) = 88$ ms
- Average Waiting Time:** $AWT = \frac{72+20+85+88}{4} = 66.25$ ms
- Completion Times:** $P_1 = 125$ ms; $P_2 = 28$ ms; $P_3 = 153$ ms; $P_4 = 112$ ms
- Average Completion Time:** $ACT = \frac{125+28+153+112}{4} = 104.5$ ms

6.3 Choosing the Optimal Time Quantum

Selecting the correct time quantum requires carefully balancing application responsiveness against system overhead:

- Quantum is configured too large ($q \rightarrow \infty$):** The Round-Robin policy degrades into standard non-preemptive First-Come, First-Served scheduling, making short tasks vulnerable to convoy effects again.
- Quantum is configured too small ($q \rightarrow 0$):** Total system throughput drops sharply due to high overhead. Because the CPU spends a large percentage of its time executing context switches rather than running application instructions, the system bogs down in pure overhead.
- The 1% Engineering Standard:** In modern production kernels, a typical time quantum is configured between **10ms and 100ms**. Given that hardware context-switching overhead typically requires between **0.1ms and 1.1ms**, this ratio ensures that context switches consume roughly **1% or less** of the total CPU capacity, keeping the system highly efficient.

Section 7: Structural Policy Performance Trade-offs

Evaluating scheduling policies requires analyzing how they behave under different workload distributions.

7.1 Performance Matrix under Homogeneous Workloads

Suppose a system receives 10 identical, CPU-bound jobs simultaneously, where each job requires exactly 100 seconds of computation. We compare how FCFS and Round-Robin (with a 1-second quantum) handle this workload:

Job Completion Order	FIFO Completion Mark (Seconds)	Round-Robin Completion Mark (Seconds)
Job 1	100 s	991 s
Job 2	200 s	992 s
Job 3	300 s	993 s
Job 9	900 s	999 s

Job Completion Order	FIFO Completion Mark (Seconds)	Round-Robin Completion Mark (Seconds)
Job 10	1000 s	1000 s

Comparative Analysis

While both policies complete the entire batch of work at the exact same 1000-second mark, their average completion times are wildly different:

- **FIFO Average Completion Time:** $ACT_{FIFO} = \frac{100+200+\dots+1000}{10} = 550$ seconds
- **Round-Robin Average Completion Time:** $ACT_{RR} = \frac{991+992+\dots+1000}{10} = 995.5$ seconds

Exam Design Takeaway: This demonstrates that Round-Robin performs poorly when handling jobs of identical length. Because RR constantly interleaves execution, it delays the completion of every single job until the very end of the batch.

Additionally, running a single job to completion under FCFS allows it to keep its cache lines hot in the CPU. Round-Robin, by contrast, constantly switches between threads, causing severe cache thrashing that slows down execution even if context-switching costs are zero.

Section 8: Priority-Based Scheduling, Starvation, & Priority Inversion

Real-world operating systems use **Strict Priority Scheduling** to ensure that high-importance tasks are executed ahead of background processes.

8.1 Multi-Level Priority Queue Framework

The scheduler categorizes threads into distinct priority tiers. The system enforces a strict rule: **always execute the highest-priority runnable jobs to completion** before allocating CPU cycles to lower priority tiers. Inside each individual priority queue, threads are multiplexed using a Round-Robin policy with an allocated time quantum.

8.2 Primary Structural Failures

1. **Starvation:** If the system faces a continuous stream of high-priority tasks, lower-priority threads will be completely starved of CPU cycles and may never get a chance to run.
2. **Deadlock via Priority Inversion:** A severe synchronization failure that occurs when a low-priority thread acquires a shared mutual exclusion lock needed by a high-priority thread. Because the high-priority thread blocks waiting for the lock, it yields the CPU. However, if an intermediate-priority thread arrives that does not need the lock, the scheduler will run it ahead of the low-priority thread. As a result, the low-priority thread is blocked from executing and cannot release the lock, permanently stalling the high-priority task.

8.3 Fixes and Mitigation Strategies

To prevent starvation and priority inversion, systems use **Dynamic Priorities** to adjust a thread's base priority level on the fly based on its execution history:

- **Aging Interactivity Heuristics:** Some operating systems monitor threads and dynamically boost the priority of jobs that have been starved of CPU time.
- **The Limitations of Ad-Hoc Adjustments:** These adjustments can be difficult to tune. If the system becomes overloaded, every thread's priority eventually gets boosted, which degrades interactive performance and causes mobile devices or responsive applications to stutter.
- **Priority Inheritance (Pintos Protocol):** To solve priority inversion, when a high-priority thread blocks on a lock held by a lower-priority thread, the lock holder temporarily inherits the high priority level of the blocked thread, allowing it to run, finish its critical section, and quickly release the lock.

Section 9: Predictive Scheduling: SJF and SRTF

If an operating system could predict the future, it could optimize average response times using task duration metrics.

9.1 Core Policies

- **Shortest Job First (SJF):** A non-preemptive policy that selects the runnable thread with the shortest upcoming CPU burst duration. SJF is provably optimal for minimizing average waiting times among non-preemptive algorithms.
- **Shortest Remaining Time First (SRTF):** The preemptive variant of Shortest Job First. If a new job arrives with a shorter total time to completion than the remaining time on the currently running job, the scheduler immediately preempts the CPU and switches to the new job. SRTF is provably optimal among preemptive scheduling models.

9.2 Complex Three-Job Allocation Scenario

To see the benefits of SRTF, consider a system with three threads and a single shared storage disk:

- **Job A and Job B:** Completely CPU-bound tasks that require a full week of continuous calculation.
- **Job C:** A heavily I/O-bound task that alternates between 1 ms of CPU calculation and 9 ms of disk read operations.

Policy Comparison

- **Under FCFS:** If Job A or Job B captures the CPU first, it will run continuously for a week. Job C is trapped in the ready queue, forcing the physical disk to sit completely idle and dragging down system utilization.
- **Under Round-Robin (with a 100ms quantum):** Job A runs for 100ms, then Job B runs for 100ms. Job C finally gets the CPU, executes its 1ms of calculation, and blocks for 9ms to read the disk. Once the disk read completes, Job C must wait for the CPU-bound jobs to finish their 100ms time slices before it can run again. Total disk utilization drops to roughly **4.5%** ($\frac{9 \text{ ms}}{201 \text{ ms}}$), wasting disk bandwidth.
- **Under Round-Robin (with a 1ms quantum):** Interleaving threads at a 1ms interval improves disk utilization to **90%**, but it forces the system to execute thousands of context switches per second, creating heavy kernel overhead.
- **Under SRTF:** The scheduler recognizes that Job C has the shortest remaining CPU burst requirement (only 1 ms). It gives Job C the CPU immediately. Job C runs for 1ms and blocks on its 9ms disk read, freeing up the CPU. The scheduler then runs Job A or Job B. As soon as Job C's disk operation finishes, it is placed back on the ready queue; because its remaining burst is shorter than the background jobs, the scheduler immediately preempts the CPU-bound task to let Job C run. This pattern achieves **90% disk utilization** with minimal context-switching overhead.

Section 10: Adaptive Burst Forecasting: The Exponential Moving Average (EMA)

Because an operating system cannot truly know a job's future runtime, it must predict upcoming CPU bursts by observing past execution history. This adaptive approach works because programs exhibit predictable behavior over time; a thread that was I/O bound in the past is highly likely to remain I/O bound in the near future.

10.1 Mathematical Formulation

The kernel tracks a thread's historical runtimes and forecasts its next CPU burst length using an **Exponential Moving Average (EMA)** formula:

$$\tau_n = \alpha t_{n-1} + (1 - \alpha)\tau_{n-1}$$

Where:

- t_{n-1} is the actual duration of the most recent physical CPU burst.
- τ_{n-1} is the historical predicted guess for that most recent burst.
- τ_n is the newly calculated prediction for the *next* upcoming CPU burst duration.

- α is a smoothing weight factor bounded strictly between 0 and 1 ($0 < \alpha \leq 1$).

10.2 Expanding the Recurrence Relation

To understand how the smoothing factor weights past history, we can expand the recurrence relation by substituting past terms:

$$\tau_n = \alpha t_{n-1} + (1 - \alpha)[\alpha t_{n-2} + (1 - \alpha)\tau_{n-2}] \tau_n = \alpha t_{n-1} + \alpha(1 - \alpha)t_{n-2} + \alpha(1 - \alpha)^2 t_{n-3} + \cdots + (1 - \alpha)^k \tau_{n-k}$$

Because $(1 - \alpha)$ is less than 1, its exponent increases with each step backward in time, meaning older CPU bursts have an exponentially smaller impact on the current prediction.

- Configuring $\alpha = 0.5$ balances recent history and long-term trends equally.
- Configuring $\alpha = 1.0$ makes the formula completely forget long-term history, basing the prediction entirely on the single most recent burst duration (t_{n-1}).

Section 11: Comprehensive Review Problems with Explanations

Question 1: Round-Robin Waiting Time Tracing

A Round-Robin scheduler uses a time quantum $q = 10$ ms to execute three processes that arrive simultaneously at time 0:

- P_1 : Burst requirement = 25 ms
- P_2 : Burst requirement = 7 ms
- P_3 : Burst requirement = 14 ms

1. Sketch the text-based Gantt chart tracing execution.
2. Calculate the exact individual waiting times and final average waiting time.

Solution

1. **Gantt Chart Generation:** The scheduler steps through the processes in a round-robin loop:

- Interval 0-10 : P_1 executes for 10 ms (requires 15 ms remaining).
- Interval 10-17 : P_2 executes for 7 ms and terminates cleanly.
- Interval 17-27 : P_3 executes for 10 ms (requires 4 ms remaining).
- Interval 27-37 : P_1 returns to execute for 10 ms (requires 5 ms remaining).
- Interval 37-41 : P_3 returns to execute its final 4 ms and terminates.
- Interval 41-46 : P_1 returns to execute its final 5 ms and terminates.

Gantt Chart Layout:

```
|—P1—|—P2—|—P3—|—P1—|—P3—|—P1—|
0    10   17   27   37   41   46
```

2. **Waiting Time Calculations:** We calculate total waiting times by tracking intervals spent in the ready queue:

- **Waiting Time P_1 :** $(27 - 10) + (41 - 37) = 17 + 4 = 21$ ms
- **Waiting Time P_2 :** $(10 - 0) = 10$ ms
- **Waiting Time P_3 :** $(17 - 0) + (37 - 27) = 17 + 10 = 27$ ms
- **Average Waiting Time:** $AWT = \frac{21+10+27}{3} = \frac{58}{3} = 19.33$ ms

Question 2: Exponential Moving Average Prediction Analysis

A thread's scheduling history has an estimated burst duration $\tau_4 = 8$ ms. The smoothing weight factor is configured to $\alpha = 0.4$. The thread executes its next burst, and its actual physical runtime turns out to be $t_4 = 15$ ms.

Calculate the predicted burst duration estimate (τ_5) for the thread's next upcoming execution phase.

Solution

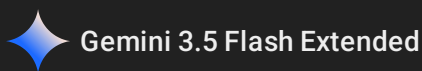
We apply the Exponential Moving Average formula directly:

$$\tau_n = \alpha t_{n-1} + (1 - \alpha)\tau_{n-1}$$

Substituting our parameters into the equation ($n = 5$):

$$\tau_5 = (0.4 \times t_4) + (1 - 0.4) \times \tau_4 \quad \tau_5 = (0.4 \times 15) + (0.6 \times 8) \quad \tau_5 = 6.0 + 4.8 = \mathbf{10.8 \text{ ms}}$$

The system adapts to the long burst by adjusting its next prediction upward to **10.8 ms**.



CS162 Operating Systems & Systems Programming

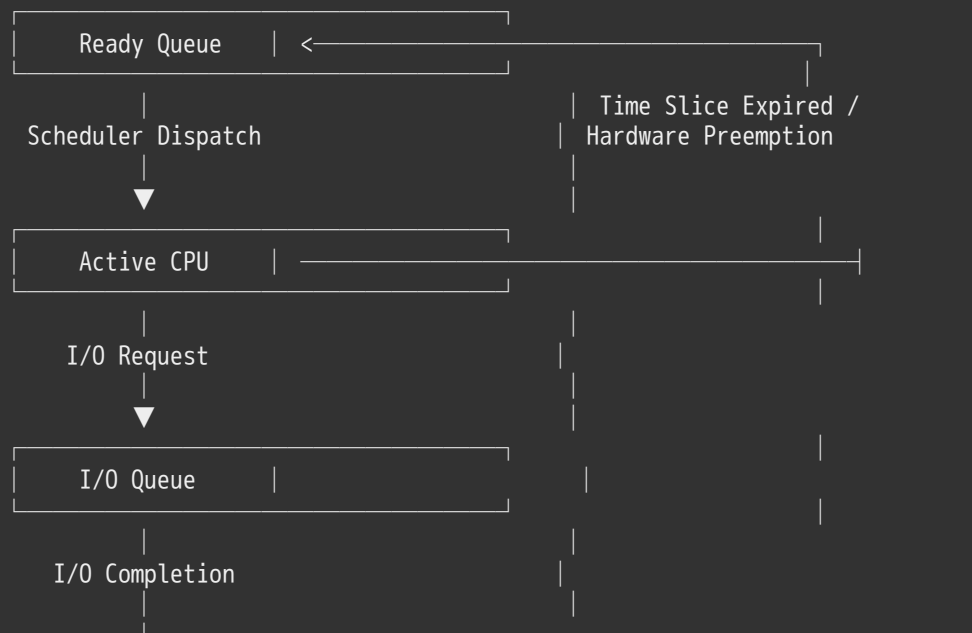
Lecture 12: Scheduling 2: Classic Policies (Con't), Case Studies, Starvation, Priority Inversion (Detailed Exam Notes)

Module 1: Core Scheduling Concepts and Metrics Review

1.1 Formal Definition of Scheduling

In computer systems architecture, **Scheduling** refers to the set of algorithmic decisions that dictate how shared resources are allocated among competing execution threads from moment to moment. While scheduling is most commonly discussed in the context of central processing unit (CPU) allocation, the fundamental engineering principles apply directly to other key hardware infrastructure components, including shared communication networks (network bandwidth) and persistent storage devices (disk read/write access).

The core design problem for an operating system's scheduler is managing the transition of tasks across various system queues as resources become available:



The workflow of thread transitions managed by a work-conserving kernel scheduler.

1.2 Mathematical Analysis of Round-Robin Scheduling

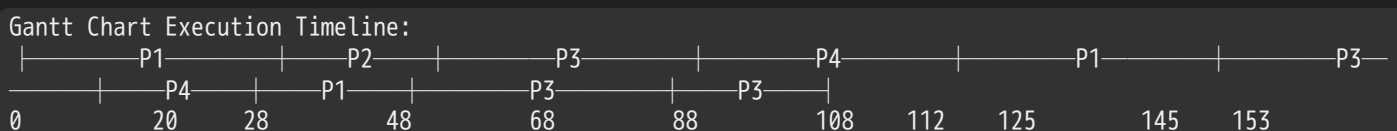
Round-Robin (RR) scheduling is a preemptive policy where each process is given a small, fixed window of CPU time called a **Time Quantum (Q)**. If a process's computation burst exceeds this quantum, a hardware timer interrupt fires, the process is preempted, and its thread control block is appended to the back of the ready queue.

Workload Specification

Consider a set of four processes arriving simultaneously at time $T = 0$, requiring the following CPU burst times:

- Process 1 (P_1): 53 ms
- Process 2 (P_2): 8 ms
- Process 3 (P_3): 68 ms
- Process 4 (P_4): 24 ms

Executing this workload under a Round-Robin policy with a time quantum $Q = 20$ ms produces the following execution timeline:



Chronological tracking of process allocations.

Chronological Breakdown of Time Allocations

- 0 to 20 ms : P_1 runs for the full time quantum. Remaining burst = $53 - 20 = 33$ ms.
- 20 to 28 ms : P_2 runs. Because its burst requirement (8 ms) is less than the quantum, it completes early and yields the CPU at 28 ms.
- 28 to 48 ms : P_3 runs for the full quantum. Remaining burst = $68 - 20 = 48$ ms.
- 48 to 68 ms : P_4 runs for the full quantum. Remaining burst = $24 - 20 = 4$ ms.
- 68 to 88 ms : P_1 returns to run for its second quantum. Remaining burst = $33 - 20 = 13$ ms.
- 88 to 108 ms : P_3 returns to run for its second quantum. Remaining burst = $48 - 20 = 28$ ms.

7. 108 to 112 ms : P_4 returns to execute its remaining 4 ms, terminating at 112 ms.
8. 112 to 125 ms : P_1 returns to execute its final 13 ms, terminating cleanly at 125 ms.
9. 125 to 145 ms : P_3 returns to run for its third quantum. Remaining burst = $28 - 20 = 8$ ms.
10. 145 to 153 ms : P_3 finishes its remaining 8 ms of calculation, terminating the workload at 153 ms.

Mathematical Calculation of Waiting Times

The waiting time for each process is calculated by tracking the total time it spent sitting in the ready queue:

Waiting Time = Completion Time – Arrival Time – Total CPU Burst Time

$$\text{Wait}(P_1) = 125 - 0 - 53 = (68 - 20) + (112 - 88) = \mathbf{72\ ms}$$

$$\text{Wait}(P_2) = 28 - 0 - 8 = (20 - 0) = \mathbf{20\ ms}$$

$$\text{Wait}(P_3) = 153 - 0 - 68 = (28 - 0) + (88 - 48) + (125 - 108) = \mathbf{85\ ms}$$

$$\text{Wait}(P_4) = 112 - 0 - 24 = (48 - 0) + (108 - 68) = \mathbf{88\ ms}$$

Average Workload Metrics

- **Average Waiting Time (AWT):** $\text{AWT} = \frac{72+20+85+88}{4} = \frac{265}{4} = \mathbf{66\frac{1}{4}\ ms}$
- **Average Completion Time (ACT):** $\text{ACT} = \frac{125+28+153+112}{4} = \frac{418}{4} = \mathbf{104\frac{1}{2}\ ms}$

Architectural Trade-offs

- **Advantages:** Round-Robin is fair and prevents long-running jobs from monopolizing the processor, significantly improving responsiveness for short, interactive tasks.
- **Disadvantages:** If the time quantum is small relative to the system's hardware limits, performance drops sharply because the CPU spends too much time saving and restoring registers during context switches.

1.3 Parametric Quantum Optimization Analysis

Choosing the right time quantum requires carefully balancing application responsiveness against system overhead. The table below analyzes how different time quantum alter waiting times and completion times for the identical workload ($P_1 = 53, P_2 = 8, P_3 = 68, P_4 = 24$) compared to First-Come, First-Served (FCFS) baselines.

Scheduling Scheme / Quantum Size	P_1 Wait / Comp (ms)	P_2 Wait / Comp (ms)	P_3 Wait / Comp (ms)	P_4 Wait / Comp (ms)	Average Wait / Comp (ms)
Best FCFS (Order: P_2, P_4, P_1, P_3)	32 / 85	0 / 8	85 / 153	8 / 32	$31\frac{1}{4}$ / $69\frac{1}{2}$
Round-Robin ($Q = 1$)	84 / 137	22 / 30	85 / 153	57 / 81	62 / $100\frac{1}{4}$
Round-Robin ($Q = 5$)	82 / 135	20 / 28	85 / 153	58 / 82	$61\frac{1}{4}$ / $99\frac{1}{2}$
Round-Robin ($Q = 8$)	80 / 133	8 / 16	85 / 153	56 / 80	$57\frac{1}{4}$ / $95\frac{1}{2}$
Round-Robin ($Q = 10$)	82 / 135	10 / 18	85 / 153	68 / 92	$61\frac{1}{4}$ / $99\frac{1}{2}$
Round-Robin ($Q = 20$)	72 / 125	20 / 28	85 / 153	88 / 112	$66\frac{1}{4}$ / $104\frac{1}{2}$

Scheduling Scheme / Quantum Size	P_1 Wait / Co mp (ms)	P_2 Wait / Co mp (ms)	P_3 Wait / Co mp (ms)	P_4 Wait / Co mp (ms)	Average Wait / Co mp (ms)
Worst FCFS (Order: P_3, P_1, P_4, P_2)	68 / 121	145 / 153	0 / 68	121 / 145	$83\frac{1}{2}$ / $121\frac{3}{4}$

Module 2: Predictive and Adaptive Scheduling Algorithms

2.1 Shortest Job First (SJF) and Shortest Remaining Time First (SRTF)

If an operating system could predict the future, it could optimize waiting times by factoring task durations directly into scheduling decisions.

[Image illustrating Predictive Scheduling via Shortest Job First versus Shortest Remaining Time First models]

- **Shortest Job First (SJF):** A non-preemptive scheduling policy that selects the runnable task with the shortest total computation requirement. SJF is provably optimal for minimizing average waiting times among non-preemptive algorithms. It is also known as Shortest Time to Completion First (STCF).
- **Shortest Remaining Time First (SRTF):** The preemptive variant of Shortest Job First. If a newly arrived task has a shorter remaining execution time than the task currently running on the CPU, the scheduler immediately preempts the running task. SRTF is provably optimal for minimizing average response times across all preemptive models. It is also known as Shortest Remaining Time to Completion First (SRTCF).

Workload Invariant Behavior

- If all tasks in the ready queue have identical execution lengths, SRTF results in the exact same scheduling timeline as standard non-preemptive FCFS.
- When task lengths vary, SRTF maximizes responsiveness for short tasks by preventing them from getting stuck behind long-running calculations.

2.2 Case Study: Maximizing System and Device Utilization via SRTF

To see the benefits of SRTF, consider a multi-device system running three processes:

- **Process A and Process B:** Completely CPU-bound tasks, where each requires an entire week of continuous execution.
- **Process C:** A heavily I/O-bound task that executes in a short loop: 1 ms of CPU computation followed immediately by 9 ms of disk storage operations.

If these processes run independently without competition, Process C utilizes 90% of the disk device ($\frac{9 \text{ ms}}{1 \text{ ms} + 9 \text{ ms}}$), while Process A or Process B can saturate 100% of the processor core.

1. First-Come, First-Served Execution Behavior

If either Process A or Process B captures the CPU first under an FCFS policy, it will run continuously for a full week. As a result, the interactive Process C sits starved in the ready queue, forcing the system's physical storage disk to sit completely idle for the entire week and severely degrading hardware utilization.

2. Round-Robin Execution Behavior with a 100 ms Time Quantum

If the system uses a standard Round-Robin policy with a 100 ms time quantum, execution alternates as follows:

- Process A runs for 100 ms, then Process B runs for 100 ms.
- Process C finally gains the CPU, executes its 1 ms calculation burst, and blocks on its 9 ms disk I/O operation.

- When the disk operation completes, Process C is moved back to the ready queue. However, it must wait for both Process A and Process B to complete their full 100 ms time slices before it can run again.
- **Hardware Efficiency:** The disk device can only ingest data for 9 ms out of every 201 ms cycle (100 ms + 100 ms + 1 ms). This drops the overall disk utilization to a meager **4.5%** ($\frac{9}{201}$), wasting significant hardware bandwidth.

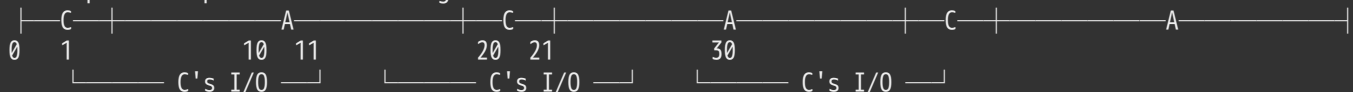
3. Round-Robin Execution Behavior with a 1 ms Time Quantum

To improve disk utilization, the time quantum could be dropped to 1 ms. This allows Process C to run almost immediately when its disk I/O completes, increasing disk utilization back to **90%**. However, this configuration forces the system to execute a massive number of context switches per second, creating heavy kernel overhead that bogs down CPU throughput.

4. Shortest Remaining Time First Execution Behavior

Under an SRTF policy, the scheduler recognizes that Process C has the shortest remaining CPU burst requirement (only 1 ms).

SRTF Optimal Pipeline Interleaving Timeline:



Process C preempts the background calculation immediately upon completing its I/O burst.

When Process C's disk operation finishes, it enters the ready queue. Because its upcoming computational burst (1 ms) is significantly shorter than the remaining runtimes of Process A or Process B, the scheduler immediately preempts the running background job. Process C runs for 1 ms and blocks on its next disk operation, freeing up the CPU. This pattern achieves **90% disk utilization** with minimal context-switching overhead, keeping the hardware highly efficient.

2.3 Adaptive Burst Forecasting via Exponential Moving Averages

Because an operating system cannot truly know an application's future runtime, it must predict upcoming CPU bursts by observing past execution history. This adaptive approach works because programs exhibit predictable behavior over time; a thread that was recently I/O bound is highly likely to remain I/O bound in its next execution phase.

[ADAPTIVE BURST LENGTH ESTIMATION]
 Historical Record of Past Burst Lengths: $t_{n-1}, t_{n-2} \dots$

[Exponential Moving Average]
 • Formulates next predicted guess: τ_n
 • Mathematically blends recent and old data

[Smoothing Factor (α) balances current vs past trends]

The scheduler tracks an application's execution history and forecasts its next CPU burst length (τ_n) using an **Exponential Moving Average (EMA)** recurrence relation:

$$\tau_n = \alpha t_{n-1} + (1 - \alpha) \tau_{n-1}$$

Where:

- t_{n-1} is the actual measured duration of the most recent CPU burst phase.
- τ_{n-1} is the historical predicted estimate calculated for that most recent burst.
- τ_n is the newly calculated prediction for the *next* upcoming CPU burst duration.
- α is a smoothing weight factor bounded strictly between 0 and 1 ($0 < \alpha \leq 1$).

Expanding the Recurrence Relation

To understand how the smoothing factor weights past history, we expand the mathematical relation by substituting past terms:

$$\tau_n = \alpha t_{n-1} + (1 - \alpha)[\alpha t_{n-2} + (1 - \alpha)\tau_{n-2}] \tau_n = \alpha t_{n-1} + \alpha(1 - \alpha)t_{n-2} + \alpha(1 - \alpha)^2 t_{n-3} + \dots + (1 - \alpha)^k \tau_{n-k}$$

Because $(1 - \alpha)$ is less than 1, its exponent increases with each step backward in time. This means older CPU bursts have an exponentially smaller impact on the current prediction, allowing the scheduler to adapt quickly to recent changes in application behavior.

Module 3: High-Level Schedulers: MLFQ and the Linux O(1) Architecture

3.1 Multi-Level Feedback Queue (MLFQ) Architecture

The Multi-Level Feedback Queue is an adaptive scheduling framework designed to automatically distinguish between interactive, I/O-bound tasks and long-running, compute-bound tasks without requiring explicit statements from the user.

Core Structural Invariants

- Priority Tiers:** The scheduler maintains multiple distinct ready queues, where each queue is assigned a different priority level. Queues near the top are designated for high-priority foreground tasks, while queues at the bottom handle low-priority background tasks.
- Dynamic Promotion and Demotion Loops:**
 - Every newly created or awakened task starts at the highest priority queue.
 - If a task uses up its entire allocated time quantum without blocking for I/O, the scheduler assumes it is a compute-bound job and demotes it down one level to a lower priority queue.
 - If a task blocks for I/O and yields the CPU before its time quantum expires, the scheduler assumes it is an interactive task and retains or promotes its position near the top of the priority tiers.
- Variable Quantum Scaling:** To maximize throughput, the length of the time quantum increases exponentially at lower priority levels (e.g., top queue $Q = 8$ ms, middle queue $Q = 16$ ms, bottom queue uses FCFS execution).

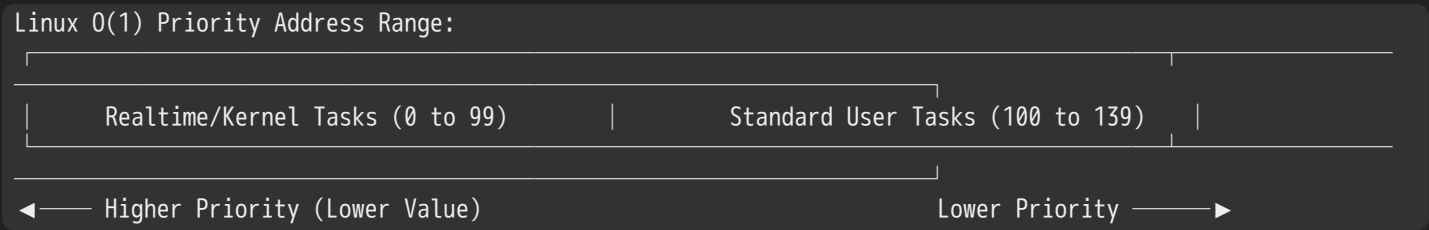
System Gaming Exploits and Countermeasures

Because MLFQ adjusts priorities based on time slice expirations, a clever application can "game" the scheduler to maintain artificially high priority.

If a compute-bound task executes for 99% of its time quantum and then issues a brief, meaningless I/O operation (such as writing a single character to a terminal log via a `printf` wrapper), it yields the CPU voluntarily. The scheduler sees that the task yielded before timeout and retains its position in the highest-priority foreground tier. This allows the task to run significantly faster than its competitors by monopolizing the top queue. To prevent this exploit, modern MLFQ implementations track a task's *total accumulated runtime* across its time slice rather than resetting the window on every yield.

3.2 Case Study: The Linux O(1) Kernel Scheduler

The Linux $O(1)$ scheduler is a priority-based scheduling architecture designed to make scheduling choices in absolute constant time ($O(1)$ complexity), regardless of how many active tasks are running on the system.



The strict division of priority ranges inside the O(1) scheduling layout.

Core Mechanical Components

- **Priority Array Layout:** The scheduler supports exactly 140 unique priority levels. Levels 0 through 99 are strictly reserved for high-priority real-time and kernel tasks, while levels 100 through 139 map to standard user-space applications (configured via the system `nice` utility).
- **Constant-Time Selection Table:** To achieve $O(1)$ selection speeds, the scheduler maps active tasks onto a **140-bit hardware bitmask table**. Each bit tracks whether its corresponding priority queue contains runnable tasks. The CPU uses a specialized hardware instruction (such as `sched_find_first_bit` or `find-first-set` asm) to locate the highest-priority active bit in constant time, entirely avoiding the need to scan through long runqueues.
- **Dual Runqueue Swapping:** The scheduler maintains two separate priority arrays: the **Active Array** and the **Expired Array**.
 1. The scheduler always selects and executes tasks from the highest-priority queue within the active array.
 2. When a task consumes its entire allocated time quantum, the scheduler calculates its new priority and appends it to the appropriate queue inside the expired array.
 3. Once the active array's bitmask hits zero (meaning all runnable tasks have exhausted their time slices), the scheduler performs a constant-time pointer swap, turning the expired array into the new active array and vice versa.

Interactive Heuristics Formulas

To keep user-space applications responsive, the scheduler adjusts a task's base priority level by up to ± 5 points based on its execution history. The scheduler tracks interactivity using an internal tracking metric:

$$\Delta_{\text{Priority}} = f(\text{sleep_avg}) \quad \text{sleep_avg} = \text{Time}_{\text{Biologically Asleep}} - \text{Time}_{\text{Actively Running}}$$

Tasks with a high `sleep_avg` are classified as I/O-bound or interactive, prompting the scheduler to reduce their priority number (boosting their execution priority).

To prevent temporary changes in behavior from causing rapid priority shifts, the scheduler uses an **Interactive Credit (IC)** tracking metric. Tasks earn credit when they sleep for extended periods and spend credit when they execute long calculations. This credit acts as a buffer (hysteresis), stabilizing the task's priority mapping.

Module 4: Multiprocessor and Multi-Core Coordination Layouts

4.1 Thread versus Process Scheduling Costs

Modern operating systems schedule individual execution **threads** or task groups rather than full processes. When a core switches contexts between tasks, the performance cost depends heavily on whether the tasks share an address space:

- **Switching Threads within the Same Process:** The kernel only needs to save and restore general-purpose registers, program counters, and stack pointers. This is fast and efficient because the active memory mapping remains unchanged.
- **Switching Processes:** The kernel must completely change the active memory mapping by reloading page table pointers (e.g., register `CR3` on x86). This is expensive because it invalidates local Translation Lookaside Buffers (TLBs) and flushes hardware caches, causing cache misses that degrade system throughput.

4.2 Multi-Core Optimization Vectors

Multiprocessor and multi-core scheduling introduces unique performance challenges, particularly around cache line contention and data sharing.

- **Core-Local Runqueues:** Rather than using a single global ready queue—which would cause heavy lock contention as cores compete to pull tasks—modern schedulers assign a private, independent set of priority queues to each individual CPU core. This allows each core to schedule tasks independently, using background load-balancing algorithms to periodically redistribute work across cores if one becomes overloaded.

- **Affinity Scheduling:** When a thread finishes an I/O operation and returns to the ready queue, the scheduler attempts to re-schedule it onto the **exact same physical CPU core** it was running on previously. This is known as affinity scheduling, and it improves performance by maximizing local cache reuse and preserving branch prediction histories.
- **Gang Scheduling:** Tightly-coupled parallel applications often split work across multiple threads that communicate frequently via synchronization barriers. If the scheduler picks these threads independently, a thread can easily waste its time slice busy-waiting on a barrier for a peer thread that isn't currently running. To prevent this, the scheduler implements **Gang Scheduling**, identifying related threads and scheduling them to run *simultaneously* across separate physical CPU cores. This keeps spin-locks efficient and significantly improves parallel execution throughput.

Gang Scheduling Visual Interleaving Pattern

```
Without Gang Scheduling (Inefficient):
Core 1: [ Thread 1 (Active) ][ Thread X (Compute) ]
Core 2: [ Thread Y (Compute) ][ Thread 2 (Active) ]
Result: Thread 1 wastes its time slice spinning on a barrier because Thread 2 is suspended.

With Gang Scheduling (Optimal Integration):
Core 1: [ Thread 1 (Active) ]
Core 2: [ Thread 2 (Active) ]
Result: Both threads run concurrently, allowing them to pass the barrier immediately.
```

4.3 Advanced Synchronization Primitive Code Implementation

When building synchronization primitives for multiprocessor systems, systems programmers must choose between letting a thread sleep or forcing it to busy-wait based on the expected wait time.

Code Example: The Basic Multiprocessor Spinlock

A basic spinlock uses an atomic hardware instruction to lock a variable without entering the kernel.

```
c

// Basic test&set Spinlock Verification Implementation
int lock_state = 0; // 0 = FREE, 1 = BUSY

void Spinlock_Acquire(int *lock_ptr) {
    // Every call to test&set executes an atomic write instruction.
    // Under cache coherence protocols, a write requires exclusive access,
    // which invalidates that cache line across all other CPU cores.
    while (test_and_set(lock_ptr)) {
        // Busy-wait loop: consumes 100% CPU cycles
    };
}

void Spinlock_Release(int *lock_ptr) {
    // Atomic store resets the lock value
    *lock_ptr = 0; //
}
```

Code Example: The Optimized test&test&set Spinlock

If multiple cores compete for the basic spinlock above, their constant write attempts force the cache line to bounce back and forth between core-local caches ("cache line ping-ponging"), consuming massive interconnect network bandwidth.

To eliminate this overhead, systems programmers use an optimized pattern called a **test&test&set lock**.

```
c
```

```
// Optimized test&test&set Spinlock Implementation
void TestAndTestAndSet_Acquire(volatile int *lock_ptr) {
    do {
        // 1. Pure Read Spin: Loop locally on a plain read instruction.
        // This allows the cache line to remain in a shared, read-only state across cores.
        // Caller reads the value directly from its local L1/L2 cache without generating bus
        traffic.
        while (*lock_ptr) {
            // Spin locally in hardware cache line space
        }

        // 2. Atomic Try: Only attempt an actual write when the lock appears to be free.
    } while (test_and_set(lock_ptr)); //
}
```

Module 5: Starvation and Progress Anomalies

An operating system's scheduler must ensure that all runnable tasks make forward progress over time.

5.1 Starvation versus Deadlock Definitions

- **Starvation:** A progress anomaly where a fully operational, runnable thread is prevented from making forward progress for an indefinite period because the scheduler continuously chooses to allocate resources to other tasks. Starvation is distinct from a deadlock because it *can* resolve on its own if system conditions change (e.g., if high-priority traffic clears).
- **Deadlock:** A structural system failure where two or more threads are permanently blocked because they are caught in a circular dependency, each waiting for a resource held by another. Deadlocks are unresolvable and cannot be cleared without manual intervention or a system reset.

5.2 Comparative Policy Starvation Vulnerability Matrix

The table below analyzes the starvation risks of the different scheduling policies covered across lectures.

Scheduling Policy / Model	Prone to Starvation?	Primary Cause of Starvation / Progress Anomalies
Non-Work-Conserving Scheduler	Yes	Leaves the CPU core idle even when there are valid, runnable tasks waiting in the system queues.
Last-Come, First-Served (LCFS / Stack)	Yes	If the task arrival rate exceeds the system's processing speed, late-arriving jobs saturate the CPU, trapping early-arriving tasks at the bottom of the stack indefinitely.
First-Come, First-Served (FCFS / FIFO)	Yes	If a task falls into an infinite loop or encounters a bug in a non-preemptive environment, it will hog the CPU forever, starving all other tasks in the queue.
Round-Robin (RR)	No	Guaranteed fairness. With N active processes and a time quantum Q , every task is guaranteed to run again within a predictable window of at most $(N - 1) \times Q$ ms.

Scheduling Policy / Model	Prone to Starvation?	Primary Cause of Starvation / Progress Anomalies
Shortest Remaining Time First (SRTF)	Yes	If the system faces a continuous stream of short, rapid execution tasks, long-running compute jobs will be starved indefinitely.
Multi-Level Feedback Queue (MLFQ)	Yes	Because MLFQ uses runtime metrics to approximate SRTF, long-running compute tasks are continuously demoted to the bottom queues, where they starve if top-tier jobs keep arriving.
Strict Priority Scheduling	Yes	The scheduler always selects the thread with the highest priority. Lower-priority background tasks are blocked from executing as long as high-priority tasks are active.

Module 6: Priority Inversion and Priority Donation Protocols

6.1 The Mechanics of Priority Inversion

Priority Inversion is a critical scheduling failure that occurs in priority-based systems when a high-priority thread is indirectly blocked from running by a lower-priority task, violating the system's priority rules.

Step-by-Step Breakdown of a Priority Inversion

Consider three jobs running under a strict priority scheduler:

- **Job 3 (J_3):** High Priority
- **Job 2 (J_2):** Medium Priority
- **Job 1 (J_1):** Low Priority

Execution Step Timeline:

1. J1 (Low) runs naturally and acquires a mutual exclusion lock.

▼

2. J3 (High) wakes up, preempts J1, and attempts to acquire the same lock. Because J1 holds the lock, J3 blocks and yields the CPU.

▼

3. J2 (Medium) wakes up. Since J2's priority is higher than J1, the scheduler runs J2 immediately. J2 completely blocks J1 from executing.

▼

[CRITICAL BREAKDOWN]: Because J1 cannot run, it can never complete its critical section to release the lock. As a result, the medium-priority J2 successfully starves the high-priority J3 via J1.

6.2 The Priority Donation (Inheritance) Mitigation Protocol

To resolve priority inversion, modern operating systems implement a coordination protocol called **Priority Donation** (or Priority Inheritance).

[Image tracing the Priority Donation flow where Job 3 temporarily boosts Job 1's scheduling priority]

The Donation Workflow

1. When a high-priority thread (J_3) blocks on a mutual exclusion lock currently held by a low-priority thread (J_1), the scheduler steps in.
 2. The scheduler temporarily boosts J_1 's priority, granting it the **high priority level of J_3** .
 3. This temporary boost allows J_1 to run ahead of any intermediate medium-priority tasks (J_2), ensuring it can execute its critical section without interference.
 4. As soon as J_1 completes its critical section and calls `Release()`, its priority drops back down to its base level. J_3 immediately acquires the lock and resumes execution, maintaining the correct priority sequence.
-

6.3 Case Study: The Martian Pathfinder Mission Recovery

On July 4, 1997, NASA's Martian Pathfinder rover successfully landed on the surface of Mars, delivering the first mobile exploration vehicle to the planet using a novel air-filled balloon system.

However, a few days into the mission, the rover began experiencing random, unprovoked system resets that caused the real-time operating system (VxWorks) to reboot, losing critical scientific progress and telemetry data.

The Failure Core (Priority Inversion)

The rover's software tasks competed for access to a shared tracking memory architecture (the information bus) protected by a mutual exclusion lock inside the VxWorks kernel:

- **High-Priority Task:** The Data Distribution Task, responsible for routing telemetry and mission critical system parameters. This task executed on a strict real-time deadline.
- **Medium-Priority Tasks:** A collection of standard background flight controls, telemetry encoders, and communication handlers.
- **Low-Priority Task:** The ASI/MET atmospheric instrument data collector task, which periodically logged basic meteorological sensor readings into the memory bus.

The system encountered the exact priority inversion sequence described in Section 6.1:

1. The low-priority atmospheric task acquired the lock on the shared memory bus.
2. The high-priority data distribution task awoke and attempted to acquire the bus lock, forcing it to block and enter a suspended state.
3. In the meantime, a collection of medium-priority background flight tasks awoke. Since their priority was higher than the atmospheric task, the scheduler ran them immediately, completely starving the low-priority task.
4. Because the atmospheric task was blocked from running, it could never release the memory bus lock.
5. **The Watchdog Trigger:** The real-time system's **Watchdog Timer** noticed that the critical high-priority data distribution task had not run for an extended period, missing its real-time deadline. Recognizing this lack of forward progress as a critical system hang, the watchdog forced a hard hardware reset to return the rover to a safe state, wiping out active calculations.

The Engineering Fix

The software team traced the issue back to a specific design choice: the original developers had explicitly **turned off priority donation** inside the VxWorks lock configuration because they were worried about the slight memory and processing overhead required to update priority metadata.

To fix the rover, the engineering team uploaded a remote software patch that re-enabled priority inheritance for the memory bus lock, allowing the atmospheric task to quickly clear its critical section and preventing further system resets.

Module 7: Comprehensive Practice Problems with Detailed Solutions

Use these practice problems to test your understanding of the concepts covered in Lecture 12.

Problem 1: Priority Inversion Cascade Analysis

Suppose a strict priority scheduler coordinates five active jobs with priorities ranging from 1 (lowest) to 5 (highest):

- Job J_1 : Priority = 1
- Job J_2 : Priority = 2
- Job J_3 : Priority = 3
- Job J_4 : Priority = 4
- Job J_5 : Priority = 5

The system contains two distinct mutual exclusion locks, `LockA` and `LockB`. Consider the following execution timeline:

1. At $T = 0$, J_1 is the only active thread. It runs and calls `Acquire(&LockA)`.
2. At $T = 10$, J_2 wakes up, preempts J_1 , and calls `Acquire(&LockB)`.
3. At $T = 20$, J_5 wakes up, preempts J_2 , and calls `Acquire(&LockA)`. It blocks immediately because `LockA` is held by J_1 .
4. At $T = 30$, J_4 wakes up and wants to execute a long computation burst. It does not require any locks.
5. At $T = 40$, J_3 wakes up and calls `Acquire(&LockB)`. It blocks immediately because `LockB` is held by J_2 .

Question

If the system does **not** implement priority donation, trace which job the scheduler will choose to run at time $T = 45$, and explain which tasks are currently starved.

Solution

To find the active thread at $T = 45$, we analyze the state of each job sequentially:

- **Job 5 (J_5 , Priority 5):** Blocked since $T = 20$, waiting on `LockA` which is held by J_1 .
- **Job 4 (J_4 , Priority 4):** Fully runnable and ready to execute its computation burst.
- **Job 3 (J_3 , Priority 3):** Blocked since $T = 40$, waiting on `LockB` which is held by J_2 .
- **Job 2 (J_2 , Priority 2):** Fully runnable but currently preempted. It holds `LockB`.
- **Job 1 (J_1 , Priority 1):** Fully runnable but currently preempted. It holds `LockA`.

The scheduler evaluates the ready tasks (J_4, J_2, J_1) and selects the one with the highest priority, which is **Job 4 (J_4)**.

Starvation Impact Analysis

- J_5 (the highest priority task) is starved because it is blocked on J_1 , which is completely preempted by J_4 .
- J_3 is starved because it is blocked on J_2 , which is also preempted by J_4 .
- J_2 and J_1 are starved because J_4 has a higher priority and will continue to hog the CPU until its long computation completes.
- **Conclusion:** This is a classic priority inversion failure. The un-synchronized, un-related J_4 is successfully starving the highest priority task J_5 by blocking J_1 from running to release the lock.

Problem 2: Exponential Moving Average Multi-Step Prediction Trace

A scheduler uses an Exponential Moving Average (EMA) to predict the length of a thread's next CPU burst. The smoothing factor is configured to $\alpha = 0.5$, and the initial prediction guess is set to $\tau_1 = 10$ ms.

The thread executes three sequential CPU burst phases, resulting in the following actual physical runtimes:

- Actual burst 1 (t_1): 6 ms
- Actual burst 2 (t_2): 4 ms
- Actual burst 3 (t_3): 14 ms

Question

Calculate the step-by-step predictions (τ_2, τ_3, τ_4) and determine the final estimated burst length τ_4 that the scheduler will use to evaluate the thread's next execution phase.

Solution

We calculate the adaptive predictions step-by-step using the EMA formula: $\tau_n = \alpha t_{n-1} + (1 - \alpha)\tau_{n-1}$

Given $\alpha = 0.5$, the formula simplifies to a straightforward average of the most recent actual burst and the previous prediction:

$$\tau_n = 0.5t_{n-1} + 0.5\tau_{n-1}$$

Step 1: Calculate Prediction τ_2

Using the actual length of the first burst ($t_1 = 6$ ms) and the initial guess ($\tau_1 = 10$ ms): $\tau_2 = (0.5 \times t_1) + (0.5 \times \tau_1)$
 $\tau_2 = (0.5 \times 6) + (0.5 \times 10) = 3 + 5 = 8.0$ ms

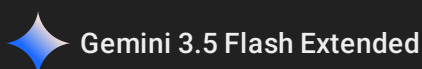
Step 2: Calculate Prediction τ_3

Using the actual length of the second burst ($t_2 = 4$ ms) and the previous prediction ($\tau_2 = 8.0$ ms): $\tau_3 = (0.5 \times t_2) + (0.5 \times \tau_2)$
 $\tau_3 = (0.5 \times 4) + (0.5 \times 8.0) = 2 + 4 = 6.0$ ms

Step 3: Calculate Prediction τ_4

Using the actual length of the third burst ($t_3 = 14$ ms) and the previous prediction ($\tau_3 = 6.0$ ms): $\tau_4 = (0.5 \times t_3) + (0.5 \times \tau_3)$
 $\tau_4 = (0.5 \times 14) + (0.5 \times 6.0) = 7 + 3 = 10.0$ ms

The adaptive scheduler adjusts its final estimate to 10.0 ms, factoring in the sudden long burst while smoothing out short-term spikes.



CS162 Operating Systems & Systems Programming

Lecture 13: Proportional Share Scheduling, Real-Time Scheduling, and Deadlocks (Comprehensive Study Notes)

Section 1: Multi-Level Feedback Queue Review & Shifting Scheduling Horizons

1.1 Algorithmic Framework of Multi-Level Feedback Queues (MLFQ)

The **Multi-Level Feedback Queue (MLFQ)** is an adaptive scheduling framework designed to analyze an application's past execution history to predict its upcoming CPU burst requirements. First implemented in the Compatible Time-Sharing System (CTSS), MLFQ addresses a core systems problem: optimizing performance without forcing applications to pre-declare their compute parameters.

Core Structural Invariants

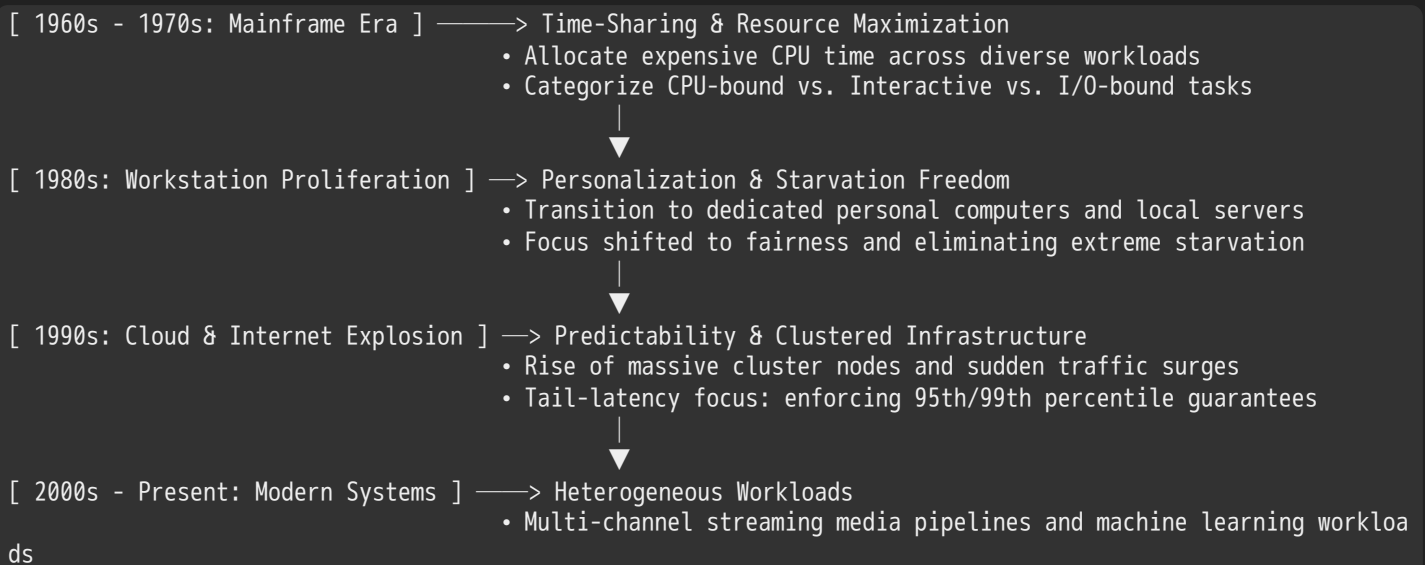
- **Priority-Tiered Layout:** The scheduler maintains multiple distinct ready queues, where each queue is assigned a unique priority level. Queues near the top handle high-priority foreground workloads, while queues at the bottom process low-

priority background workloads.

- **Divergent Scheduling Disciplines:** Each individual queue can run a different scheduling policy. For instance, high-priority foreground queues typically use a preemptive **Round-Robin (RR)** policy to ensure responsiveness, while the lowest-priority background queue uses a non-preemptive **First-Come, First-Served (FCFS)** policy to maximize raw instruction throughput.
- **Exponential Quantum Scaling:** To prevent long-running tasks from causing frequent context switches, the time quantum increases exponentially at lower priority levels (e.g., top queue $Q = 8$ ms, middle queue $Q = 16$ ms, cascading down to an un-quantized FCFS baseline).
- **Dynamic Priority Adjustments:** 1. A newly created or awakened thread is placed at the back of the highest-priority queue. 2. If the thread consumes its entire allocated time quantum without blocking for I/O, the scheduler assumes it is a compute-bound task and demotes it down one level to a lower priority queue. 3. If the thread blocks for I/O or yields the CPU voluntarily before its time slice expires, the scheduler identifies it as an interactive or I/O-bound task, retaining or promoting its position near the top of the priority tiers.

1.2 The Changing Landscape of Operating System Workloads

As computer hardware has evolved, the primary goals of scheduling algorithms have fundamentally shifted to meet new system requirements:



Section 2: Proportional-Share Paradigms: Lottery & Stride Scheduling

2.1 Philosophy of Proportional Sharing

Traditional priority-based scheduling algorithms always favor high-priority jobs, which means lower-priority background tasks can be starved of CPU cycles indefinitely if high-priority tasks continue to arrive. However, in many systems, priority is simply a tool to keep the system responsive rather than an absolute rule.

Proportional-Share Scheduling replaces strict preemption hierarchies with a resource-sharing model. Instead of completely blocking lower-priority tasks, the scheduler guarantees each job a specific, predictable percentage of total CPU cycles based on its assigned weight. Because every active job is guaranteed a non-zero portion of the CPU, starvation is structurally eliminated, ensuring that all tasks make forward progress. This model is closely related to the principle of **Fair Queueing** used in computer networking.

2.2 Lottery Scheduling Mechanics

Lottery Scheduling is a randomized approach to proportional-share scheduling. The operating system represents CPU allocation fractions as virtual **Lottery Tickets**.

Core Selection Logic

- Every active job is assigned a specific number of lottery tickets based on its importance or weight.
- At each new time slice interval, the scheduler generates a random number within the range of total outstanding tickets.
- The task that holds the winning ticket number is awarded the CPU core for that time slice.
- **Statistical Convergence:** While any individual draw is random, over time, the total CPU allocation converges mathematically so that each job's runtime is directly proportional to its number of tickets.

Approximating Shortest Remaining Time First (SRTF)

To maximize responsiveness without running into starvation issues, the system can adjust ticket counts dynamically based on execution characteristics:

- Short, interactive, or I/O-bound jobs are awarded a large pool of tickets, which gives them a high probability of gaining the CPU quickly and keeps the system responsive.
- Long-running, compute-bound jobs are given a smaller pool of tickets. They run less frequently but are still guaranteed to make progress since they retain at least one ticket.
- **Graceful Scaling under Load:** Unlike strict priority systems, adding or deleting jobs in a lottery scheduler scales resource allocation gracefully and proportionally across all active tasks, entirely independent of the total number of outstanding tickets in the system.

2.3 Mathematical Allocation Analysis under Variable Workload Mixes

To calculate a task's expected CPU share, we use the following probability formula: Expected CPU Share $U_i = \frac{N_i}{\sum_{j=1}^M N_j}$

Where N_i represents the number of tickets allocated to target job i , and M tracks the total number of active threads in the ready queue.

Suppose the system uses a configuration where short, interactive tasks are assigned **10 tickets** and long, compute-bound tasks are assigned **1 ticket**. The table below analyzes the expected CPU allocation across different workload combinations:

Active Job Mix Layout (# Short / # Long)	Expected CPU Share (Per Short Job)	Expected CPU Share (Per Long Job)	Derivation and Probability Formulas
1 / 1	91%	9%	Total = 10 + 1 = 11 tickets. Short Share = $\frac{10}{11} \approx 90.9\%$; Long Share = $\frac{1}{11} \approx 9.09\%$.
0 / 2	N/A	50%	Total = 1 + 1 = 2 tickets. Long Share = $\frac{1}{2} = 50\%$ per process.
2 / 0	50%	N/A	Total = 10 + 10 = 20 tickets. Short Share = $\frac{10}{20} = 50\%$ per process.
10 / 1	9.9%	0.99%	Total = (10 × 10) + 1 = 101 tickets. Short Share = $\frac{10}{101} \approx 9.9\%$; Long Share = $\frac{1}{101} \approx 0.99\%$.

Active Job Mix Layout (# Short / # Long)	Expected CPU Share (Per Short Job)	Expected CPU Share (Per Long Job)	Derivation and Probability Formulas
1 / 10	50%	5%	Total = 10 + (10 × 1) = 20 tickets. Short Share = $\frac{10}{20} = 50\%$; Long Share = $\frac{1}{20} = 5\%$.

Handling System Overload Bounds

If the system faces extreme overload (for example, a load average of 100 or more concurrent short tasks), individual time allocations drop significantly, making it difficult for any single application to make reasonable forward progress. To protect system stability in this scenario, administrative policies must intervene to drop the active load, such as force-terminating non-critical background jobs or logging off inactive users.

2.4 Code Example: Lottery Ticket Selection Loop

This code shows how a lottery scheduler selects the next winning task by traversing an internal array of active threads.

```

c

#include <stdio.h>
#include <stdlib.h>

typedef struct thread {
    char name[16];
    int ticket_count;
} thread_t;

// Selects the winning thread by accumulating tickets sequentially
thread_t* schedule_lottery_winner(thread_t* active_threads, int dynamic_task_count) {
    int total_system_tickets = 0; //

    // Accumulate the sum of all outstanding tickets across active tasks
    for (int i = 0; i < dynamic_task_count; i++) {
        total_system_tickets += active_threads[i].ticket_count; //
    }

    if (total_system_tickets == 0) return NULL;

    // Pick a random "dart" value d within the range [1, total_system_tickets]
    int dart = (rand() % total_system_tickets) + 1; //
    int running_ticket_sum = 0;

    // Step through the tasks sequentially; select the first task that pushes the sum past the
    dart
    for (int j = 0; j < dynamic_task_count; j++) {
        running_ticket_sum += active_threads[j].ticket_count; //
        if (running_ticket_sum >= dart) { //
            return &active_threads[j]; // Return pointer to the winning thread
        }
    }
}

```

```

return &active_threads[dynamic_task_count - 1];
}

```

2.5 Stride Scheduling: Non-Random Proportional Sharing

While lottery scheduling is simple, it suffers from the **Law of Small Numbers**: over short time frames, the actual CPU allocation can deviate significantly from the intended proportions, leading to short-term unfairness. To eliminate this randomness while maintaining proportional allocation, systems developers use **Stride Scheduling**.

[STRIDE SCHEDULING FORMULATION]

Compute Stride length for each active task: $\text{Stride} = \text{Big\# } W / \text{Tickets}$

- High-ticket tasks receive a small stride.
- Low-ticket tasks receive a large stride.



[Selection Step: Choose the active task with the lowest Pass Counter]

- Run selected task for a single time quantum.
- Update tracking register: $\text{Pass_Counter} = \text{Pass_Counter} + \text{Stride}$.

Step-by-Step Mathematical Walkthrough

The scheduler calculates an execution **Stride** for each job, which is inversely proportional to its ticket allocation: $\text{Stride } S_i = \frac{\text{Large Constant } W}{N_i}$

Where N_i represents the number of tickets assigned to job i . Every job tracks its execution progress using a local register called a **Pass Counter**. At each scheduling interval, the scheduler selects the job with the lowest pass counter, runs it for a single time slice, and increments its pass counter by its stride value. This mechanism ensures that jobs with smaller strides (more tickets) are selected more frequently, matching their target allocations exactly over short time frames.

Consider a system with a constant $W = 10,000$ and three active jobs with the following ticket allocations:

- Job A: 100 tickets $\implies \text{Stride } S_A = \frac{10,000}{100} = 100$
- Job B: 50 tickets $\implies \text{Stride } S_B = \frac{10,000}{50} = 200$
- Job C: 250 tickets $\implies \text{Stride } S_C = \frac{10,000}{250} = 40$

Chronological Scheduling Execution Matrix

Assuming all pass counters are initially zero, the table below traces the first 6 scheduling intervals:

Step Allocation	Job Selected (Lowest Pass)	Pass Counter A (Before $\rightarrow$ After)	Pass Counter B (Before $\rightarrow$ After)	Pass Counter C (Before $\rightarrow$ After)	Invariant Status Summary
Initial	—	0	0	0	All pass counters tied at 0.
Step 1	Job A (Tied-break)	0 $\rightarrow$ 100	0	0	Job A runs; its counter increases by its stride (+100).
Step 2	Job B (Tied-break)	100	0 $\rightarrow$ 200	0	Job B runs; its counter increases by its stride (+200).

Step Allocation	Job Selected (Lowest Pass)	Pass Counter A (Before → After)	Pass Counter B (Before → After)	Pass Counter C (Before → After)	Invariant Status Summary
Step 3	Job C	100	200	0 → 40	Job C runs; its counter increases by its stride (+40).
Step 4	Job C	100	200	40 → 80	Job C still has the lowest counter (80 < 100); it runs again.
Step 5	Job C	100	200	80 → 120	Job C runs for a third time (120).
Step 6	Job A	100 → 200	200	120	Job A has the lowest counter (100 < 120); it is selected.

This deterministic execution flow ensures that Job C runs exactly twice as often as Job A, and Job A runs exactly twice as often as Job B, matching their ticket ratios (250 : 100 : 50 $\implies$ 5 : 2 : 1) without any of the statistical variance seen in lottery scheduling.

Section 3: The Linux Completely Fair Scheduler (CFS) Deep Dive

3.1 Core Philosophy and the Virtualization Model

The Linux **Completely Fair Scheduler (CFS)** is an elegant, multi-tier implementation of proportional-share scheduling used in modern production kernels.

Idealized Multi-Threading Model (Simultaneous Execution):
Core: [Thread 1 (1/N)] [Thread 2 (1/N)] [Thread 3 (1/N)] $\longrightarrow$ Simultaneous continuous cycles

Hardware Multi-Threading Model (Time-Slices):
Core: [Thread 1 (Q)] [Thread 2 (Q)] [Thread 3 (Q)] $\longrightarrow$ Discrete interleaved chunks

The fluid conceptual multi-threading model vs. discrete hardware execution.

CFS is built around a specific ideal model: a **perfectly subdivided hardware processor** where N active threads run simultaneously on the same core, with each thread receiving an exact, continuous $\frac{1}{N}$ fraction of total CPU instruction cycles.

Since real, commodity hardware cannot execute an arbitrary number of threads simultaneously in infinitely small subdivisions, the operating system must allocate the full CPU to a single thread for discrete time slices. The core goal of CFS is to minimize execution variance, tracking how much time each thread has run to keep them roughly in sync with the idealized model.

3.2 CFS Responsiveness Boundaries: Target Latency & Minimum Granularity

To maintain predictability and prevent starvation, CFS manages context switches using two key bounds:

1. Target Latency (sched_latency)

The target latency is the fixed window of time over which **every single active runnable process must be scheduled at least once**. Under standard equal sharing with n active processes, the base time quantum Q_i allocated to a task is calculated as:

$$Q_i = \frac{\text{Target_Latency}}{n}$$

- Consider a system with a target latency of 20 ms running 4 active processes. Each process is awarded an equal time slice of $\frac{20\text{ ms}}{4} = 5\text{ ms}$.
- If the workload surges to 200 active processes, a naive application of the formula drops each individual time slice to $\frac{20\text{ ms}}{200} = 0.1\text{ ms}$. This micro-slice causes severe performance issues, as the system spends almost all its time

executing context switches rather than running application instructions.

2. Minimum Granularity (`sched_min_granularity`)

To protect system throughput from context-switching overhead, CFS enforces a lower bound called the **Minimum Granularity**. This parameter defines the absolute shortest time slice any individual thread can be assigned during a scheduling cycle.

- If a system has a target latency of 20 ms and a minimum granularity floor of 1 ms:
- When handling 200 concurrent processes, CFS drops the target latency constraint and overrides the time slice calculation, awarding each process the minimum floor of 1 ms. This pushes the total scheduling period out to 200 ms ($200 \times 1 \text{ ms}$), protecting system throughput at the cost of longer response times under heavy load.

3.3 Nice Values, Weights, and the `vruntime` Transformation

To support proportional-share allocation, CFS maps traditional UNIX **nice values** (which range from -20 for high priority to 19 for low priority) to an exponential scale of internal weights:

$$\text{Weight } W_i = \frac{1024}{(1.25)^{\text{nice}}}$$

This exponential mapping ensures that a relative difference of 5 nice points always translates to a fixed, predictable ratio of CPU cycles, regardless of the absolute nice values chosen. Because $(1.25)^5 \approx 3$, two tasks whose nice values differ by 5 will always maintain a **3 : 1 ratio of weights and CPU cycles**.

Using these weights, the scheduler shifts from tracking raw physical CPU runtime to tracking a normalized metric called **Virtual Runtime** (`vruntime`):

$$\text{vruntime}_i = \text{Real Physical CPU Execution Time} \times \frac{\text{Baseline Weight (1024)}}{W_i}$$

Operational Implications of the Formula

- **High-Priority Tasks (Low Nice Value / Large Weight):** For a high-priority task, $W_i > 1024$, which means its virtual runtime accumulates **slower** than real physical time.
- **Low-Priority Tasks (High Nice Value / Small Weight):** For a low-priority task, $W_i < 1024$, which means its virtual runtime accumulates **faster** than real physical time.
- **Selection Policy:** The scheduler always selects the thread with the **absolute lowest virtual runtime** to run next. This automatically rewards high-priority tasks with longer and more frequent physical time slices while keeping all tasks synchronized over time.

3.4 C++ Simulation: CFS Allocation Loop

This simulation demonstrates how CFS calculates time slices and virtual runtime accumulation for a group of competing threads.

```
cpp

#include <iostream>
#include <vector>
#include <cmath>
#include <string>

struct CFSTask {
    std::string name;
    int nice_value;
    double weight;
```

```

    double physical_cpu_time;
    double vruntime;
};

void calculate_cfs_quantums(std::vector<CFSTask>& tasks, double target_latency, double
min_granularity) {
    double total_system_weight = 0.0;
    for (const auto& task : tasks) {
        total_system_weight += task.weight;
    }

    std::cout << "--- CFS Scheduling Period Allocation Loop ---\n";
    for (auto& task : tasks) {
        // Compute proportional time slice quantum based on relative task weight
        double quantum = target_latency * (task.weight / total_system_weight); //

        // Enforce the minimum granularity floor to protect system throughput
        if (quantum < min_granularity) {
            quantum = min_granularity; //
        }

        // Simulate execution: add quantum to raw physical time
        task.physical_cpu_time += quantum;

        // Update virtual runtime: vruntime = real_time * (baseline_weight / task_weight)
        task.vruntime += quantum * (1024.0 / task.weight); //

        std::cout << "Task: " << task.name
                    << " | Nice: " << task.nice_value
                    << " | Allocated Quantum: " << quantum << " ms"
                    << " | New vruntime: " << task.vruntime << " ms\n";
    }
}

int main() {
    std::vector<CFSTask> tasks = {
        // Weight = 1024 / (1.25)^nice
        {"Thread_A", 0, 1024.0, 0.0, 0.0}, // Nice 0 baseline
        {"Thread_B", 5, 1024.0 / std::pow(1.25, 5), 0.0, 0.0} // Nice 5 (~3x less weight)
    };

    calculate_cfs_quantums(tasks, 20.0, 1.0); // Target Latency = 20ms, Min Granularity = 1ms
    return 0;
}

```

3.5 Caching and Lookups via Red-Black Trees

To track and select tasks efficiently, CFS stores all runnable threads in a self-balancing **Red-Black Tree**, sorted strictly by their virtual runtimes (`vruntime`).

- **Algorithmic Complexity:** Inserting a thread or removing it from the tree requires balanced path traversals that run in logarithmic time, $O(\log N)$, where N is the number of active threads.
- **Leftmost Node Caching:** Because the scheduler needs to access the task with the absolute lowest `vruntime` at every interval, modern implementations optimize lookups by **caching a pointer directly to the leftmost leaf node**. This allows the

scheduler to select the next thread to run in instant constant time, $O(1)$, avoiding the need to traverse the tree structure on every context switch.

- **Automatic Interactivity Credits:** When an interactive thread blocks for I/O, it is removed from the tree, and its `vruntime` stops accumulating. While it sleeps, the `vruntime` values of active, compute-bound threads continue to increase. When the interactive thread wakes up, its historical `vruntime` is significantly lower than its competitors. As a result, it is immediately placed at the far left of the Red-Black tree, allowing it to preempt the compute-bound tasks and run immediately. This elegant design handles interactive tasks automatically without requiring complex heuristics.

Section 4: Real-Time Scheduling Systems & Workload Models

4.1 Philosophy of Real-Time Systems

In conventional operating systems, the scheduler optimizes for system metrics like maximum throughput or average response times.

In a **Real-Time System (RTS)**, the primary goal is **Predictability of Performance**. The system must guarantee with complete confidence that critical operations will always complete within strict, worst-case timing deadlines imposed by the physical world. Real-time scheduling is not about raw processing speed; it is about enforcing absolute timing predictability.

- **Hard Real-Time Systems:** Used for mission-critical, safety-oriented systems (such as medical pacemakers, automotive brake controllers, or automated flight systems). Missing a single deadline is treated as a fatal system failure. The system must prove *a priori* (before deployment) that all tasks will meet their deadlines under worst-case scenarios.
- **Soft Real-Time Systems:** Used for applications like live multimedia streaming or video playback. Missing a deadline drops a frame but does not cause a fatal failure; the system simply aims to meet deadlines with high probability.

4.2 Real-Time Workload Formulations

A real-time workload model represents tasks as a series of repeating execution requirements defined by specific constraints:

- Tasks are assumed to be independent, fully preemptable, and can arrive at arbitrary release times.
- Each task i is defined by a tuple: (P_i, C_i) , where P_i represents its recurring **Period** window, and C_i specifies its maximum **Worst-Case Computation Time**.
- **Periodic Arrivals:** Tasks arrive cyclically at fixed intervals: $A_i^{t+1} = A_i^t + P_i$.
- **Relative Deadlines:** Each task must finish its computation burst before the end of its current period: $D_i^t = A_i^t + P_i$.

4.3 Why Conventional Scheduling Fails Real-Time Deadlines

Traditional scheduling policies like Round-Robin are completely unsuited for real-time systems because they make scheduling choices without considering task deadlines.

If multiple real-time tasks are multiplexed using a uniform time quantum, a task with a long period can easily starve a short-period task with an immediate deadline, causing the system to fail. To handle real-time workloads safely, scheduling policies must factor deadlines directly into their selection logic.

Section 5: Hard Real-Time Policies: Rate Monotonic & Earliest Deadline First

5.1 Rate Monotonic Scheduling (RMS)

Rate Monotonic Scheduling (RMS) is a foundational static priority scheduling policy for periodic real-time workloads.

Algorithmic Assignment Invariant

- Priorities are assigned statically based on the length of each task's period: **the shorter the period, the higher its scheduling priority**.
- Mathematically: $P_i < P_j \implies \pi_i > \pi_j$, where π represents priority.
- At any scheduling interval, the scheduler always executes the active task with the highest priority.

The Liu & Layland Schedulability Bound Theorem

To determine if a set of periodic tasks can be scheduled safely under RMS without missing deadlines, we evaluate the **Liu & Layland Utilization Bound Test**. The processor utilization U of a task set is the sum of the ratios of computation time to period for all tasks:

$$U = \sum_{i=1}^n \left(\frac{C_i}{P_i} \right)$$

A set of n periodic tasks is guaranteed to be fully schedulable under RMS if its total processor utilization satisfies the following inequality:

$$\sum_{i=1}^n \left(\frac{C_i}{P_i} \right) \leq n (2^{1/n} - 1)$$

- **Sufficient but Not Necessary:** This utilization test is *sufficient* but not *necessary*. If a task set passes the test, it is guaranteed to meet all deadlines. If it fails, it might still be schedulable, depending on the exact alignment of task periods. For example, if all periods are perfectly harmonic (meaning each period evenly divides the larger ones), the utilization bound scales up to a full **100%**.
- **Asymptotic Limit:** As the number of independent tasks grows toward infinity, the utilization bound converges to a fixed mathematical limit: $\lim_{n \rightarrow \infty} n (2^{1/n} - 1) = \ln 2 \approx \mathbf{69.3\%}$

5.2 RMS Execution Failure Case Study

Consider a real-time system running three periodic tasks with parameters $T_i = (P_i, C_i)$:

- Task 1: $T_1 = (4, 1) \implies U_1 = \frac{1}{4} = \mathbf{0.25}$
- Task 2: $T_2 = (5, 2) \implies U_2 = \frac{2}{5} = \mathbf{0.40}$
- Task 3: $T_3 = (7, 2) \implies U_3 = \frac{2}{7} \approx \mathbf{0.286}$

Load Verification

We sum the individual task utilizations to check the total processor load: $U_{\text{Total}} = 0.25 + 0.40 + 0.286 = \mathbf{0.936}$

Because $U_{\text{Total}} \leq 1.0$, the workload does not physically overload the processor, meaning a valid schedule is theoretically possible.

Applying the Schedulability Test

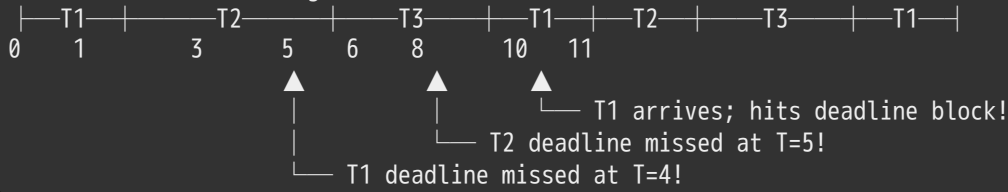
Next, we evaluate the Liu & Layland bound for $n = 3$ tasks: $\text{Bound} = 3 \times (2^{1/3} - 1) \approx 3 \times (1.2599 - 1) \approx \mathbf{0.780}$

Since $0.936 > 0.780$, the task set fails the sufficient utilization test. This tells us that under a static priority RMS policy, deadlines are likely to be missed.

Step-by-Step Chronological Execution Trace

Under RMS, priorities are assigned statically based on period lengths ($P_1 = 4 < P_2 = 5 < P_3 = 7$), resulting in the fixed priority hierarchy: $\pi_1 > \pi_2 > \pi_3$.

RMS Failure Interleaving Timeline:



Chronological trace of the static priority execution failure.

- **Interval 0 to 1** : T_1 has the highest priority and runs for 1 ms, completing its computation cleanly.
- **Interval 1 to 3** : With T_1 finished, T_2 runs for its required 2 ms burst, completing at 3 ms.
- **Interval 3 to 4** : T_3 runs for 1 ms. At time 4 ms, the period for T_1 expires, prompting it to arrive again and immediately preempt T_3 due to its higher priority.
- **Interval 4 to 5** : T_1 runs for 1 ms. At time 5 ms, the period for T_2 expires, causing it to arrive again.
- **Interval 5 to 7** : T_2 runs for its 2 ms burst, completing at 7 ms.
- **Interval 7 to 8** : At time 7 ms, **Task 3 (T_3) hits its relative deadline!** However, it has only executed for 1 ms out of its required 2 ms computation burst.
- **The Failure**: Because static priorities forced T_3 to be preempted by arriving higher-priority tasks, it misses its deadline at 7 ms, causing a hard real-time system failure.

5.3 Earliest Deadline First (EDF) Scheduling

To avoid the utilization limitations of static priority policies, real-time systems use **Earliest Deadline First (EDF)**. EDF is a dynamic priority scheduling policy where priorities are updated on the fly based on changing task deadlines.

Algorithmic Selection Invariant

At any scheduling interval, the scheduler evaluates all active tasks and allocates the CPU to the task with the **closest absolute deadline** (D_i^t).

The Schedulability Bound Theorem

A set of periodic real-time tasks is guaranteed to meet all deadlines under an EDF policy if and only if its total processor utilization satisfies the following constraint:

$$\sum_{i=1}^n \left(\frac{C_i}{P_i} \right) \leq 1.0$$

This is an **exact schedulability test** (both *necessary* and *sufficient*). As long as the workload does not physically overload the processor, EDF is mathematically guaranteed to find a valid schedule that meets every deadline.

Applying this to our previous counter-example workload ($U_{\text{Total}} = 0.936 \leq 1.0$), EDF successfully schedules the tasks without missing any deadlines.

5.4 The Domino Effect Performance Anomaly

While EDF is optimal for underloaded systems ($U \leq 1.0$), it exhibits a critical performance bottleneck under short-term overloads ($U > 1.0$) known as the **Domino Effect**.

If an unexpected hardware spike or an extra task (T_{new}) pushes the total processor utilization past 1.0, the closest-deadline task will consume extra cycles, causing it to miss its deadline slightly.

Because EDF always prioritizes the task whose deadline is closest, the scheduler will cascade this delay down the line. It switches to the next closest task, which now misses its deadline as well, triggering a chain reaction where **every subsequent**

task in the queue misses its deadline sequentially. A minor overload can thus cause a complete collapse of deadline compliance across the entire system.

Section 6: Methodologies for Scheduling Evaluation & System Load Curves

6.1 Evaluation Frameworks

Systems engineers use three complementary techniques to evaluate and validate scheduling algorithms:

- 1. Deterministic Modeling:** Takes a fixed, predetermined workload profile and calculates the exact performance metrics for each policy. While simple and useful for core validation, it only applies to that specific workload profile.
- 2. Queueing Models:** Uses stochastic probability distributions (such as Poisson arrival rates and exponential service times) to analyze system behavior mathematically. This provides useful long-term averages but can struggle with complex, non-linear real-world workloads.
- 3. Trace-Driven Implementation and Simulation:** Records actual execution logs from production servers onto a "trace tape" and replays them inside a simulator to test different scheduling algorithms. This provides a flexible and realistic environment to evaluate policies against real-world data before deploying them to production kernels.

6.2 Strategic Selection Synthesis Table

The optimal scheduling policy depends entirely on the primary performance goals of the system:

Primary Optimization Core Target	Recommended Scheduling Policy Selection	Architectural Justification and Mechanics
Maximize Raw CPU Throughput	FCFS	Eliminates all preemption and context-switching overhead, keeping the CPU focused entirely on application code.
Minimize Average Response Time	SRTF Approximation	Provably optimal framework that gets short jobs out of the system quickly.
Maximize I/O Device Throughput	SRTF Approximation	Prioritizes interactive tasks, allowing them to issue I/O requests quickly and keep peripheral devices fully utilized.
Enforce Strict CPU Time Fairness	Linux CFS	Uses virtual runtime tracking (<code>vruntime</code>) to keep all executing threads synchronized with a perfectly subdivided processor model.
Enforce Fair Ready-Queue Wait Times	Round-Robin	Uses fixed time slices to cycle through the ready queue sequentially, ensuring no thread is left waiting for long.
Guarantee Real-Time Deadlines	EDF	Dynamic priority policy that scales up to 100% utilization, always prioritizing the task closest to its deadline.
Favor High-Importance System Tasks	Strict Priority	Enforces a rigid execution hierarchy, guaranteeing that high-priority system tasks always run ahead of background processes.

6.3 The Non-Linear System Load Curve

A common misconception in systems engineering is that a hardware resource (such as a CPU, network link, or storage interface) should be driven to 100% utilization to maximize efficiency.

In reality, response times follow a highly non-linear load curve : as resource utilization approaches **100%**, response times escape toward **infinity**. This occurs because as the system fills up, any minor variation in task arrivals causes long queues to build up faster than they can drain.

Most scheduling algorithms work fine within the stable, linear portion of the curve, but performance degrades rapidly once load passes the "**Knee**" of the curve (typically around 70% to 80% utilization). When a system hits this inflection point, the correct solution is not to tune the scheduling policy, but to expand the physical hardware capacity to return the system to its stable operating range.

Section 7: Deadlock Architecture: Principles, Case Studies, & Multi-Resource Traps

7.1 Deadlock vs. Starvation

- **Starvation:** A progress anomaly where a fully operational, runnable thread is delayed indefinitely because the scheduler continuously chooses to allocate resources to other tasks. Starvation is a policy issue and *can* resolve on its own if system conditions change.
- **Deadlock:** A structural failure where a set of threads is permanently blocked because they are caught in a circular dependency, each waiting for a resource held by another. Deadlocks are unresolvable and can never break on their own without external intervention, such as force-terminating a process or restarting the system.

7.2 Case Study: The Single-Lane Bridge

The single-lane bridge crossing is a classic physical analogy for resource deadlocks:

- **Resource Mapping:** Each distinct section of the single-lane road acts as an exclusive hardware resource. A vehicle must control the segment beneath it and successfully acquire the upcoming segment before it can move forward.
- **Deadlock State:** If two vehicles traveling in opposite directions meet in the middle of the bridge, a deadlock occurs. Each vehicle controls one half of the bridge and is waiting for the other half to clear; neither can move forward, creating a permanent stall.
- **Recovery via Preemption:** To break this deadlock, an external force must intervene and force one of the vehicles to back up, rolling back its execution state and preempting its resource so the other vehicle can pass. If a long line of cars has built up behind the moving vehicle, multiple cars may have to be backed up sequentially to clear the jam.
- **Starvation Contradiction:** If a continuous stream of east-bound vehicles crosses the bridge at high speed, west-bound vehicles will sit stalled at the bank indefinitely. This is an example of **Starvation**, not deadlock; the system is still making forward progress, and the stall will clear naturally once the east-bound traffic thins out.

7.3 Non-Deterministic Lock Deadlocks

In multithreaded applications, deadlocks regularly manifest as non-deterministic timing bugs that depend on the exact order in which instructions interleave.

```
c

// Cross-Locking Deadlock Pattern Architecture

pthread_mutex_t lock_x = PTHREAD_MUTEX_INITIALIZER;
pthread_mutex_t lock_y = PTHREAD_MUTEX_INITIALIZER;

void *Thread_A_Routine(void *arg) {
    pthread_mutex_lock(&lock_x);    // Acquire Lock X successfully
    // If a context switch occurs here, Thread B can run and grab Lock Y
```

```

pthread_mutex_lock(&lock_y);    // Attempt to acquire Lock Y; stalls if held by B

/* Execute Critical Work */
pthread_mutex_unlock(&lock_y);
pthread_mutex_unlock(&lock_x);
return NULL;
}

void *Thread_B_Routine(void *arg) {
    pthread_mutex_lock(&lock_y);    // Acquire Lock Y successfully
    pthread_mutex_lock(&lock_x);    // Attempt to acquire Lock X; stalls if held by A

    /* Execute Critical Work */
    pthread_mutex_unlock(&lock_x);
    pthread_mutex_unlock(&lock_y);
    return NULL;
}

```

This code pattern creates a classic **"Unlucky Case"** race condition. If Thread A acquires Lock X and is immediately context-switched out before it can request Lock Y, Thread B can run and acquire Lock Y.

When both threads resume, Thread A will stall waiting for Lock Y, and Thread B will stall waiting for Lock X. Neither thread can make forward progress, locking up both paths permanently. This bug is exceptionally difficult to diagnose because it is non-deterministic; under a **"Lucky"** scheduling sequence, one thread may clear both locks before the second thread starts, allowing the program to complete successfully without any signs of a fault.

7.4 Multi-Resource Vectors: Space and Sockets

Deadlocks are not limited to software locks; they can occur across any finite system resources, including physical memory or network connections.

Memory Space Deadlocks

Consider a server with exactly 2 MB of available physical memory running two threads:

- **Thread A:** Calls `AllocateOrWait(1 MB)`, succeeds, and is context-switched out.
- **Thread B:** Calls `AllocateOrWait(1 MB)`, succeeds, and consumes the remaining memory.
- Next, both threads attempt to allocate an additional 1 MB of memory before releasing their initial allocations. Because the system's memory is fully exhausted, both threads enter a blocked state waiting for space to clear. Since neither thread can proceed to call `Free()`, the system enters a permanent memory deadlock.

Section 8: Coffman Conditions & Resource-Allocation Graphs (RAG)

8.1 The Four Coffman Conditions

For a deadlock to occur, all four of the following physical conditions must hold simultaneously within the system:

1. **Mutual Exclusion:** Only one thread can use a specific resource at any given time. If a resource is fully shareable, it cannot participate in a deadlock.
2. **Hold and Wait:** A thread must currently hold at least one resource while waiting to acquire additional resources that are held by other active threads.
3. **No Preemption:** Resources cannot be forcefully reclaimed from a thread. A resource can only be released voluntarily by the thread holding it after its task finishes.

4. **Circular Wait:** There must exist a closed, circular loop of waiting threads, $\{T_1, T_2, \dots, T_n\}$, where T_1 is waiting for a resource held by T_2 , T_2 is waiting for a resource held by T_3 , and T_n is waiting for a resource held by T_1 .

8.2 Resource-Allocation Graph (RAG) Modeling

The operating system can model and detect deadlocks at runtime using a directed graph structure called a **Resource-Allocation Graph (RAG)**.

Mathematical Graph Formulation

- The graph consists of a set of vertices V and directed edges E .
- The vertices are partitioned into two distinct classes:
 - Thread Nodes: $T = \{T_1, T_2, \dots, T_n\}$, represented as circles.
 - Resource Type Nodes: $R = \{R_1, R_2, \dots, R_m\}$, represented as squares. Internal dots within a square indicate the number of available instances of that resource type.
- **Directed Request Edge** ($T_i \rightarrow R_j$): Indicates that thread T_i is currently blocked and waiting to acquire an instance of resource type R_j .
- **Directed Assignment Edge** ($R_j \rightarrow T_i$): Indicates that a specific instance of resource type R_j has been allocated to and is currently held by thread T_i .

Analyzing Cycles and Deadlocks

- **Single-Instance Resources:** If every resource type node contains exactly one instance, **the presence of a cycle in the RAG is a necessary and sufficient condition for a deadlock**. If a cycle exists, the system is deadlocked; if no cycle exists, the system is safe.
- **Multi-Instance Resources:** If resource nodes contain multiple instances, a cycle is a *necessary* but *not sufficient* condition for a deadlock. If a cycle forms but an external thread outside the loop holds an instance of the same resource, that external thread can complete its work, release its instance, and break the cycle, resolving the stall without deadlock.

Section 9: Algorithmic Deadlock Detection & Strategic Management Frameworks

9.1 The Deadlock Detection Vector Algorithm

For systems with multi-instance resources, the kernel uses a vector-based bank detection algorithm to scan the system for deadlocks. The algorithm attempts to find a valid execution sequence that allows all current threads to finish.

Vector Parameters

Let m be the number of resource types in the system.

- **[Available]** : A vector of length m tracking the current count of free, unallocated resources of each type.
- **[Allocation_x]** : A vector of length m tracking the resources currently held by thread X .
- **[Request_x]** : A vector of length m tracking the additional resources currently requested by thread X .

Algorithmic Execution Steps

```

1. Initialize a working tracking vector: [Avail] = [FreeResources].
2. Construct a list named UNFINISHED containing all active system nodes.

3. LOOP:
  done = true
  Foreach node in UNFINISHED {
    // Check if the node's resource request can be satisfied by the available pool
    if ([Request_node] <= [Avail]) {
      remove node from UNFINISHED; // Assume the task can finish and return its resources
      [Avail] = [Avail] + [Allocation_node]; // Reclaim all resources held by this node
      done = false
    }
  }
  if (done == false) GOTO LOOP; // Continue iterating if progress was made

4. EVALUATION:
  if (UNFINISHED list is NOT empty) {
    The remaining nodes left inside the UNFINISHED list are formally DEADLOCKED.
  }

```

9.2 Complete Step-by-Step Matrix Simulation

Consider a system running three threads (T_1, T_2, T_3) and managing two resource types (R_1, R_2). Let's trace the deadlock detection algorithm against a specific resource allocation state.

Initial Vector Configurations

[FreeResources] = [0, 1] (0 instances of R1, 1 instance of R2 are free)

Current Allocation State:

[Alloc_T1] = [1, 0] | [Request_T1] = [0, 1]
 [Alloc_T2] = [0, 1] | [Request_T2] = [1, 0]
 [Alloc_T3] = [0, 1] | [Request_T3] = [0, 0]

We step through the detection algorithm to evaluate system safety:

Iteration Step	[Avail] Vector State	UNFINISHED List Contents	Node Evaluated	Conditional Check Status	Verification Results & Actions
Step 0	[0, 1]	[T1, T2, T3]	—	—	Initialization; copy free resources to [Avail].
Step 1	[0, 1]	[T1, T2, T3]	T_1	$[0, 1] \leq [0, 1]$ (TRUE)	T_1 's request can be satisfied. Move T_1 out of the list and reclaim its resources: [Avail] = [0, 1] + [1, 0] = [1, 1].
Step 2	[1, 1]	[T2, T3]	T_2	$[1, 0] \leq [1, 1]$ (TRUE)	T_2 's request can be satisfied. Move T_2 out of the list and reclaim its resources: [Avail] = [1, 1] + [0, 1] = [1, 2].
Step 3	[1, 2]	[T3]	T_3	$[0, 0] \leq [1, 2]$ (TRUE)	T_3 's request is empty and passes. Move T_3 out of the list: [Avail] = [1, 2] + [0, 1] = [1, 3].
Step 4	[1, 3]	Empty	—	—	Loop completes. Since UNFINISHED is completely empty, the system is safe and free of deadlocks.

9.3 Strategic Management Frameworks

Operating systems use four core paradigms to manage the risk of resource deadlocks:

1. **Deadlock Prevention:** Design the system's code and software structures so that it is physically impossible for at least one of the four Coffman conditions to form. For example, forcing all threads to acquire locks in a strict, pre-defined global order completely eliminates the risk of circular waits.
2. **Deadlock Avoidance:** The kernel monitors resource requests dynamically at runtime. If a request is correct but has the potential to lead to a circular wait down the line, the kernel deliberately delays the thread's request until safety is restored. The **Banker's Algorithm** is the classic algorithmic framework used to implement deadlock avoidance.
3. **Deadlock Recovery:** Allows deadlocks to happen naturally, using a background detection thread to spot cycles in the resource-allocation graph. Once a deadlock is detected, the system recovers by force-terminating the deadlocked threads or rolling back their execution states to free up resources.
4. **Deadlock Denial (The Ostrich Algorithm):** Completely ignores the possibility of deadlocks. If deadlocks are rare and the engineering cost of prevention or avoidance is high, the system simply assumes deadlocks won't happen, relying on users to restart their applications if a stall occurs. Nearly all major general-purpose operating systems, including Linux, macOS, and Windows, use this pragmatic approach.

Section 10: High-Yield Practice Exam Problems with Detailed Solutions

Problem 1: Hard Real-Time Schedulability Testing

A hard real-time system is configured to coordinate three independent periodic tasks with parameters $T_i = (P_i, C_i)$:

- Task 1: $T_1 = (5, 2)$
 - Task 2: $T_2 = (6, 1)$
 - Task 3: $T_3 = (10, 1)$
1. Calculate the total processor utilization U_{Total} for this task set.
 2. Determine if this task set is guaranteed to meet all deadlines under **Rate Monotonic Scheduling (RMS)**.
 3. Determine if this task set is guaranteed to meet all deadlines under **Earliest Deadline First (EDF)**.

Solution

1. **Calculate Total Processor Utilization:** We compute individual utilizations ($U_i = C_i/P_i$) and sum them up: $U_1 = \frac{2}{5} = 0.400$ $U_2 = \frac{1}{6} \approx 0.167$ $U_3 = \frac{1}{10} = 0.100$ $U_{\text{Total}} = 0.400 + 0.167 + 0.100 = \mathbf{0.667}$
2. **Evaluate Rate Monotonic Schedulability:** We apply the Liu & Layland utilization bound test for $n = 3$ tasks: $\text{Bound} = 3 \times (2^{1/3} - 1) \approx 3 \times (1.2599 - 1) \approx \mathbf{0.780}$
 - **RMS Analysis:** Comparing our values, we see that $U_{\text{Total}} = 0.667 \leq \text{Bound} = 0.780$. Because the total utilization is below the sufficient threshold, **the task set is guaranteed to meet every deadline under RMS.**
3. **Evaluate Earliest Deadline First Schedulability:** We check the EDF schedulability condition ($\sum U_i \leq 1.0$):
 - **EDF Analysis:** Since $U_{\text{Total}} = 0.667 \leq 1.0$, the workload satisfies the exact condition. **The task set is guaranteed to be fully schedulable under EDF.**

Problem 2: Tracking Strides and Pass Counters

A stride scheduler coordinates three active threads (Thread A, Thread B, Thread C) using a global constant $W = 60,000$. The threads are assigned the following ticket counts:

- Thread A: 300 tickets
- Thread B: 100 tickets
- Thread C: 200 tickets

1. Calculate the execution stride length for each thread.
2. Assuming all pass counters are initially zero, trace the scheduler's choices and update the counters step-by-step for the first 4 scheduling intervals.

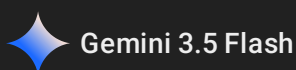
Solution

1. **Calculate Stride Lengths:** Using the formula $S_i = W / N_i$, we compute the stride lengths: Stride $S_A = \frac{60,000}{300} = 200$
 Stride $S_B = \frac{60,000}{100} = 600$ Stride $S_C = \frac{60,000}{200} = 300$

2. **Trace Scheduling Decisions and Counter Updates:** At each interval, the scheduler selects the thread with the lowest pass counter, runs it for a single time quantum, and adds its stride to its counter.

- **Interval 1:** All pass counters are tied at 0 (Pass_A = 0, Pass_B = 0, Pass_C = 0).
 - **Selection:** **Thread A** is chosen (resolving the tie arbitrarily).
 - **Update:** Pass_A = 0 + 200 = 200 .
 - **State:** [Pass_A = 200, Pass_B = 0, Pass_C = 0] .
- **Interval 2:** Evaluating counters [200, 0, 0] , Thread B and Thread C are tied for the lowest value (0).
 - **Selection:** **Thread B** is chosen.
 - **Update:** Pass_B = 0 + 600 = 600 .
 - **State:** [Pass_A = 200, Pass_B = 600, Pass_C = 0] .
- **Interval 3:** Evaluating counters [200, 600, 0] , Thread C has the lowest value (0).
 - **Selection:** **Thread C** is chosen.
 - **Update:** Pass_C = 0 + 300 = 300 .
 - **State:** [Pass_A = 200, Pass_B = 600, Pass_C = 300] .
- **Interval 4:** Evaluating counters [200, 600, 300] , Thread A has the lowest value (200).
 - **Selection:** **Thread A** is chosen.
 - **Update:** Pass_A = 200 + 200 = 400 .
 - **State:** [Pass_A = 400, Pass_B = 600, Pass_C = 300] .

This deterministic tracking ensures that over time, the frequency of thread execution matches their ticket ratios (300 : 100 : 200 $\implies$ 3 : 1 : 2) without any statistical variance.



CS162 Operating Systems & Systems Programming

Lecture 13: Proportional Share Scheduling, Real-Time Scheduling, and Deadlocks (Comprehensive Study Notes)

Section 1: Multi-Level Feedback Queue Review & Shifting Scheduling Horizons

1.1 Algorithmic Framework of Multi-Level Feedback Queues (MLFQ)

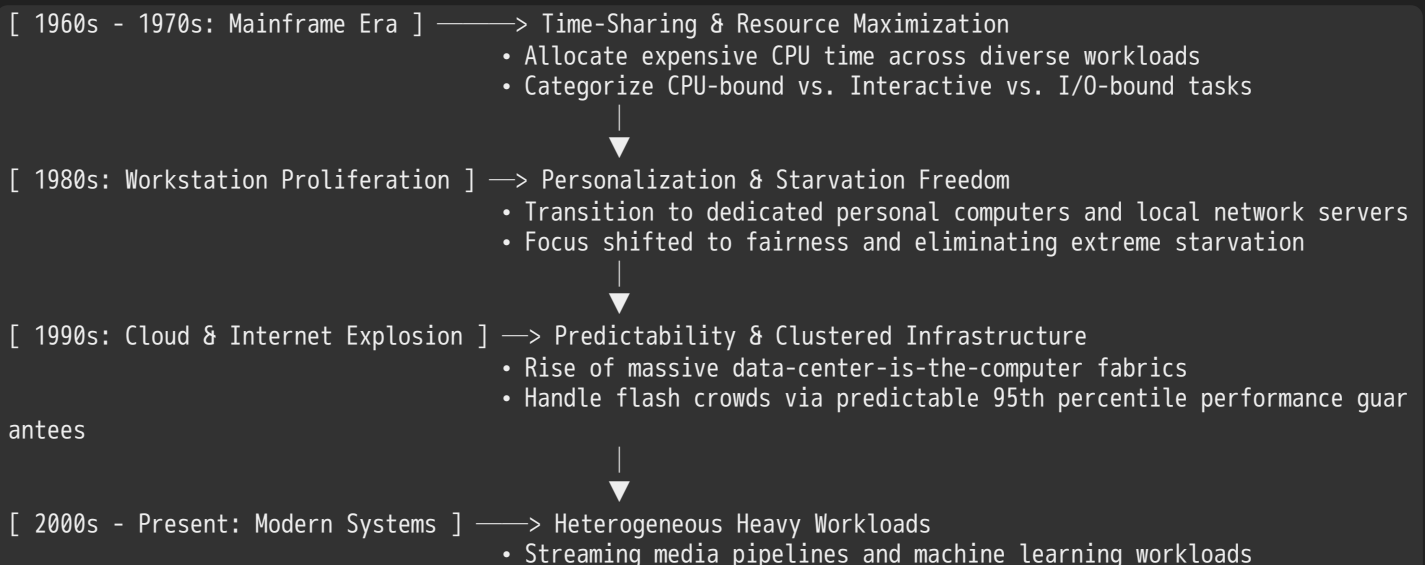
The **Multi-Level Feedback Queue (MLFQ)** is an adaptive scheduling framework designed to analyze an application's past execution history to predict its upcoming CPU burst requirements. Originally implemented in the Compatible Time-Sharing System (CTSS), MLFQ addresses a core systems challenge: optimizing scheduling decisions without forcing applications to pre-declare their compute parameters.

Core Structural Invariants

- **Priority-Tiered Layout:** The scheduler maintains multiple distinct ready queues, where each queue is assigned a unique priority level. Queues near the top handle high-priority foreground workloads, while queues at the bottom process low-priority background workloads.
- **Divergent Scheduling Disciplines:** Each individual queue can run its own scheduling policy. For instance, high-priority foreground queues typically use a preemptive **Round-Robin (RR)** policy to ensure responsiveness, while the lowest-priority background queue uses a non-preemptive **First-Come, First-Served (FCFS)** policy to maximize raw instruction throughput.
- **Exponential Quantum Scaling:** To prevent long-running tasks from causing excessive context-switching overhead, the time quantum increases exponentially at lower priority levels (e.g., top queue $Q = 8$ ms, middle queue $Q = 16$ ms, cascading down to an un-quantized FCFS baseline).
- **Dynamic Priority Adjustments:** 1. A newly created or awakened thread is placed at the back of the highest-priority queue. 2. If the thread consumes its entire allocated time quantum without blocking for I/O, the scheduler assumes it is a compute-bound task and demotes it down one level to a lower priority queue. 3. If the thread blocks for I/O or yields the CPU voluntarily before its time slice expires, the scheduler identifies it as an interactive or I/O-bound task, retaining or promoting its position near the top of the priority tiers.

1.2 The Changing Landscape of Operating System Workloads

As computer systems have evolved from central mainframes to hyper-scale cloud environments, the primary goals of scheduling algorithms have fundamentally shifted to meet new architecture profiles:



Section 2: Proportional-Share Paradigms: Lottery & Stride Scheduling

2.1 Philosophy of Proportional Sharing

Traditional scheduling policies (such as strict priority scheduling or MLFQ) prefer to give the CPU entirely to a prioritized job. Consequently, non-prioritized or lower-priority tasks can be starved of CPU cycles indefinitely if high-priority tasks continue to arrive. However, in general-purpose computing, priorities are often a means to an end (such as boosting interactive tasks for responsiveness), not the end goal itself.

Proportional-Share Scheduling replaces strict preemption hierarchies with a resource-sharing model. Instead of completely blocking lower-priority tasks, the scheduler guarantees each job a specific, predictable percentage of total CPU cycles based on its assigned weight. Because every active job is guaranteed a non-zero portion of the CPU, starvation is structurally eliminated, ensuring that all tasks make steady forward progress. This model is closely related to the principle of **Fair Queuing** used in computer networking.

2.2 Lottery Scheduling Mechanics

Lottery Scheduling is a randomized approach to proportional-share scheduling. The operating system represents CPU allocation fractions as virtual **Lottery Tickets**.

Core Selection Logic

- Every active job is assigned a specific number of lottery tickets.
- At each new time slice interval, the scheduler draws a random ticket number from the pool of outstanding tickets.
- The task that holds the winning ticket number is awarded the CPU core for that time slice.
- **Statistical Convergence:** While any individual draw is random, over time, the total CPU allocation converges mathematically so that each job's runtime is directly proportional to its number of tickets.

Approximating Shortest Remaining Time First (SRTF)

To maximize responsiveness without running into starvation issues, the system can adjust ticket counts dynamically based on execution characteristics:

- Short, interactive, or I/O-bound jobs are awarded a large pool of tickets, which gives them a high probability of gaining the CPU quickly and keeps the system responsive.
- Long-running, compute-bound jobs are given a smaller pool of tickets. They run less frequently but are still guaranteed to make progress since they retain at least one ticket, preventing starvation.
- **Graceful Scaling under Load:** Unlike strict priority systems, adding or deleting jobs in a lottery scheduler scales resource allocation gracefully and proportionally across all active tasks, entirely independent of the total number of outstanding tickets in the system.

2.3 Mathematical Allocation Analysis under Variable Workload Mixes

To calculate a task's expected CPU share, we use the following probability formula: Expected CPU Share $U_i = \frac{N_i}{\sum_{j=1}^M N_j}$

Where N_i represents the number of tickets allocated to target job i , and M tracks the total number of active threads in the ready queue.

Suppose the system uses a configuration where short, interactive tasks are assigned **10 tickets** and long, compute-bound tasks are assigned **1 ticket**. The table below analyzes the expected CPU allocation across different workload combinations:

Active Job Mix Layout (# Short / # Long)	Expected CPU Share (Per Short Job)	Expected CPU Share (Per Long Job)	Derivation and Probability Formulas
1 / 1	91%	9%	Total = 10 + 1 = 11 tickets. Short Share = $\frac{10}{11} \approx 90.9\%$; Long Share = $\frac{1}{11} \approx 9.09\%$.
0 / 2	N/A	50%	Total = 1 + 1 = 2 tickets. Long Share = $\frac{1}{2} = 50\%$ per process.
2 / 0	50%	N/A	Total = 10 + 10 = 20 tickets. Short Share = $\frac{10}{20} = 50\%$ per process.
10 / 1	9.9%	0.99%	Total = (10 × 10) + 1 = 101 tickets. Short Share = $\frac{10}{101} \approx 9.9\%$; Long Share = $\frac{1}{101} \approx 0.99\%$.
1 / 10	50%	5%	Total = 10 + (10 × 1) = 20 tickets. Short Share = $\frac{10}{20} = 50\%$; Long Share = $\frac{1}{20} = 5\%$.

Handling System Overload Bounds

If the system faces extreme overload (for example, a load average of 100 or more concurrent short tasks), individual time allocations drop significantly, making it difficult for any single application to make reasonable forward progress. To protect system stability in this scenario, administrative policies must intervene to drop the active load, such as force-terminating non-critical background jobs or logging out inactive users.

2.4 Code Example: Lottery Ticket Selection Loop

This code shows how a lottery scheduler selects the next winning task by traversing an internal array of active threads.

```

c

#include <stdio.h>
#include <stdlib.h>

typedef struct thread {
    char name[16];
    int ticket_count;
} thread_t;

// Selects the winning thread by accumulating tickets sequentially
thread_t* schedule_lottery_winner(thread_t* active_threads, int dynamic_task_count) {
    int total_system_tickets = 0; //

    // Accumulate the sum of all outstanding tickets across active tasks
    for (int i = 0; i < dynamic_task_count; i++) {
        total_system_tickets += active_threads[i].ticket_count; //
    }
}

```

```

if (total_system_tickets == 0) return NULL;

// Pick a random "dart" value d within the range [1, total_system_tickets]
int dart = (rand() % total_system_tickets) + 1; //
int running_ticket_sum = 0;

// Step through the tasks sequentially; select the first task that pushes the sum past the
dart
for (int j = 0; j < dynamic_task_count; j++) {
    running_ticket_sum += active_threads[j].ticket_count; //
    if (running_ticket_sum >= dart) { //
        return &active_threads[j]; // Return pointer to the winning thread
    }
}
return &active_threads[dynamic_task_count - 1];
}

```

2.5 Lottery Scheduling Unfairness Metric

To evaluate the short-term performance of lottery scheduling, researchers define an **Unfairness Metric** (U). Consider two identical jobs, A and B, running for a specific number of time quanta, where each is assigned 50% of the ticket pool.

Unfairness is defined as the ratio of the completion times of the two jobs: $U = \frac{\text{Termination Time of the First Completing Job}}{\text{Termination Time of the Last Completing Job}}$

As a function of job length (measured in total executed time slices), the unfairness metric exhibits a clear curve:

- For short job lengths, U is low (around 0.5), meaning randomness can cause one job to complete significantly ahead of its peer due to short-term luck.
- As job length increases toward 100 or 1000 time slices, the unfairness metric converges toward 1.0. This demonstrates that while lottery scheduling can be unfair over short durations, it achieves perfect, deterministic proportional fairness over long-running workloads.

2.6 Stride Scheduling: Non-Random Proportional Sharing

While lottery scheduling is simple, its short-term unfairness can cause issues for workloads that require steady resource allocation. To eliminate this randomness while maintaining proportional share guarantees, systems developers use **Stride Scheduling**.

[STRIDE SCHEDULING FORMULATION]

Compute Stride length for each active task: $\text{Stride} = \text{Big\#} \ W / \text{Tickets}$

- High-ticket tasks receive a small stride.
- Low-ticket tasks receive a large stride.



[Selection Step: Choose the active task with the lowest Pass Counter]

- Run selected task for a single time quantum.
- Update tracking register: $\text{Pass_Counter} = \text{Pass_Counter} + \text{Stride}$.

Step-by-Step Mathematical Walkthrough

The scheduler calculates an execution **Stride** for each job, which is inversely proportional to its ticket allocation: $\text{Stride } S_i = \frac{\text{Large Constant } W}{N_i}$

Where N_i represents the number of tickets assigned to job i . Every job tracks its execution progress using a local register called a **Pass Counter**. At each scheduling interval, the scheduler selects the job with the lowest pass counter, runs it for a

single time slice, and increments its pass counter by its stride value. This mechanism ensures that jobs with smaller strides (more tickets) are selected more frequently, matching their target allocations exactly.

Consider a system with a constant $W = 10,000$ and three active jobs with the following ticket allocations:

- Job A: 100 tickets $\implies$ Stride $S_A = \frac{10,000}{100} = \mathbf{100}$
- Job B: 50 tickets $\implies$ Stride $S_B = \frac{10,000}{50} = \mathbf{200}$
- Job C: 250 tickets $\implies$ Stride $S_C = \frac{10,000}{250} = \mathbf{40}$

Chronological Scheduling Execution Matrix

Assuming all pass counters are initially zero, the table below traces the first 6 scheduling intervals:

Step Allocation	Job Selected (Lowest Pass)	Pass Counter A (Before $\rightarrow$ After)	Pass Counter B (Before $\rightarrow$ After)	Pass Counter C (Before $\rightarrow$ After)	Invariant Status Summary
Initial	—	0	0	0	All pass counters tied at 0.
Step 1	Job A (Tie-break)	0 $\rightarrow$ 100	0	0	Job A runs; its counter increases by its stride (+100).
Step 2	Job B (Tie-break)	100	0 $\rightarrow$ 200	0	Job B runs; its counter increases by its stride (+200).
Step 3	Job C	100	200	0 $\rightarrow$ 40	Job C runs; its counter increases by its stride (+40).
Step 4	Job C	100	200	40 $\rightarrow$ 80	Job C still has the lowest counter (80 < 100); it runs again.
Step 5	Job C	100	200	80 $\rightarrow$ 120	Job C runs for a third time (120).
Step 6	Job A	100 $\rightarrow$ 200	200	120	Job A has the lowest counter (100 < 120); it is selected.

This deterministic execution flow ensures that Job C runs exactly twice as often as Job A, and Job A runs exactly twice as often as Job B, matching their ticket ratios (250 : 100 : 50 $\implies$ 5 : 2 : 1) without any of the statistical variance seen in lottery scheduling.

Note: Implementing stride scheduling in production requires additional mechanisms to handle integer counter wrap-around and to initialize the pass counters of newly created or arriving tasks.

Section 3: The Linux Completely Fair Scheduler (CFS) Deep Dive

3.1 Core Philosophy and the Virtualization Model

The Linux **Completely Fair Scheduler (CFS)** is an elegant implementation of proportional-share scheduling used in modern production kernels.

Idealized Multi-Threading Model (Simultaneous Execution):

Core: [Thread 1 (1/N)][Thread 2 (1/N)][Thread 3 (1/N)] —→ Simultaneous continuous cycles

Hardware Multi-Threading Model (Time-Slices):

Core: [Thread 1 (Q)][Thread 2 (Q)][Thread 3 (Q)] —→ Discrete interleaved chunks

The fluid conceptual multi-threading model vs. discrete hardware execution.

CFS is built around an idealized fluid model: a **perfectly subdivided hardware processor** where N active threads run simultaneously on the same core, with each thread receiving an exact, continuous $\frac{1}{N}$ fraction of total CPU instruction cycles.

Since real hardware cannot execute an arbitrary number of threads simultaneously in infinitely small subdivisions, the operating system must allocate the full CPU to a single thread for discrete time slices. The core goal of CFS is to minimize execution variance, tracking how much time each thread has run to keep them roughly in sync with the idealized model.

3.2 CFS Responsiveness Boundaries: Target Latency & Minimum Granularity

To maintain predictability and prevent starvation, CFS manages context switches using two key bounds:

1. Target Latency (`sched_latency`)

The target latency is the fixed window of time over which **every single active runnable process must be scheduled at least once**. Under standard equal sharing with n active processes, the base time quantum Q_i allocated to a task is calculated as:

$$Q_i = \frac{\text{Target_Latency}}{n}$$

- Consider a system with a target latency of 20 ms running 4 active processes. Each process is awarded an equal time slice of $\frac{20 \text{ ms}}{4} = 5 \text{ ms}$.
- If the workload surges to 200 active processes, a naive application of the formula drops each individual time slice to $\frac{20 \text{ ms}}{200} = 0.1 \text{ ms}$. This micro-slice causes severe performance issues, as the system spends almost all its time executing context switches rather than running application instructions.

2. Minimum Granularity (`sched_min_granularity`)

To protect system throughput from context-switching overhead, CFS enforces a lower bound called the **Minimum Granularity**. This parameter defines the absolute shortest time slice any individual thread can be assigned during a scheduling cycle.

- If a system has a target latency of 20 ms and a minimum granularity floor of 1 ms:
- When handling 200 concurrent processes, CFS drops the target latency constraint and overrides the time slice calculation, awarding each process the minimum floor of 1 ms. This pushes the total scheduling period out to 200 ms ($200 \times 1 \text{ ms}$), protecting system throughput at the cost of longer response times under heavy load.

3.3 Nice Values, Weights, and the `vruntime` Transformation

To support proportional-share allocation, CFS maps traditional UNIX **nice values** (which range from -20 for high priority to 19 for low priority) to an exponential scale of internal weights:

$$\text{Weight } W_i = \frac{1024}{(1.25)^{\text{nice}}}$$

This exponential mapping ensures that a relative difference of 5 nice points always translates to a fixed, predictable ratio of CPU cycles, regardless of the absolute nice values chosen. Because $(1.25)^5 \approx 3$, two tasks whose nice values differ by 5 will always maintain a **3 : 1 ratio of weights and CPU cycles**.

Using these weights, the scheduler shifts from tracking raw physical CPU runtime to tracking a normalized metric called **Virtual Runtime (`vruntime`)**:

$$\text{vruntime}_i = \text{Real Physical CPU Execution Time} \times \frac{\text{Baseline Weight (1024)}}{W_i}$$

Operational Implications of the Formula

- **High-Priority Tasks (Low Nice Value / Large Weight):** For a high-priority task, $W_i > 1024$, which means its virtual runtime accumulates **slower** than real physical time.
- **Low-Priority Tasks (High Nice Value / Small Weight):** For a low-priority task, $W_i < 1024$, which means its virtual runtime accumulates **faster** than real physical time.
- **Selection Policy:** The scheduler always selects the thread with the **absolute lowest virtual runtime** to run next. This automatically rewards high-priority tasks with longer and more frequent physical time slices while keeping all tasks synchronized over time.

Chronological Scheduling Example

Consider a system with a target latency of 20 ms running two CPU-bound threads:

- Thread A has a weight of 1 ($W_A = 1$).
- Thread B has a weight of 4 ($W_B = 4$).
- Total weight = $1 + 4 = 5$.
- Time quantum for A: $Q_A = 20 \text{ ms} \times \frac{1}{5} = 4 \text{ ms}$.
- Time quantum for B: $Q_B = 20 \text{ ms} \times \frac{4}{5} = 16 \text{ ms}$.

CFS Proportional Runtime Mapping:

Thread A (Weight = 1): Physical Time = 4ms $\longrightarrow$ Virtual Runtime = $4\text{ms} * (1024 / 1) = 4096\text{ms}$

Thread B (Weight = 4): Physical Time = 16ms $\longrightarrow$ Virtual Runtime = $16\text{ms} * (1024 / 4) = 4096\text{ms}$

Both threads accumulate virtual runtime at matching rates, keeping them in sync.

3.4 Caching and Lookups via Red-Black Trees

To track and select tasks efficiently, CFS stores all runnable threads in a self-balancing **Red-Black Tree**, sorted strictly by their virtual runtimes (`vruntime`).

- **Algorithmic Complexity:** Inserting a thread or removing it from the tree requires balanced path traversals that run in logarithmic time, $O(\log N)$, where N is the number of active threads.
- **Leftmost Node Caching:** Because the scheduler needs to access the task with the absolute lowest `vruntime` at every interval, modern implementations optimize lookups by **caching a pointer directly to the leftmost leaf node**. This allows the scheduler to select the next thread to run in instant constant time, $O(1)$, avoiding the need to traverse the tree structure on every context switch.
- **Automatic Interactivity Credits:** When an interactive thread blocks for I/O, it is removed from the tree, and its `vruntime` stops accumulating. While it sleeps, the `vruntime` values of active, compute-bound threads continue to increase. When the interactive thread wakes up, its historical `vruntime` is significantly lower than its competitors. As a result, it is immediately placed at the far left of the Red-Black tree, allowing it to preempt the compute-bound tasks and run immediately. This design handles interactive tasks automatically without requiring complex heuristics.

Section 4: Real-Time Scheduling Systems & Workload Models

4.1 Philosophy of Real-Time Systems

In conventional operating systems, the scheduler optimizes for system metrics like maximum throughput or average response times.

In a **Real-Time System (RTS)**, the primary goal is **Predictability of Performance**. The system must guarantee with complete confidence that critical operations will complete within strict, worst-case timing deadlines imposed by the physical world. Real-time scheduling is not about raw processing speed; it is about enforcing absolute timing predictability.

- **Hard Real-Time Systems:** Used for mission-critical, safety-oriented systems (such as medical pacemakers, automotive brake controllers, or automated flight systems). Missing a single deadline is treated as a fatal system failure. The system must prove *a priori* (before deployment) that all tasks will meet their deadlines under worst-case scenarios.
- **Soft Real-Time Systems:** Used for applications like live multimedia streaming or video playback. Missing a deadline drops a frame but does not cause a fatal failure; the system simply aims to meet deadlines with high probability.

4.2 Real-Time Workload Formulations

A real-time workload model represents tasks as a series of repeating execution requirements defined by specific constraints:

- Tasks are assumed to be independent, fully preemptable, and can arrive at arbitrary release times.
- Each task i is defined by a tuple: (P_i, C_i) , where P_i represents its recurring **Period** window, and C_i specifies its maximum **Worst-Case Computation Time**.
- **Periodic Arrivals:** Tasks arrive cyclically at fixed intervals: $A_i^{t+1} = A_i^t + P_i$.
- **Relative Deadlines:** Each task must finish its computation burst before the end of its current period: $D_i^t = A_i^t + P_i$.

4.3 Why Conventional Scheduling Fails Real-Time Deadlines

Traditional scheduling policies like Round-Robin are completely unsuited for real-time systems because they make scheduling choices without considering task deadlines.

As shown in the execution trace above, if multiple real-time tasks are multiplexed using a uniform time quantum, a task with a long period can easily starve a short-period task with an immediate deadline, causing the system to miss its deadline and fail. To handle real-time workloads safely, scheduling policies must factor deadlines directly into their selection logic.

Section 5: Hard Real-Time Policies: Rate Monotonic & Earliest Deadline First

5.1 Rate Monotonic Scheduling (RMS)

Rate Monotonic Scheduling (RMS) is a foundational static priority scheduling policy for periodic real-time workloads.

Algorithmic Assignment Invariant

- Priorities are assigned statically based on the length of each task's period: **the shorter the period, the higher its scheduling priority**.
- Mathematically: $P_i < P_j \implies \pi_i > \pi_j$, where π represents priority.
- At any scheduling interval, the scheduler always executes the active task with the highest priority. This can be implemented using a strict priority scheduler, such as the real-time priority levels in Linux.

The Liu & Layland Schedulability Bound Theorem

To determine if a set of periodic tasks can be scheduled safely under RMS without missing deadlines, we evaluate the **Liu & Layland Utilization Bound Test**. The processor utilization U of a task set is the sum of the ratios of computation time to period for all tasks: $U = \sum_{i=1}^n \left(\frac{C_i}{P_i} \right)$

A set of n periodic tasks is guaranteed to be fully schedulable under RMS if its total processor utilization satisfies the following inequality: $\sum_{i=1}^n \left(\frac{C_i}{P_i} \right) \leq n (2^{1/n} - 1)$

- **Sufficient but Not Necessary:** This utilization test is *sufficient* but not *necessary*. If a task set passes the test, it is guaranteed to meet all deadlines. If it fails, it might still be schedulable, depending on the exact alignment of task periods. For example, if all periods are perfectly harmonic (meaning each period evenly divides the larger ones), the utilization bound scales up to a full **100%**.

- **Asymptotic Limit:** As the number of independent tasks grows toward infinity, the utilization bound converges to a fixed mathematical limit: $\lim_{n \rightarrow \infty} n(2^{1/n} - 1) = \ln 2 \approx \mathbf{69.3\%}$

5.2 RMS Execution Failure Case Study

Consider a real-time system running three periodic tasks with parameters $T_i = (P_i, C_i)$:

- Task 1: $T_1 = (4, 1) \implies U_1 = \frac{1}{4} = \mathbf{0.25}$
- Task 2: $T_2 = (5, 2) \implies U_2 = \frac{2}{5} = \mathbf{0.40}$
- Task 3: $T_3 = (7, 2) \implies U_3 = \frac{2}{7} \approx \mathbf{0.286}$

Load Verification

We sum the individual task utilizations to check the total processor load: $U_{\text{Total}} = 0.25 + 0.40 + 0.286 = \mathbf{0.936}$

Because $U_{\text{Total}} \leq 1.0$, the workload does not physically overload the processor, meaning a valid schedule is theoretically possible.

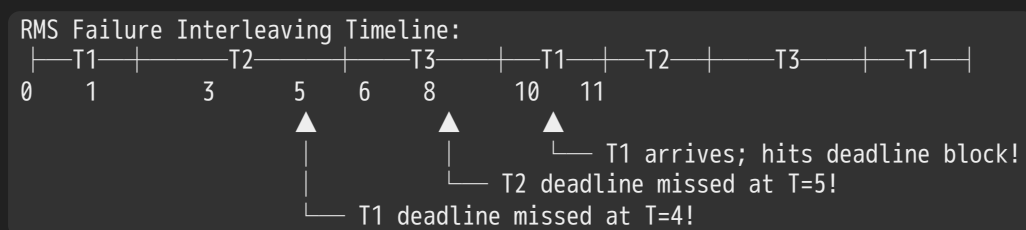
Applying the Schedulability Test

Next, we evaluate the Liu & Layland bound for $n = 3$ tasks: $\text{Bound} = 3 \times (2^{1/3} - 1) \approx 3 \times (1.2599 - 1) \approx \mathbf{0.780}$

Since $0.936 > 0.780$, the task set fails the sufficient utilization test. This tells us that under a static priority RMS policy, deadlines are likely to be missed.

Step-by-Step Chronological Execution Trace

Under RMS, priorities are assigned statically based on period lengths ($P_1 = 4 < P_2 = 5 < P_3 = 7$), resulting in the fixed priority hierarchy: $\pi_1 > \pi_2 > \pi_3$.



Chronological trace of the static priority execution failure.

- **Interval 0 to 1** : T_1 has the highest priority and runs for 1 ms, completing its computation cleanly.
- **Interval 1 to 3** : With T_1 finished, T_2 runs for its required 2 ms burst, completing at 3 ms.
- **Interval 3 to 4** : T_3 runs for 1 ms. At time 4 ms, the period for T_1 expires, prompting it to arrive again and immediately preempt T_3 due to its higher priority.
- **Interval 4 to 5** : T_1 runs for 1 ms. At time 5 ms, the period for T_2 expires, causing it to arrive again.
- **Interval 5 to 7** : T_2 runs for its 2 ms burst, completing at 7 ms.
- **Interval 7 to 8** : At time 7 ms, **Task 3 (T_3) hits its relative deadline!** However, it has only executed for 1 ms out of its required 2 ms computation burst.
- **The Failure:** Because static priorities forced T_3 to be preempted by arriving higher-priority tasks, it misses its deadline at 7 ms, causing a hard real-time system failure.

5.3 Earliest Deadline First (EDF) Scheduling

To avoid the utilization limitations of static priority policies, real-time systems use **Earliest Deadline First (EDF)**. EDF is a dynamic priority scheduling policy where priorities are updated on the fly based on changing task deadlines.

Algorithmic Selection Invariant

At any scheduling interval, the scheduler evaluates all active tasks and allocates the CPU to the task with the **closest absolute deadline** (D_i^t).

The Schedulability Bound Theorem

A set of periodic real-time tasks is guaranteed to meet all deadlines under an EDF policy if and only if its total processor utilization satisfies the following constraint: $\sum_{i=1}^n \left(\frac{C_i}{P_i} \right) \leq 1.0$

This is an **exact schedulability test** (both *necessary* and *sufficient*). As long as the workload does not physically overload the processor, EDF is mathematically guaranteed to find a valid schedule that meets every deadline.

Applying this to our previous counter-example workload ($U_{Total} = 0.936 \leq 1.0$), EDF successfully schedules the tasks without missing any deadlines.

5.4 The Domino Effect Performance Anomaly

While EDF is optimal for underloaded systems ($U \leq 1.0$), it exhibits a critical performance bottleneck under short-term overloads ($U > 1.0$) known as the **Domino Effect**.

If an unexpected hardware spike or an extra task (T_{new}) pushes the total processor utilization past 1.0, the closest-deadline task will consume extra cycles, causing it to miss its deadline slightly.

Because EDF always prioritizes the task whose deadline is closest, the scheduler will cascade this delay down the line. It switches to the next closest task, which now misses its deadline as well, triggering a chain reaction where **every subsequent task in the queue misses its deadline sequentially**. A minor overload can thus cause a complete collapse of deadline compliance across the entire system.

Section 6: Methodologies for Scheduling Evaluation & System Load Curves

6.1 Evaluation Frameworks

Systems engineers use three complementary techniques to evaluate and validate scheduling algorithms:

- 1. **Deterministic Modeling:** Takes a fixed, predetermined workload profile and calculates the exact performance metrics for each policy. While simple and useful for core validation, it only applies to that specific workload profile.
- 2. **Queueing Models:** Uses stochastic probability distributions (such as Poisson arrival rates and exponential service times) to analyze system behavior mathematically. This provides useful long-term averages but can struggle with complex, non-linear real-world workloads.
- 3. **Trace-Driven Implementation and Simulation:** Records actual execution logs from production servers onto a "trace tape" and replays them inside a simulator to test different scheduling algorithms. This provides a flexible and realistic environment to evaluate policies against real-world data before deploying them to production kernels.

6.2 Strategic Selection Synthesis Table

The optimal scheduling policy depends entirely on the primary performance goals of the system:

Primary Optimization Core Target	Recommended Scheduling Policy Selection	Architectural Justification and Mechanics
Maximize Raw CPU Throughput	FCFS	Eliminates all preemption and context-switching overhead, keeping the CPU focused entirely on application code.

Primary Optimization Core Target	Recommended Scheduling Policy Selection	Architectural Justification and Mechanics
Minimize Average Response Time	SRTF Approximation	Provably optimal framework that gets short jobs out of the system quickly.
Maximize I/O Device Throughput	SRTF Approximation	Prioritizes interactive tasks, allowing them to issue I/O requests quickly and keep peripheral devices fully utilized.
Enforce Strict CPU Time Fairness	Linux CFS	Uses virtual runtime tracking (<code>vruntime</code>) to keep all executing threads synchronized with a perfectly subdivided processor model.
Enforce Fair Ready-Queue Wait Times	Round-Robin	Uses fixed time slices to cycle through the ready queue sequentially, ensuring no thread is left waiting for long.
Guarantee Real-Time Deadlines	EDF	Dynamic priority policy that scales up to 100% utilization, always prioritizing the task closest to its deadline.
Favor High-Importance System Tasks	Priority	Enforces a rigid execution hierarchy, guaranteeing that high-priority system tasks always run ahead of background processes.

6.3 The Non-Linear System Load Curve

A common misconception in systems engineering is that a hardware resource (such as a CPU, network link, or storage interface) should be driven to 100% utilization to maximize efficiency.

In reality, response times follow a highly non-linear load curve: as resource utilization approaches **100%**, response times escape toward **infinity**. This occurs because as the system fills up, any minor variation in task arrivals causes long queues to build up faster than they can drain.

Most scheduling algorithms work fine within the stable, linear portion of the curve, but performance degrades rapidly once load passes the **"Knee" of the curve**. When a system hits this inflection point, the correct solution is not to tune the scheduling policy, but to expand the physical hardware capacity to return the system to its stable operating range.

Section 7: Deadlock Architecture: Principles, Case Studies, & Multi-Resource Traps

7.1 Deadlock vs. Starvation

- **Starvation:** A progress anomaly where a fully operational, runnable thread is delayed indefinitely because the scheduler continuously chooses to allocate resources to other tasks. Starvation is a policy issue and *can* resolve on its own if system conditions change (e.g., if high-priority traffic clears).
- **Deadlock:** A structural failure where a set of threads is permanently blocked because they are caught in a circular dependency, each waiting for a resource held by another. Deadlocks are unresolvable and can never break on their own without external intervention, such as force-terminating a process or restarting the system.

7.2 Case Study: The Single-Lane Bridge

The single-lane bridge crossing is a classic physical analogy for resource deadlocks:

- **Resource Mapping:** Each distinct section of the single-lane road acts as an exclusive hardware resource. A vehicle must control the segment beneath it and successfully acquire the upcoming segment before it can move forward.

- **Deadlock State:** If two vehicles traveling in opposite directions meet in the middle of the bridge, a deadlock occurs. Each vehicle controls one half of the bridge and is waiting for the other half to clear; neither can move forward, creating a permanent stall.
- **Recovery via Preemption:** To break this deadlock, an external force must intervene and force one of the vehicles to back up, rolling back its execution state and preempting its resource so the other vehicle can pass. If a long line of cars has built up behind the moving vehicle, multiple cars may have to be backed up sequentially to clear the jam.
- **Starvation Contradiction:** If a continuous stream of east-bound vehicles crosses the bridge at high speed, west-bound vehicles will sit stalled at the bank indefinitely. This is an example of **Starvation**, not deadlock; the system is still making forward progress, and the stall will clear naturally once the east-bound traffic thins out.

7.3 Non-Deterministic Lock Deadlocks

In multithreaded applications, deadlocks regularly manifest as non-deterministic timing bugs that depend on the exact order in which instructions interleave.

```
c

// Cross-Locking Deadlock Pattern Architecture

pthread_mutex_t lock_x = PTHREAD_MUTEX_INITIALIZER;
pthread_mutex_t lock_y = PTHREAD_MUTEX_INITIALIZER;

void *Thread_A_Routine(void *arg) {
    pthread_mutex_lock(&lock_x); // Acquire Lock X successfully
    // If a context switch occurs here, Thread B can run and grab Lock Y
    pthread_mutex_lock(&lock_y); // Attempt to acquire Lock Y; stalls if held by B

    /* Execute Critical Work */
    pthread_mutex_unlock(&lock_y); //
    pthread_mutex_unlock(&lock_x); //
    return NULL;
}

void *Thread_B_Routine(void *arg) {
    pthread_mutex_lock(&lock_y); // Acquire Lock Y successfully
    pthread_mutex_lock(&lock_x); // Attempt to acquire Lock X; stalls if held by A

    /* Execute Critical Work */
    pthread_mutex_unlock(&lock_x); //
    pthread_mutex_unlock(&lock_y); //
    return NULL;
}
```

This code pattern creates a classic "**Unlucky Case**" race condition. If Thread A acquires Lock X and is immediately context-switched out before it can request Lock Y, Thread B can run and acquire Lock Y.

When both threads resume, Thread A will stall waiting for Lock Y, and Thread B will stall waiting for Lock X. Neither thread can make forward progress, locking up both paths permanently. This bug is exceptionally difficult to diagnose because it is non-deterministic; under a "**Lucky**" scheduling sequence, one thread may clear both locks before the second thread starts, allowing the program to complete successfully without any signs of a fault.

7.4 Multi-Resource Vectors: Space and Sockets

Deadlocks are not limited to software locks; they can occur across any finite system resources, including physical memory or network connections.

Memory Space Deadlocks

Consider a server with exactly 2 MB of available physical memory running two threads:

- **Thread A:** Calls `AllocateOrWait(1 MB)`, succeeds, and is context-switched out.
- **Thread B:** Calls `AllocateOrWait(1 MB)`, succeeds, and consumes the remaining memory.
- Next, both threads attempt to allocate an additional 1 MB of memory before releasing their initial allocations. Because the system's memory is fully exhausted, both threads enter a blocked state waiting for space to clear. Since neither thread can proceed to call `Free()`, the system enters a permanent memory deadlock.

7.5 The Dining Lawyers Problem

The Dining Lawyers problem (adapted from the classical Dining Philosophers problem) serves as a core case study for multi-resource concurrency constraints.

- **System Setup:** Five lawyers sit around a circular table with exactly five chopsticks placed between them. A lawyer needs to acquire **two chopsticks** (the one to their left and the one to their right) to eat a bowl of rice.
- **The Deadlock Condition:** If every lawyer at the table reaches out simultaneously and grabs their right chopstick, every individual chopstick becomes held. When the lawyers then attempt to acquire their left chopstick, they find it held by their neighbor. Every lawyer enters a blocked wait state holding one resource and waiting for another, creating a permanent circular deadlock.
- **Algorithmic Prevention Check:** To prevent this deadlock safely, the system can enforce an access rule: *Never let a lawyer take the last available chopstick unless at least one hungry lawyer is guaranteed to hold two chopsticks immediately afterward.*

Section 8: Coffman Conditions & Resource-Allocation Graphs (RAG)

8.1 The Four Coffman Conditions

For a deadlock to occur, all four of the following physical conditions must hold simultaneously within the system:

1. **Mutual Exclusion:** Only one thread can use a specific resource at any given time. If a resource is fully shareable, it cannot participate in a deadlock.
2. **Hold and Wait:** A thread must currently hold at least one resource while waiting to acquire additional resources that are held by other active threads.
3. **No Preemption:** Resources cannot be forcefully reclaimed from a thread. A resource can only be released voluntarily by the thread holding it after its task finishes.
4. **Circular Wait:** There must exist a closed, circular loop of waiting threads, $\{T_1, T_2, \dots, T_n\}$, where T_1 is waiting for a resource held by T_2 , T_2 is waiting for a resource held by T_3 , and T_n is waiting for a resource held by T_1 .

8.2 Resource-Allocation Graph (RAG) Modeling

The operating system can model and detect deadlocks at runtime using a directed graph structure called a **Resource-Allocation Graph (RAG)**.

Mathematical Graph Formulation

- The graph consists of a set of vertices V and directed edges E .

- The vertices are partitioned into two distinct classes:
 - Thread Nodes: $T = \{T_1, T_2, \dots, T_n\}$, represented as circles.
 - Resource Type Nodes: $R = \{R_1, R_2, \dots, R_m\}$, represented as squares. Internal dots within a square indicate the number of available instances of that resource type.
- **Directed Request Edge** ($T_i \rightarrow R_j$): Indicates that thread T_i is currently blocked and waiting to acquire an instance of resource type R_j .
- **Directed Assignment Edge** ($R_j \rightarrow T_i$): Indicates that a specific instance of resource type R_j has been allocated to and is currently held by thread T_i .

Analyzing Cycles and Deadlocks

- **Single-Instance Resources:** If every resource type node contains exactly one instance, **the presence of a cycle in the RAG is a necessary and sufficient condition for a deadlock**. If a cycle exists, the system is deadlocked; if no cycle exists, the system is safe.
- **Multi-Instance Resources:** If resource nodes contain multiple instances, a cycle is a *necessary but not sufficient* condition for a deadlock. If a cycle forms but an external thread outside the loop holds an instance of the same resource, that external thread can complete its work, release its instance, and break the cycle, resolving the stall without deadlock.

Section 9: Algorithmic Deadlock Detection & Strategic Management Frameworks

9.1 The Deadlock Detection Vector Algorithm

For systems with multi-instance resources, the kernel uses a vector-based bank detection algorithm to scan the system for deadlocks. The algorithm attempts to find a valid execution sequence that allows all current threads to finish.

Vector Parameters

Let m be the number of resource types in the system.

- **[FreeResources]** : A vector of length m tracking the current count of free, unallocated resources of each type.
- **[Alloc_x]** : A vector of length m tracking the resources currently held by thread X .
- **[Request_x]** : A vector of length m tracking the additional resources currently requested by thread X .

Algorithmic Execution Steps

```

1. Initialize a working tracking vector: [Avail] = [FreeResources].
2. Construct a list named UNFINISHED containing all active system nodes.
3. do {
    done = true //
    Foreach node in UNFINISHED { //
        // Check if the node's resource request can be satisfied by the available pool
        if ([Request_node] <= [Avail]) { //
            remove node from UNFINISHED; // Assume the task can finish and return its resources
            [Avail] = [Avail] + [Alloc_node]; // Reclaim all resources held by this node
            done = false //
        }
    }
} until (done) // Continue iterating if progress was made

4. EVALUATION:
if (Nodes are left inside the UNFINISHED list) {
    The remaining nodes are formally DEADLOCKED.
}

```

9.2 Strategic Management Frameworks

Operating systems use four core paradigms to manage the risk of resource deadlocks:

1. **Deadlock Prevention:** Design the system's code and software structures so that it is physically impossible for at least one of the four Coffman conditions to form. For example, forcing all threads to acquire locks in a strict, pre-defined global order completely eliminates the risk of circular waits.
2. **Deadlock Avoidance:** The kernel monitors resource requests dynamically at runtime. If a request is correct but has the potential to lead to a circular wait down the line, the kernel deliberately delays the thread's request until safety is restored. The **Banker's Algorithm** is the classic algorithmic framework used to implement deadlock avoidance.
3. **Deadlock Recovery:** Allows deadlocks to happen naturally, using a background detection thread to spot cycles in the resource-allocation graph. Once a deadlock is detected, the system recovers by force-terminating the deadlocked threads or rolling back their execution states to free up resources.
4. **Deadlock Denial (The Ostrich Algorithm):** Completely ignores the possibility of deadlocks. If deadlocks are rare and the engineering cost of prevention or avoidance is high, the system simply assumes deadlocks won't happen, relying on users to restart their applications if a stall occurs. Nearly all major general-purpose operating systems, including Linux, macOS, and Windows, use this pragmatic approach.

Section 10: High-Yield Practice Exam Problems with Detailed Solutions

Problem 1: Hard Real-Time Schedulability Testing

A hard real-time system is configured to coordinate three independent periodic tasks with parameters $T_i = (P_i, C_i)$:

- Task 1: $T_1 = (5, 2)$
 - Task 2: $T_2 = (6, 1)$
 - Task 3: $T_3 = (10, 1)$
1. Calculate the total processor utilization U_{Total} for this task set.
 2. Determine if this task set is guaranteed to meet all deadlines under **Rate Monotonic Scheduling (RMS)**.
 3. Determine if this task set is guaranteed to meet all deadlines under **Earliest Deadline First (EDF)**.

Solution

1. **Calculate Total Processor Utilization:** We compute individual utilizations ($U_i = C_i/P_i$) and sum them up: $U_1 = \frac{2}{5} = 0.400$ $U_2 = \frac{1}{6} \approx 0.167$ $U_3 = \frac{1}{10} = 0.100$ $U_{\text{Total}} = 0.400 + 0.167 + 0.100 = \mathbf{0.667}$
2. **Evaluate Rate Monotonic Schedulability:** We apply the Liu & Layland utilization bound test for $n = 3$ tasks: $\text{Bound} = 3 \times (2^{1/3} - 1) \approx 3 \times (1.2599 - 1) \approx \mathbf{0.780}$
 - **RMS Analysis:** Comparing our values, we see that $U_{\text{Total}} = 0.667 \leq \text{Bound} = 0.780$. Because the total utilization is below the sufficient threshold, **the task set is guaranteed to meet every deadline under RMS.**
3. **Evaluate Earliest Deadline First Schedulability:** We check the EDF schedulability condition ($\sum U_i \leq 1.0$):
 - **EDF Analysis:** Since $U_{\text{Total}} = 0.667 \leq 1.0$, the workload satisfies the exact condition. **The task set is guaranteed to be fully schedulable under EDF.**

Problem 2: Tracking Strides and Pass Counters

A stride scheduler coordinates three active threads (Thread A, Thread B, Thread C) using a global constant $W = 60,000$. The threads are assigned the following ticket counts:

- Thread A: 300 tickets
- Thread B: 100 tickets
- Thread C: 200 tickets

1. Calculate the execution stride length for each thread.
2. Assuming all pass counters are initially zero, trace the scheduler's choices and update the counters step-by-step for the first 4 scheduling intervals.

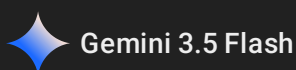
Solution

1. **Calculate Stride Lengths:** Using the formula $S_i = W / N_i$, we compute the stride lengths: Stride $S_A = \frac{60,000}{300} = 200$
 Stride $S_B = \frac{60,000}{100} = 600$ Stride $S_C = \frac{60,000}{200} = 300$

2. **Trace Scheduling Decisions and Counter Updates:** At each interval, the scheduler selects the thread with the lowest pass counter, runs it for a single time quantum, and adds its stride to its counter.

- **Interval 1:** All pass counters are tied at 0 (Pass_A = 0, Pass_B = 0, Pass_C = 0).
 - **Selection:** **Thread A** is chosen (resolving the tie arbitrarily).
 - **Update:** Pass_A = 0 + 200 = 200 .
 - **State:** [Pass_A = 200, Pass_B = 0, Pass_C = 0] .
- **Interval 2:** Evaluating counters [200, 0, 0] , Thread B and Thread C are tied for the lowest value (0).
 - **Selection:** **Thread B** is chosen.
 - **Update:** Pass_B = 0 + 600 = 600 .
 - **State:** [Pass_A = 200, Pass_B = 600, Pass_C = 0] .
- **Interval 3:** Evaluating counters [200, 600, 0] , Thread C has the lowest value (0).
 - **Selection:** **Thread C** is chosen.
 - **Update:** Pass_C = 0 + 300 = 300 .
 - **State:** [Pass_A = 200, Pass_B = 600, Pass_C = 300] .
- **Interval 4:** Evaluating counters [200, 600, 300] , Thread A has the lowest value (200).
 - **Selection:** **Thread A** is chosen.
 - **Update:** Pass_A = 200 + 200 = 400 .
 - **State:** [Pass_A = 400, Pass_B = 600, Pass_C = 300] .

This deterministic tracking ensures that over time, the frequency of thread execution matches their ticket ratios (300 : 100 : 200 $\implies$ 3 : 1 : 2) without any statistical variance.



CS162 Operating Systems & Systems Programming

Lecture 15: Memory Management — Abstractions, Segmentation, and Paging (Detailed Exam Notes)

1. The Virtual Address Space & Runtime Memory Invariants

An **Address Space** is formally defined as the complete set of accessible virtual memory addresses and the architectural state associated with them.

On a standard 32-bit computer architecture, the processor registers can index up to 2^{32} distinct memory bytes, providing a theoretical memory space of exactly **4 Gibibytes (GiB)**.

1.1 Memory Sizing & Word Distribution Mathematics

To understand the capacity of this address space, we parse its components using binary powers:

- **Base Metric Definitions:**
 - 1 KiB (Kibibyte) = 2^{10} bytes = 1,024 bytes $\approx 10^3$ bytes
 - 1 MiB (Mebibyte) = 2^{20} bytes = 1,048,576 bytes $\approx 10^6$ bytes
 - 1 GiB (Gibibyte) = 2^{30} bytes = 1,073,741,824 bytes $\approx 10^9$ bytes

If individual data words are aligned to a standard 32-bit (4-byte) physical dimension, the total number of words that can fit into a 4 GiB virtual address space is calculated as: $\text{Total 32-bit Words} = \frac{2^{32} \text{ bytes}}{4 \text{ bytes/word}} = \frac{2^{32}}{2^2} = 2^{30} \approx 1 \text{ Billion Words}$

1.2 Processor Memory Interactions

When a physical central processing unit execution core executes an instruction that issues a memory read or write command to a specific target address, the system maps the action to one of four paths:

1. **Regular Cache/RAM Access:** The target address maps to a valid memory location containing standard application data, instructions, or variables.
2. **Memory-Mapped I/O (MMIO):** The target address bypasses physical RAM and interacts directly with device registers on hardware peripheral controllers (such as storage disks, network interfaces, or graphics adapters).
3. **Hardware Execution Fault (Segmentation Fault):** The address violates permission bit settings or falls outside the process's allowed boundaries, prompting the hardware to trap into the kernel and abort the application.
4. **Inter-Process Communication (IPC):** The address links directly to a shared memory segment, allowing separate applications to communicate without kernel overhead.

1.3 Layout Invariants of a Process Address Space

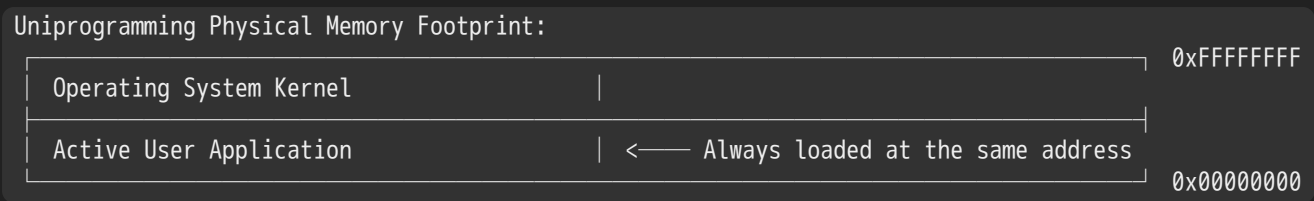
To simplify compilation and runtime execution, the operating system organizes a process's virtual memory into a standard hierarchical layout:

- **The Code Segment:** Positioned at the base of the user-space address space (starting near address `0x00000000`). It stores the application's compiled, read-only machine instructions. The Program Counter (`PC`) register tracks execution within this zone.
- **The Static Data Segment:** Positioned directly above the code segment. It stores global variables and static values initialized by the developer at compile time.
- **The Heap Segment:** Used for dynamic runtime allocations (e.g., via C's `malloc()`). It starts directly above the static data segment and grows **upward** toward higher memory addresses. Applications request more heap space from the kernel using the `sbrk` system call, which adjusts the system's "program break" line.
- **The Stack Segment:** Positioned at the very top of the user-space boundary (ending near address `0xFFFFFFFF` in a 32-bit address space). It grows **downward** toward lower addresses, storing function activation records, local variables, parameters, and return addresses. The Stack Pointer (`SP`) register tracks the active frame of this segment.

2. Early Memory Management Models: Uniprogramming & Relocation

2.1 Uniprogramming Systems (No Relocation or Protection)

In early computer systems, only a single application could execute at any given time—a paradigm known as **Uniprogramming**.



The absolute alignment model used in uniprogramming architectures.

- **Absolute Memory Alignment:** Because only one application runs at a time, it is always loaded into the exact same location in physical memory.
- **Lack of Protection:** The system does not use any address translation or protection mechanisms. The active application operates directly on physical memory addresses, allowing it to modify any memory location, including the operating system kernel itself.
- **The Dedicated Illusion:** The application is given the illusion of exclusive access to a dedicated machine by the simple physical reality of being the only program running on the hardware.

2.2 Primitive Multiprogramming (Linker-Loader Relocation)

To run multiple applications concurrently, early systems like Microsoft Windows 3.1 used a model called **Primitive Multiprogramming**. This model allowed multiple programs to sit in physical memory simultaneously without using hardware translation or protection features.

- **Software Relocation Mechanism:** To prevent independent applications from trampling over each other's memory spaces, the system used a **Linker-Loader**. When an application was loaded into memory, the loader modified all of its absolute memory references (such as direct data loads, stores, and branch jumps) to align with its assigned physical memory base.
- **The Binding Timeline:** Addresses could be bound to final values at three different stages during a program's lifecycle:
 1. **Compile Time:** The compiler generates fixed, absolute machine code references. This requires knowing exactly where the program will sit in physical memory before it runs.
 2. **Load Time:** The compiler generates relocatable object code. The linkage editor and system loader then modify memory references on the fly as the program is loaded into RAM.
 3. **Execution Time (Dynamic Relocation):** Memory references are kept flexible while loading and are only bound to final values at runtime, often using specialized hardware registers or dynamic link stubs.
- **Dynamic Library Stubs:** When using dynamic libraries, the linker inserts a small placeholder code fragment called a **Stub** into the application binary instead of compiling the full library code. When the program runs, this stub checks whether the requested library routine is already resident in memory. If it is not, the stub loads the library, updates its own references with the routine's active address, and branches directly into the library code.

Memory Binding Analysis: Single vs. Dual Instance Limitations

Consider an assembly program compiled to run at base address `0x0000` . The program includes a data label `data1` at address `0x0300` , and an initial load instruction `lw r1, 0(data1)` positioned at address `0x0900` . At compile time, the instruction's target address is bound directly to the value `0x0300` .

Process View (Compiled Baseline):		Physical RAM (Loader Adjusted):	
Address	Instruction	Address	Instruction
0x0300:	[Data Word: 32]	0x0300:	[Data Word: 32]
...		...	
0x0900:	lw r1, 0(0x0300)	0x0900:	lw r1, 0(0x0300)

The software relocation adjustment for a single program instance.

If the system attempts to run a **second copy** of this exact same program concurrently, the loader cannot place it at physical base address `0x0000` because that space is already occupied by the first instance. Instead, it relocates the second copy to a different physical base address, such as `0x1000`.

Second Copy Physical RAM Footprint:		
Address	Instruction	
0x1300:	[Data Word: 32]	(Relocated from 0x0300)
...		
0x1900:	lw r1, 0(0x1300)	(Relocated from lw r1, 0(0x0300))

The software relocation adjustment for a second program instance.

- **The Core Flaw:** While software relocation allows multiple programs to co-exist, it provides **zero memory protection**. Because applications operate directly on physical RAM addresses, a single pointer bug or malicious instruction in one program can overwrite the code or data of another program or the operating system, crashing the entire system.

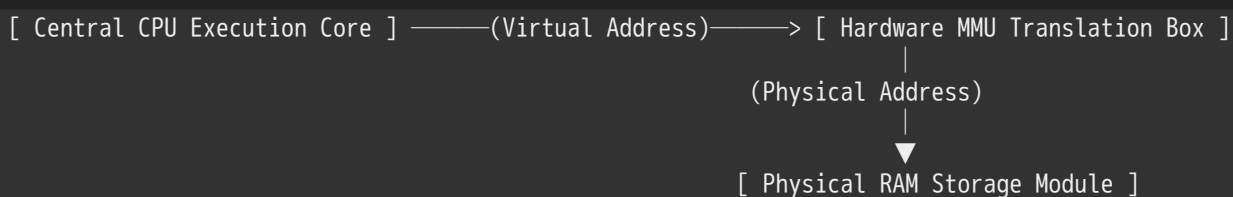
3. Hardware-Enforced Relocation: Memory Management Units (MMU)

To provide reliable isolation and safety, modern computer architectures shift the responsibility of address translation from software loaders to specialized processor hardware: the **Memory Management Unit (MMU)**.

3.1 The Dual Views of Memory

With an MMU in place, the system separates memory into two distinct conceptual views:

1. **The Virtual Memory View (CPU View):** The logical address layout visible to the programmer and the executing application. Every program can be compiled and loaded into the exact same region of user address space, completely oblivious to where it actually sits in physical RAM.
2. **The Physical Memory View (RAM View):** The actual hardware storage blocks on physical memory chips.



The hardware translation pipeline managed by the Memory Management Unit.

By intercepting every memory reference and translating virtual addresses to physical ones on the fly, the MMU enforces absolute protection. If Process A has no virtual address mappings pointing to Process B's physical memory, it is physically impossible for Process A to access or corrupt Process B's data.

3.2 The Base and Bound (B&B) Relocation Model

The simplest historical approach to hardware address translation is the **Base and Bound** model.

Hardware Translation Mechanics

The MMU uses two internal hardware registers to protect and relocate a process's memory:

- **The Base Register:** Stores the physical memory address where the process's allocated block begins.

- **The Bound Register:** Stores the maximum virtual size limit of the process's address space.

When an application issues a virtual memory address, the hardware executes a two-step validation and relocation sequence in parallel:

1. **Boundary Limit Check:** The MMU verifies that the incoming virtual address is strictly within the process's bounds:
 $\text{Virtual Address} < \text{Bound}$ If the virtual address matches or exceeds this limit, the MMU blocks the operation and triggers a memory protection fault trap into the kernel (a Segmentation Fault).
2. **Physical Address Relocation:** If the address is within bounds, the MMU adds the value of the base register to compute the final physical address: $\text{Physical Address} = \text{Virtual Address} + \text{Base}$

Limitations of the Simple Base and Bound Model

While fast and simple, the simple B&B model has severe drawbacks for general-purpose computing:

- **Physical Memory Fragmentation:** Over time, as processes of different sizes are loaded and terminated, physical memory becomes divided into a patchwork of small, unusable holes—a problem known as **External Fragmentation**.
- **No Support for Sparse Address Spaces:** A process's virtual address space often contains large, unmapped holes between segments (such as the empty space between the heap and the stack). Under a simple B&B model, the operating system must allocate a single, contiguous block of physical memory that spans the entire space from address 0 to the top of the stack, wasting massive amounts of physical RAM on empty space.
- **No Memory Sharing:** Because a process is strictly confined to a single, contiguous base-and-bounds window, two processes cannot share memory pages or read-only library code directly.

4. Advanced Abstractions: Segmentation Architecture

To support sparse address spaces and allow memory sharing, systems designers evolved the simple B&B model into **Segmentation Architecture**.

4.1 The Logical Concept of Segments

Instead of treating an application's memory as a single contiguous block, segmentation breaks the virtual address space down into multiple, logically independent components called **Segments** (such as the Code segment, Data segment, Stack segment, or explicit Shared Memory segments).

- **Independent Relocation:** Each segment is allocated its own contiguous region of physical memory with its own independent base address and size limit. This allows a process's segments to be scattered anywhere across physical RAM, eliminating the need to allocate contiguous memory for unmapped holes.

4.2 Segmentation Address Translation Mechanics

Under a segmentation model, a virtual address is divided into two separate components: a **Segment Identifier (Seg ID #)** and an **Offset**.

Hardware Lookup Logic

1. **Segment Table Indexing:** The MMU uses the high-order bits of the virtual address (the **Seg ID #**) as an index into an internal hardware array called the **Segment Table**. Each entry in this table stores three tracking metrics:
 - **Base:** The physical starting address of that specific segment.
 - **Limit:** The maximum virtual size allocated to that segment.
 - **Valid / Invalid Bit (V/N):** A flag indicating whether the segment is currently active and mapped to valid memory.
2. **Validation Checks:** The MMU verifies that the segment is marked valid (**V**) and that the incoming offset is strictly within the segment's size limits: $\text{Offset} < \text{Limit}$ If the segment is invalid (**N**) or the offset is out of range, the MMU stops

execution and triggers an access error fault.

3. **Physical Address Calculation:** If all checks pass, the MMU adds the segment's base address to the offset to generate the final physical memory address: $\text{Physical Address} = \text{Offset} + \text{Base}$

4.3 Case Study: Intel x86 Segment Descriptors & flat memory models

The classic Intel x86 processor architecture provides a sophisticated hardware implementation of segmentation known as **Protected Mode Segmentation**.

Intel 16-bit Segment Selector Register Bit Fields:



Index: Selects 1 of 8192 unique table entries TI: Table Indicator (0=GDT, 1=LDT)

RPL: Requestor's Privilege Level (0=Kernel, 3=User)

Segment Selection & Table Registers

The x86 architecture contains six dedicated hardware segment registers: **CS** (Code Segment), **DS** (Data Segment), **SS** (Stack Segment), **ES**, **FS**, and **GS** (Extra Segments). Each register stores a 16-bit **Segment Selector** that acts as a pointer to an actual segment description in memory.

- **Table Indicator (TI):** Bit 2 selects between two system tables in memory: the Global Descriptor Table (**GDT** , **TI=0**) or the Local Descriptor Table (**LDT** , **TI=1**).
- **System Pointers:** The physical base addresses of these descriptor tables are tracked by dedicated processor registers: the Global Descriptor Table Register (**GDTR**) and the Local Descriptor Table Register (**LDTR**).

The 64-bit Segment Descriptor Format

Each entry inside the GDT or LDT is a 64-bit hardware record defining the segment's properties:

- **Base Address (32 bits):** Split across bits 0-15, 16-23, and 24-31 due to historical backward-compatibility constraints.
- **Segment Limit (20 bits):** Defines the size of the segment, split across bits 0-15 and 16-19.
- **Granularity Bit (G):** Dictates how the limit field is interpreted. If $G = 0$, the limit is measured in exact byte increments. If $G = 1$, the limit is measured in clusters of **4 KiB units**, allowing a 20-bit field to span up to a full 4 GiB address space.
- **Descriptor Privilege Level (DPL):** A 2-bit field tracking the segment's required privilege tier (Ring 0 for Kernel, Ring 3 for User space). Access is only granted if the Current Privilege Level (**CPL**) satisfies the inequality: $\text{Max}(\text{CPL}, \text{RPL}) \leq \text{DPL}$

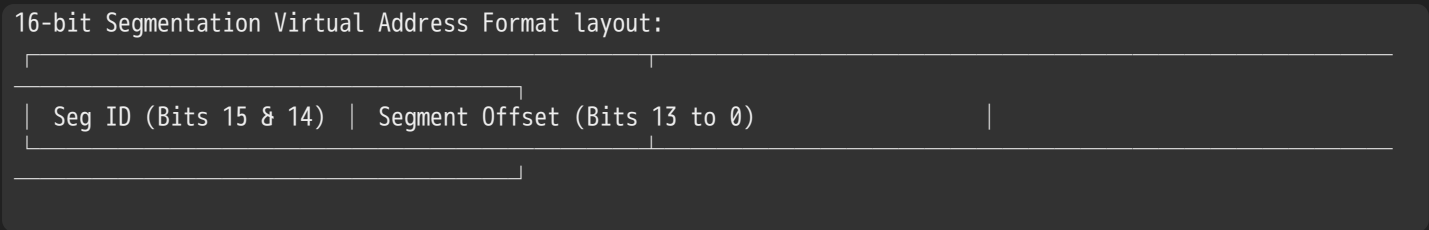
Bypassing Segmentation: The Flat Memory Model

Modern operating systems like Linux bypass x86 hardware segmentation to rely entirely on page-based memory management. However, because segmentation cannot be fully disabled on x86 hardware, the kernel works around it by configuring a **Flat Memory Model**.

To do this, the kernel sets the base address of all major segments (**CS** , **DS** , **SS**) to **0x00000000** and sets their limits to the absolute maximum of **0xFFFFFFFF** (4 GiB). Under this configuration, the offset matches the virtual address exactly ($\text{Physical Address} = \text{Virtual Address} + 0$), effectively bypassing segmentation so the OS can handle translation via paging.

4.4 Comprehensive Segmentation Mapping Mathematical Trace

To see how segments translate addresses in practice, consider a 16-bit address space where the top two bits define the segment ID ($2^2 = 4$ unique segments), leaving the remaining 14 bits to serve as the segment offset ($2^{14} = 16$ KiB maximum size limit).



System Segment Table Configuration

The operating system configures the process's segment table with the following parameters:

- Segment 0 (Code): Base = 0x4000 , Limit = 0x0800
- Segment 1 (Data): Base = 0x4800 , Limit = 0x1400
- Segment 2 (Shared): Base = 0xF000 , Limit = 0x1000
- Segment 3 (Stack): Base = 0x0000 , Limit = 0x3000

Let's trace how the hardware evaluates and translates four different virtual addresses:

Address 1: Virtual Address 0x0240

1. **Binary Extraction:** Convert 0x0240 to a 16-bit binary string: 0000 0010 0100 0000
2. **Field Parsing:** Extract the top two bits for the segment ID, and use the remaining bits for the offset:
 - Seg ID = 00 = Segment 0
 - Offset = 00 0010 0100 0000 = 0x0240
3. **Validation Check:** Verify that the offset is within the segment's size limits: Offset (0x0240) < Limit (0x0800) $\implies$ Valid
4. **Physical Address Calculation:** Add the segment's base address to the offset to find the final physical address:
Physical Address = 0x0240 + 0x4000 = 0x4240

Address 2: Virtual Address 0x4050

1. **Binary Extraction:** Convert 0x4050 to a 16-bit binary string: 0100 0000 0101 0000
2. **Field Parsing:** Extract the fields:
 - Seg ID = 01 = Segment 1
 - Offset = 00 0000 0101 0000 = 0x0050
3. **Validation Check:** Verify that the offset is within the segment's limits: Offset (0x0050) < Limit (0x1400) $\implies$ Valid
4. **Physical Address Calculation:** Add the base address to the offset to find the final physical address:
Physical Address = 0x0050 + 0x4800 = 0x4850

Address 3: Virtual Address 0x0900

1. **Binary Extraction:** Convert 0x0900 to a 16-bit binary string: 0000 1001 0000 0000
2. **Field Parsing:** Extract the fields:
 - Seg ID = 00 = Segment 0
 - Offset = 00 1001 0000 0000 = 0x0900

3. **Validation Check:** Verify that the offset is within the segment's limits: Offset ($0x0900$) $\nless$ Limit ($0x0800$) $\implies$ **Boundary Limit Violation!**

4. **Result:** The MMU blocks the operation and triggers an access error exception (Segmentation Fault).

Address 4: Virtual Address `0x2500`

1. **Binary Extraction:** Convert `0x2500` to a 16-bit binary string: `0010 0101 0000 0000`

2. **Field Parsing:** Extract the fields:

- Seg ID = `00` = **Segment 0**
- Offset = `00 1001 0100 0000` = **`0x0500`**

3. **Validation Check:** Verify that the offset is within the segment's limits: Offset ($0x0500$) $<$ Limit ($0x0800$) $\implies$ **Valid**

4. **Physical Address Calculation:** Add the base address to the offset to find the final physical address:

$$\text{Physical Address} = 0x0500 + 0x4000 = \mathbf{0x4500}$$

4.5 Runtime Performance Costs & Memory Swapping

While segmentation handles sparse address spaces well, it introduces significant performance overhead when physical memory fills up.

- **The Allocation Overhead:** Because segments must be stored in contiguous blocks of physical memory, the operating system must frequently search for and shuffle segments around to fit new resource requests, causing heavy fragmentation overhead.
- **The Swapping Trap:** If a process requests a segment that cannot fit into the available physical memory, the operating system must perform **Swapping**. The kernel selects an inactive process, copies its entire segment allocation out of RAM onto slower persistent storage (the disk backing store), marks its segment table entries invalid, and reallocates the freed RAM to the active process.
- **The Cost:** Copying large, multi-megabyte contiguous segments to and from disk takes millions of clock cycles, creating a massive performance bottleneck during context switches. To keep systems responsive, operating systems need a finer-grained way to manage physical memory.

5. Finer-Grained Memory Management: Paging Architecture

To eliminate the overhead of variable-sized segments and avoid external fragmentation, modern operating systems use **Paging Architecture**.

5.1 Fixed-Size Chunks vs. Dynamic Segments

Instead of allocating memory in variable-sized blocks, paging divides both virtual and physical memory into fixed-size, uniform blocks:

- **Pages:** Fixed-size blocks of **Virtual Memory** viewed by the CPU.
- **Page Frames:** Matching fixed-size blocks of **Physical Memory** viewed by RAM.
- **Simplified Allocation:** Because every page frame is identical in size, any virtual page can be mapped to any available physical frame, completely eliminating external fragmentation. The kernel can track free memory using a simple bit vector, where each bit represents the status of a specific frame (1 for allocated, 0 for free).
- **Granularity Control:** To avoid wasting space on internal fragmentation (wasted space *inside* an allocated block), pages are kept relatively small, typically ranging between 1 KiB and 16 KiB (with 4 KiB being the industry standard). This means a single logical segment (like the heap) is split across multiple independent pages.

5.2 Single-Level Paging Translation Mechanics

Under a paging model, a virtual address is divided into two components: a **Virtual Page Number (VPN)** and an **Offset**.

Hardware Translation Loop

1. **The Offset Invariant:** Because pages and page frames have identical, fixed sizes, the internal byte offset remains completely unchanged during translation. The offset bits are copied directly from the virtual address to the final physical address. For example, a 12-bit offset maps to a fixed $2^{12} = 4$ KiB page boundary.
2. **Page Table Indexing:** The MMU uses the high-order bits of the virtual address (the **VPN**) as an index into the process's **Page Table** to look up the corresponding translation entry.
3. **Physical Page Mapping:** The MMU extracts the matching **Physical Page Number (PPN)** (or Page Frame Number) from the table entry and combines it with the original offset bits to generate the final physical memory address.

5.3 The Critical Problem: Page Table Memory Explosion

While single-level paging simplifies memory allocation, it introduces a massive scaling issue: the memory required to store the page tables themselves can quickly overwhelm the system.

1. Sizing Analysis for a 32-bit Architecture

Consider a standard 32-bit address space ($2^{32} = 4$ GiB) using standard 4 KiB (2^{12} bytes) pages.

- **Calculating the Offset Bits:** A 4 KiB page requires a 12-bit offset field to address every byte within the page ($2^{12} = 4096$).
- **Calculating the Virtual Page Number Bits:** The remaining bits are used for the virtual page number: $\text{VPN Bits} = 32 \text{ bits} - 12 \text{ offset bits} = 20 \text{ bits}$
- **Calculating Total Page Table Entries:** A 20-bit VPN field can index up to 2^{20} distinct virtual pages, requiring an array with **1,048,576 unique entries**.
- **Calculating Total Table Memory Space:** If each Page Table Entry (PTE) requires 4 bytes of storage, the memory needed to store a single flat page table is calculated as: $\text{Table Memory Space} = 2^{20} \text{ entries} \times 4 \text{ bytes/entry} = 2^{22} \text{ bytes} = 4 \text{ MiB}$

The Scaling Issue: Because every individual process requires its own private page table, running 100 concurrent processes would consume **400 MiB of physical RAM** just to store the translation tables, which was a massive percentage of total system memory on early machines.

2. Sizing Analysis for a 64-bit Architecture

The memory explosion becomes completely unmanageable on a modern 64-bit architecture using the same 4 KiB page size:

- **Offset Bits:** Fixed at 12 bits ($2^{12} = 4$ KiB).
- **VPN Bits:** $\text{VPN Bits} = 64 - 12 = 52 \text{ bits}$.
- **Total Page Table Entries:** 2^{52} entries.
- **Total Table Memory Space:** Assuming each 64-bit entry requires 8 bytes of storage, the memory needed for a single flat page table is: $64\text{-bit Table Memory Space} = 2^{52} \text{ entries} \times 8 \text{ bytes/entry} = 2^{55} \text{ bytes} = 32 \text{ Pebibytes (PiB)}$

This is an impossible requirement. Flat linear page tables are fundamentally broken for large, sparse address spaces because they force the system to waste petabytes of memory storing blank entries for unmapped regions.

6. Advanced Translation: Multi-Level Page Tables

To solve the memory explosion problem, modern operating systems replace flat linear page tables with hierarchical tree structures: **Multi-Level Page Tables**.

6.1 The Two-Level Tree Architecture

Instead of allocating a single, massive array for the entire address space, a two-level page table breaks the virtual address space down into a tree of smaller, fixed-size tables.

Under a classic 32-bit x86 architecture with 4 KiB pages, the 32-bit virtual address is split into a "Magic" 10-10-12 bit pattern:

- **P1 Index (Bits 31 to 22 - 10 bits):** Indexes into the top-level page table, called the **Page Directory**.
- **P2 Index (Bits 21 to 12 - 10 bits):** Indexes into the second-level **Page Table**.
- **Offset (Bits 11 to 0 - 12 bits):** Defines the target byte offset within the physical page frame.

6.2 Saving Memory space via Sparse Tables

Why does this hierarchical design save so much memory?

- **Fixed-Size Subdivisions:** Both the Page Directory and each second-level Page Table are configured to contain exactly $2^{10} = 1024$ entries. Given that each entry is 4 bytes wide, each table occupies exactly **4 KiB of memory**, matching the size of a single physical page frame.
- **On-Demand Allocation:** The master Page Directory must always remain resident in memory. However, the second-level page tables are only allocated when the process actually maps data within that region.
- If a sparse process only maps a small code segment at address `0` and a small stack segment at the top of memory, the system only needs to allocate the master Page Directory and **two second-level page tables**, reducing the total memory footprint to just 12 KiB (3×4 KiB) instead of the 4 MiB required by a flat table. Unused regions of the tree simply contain null pointers, avoiding any wasted space.

6.3 Anatomy of a Page Table Entry (PTE)

Each Page Table Entry (PTE) is a raw hardware bitmask that manages both address routing and protection flags.

Intel x86 32-bit Page Table Entry Bit Fields Layout:

Page Frame Number (Bits 31 to 12)										OS Free Bits		PS	D	A	PCD	PWT	U	W	P
-----------------------------------	--	--	--	--	--	--	--	--	--	--------------	--	----	---	---	-----	-----	---	---	---

Flags: P=Present, W=Writable, U=User, PWT=Write-Through, PCD=Cache-Disabled, A=Accessed, D=Dirty, PS=Page Size

The x86 architecture defines the following standard bit field assignments within each 4-byte entry:

- **Page Frame Number (Bits 31 to 12):** Stores the high-order bits of the physical base address of the target page frame.
- **Present Flag (P - Bit 0):** The equivalence of the validity bit. If $P = 1$, the page is resident in physical RAM. If $P = 0$, accessing the page triggers a page fault exception.
- **Read/Write Flag (W - Bit 1):** Enforces write protection. If $W = 0$, the page is read-only. If $W = 1$, write operations are permitted.
- **User/Supervisor Flag (U - Bit 2):** Enforces privilege boundaries. If $U = 0$, the page can only be accessed by kernel-mode code running at privilege level 0. If $U = 1$, user-space applications can access it.
- **Accessed Flag (A - Bit 5):** Automatically set to `1` by the hardware MMU whenever the page is read or written, helping the kernel track active page usage for replacement algorithms.

- **Dirty Flag (D - Bit 6):** Set to `1` by the hardware whenever a write operation modifies data within the page frame, indicating to the kernel that the page must be written back to disk before being cleared.
- **Page Size Flag (PS - Bit 7):** Available only in Page Directory Entries. If $PS = 1$, the hierarchy short-circuits and maps directly to a large **4 MiB page frame**, using the remaining 22 bits of the virtual address as a direct offset and bypassing the second-level table entirely.

7. Advanced Memory Mechanisms: COW, Demand Paging, and Inverted Tables

Operating systems exploit the hardware page fault lifecycle and flag bits within the PTE to implement high-performance virtual memory features.

7.1 Demand Paging Mechanics

Operating systems do not need to load a program's entire executable into physical memory before it can start running. Instead, they use a technique called **Demand Paging** to load pages only when they are actively needed.

1. **Initial State:** When a process starts, the kernel only allocates its master page directory and marks all second-level page table entries as invalid (`Present = 0`).
2. **The Page Fault:** When the CPU attempts to fetch an instruction from an unmapped page, the MMU catches the invalid present bit and triggers a **Page Fault Exception**. The kernel takes control and reads the faulting virtual address from control register `CR2` .
3. **Disk Retrieval:** The kernel checks its internal tracking structures, locates the missing page within the application's executable file on disk, finds a free frame in physical RAM, and copies the page data into memory.
4. **PTE Update:** The kernel updates the process's PTE with the new frame number, sets the present flag to `1` , and returns from the exception handler, instructing the CPU to retry the faulting instruction. This allows the program to continue executing smoothly, loading its code on demand.

7.2 Copy-On-Write (COW) Optimization

When an application calls the `fork()` system call, the operating system must create an independent copy of the parent process's address space for the child. Duplicating megabytes of physical memory immediately during the call is incredibly slow and expensive, especially since the child process often calls `exec()` right away to replace its address space with a new program.

To optimize this path, modern kernels use **Copy-On-Write (COW)**:

1. **Table Aliasing:** Instead of copying any physical data, the kernel duplicates the parent's *page tables* directly to the child. This means both the parent and child virtual address spaces point to the exact same physical memory frames.
2. **Write-Protecting Entries:** The kernel goes through both sets of page tables and forces the read/write flags of all writable pages to **Read-Only** ($W = 0$).
3. **The Allocation Fault:** As long as both processes only execute read operations, they access the shared physical memory frames simultaneously with zero overhead. However, as soon as either process attempts to modify data within a page, the MMU intercepts the write attempt due to the read-only flag and triggers a page fault exception.
4. **Duplication on Demand:** The kernel handles the fault by allocating a new, independent physical frame in RAM, copying the contents of the shared page into it, updating the faulting process's PTE to point to this new frame, and re-enabling write permissions ($W = 1$) for that page. This minimizes memory overhead by only duplicating pages that are actively modified.

7.3 Zero-Fill-On-Demand (ZFOD)

When an application requests new memory pages (such as expanding its heap via `sbrk`), security constraints require that these pages contain no historical data to prevent information leaks across processes. The kernel implements this cleanly using **Zero-Fill-On-Demand (ZFOD)**:

- **Deferred Invalidation:** Instead of spending CPU cycles clearing large blocks of memory during the system call, the kernel simply maps the new pages to an invalid PTE pointer state.
- **Fault Clearing:** When the application first attempts to write to the new page, it triggers a page fault. The kernel catches the exception, allocates a free physical page frame, fills it with zeros, updates the PTE to point to this clean frame, sets the present bit to `1` , and resumes execution. To improve performance further, modern operating systems pre-clear pages in the background when the CPU is idle, allowing the fault handler to swap in a clean page instantly.

7.4 Inverted Page Tables

To eliminate the overhead of storing multi-tier tree structures for massive address spaces, modern 64-bit systems can switch from standard multi-level hierarchies to an **Inverted Page Table** architecture.

Flat/Hierarchical Table Scaling:	Size scales with Virtual Address Space (Huge)
Inverted Page Table Scaling:	Size scales with Physical Memory Space (Fixed and bounded)

- **Physical Frame Mapping:** Instead of maintaining a separate virtual-to-physical mapping array for every process, an inverted page table maintains a **single, global table for the entire system**. Each entry in this global table corresponds to a single, physical page frame in RAM.
- **Hash-Table Lookups:** Each entry tracks which process ID (`pid`) and virtual page number (`VPN`) currently occupy that physical frame. When the CPU issues a memory reference, the MMU routes the combination of the active `pid` and `VPN` through a hardware hash table to locate the matching entry and extract its frame index.
- **Bounded Table Sizing:** Because table entries are strictly bound to physical page frames, the size of the inverted page table scales with the amount of physical RAM installed in the machine rather than the size of the virtual address space. This completely avoids the memory explosion issue on 64-bit systems.

8. Operating System Memory Layouts & Kernel Isolation Protocols

8.1 32-bit Linux Memory Layout (The 3G/1G Split)

A classic 32-bit Linux system divides the process's 4 GiB virtual address space into two primary privilege zones using a strict **3G/1G Split**:

Linux 32-bit Process Memory Space Layout (Pre-Meltdown):

0xFFFFFFFF

Kernel Space Segment (1 GiB Allocation)
Privileged mode only; stores kernel code & page mappings

0xC0000000 (TASK_SIZE)

User Space Segment (3 GiB Allocation)
Stack segment (Grows downward)
... Gaps / Memory Mapping Segment ...
Heap segment (Grows upward)
BSS & Data segments
Text / Executable Binary segment

0x00000000

- **The User Space Boundary (0 to 3 GiB):** Stores the active process's executable binary text, initialized variables, BSS static blocks, heap allocation buffers, memory-mapped files (such as standard library file `/lib/libc.so`), and stack frames.

- **The Kernel Space Boundary (3 GiB to 4 GiB):** Starts exactly at address `0xC0000000` (defined in the kernel configuration as `TASK_SIZE`). This region stores the core operating system kernel's code, page tables, tracking structures, and device driver drivers.
- **The Privilege Guard:** When a process executes in user mode, any attempt to read or write to this kernel region is blocked by the MMU, throwing an immediate segmentation fault. However, by mapping this kernel region into the upper portion of every process's page table, the system avoids the need to reload page table pointers or flush TLB caches when an application switches to kernel mode via a system call, making transitions extremely fast.

8.2 Security Protections: ASLR & Kernel Page-Table Isolation (KPTI)

As hardware architectures evolved, shared page layouts introduced dangerous security vulnerabilities that forced modern operating systems to redesign their memory isolation protocols.

1. Address Space Layout Randomization (ASLR)

To prevent attackers from exploiting memory corruption vulnerabilities (such as buffer overflows), modern operating systems implement **Address Space Layout Randomization (ASLR)**.

The kernel randomizes the starting memory offsets of the process stack, heap, and executable text segments every time a program is run. Because code locations shift randomly with every run, an attacker cannot hard-code memory jump addresses into an exploit, significantly improving system security. This mechanism relies on compiling binaries as **Position-Independent Code (PIC)**, allowing code to execute correctly from any arbitrary base location in the address space.

2. The Meltdown Flaw & Kernel Page-Table Isolation (KPTI)

Historically, mapping the entire kernel space into each process's page table was considered safe because the MMU's privilege bits blocked user-space access. However, in early 2018, security researchers discovered the **Meltdown Vulnerability**, a critical design flaw affecting out-of-order execution optimizations in modern microprocessors.

Planar Hardware Layout Context (Speculative Leakage):
 [User Code Execution Path] —> Speculatively requests a protected Kernel memory address.
 Processor executes the instruction out-of-order, fetching data into cache before the MMU privilege check fails and blocks it.

↓

Side-channel timing analysis of the cache lines allows user code to reconstruct kernel data without permissions!

Because the entire kernel address space was resident inside the active page table, a user-space application could use side-channel timing attacks to read sensitive kernel data (including passwords, encryption keys, and other processes' memory) at speeds of multiple megabytes per second.

The Architectural Solution: KPTI

To mitigate this hardware vulnerability, operating systems implemented a strict protection mechanism known as **Kernel Page-Table Isolation (KPTI)**.

KPTI breaks the historic model of sharing a single page table per process. Instead, it splits each process's memory tracking into **two distinct page tables**:

1. **The Kernel-Mode Page Table:** Retains the historic layout, mapping the full user space alongside the entire kernel memory space for supervisor operations.
2. **The User-Mode Page Table:** Extracted and trimmed down for standard execution. It maps the full user space but removes the kernel space entirely, mapping only a minimal, essential island of trampoline code needed to process interrupts and system calls.

When an application is running in user mode, the kernel space is completely unmapped from the active page table, making it invisible to speculative execution side-channel attacks. When the process triggers a system call, the trampoline code handles the initial transition, reloads page table pointers to activate the kernel-mode table, and runs the operation safely. While KPTI introduces performance overhead due to frequent page table pointer swaps and TLB flushes during kernel transitions, it provides absolute protection against speculative execution vulnerabilities.

9. Core Concepts Exam Review (Verification Sheet)

Use this high-yield concept review to test your understanding before the exam:

1. Loader/Linker Relocation vs. Hardware Translation

- Loader/Linker relocation modifies an application's absolute machine code references at load time, providing no protection boundaries between applications. Hardware translation uses a dedicated Memory Management Unit (MMU) to translate virtual addresses to physical ones on the fly, enforcing absolute security isolation.

2. Base and Bound (B&B) Mechanics

- The MMU relocates addresses by adding a base register value ($\text{Physical} = \text{Virtual} + \text{Base}$) and enforces boundaries by checking against a bound register ($\text{Virtual} < \text{Bound}$). This model is simple but suffers from external memory fragmentation and cannot support sparse address spaces.

3. Segmentation Variables

- Segmentation divides virtual memory into logically independent segments (Code, Data, Stack). The virtual address is split into a segment identifier (`Seg ID #`) and an offset. The Seg ID indexes into a table to extract a base-and-limit pair. It eliminates the need to allocate physical RAM for unmapped holes but still requires contiguous memory for each segment, leading to fragmentation and high swapping costs.

4. Flat Memory Configurations

- Modern operating systems like Linux bypass x86 hardware segmentation by configuring a Flat Memory Model, setting the base address of all segment registers to `0x00000000` and their limits to `0xFFFFFFFF`. This allows the kernel to handle all translation and protection using paging.

5. Paging Principles & Address Format

- Paging eliminates external fragmentation by allocating physical memory in fixed-size, uniform blocks called page frames. Virtual addresses are split into a Virtual Page Number (`VPN`) and an `Offset`. The VPN indexes into a page table to retrieve a Physical Page Number (`PPN`), which is combined with the original offset to generate the physical address.

6. Sizing Page Tables

- For a 32-bit system with 4 KiB pages and 4-byte entries, a flat linear page table requires 2^{20} entries, consuming **4 MiB of RAM per process**. On a 64-bit architecture, a flat linear page table would require a ridiculous **32 Pebibytes** of storage, making multi-level or inverted page tables necessary.

7. Multi-Level Page Table Optimization

- A multi-level page table organizes translation entries into a hierarchical tree structure. Under a 10-10-12 bit pattern, the top-level index points to a Page Directory, which maps to second-level Page Tables. This avoids wasting memory on unmapped regions by allocating second-level tables only for active data segments, allowing sparse address spaces to be represented efficiently.

8. Inverted Page Table Mechanics

- An inverted page table maintains a single, global table for the entire system, where each entry corresponds to a specific physical page frame in RAM. It uses a hash table to map process IDs and virtual page numbers to frame indices, bounding the size of the table to the amount of physical RAM installed in the machine.

9. Flag Bits in the PTE

- Core Page Table Entry (PTE) flags include: **Present (P)** (tracks if the page is resident in RAM), **Read/Write (W)** (enforces write protection), **User/Supervisor (U)** (protects kernel space), **Accessed (A)** (tracks reads/writes for replacement selection), and **Dirty (D)** (tracks modifications before flushing to disk).

10. Memory Protection Mechanisms (COW, ZFOD, KPTI)

- **Copy-On-Write (COW)**: Links parent and child page tables to the same physical memory frames and marks them read-only, duplicating pages only when a write operation occurs.
- **Zero-Fill-On-Demand (ZFOD)**: Defers zeroing new memory allocations by marking pages invalid, filling them with zeros only when an initial write fault occurs.
- **Kernel Page-Table Isolation (KPTI)**: Mitigates the Meltdown vulnerability by maintaining two separate page tables per process—a user-mode table that completely unmaps kernel memory, and a kernel-mode table for supervisor operations.